



**Institute for Advanced Studies
in Basic Sciences
GavaZang, Zanjan, Iran**

Deep Learning Problem Set

Prof. Parvin Razzaghi

Report for Problem Set

Faeze Ahmadi

14044141

August 2026

Introduction

This problem set is designed to bridge the gap between theoretical deep learning concepts and their practical implications. Throughout these tasks, the goal extends beyond merely implementing algorithms or achieving high accuracy scores; it is a systematic journey to understand the why behind the behavior of neural networks. By exploring different architectures, training dynamics, and evaluation methods, this series of problems aims to uncover how specific design choices—such as network depth, width, or connectivity—shape a model's ability to learn and generalize.

The structure of this project reflects the multifaceted nature of modern AI research. We begin by examining the architectural nuances of Convolutional Neural Networks (CNNs), specifically through the lens of Residual Networks (ResNets). By testing these models on datasets like CIFAR-10 and CIFAR-100, we aim to find the sweet spot where computational efficiency meets predictive power. However, high accuracy is only one side of the coin. The later sections of this problem set transition into the critical realm of robustness and security, investigating how even the most sophisticated models can be vulnerable to subtle, adversarial perturbations like those generated by the FGSM attack.

This document is more than just a collection of results; it is a narrative of the learning process. Each of the six problems presented here serves as a building block for a comprehensive understanding of deep learning systems. In the following sections, I have documented the implementation details, visualized the performance metrics, and provided a deep dive into the analysis of each result. By reflecting on the challenges encountered and the patterns observed, this report aims to provide a clear and intuitive explanation of how these models function in practice and how we can make them more reliable and resilient.

Problem 1: Systematic Exploration of Architectures and Robustness

The first problem of this assignment serves as a deep dive into the structural heart of modern computer vision: the Convolutional Neural Network (CNN). Before jumping into the code, I had to clarify what these components actually do. At its core, a CNN is designed to mimic how the human visual system works—instead of looking at every pixel individually, it uses filters to detect patterns like edges, textures, and eventually complex objects. However, as I realized during the implementation, simply making these networks deeper doesn't always make them better.

This is where ResNet (Residual Network) comes into play. One of the biggest challenges I encountered was understanding how to train very deep networks without the gradients vanishing or the model's performance degrading. ResNet solves this beautifully using Skip Connections (or shortcuts). These connections allow information to bypass certain layers, ensuring that the network can learn even as it grows to 18, 34, or 50 layers. In this task, I implemented three versions (ResNet-18, 34, and 50) to see the trade-off between the number of parameters and the final accuracy.

To test these architectures, I used two benchmark datasets: CIFAR-10 and CIFAR-100.

- CIFAR-10 consists of 60,000 tiny (32×32) color images across 10 classes, with 5,000 training images per class. It serves as a simpler baseline for checking whether a model learns useful visual features.
- CIFAR-100 is much more challenging; it has the same total number of images as CIFAR-10, but they are divided into 100 classes instead of 10. As a result, each class has far fewer training examples, which makes generalization significantly harder.

In the first part of this problem (Task A), I primarily explore how changing the depth and overall model capacity affects performance. Although width is mentioned as a theoretical factor, it is not independently varied in the implemented experiment, so the empirical analysis focuses on depth. I was looking for the Sweet Spot—that perfect point where the model is complex enough to learn the data but not so heavy that it becomes too slow or starts overfitting.

The second part (Task B) moves beyond simple accuracy and looks at Linear Mode Connectivity. This asks whether two independently trained models can be connected by a straight line in weight space without passing through a low-accuracy region. I explored how modern techniques like Batch Normalization (BN) and those Skip Connections I mentioned earlier affect this landscape.

Finally, I tackled the issue of Robustness. Even a model with 90% accuracy can be easily fooled by tiny, invisible changes to an image. Using the FGSM (Fast Gradient Sign Method) attack, I investigated how vulnerable these models are and experimented with Adversarial Training to see if we can teach the model to defend itself.

Explain the code of problem set 1:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
import time
import copy
import random

device = torch.device(cuda if torch.cuda.is_available() else cpu)
print(Device:, device)

SEED = 42
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(SEED)
```

This section imports the libraries needed for building, training, and evaluating the deep learning models. It then selects the GPU if CUDA is available (otherwise it uses the CPU) and fixes the random seed to make the experiments more reproducible.

Output:

```
Device: cuda
```

It means the code is running on the GPU.

Cell 2:

```
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

def get_dataloaders(dataset_name=cifar10, batch_size=128,
subset_fraction=1.0):
    if dataset_name == cifar10:
        train_set = torchvision.datasets.CIFAR10(root=../data, train=True,
download=True, transform=transform_train)
        test_set = torchvision.datasets.CIFAR10(root=../data, train=False,
download=True, transform=transform_test)
        num_classes = 10
    elif dataset_name == cifar100:
        train_set = torchvision.datasets.CIFAR100(root=../data, train=True,
download=True, transform=transform_train)
        test_set = torchvision.datasets.CIFAR100(root=../data, train=False,
download=True, transform=transform_test)
        num_classes = 100
    else:
        raise ValueError(dataset_name must be cifar10 or cifar100)

    if subset_fraction < 1.0:
        n_train = int(len(train_set) * subset_fraction)
        indices = list(range(len(train_set)))
        random.shuffle(indices)
        train_set = torch.utils.data.Subset(train_set, indices[:n_train])
```

```

        train_loader = torch.utils.data.DataLoader(train_set,
batch_size=batch_size, shuffle=True, num_workers=2)
        test_loader = torch.utils.data.DataLoader(test_set, batch_size=batch_size,
shuffle=False, num_workers=2)
        return train_loader, test_loader, num_classes

```

This cell handles data preparation—arguably the most important step before training. It defines two transformation pipelines: one for training and one for testing. The training pipeline applies data augmentation—a random 32×32 crop with 4 pixels of padding and a random horizontal flip—which artificially diversifies the dataset by presenting slightly different versions of each image every epoch. This acts as a regularizer and helps the model generalize instead of memorizing. Both pipelines then convert images to tensors and normalize them using the precomputed mean and standard deviation of CIFAR (0.4914, 0.4822, 0.4465 for RGB channels), which centers the pixel values around zero and keeps training numerically stable. The test pipeline deliberately omits augmentation, because evaluation must be deterministic to fairly measure performance.

The `get_dataloaders` function wraps this logic into a reusable utility: it downloads and loads either CIFAR-10 or CIFAR-100 based on the given name, optionally takes a random subset of the training data (useful for quick experiments), and returns `DataLoader` objects that feed the network with mini-batches of a specified size. Shuffling is enabled for training to prevent the model from learning order-dependent patterns, while the test loader keeps the original order. In short, this cell turns raw image files into clean, augmented, batched tensors ready for the model.

Cell 3:

```

class BasicBlock(nn.Module):
    expansion = 1
    def __init__(self, in_planes, planes, stride=1, use_skip=True):
        super().__init__()
        self.use_skip = use_skip
        self.conv1 = nn.Conv2d(in_planes, planes, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.shortcut = nn.Sequential()
        if use_skip and (stride != 1 or in_planes != planes * self.expansion):
            self.shortcut = nn.Sequential(

```

```

        nn.Conv2d(in_planes, planes=self.expansion, 1, stride,
bias=False),
        nn.BatchNorm2d(planes=self.expansion)
    )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.use_skip:
            out += self.shortcut(x)
        out = F.relu(out)
        return out

class Bottleneck(nn.Module):
    expansion = 4
    def __init__(self, in_planes, planes, stride=1, use_skip=True):
        super().__init__()
        self.use_skip = use_skip
        self.conv1 = nn.Conv2d(in_planes, planes, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, 3, stride, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.conv3 = nn.Conv2d(planes, planes=self.expansion, 1, bias=False)
        self.bn3 = nn.BatchNorm2d(planes=self.expansion)
        self.shortcut = nn.Sequential()
        if use_skip and (stride != 1 or in_planes != planes=self.expansion):
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, planes=self.expansion, 1, stride,
bias=False),
                nn.BatchNorm2d(planes=self.expansion)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = F.relu(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        if self.use_skip:
            out += self.shortcut(x)

```

```

        out = F.relu(out)
        return out

class ResNetCIFAR(nn.Module):
    def __init__(self, block, num_blocks, num_classes=10, use_bn=True,
use_skip=True):
        super().__init__()
        self.in_planes = 64
        self.use_bn = use_bn
        self.use_skip = use_skip

        self.conv1 = nn.Conv2d(3, 64, 3, 1, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(64) if use_bn else nn.Identity()
        self.layer1 = self._make_layer(block, 64, num_blocks[0], 1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], 2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], 2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], 2)
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512 + block.expansion, num_classes)

    def _make_layer(self, block, planes, num_blocks, stride):
        strides = [stride] + [1] * (num_blocks - 1)
        layers = []
        for s in strides:
            layers.append(block(self.in_planes, planes, s,
use_skip=self.use_skip))
        self.in_planes = planes + block.expansion
        return nn.Sequential(layers)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = self.avgpool(out)
        out = out.view(out.size(0), -1)
        out = self.fc(out)

```



```

        return out

def ResNet18(num_classes=10, use_bn=True, use_skip=True):
    return ResNetCIFAR(BasicBlock, [2, 2, 2, 2], num_classes, use_bn, use_skip)

def ResNet34(num_classes=10, use_bn=True, use_skip=True):
    return ResNetCIFAR(BasicBlock, [3, 4, 6, 3], num_classes, use_bn, use_skip)

def ResNet50(num_classes=10, use_bn=True, use_skip=True):
    return ResNetCIFAR(Bottleneck, [3, 4, 6, 3], num_classes, use_bn, use_skip)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

```

This cell defines a flexible ResNet architecture adapted for CIFAR images. Its main components are BasicBlock and Bottleneck, which are the building blocks used to construct different ResNet depths. A BasicBlock contains two (3×3) convolutional layers, followed by Batch Normalization and ReLU activation. It is used in ResNet-18 and ResNet-34. The Bottleneck block, used in ResNet-50, contains three convolutions: a (1×1) layer to reduce the number of channels, a (3×3) layer to learn spatial patterns, and another (1×1) layer to expand the channels. This design makes deeper networks more computationally efficient.

Both blocks optionally use a skip connection, which adds the original input to the output of the convolutional layers. If the input and output shapes differ—for example, because the number of channels changes or downsampling is applied—a (1×1) convolution is used in the shortcut path to match their dimensions. These residual connections help gradients flow through deeper models and make training more stable. The `use_skip` argument makes it possible to disable this feature, allowing a direct comparison between models with and without skip connections.

The ResNetCIFAR class combines these blocks into the full network. It first processes the RGB image with an initial convolutional layer, then passes the features through four stages with increasing numbers of channels: 64, 128, 256, and 512. The later stages use a stride of 2 to reduce the spatial size of feature maps while learning higher-level visual features. The initial Batch Normalization layer can be enabled or disabled using the `use_bn` option, which allows us to modify the normalization configuration for the subsequent experiment.

Finally, global average pooling reduces each final feature map to a single value, and a fully connected layer produces the final class scores. The helper functions ResNet18, ResNet34, and ResNet50 instantiate the corresponding ResNet variants by specifying the appropriate block type and number of blocks per stage. The `count_parameters` function

then calculates the number of trainable parameters in each model, making it easier to compare architectural complexity with accuracy and training cost.

Cell 4:

```
def train_model(model, train_loader, test_loader, epochs=8, lr=0.01):
    model = model.to(device)
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,
weight_decay=5e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
scheduler.step()
    criterion = nn.CrossEntropyLoss()

    epoch_times = []
    for epoch in range(epochs):
        model.train()
        t0 = time.time()
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
        scheduler.step()
        t1 = time.time()
        epoch_times.append(t1 - t0)
        acc = evaluate(model, test_loader)
        print(f" Epoch {epoch+1}/{epochs} - time: {t1-t0:.1f}s - test_acc:
{acc:.4f}")

    avg_epoch_time = np.mean(epoch_times)
    final_acc = evaluate(model, test_loader)
    return model, final_acc, avg_epoch_time

def evaluate(model, loader):
    model.eval()
```

```

correct, total = 0, 0
with torch.no_grad():
    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
return correct / total

```

This cell defines the training and evaluation pipeline used for all ResNet experiments. The `train_model` function first moves the model to the selected device, which is the GPU in this case. It then defines the main components needed for training: SGD as the optimizer, Cross-Entropy Loss as the classification loss function, and a Cosine Annealing learning-rate scheduler. SGD updates the model weights after each batch, while momentum helps make these updates more stable and faster. Weight decay is also included to reduce overfitting. The scheduler gradually decreases the learning rate during training, allowing the model to make larger updates early on and more refined updates toward the end.

For each epoch, the model is switched to training mode using `model.train()`. The code then iterates through the training batches, sends the images and labels to the GPU, clears gradients from the previous step, performs a forward pass, computes the loss, calculates gradients through backpropagation, and finally updates the model parameters. The execution time of each epoch is recorded, and after every epoch the model is evaluated on the test set. This makes it possible to monitor both the training speed and the test accuracy throughout the experiment.

The `evaluate` function measures the classification accuracy of a trained model. It switches the model to evaluation mode using `model.eval()` and disables gradient calculations with `torch.no_grad()`, since gradients are not needed during testing. For each batch, the model produces class scores, and the class with the highest score is selected as the prediction. The function then compares these predictions with the true labels and returns the proportion of correctly classified images.

Cell 5:

```

EPOCHS = 8
SUBSET = 1.0

train_loader_10, test_loader_10, nc10 = get_dataloaders("cifar10",
subset_fraction=SUBSET)

```

```

architectures = {
    "ResNet-18": ResNet18,
    "ResNet-34": ResNet34,
    "ResNet-50": ResNet50,
}

results_cifar10 = {}

for name, constructor in architectures.items():
    print(f"\nTraining {name} on CIFAR-10")
    set_seed(SEED)
    model = constructor(num_classes=nc10)
    n_params = count_parameters(model)
    trained_model, acc, avg_time = train_model(model, train_loader_10,
test_loader_10, epochs=EPOCHS)
    results_cifar10[name] = {
        "test_accuracy": acc,
        "avg_epoch_time": avg_time,
        "num_params": n_params,
        "model": trained_model
    }

    print(f"{name}:          acc={acc:.4f},          params={n_params:},,
avg_epoch_time={avg_time:.1f}s")

print("\nSummary (CIFAR-10)")
for name, r in results_cifar10.items():
    print(f"{name}: acc={r['test_accuracy']:.4f}, params={r['num_params']:,},
epoch_time={r['avg_epoch_time']:.1f}s")

```

This cell runs the first systematic experiment of the whole problem: comparing three ResNet variants on CIFAR-10. It starts by setting two hyperparameters—8 epochs and using 100% of the dataset—and then loads the CIFAR-10 data loaders through the utility function we defined earlier. Next, it creates a dictionary called `architectures` that maps each model name (ResNet-18, ResNet-34, and ResNet-50) to its factory function. This is a clean and convenient way to loop over all three architectures without writing repetitive code.

In the loop, each architecture is trained one by one. Before training each model, `set_seed(SEED)` is called to improve reproducibility and to keep the random conditions as consistent as possible across the experiments. This is an important detail: it keeps the comparison fair, since any difference in accuracy should come from the architecture itself and not from random luck. The model is then created with the correct number of classes, and the number of trainable parameters is counted so we can compare model sizes. After training, the final test accuracy, the average epoch time, and the parameter count are all stored in a dictionary called `results_cifar10`. Since the trained model itself is also stored, we can reuse it later for the robustness experiments. Finally, a summary of all three models is printed, giving a clear side-by-side view of accuracy, parameter count, and training time. In short, this cell produces the first key comparison table of the assignment: how depth (and model size) affects both accuracy and training speed.

Output:

```
100%|██████████| 170M/170M [25:40<00:00, 111kB/s]

Training ResNet-18 on CIFAR-10
Epoch 1/8 - time: 39.7s - test_acc: 0.5752
Epoch 2/8 - time: 40.0s - test_acc: 0.6579
Epoch 3/8 - time: 44.3s - test_acc: 0.7480
Epoch 4/8 - time: 44.3s - test_acc: 0.8055
Epoch 5/8 - time: 44.2s - test_acc: 0.8331
Epoch 6/8 - time: 44.1s - test_acc: 0.8438
Epoch 7/8 - time: 44.0s - test_acc: 0.8648
Epoch 8/8 - time: 44.2s - test_acc: 0.8735
ResNet-18: acc=0.8735, params=11,173,962, avg_epoch_time=43.1s

Training ResNet-34 on CIFAR-10
Epoch 1/8 - time: 75.4s - test_acc: 0.5595
Epoch 2/8 - time: 75.0s - test_acc: 0.6729
Epoch 3/8 - time: 75.8s - test_acc: 0.6731
Epoch 4/8 - time: 75.3s - test_acc: 0.7950
Epoch 5/8 - time: 76.0s - test_acc: 0.8180
Epoch 6/8 - time: 75.7s - test_acc: 0.8465
Epoch 7/8 - time: 75.7s - test_acc: 0.8699
Epoch 8/8 - time: 75.9s - test_acc: 0.8785
ResNet-34: acc=0.8785, params=21,282,122, avg_epoch_time=75.6s
```

```

Training ResNet-50 on CIFAR-10
Epoch 1/8 - time: 173.0s - test_acc: 0.4813
Epoch 2/8 - time: 173.1s - test_acc: 0.6068
Epoch 3/8 - time: 173.9s - test_acc: 0.6754
Epoch 4/8 - time: 173.8s - test_acc: 0.7578
Epoch 5/8 - time: 173.6s - test_acc: 0.7893
Epoch 6/8 - time: 173.8s - test_acc: 0.8043
Epoch 7/8 - time: 173.3s - test_acc: 0.8519
Epoch 8/8 - time: 173.4s - test_acc: 0.8571
ResNet-50: acc=0.8571, params=23,520,842, avg_epoch_time=173.5s

Summary (CIFAR-10)
ResNet-18: acc=0.8735, params=11,173,962, epoch_time=43.1s
ResNet-34: acc=0.8785, params=21,282,122, epoch_time=75.6s
ResNet-50: acc=0.8571, params=23,520,842, epoch_time=173.5s

```

The output shows that CIFAR-10 was downloaded successfully, and then the three ResNet models were trained one by one. What is immediately noticeable is the clear difference in training time across architectures: ResNet-18 is the fastest, ResNet-34 takes noticeably longer, and ResNet-50 is by far the slowest. This makes sense because deeper models with more layers and more parameters require more computation in every epoch. At the same time, the test accuracy steadily improves during training for all three models, which shows that the optimization process is working properly and the networks are learning meaningful features from the data.

Looking at the final results, ResNet-34 achieved the best test accuracy at 0.8785, followed very closely by ResNet-18 at 0.8735, while ResNet-50 ended slightly lower at 0.8571. This is an important result because it shows that the deepest model was not necessarily the best one for this task. In fact, ResNet-50 had the largest number of parameters and the highest training cost, but it did not outperform the smaller models on CIFAR-10. So overall, this output suggests that for this dataset, ResNet-34 gives the best trade-off between accuracy and efficiency, while ResNet-18 remains a very strong and much cheaper baseline.

Cell 6:

```

train_loader_100, test_loader_100, nc100 = get_dataloaders("cifar100",
subset_fraction=SUBSET)

results_cifar100 = {}

```

```

for name, constructor in architectures.items():
    print(f"\nTraining {name} on CIFAR-100")
    set_seed(SEED)
    model = constructor(num_classes=nc100)
    n_params = count_parameters(model)
    trained_model, acc, avg_time = train_model(model, train_loader_100,
test_loader_100, epochs=EPOCHS)
    results_cifar100[name] = {
        "test_accuracy": acc,
        "avg_epoch_time": avg_time,
        "num_params": n_params,
        "model": trained_model
    }

    print(f"{name}:          acc={acc:.4f},          params={n_params:},,
avg_epoch_time={avg_time:.1f}s")

print("\nSummary (CIFAR-100)")
for name, r in results_cifar100.items():
    print(f"{name}: acc={r['test_accuracy']:.4f}, params={r['num_params']:},,
epoch_time={r['avg_epoch_time']:.1f}s")

```

This cell repeats the same architectural comparison experiment, but this time on the more challenging CIFAR-100 dataset. It first loads CIFAR-100 using the previously defined `get_dataloaders` function. Unlike CIFAR-10, this dataset contains 100 classes instead of 10, so the classification task is considerably harder. The variable `nc100` is used when creating each model to make sure that the final fully connected layer produces scores for 100 possible classes.

The code then trains ResNet-18, ResNet-34, and ResNet-50 separately under the same training settings used for CIFAR-10. Before each run, the random seed is reset to ensure a fair and reproducible comparison. For every architecture, the code counts its trainable parameters, trains it for the specified number of epochs, and saves its final test accuracy, average epoch time, parameter count, and trained model in the `results_cifar100` dictionary. At the end, it prints a summary table that makes it easy to compare the three models.

Overall, this cell helps us examine whether the conclusions from CIFAR-10 still hold when the task becomes more difficult. Since CIFAR-100 has many more categories and fewer examples per class, it is useful for evaluating whether deeper and larger architectures can learn more detailed visual differences, and whether their additional computational cost leads to a meaningful improvement in accuracy.

Output:

```
100%|██████████| 169M/169M [19:35<00:00, 144kB/s]

Training ResNet-18 on CIFAR-100
Epoch 1/8 - time: 46.0s - test_acc: 0.2145
Epoch 2/8 - time: 43.2s - test_acc: 0.3271
Epoch 3/8 - time: 44.3s - test_acc: 0.3806
Epoch 4/8 - time: 44.3s - test_acc: 0.4569
Epoch 5/8 - time: 44.1s - test_acc: 0.5017
Epoch 6/8 - time: 44.2s - test_acc: 0.5524
Epoch 7/8 - time: 44.5s - test_acc: 0.5872
Epoch 8/8 - time: 44.2s - test_acc: 0.6064
ResNet-18: acc=0.6064, params=11,220,132, avg_epoch_time=44.4s

Training ResNet-34 on CIFAR-100
Epoch 1/8 - time: 75.1s - test_acc: 0.1846
Epoch 2/8 - time: 75.8s - test_acc: 0.2610
Epoch 3/8 - time: 75.4s - test_acc: 0.3382
Epoch 4/8 - time: 75.6s - test_acc: 0.3770
Epoch 5/8 - time: 75.5s - test_acc: 0.4666
Epoch 6/8 - time: 75.2s - test_acc: 0.5259
Epoch 7/8 - time: 75.9s - test_acc: 0.5764
Epoch 8/8 - time: 75.5s - test_acc: 0.5984
ResNet-34: acc=0.5984, params=21,328,292, avg_epoch_time=75.5s

Training ResNet-50 on CIFAR-100
Epoch 1/8 - time: 172.6s - test_acc: 0.1355
Epoch 2/8 - time: 173.3s - test_acc: 0.2112
Epoch 3/8 - time: 173.9s - test_acc: 0.2831
Epoch 4/8 - time: 173.8s - test_acc: 0.3440
Epoch 5/8 - time: 173.3s - test_acc: 0.4170
Epoch 6/8 - time: 173.1s - test_acc: 0.4764
Epoch 7/8 - time: 173.5s - test_acc: 0.5140
Epoch 8/8 - time: 173.8s - test_acc: 0.5386
ResNet-50: acc=0.5386, params=23,705,252, avg_epoch_time=173.4s

Summary (CIFAR-100)
```



```
ResNet-18: acc=0.6064, params=11,220,132, epoch_time=44.4s
ResNet-34: acc=0.5984, params=21,328,292, epoch_time=75.5s
ResNet-50: acc=0.5386, params=23,705,252, epoch_time=173.4s
```

The output shows that the three ResNet architectures were trained on CIFAR-100, which is a more difficult classification problem than CIFAR-10 because it contains 100 classes instead of 10. The dataset was downloaded successfully, and all three models improved their test accuracy throughout the eight epochs. However, the starting accuracies were relatively low, especially for ResNet-50, which began with only 13.55% accuracy. This is expected because the model starts with randomly initialized weights and must distinguish between a much larger number of classes.

Among the three models, ResNet-18 achieved the highest final test accuracy at 60.64%. ResNet-34 reached 59.84%, while ResNet-50 achieved 53.86%. Therefore, increasing the depth of the network did not improve performance in this experiment. ResNet-50 had the largest number of parameters and required the longest training time, but it produced the lowest accuracy. This may indicate that the deeper models needed more training epochs or more carefully tuned hyperparameters to fully benefit from their larger capacity. Based on these results, ResNet-18 provided the best balance of accuracy and computational efficiency for CIFAR-100, while the deeper models introduced considerable additional cost without producing better results.

Cell 7:

```
names = list(results_cifar10.keys())
params = [results_cifar10[n]["num_params"] / 1e6 for n in names]
accs = [results_cifar10[n]["test_accuracy"] for n in names]
times = [results_cifar10[n]["avg_epoch_time"] for n in names]

fig, ax1 = plt.subplots(figsize=(9, 6))

color1 = "tab:blue"
ax1.set_xlabel("Model")
ax1.set_ylabel("Test Accuracy", color=color1)
ax1.plot(names, accs, "o-", color=color1, label="Accuracy vs Params (see markers)")
ax1.tick_params(axis="y", labelcolor=color1)

for i, n in enumerate(names):
```

```

        ax1.annotate(f"{params[i]:.1f}M   paramsn{times[i]:.1f}s/epoch", (n,
accs[i]),
                    textcoords="offset points", xytext=(10, 5), fontsize=8)

ax2 = ax1.twinx()
color2 = "tab:red"
ax2.set_ylabel("Params (M) / Epoch Time (s)", color=color2)
ax2.plot(names, params, "s--", color="tab:green", label="Params (M)")
ax2.plot(names, times, "^--", color=color2, label="Epoch Time (s)")
ax2.tick_params(axis="y", labelcolor=color2)

fig.suptitle("Accuracy vs Parameter Count & Training Time (CIFAR-10)")
fig.legend(loc="upper left", bbox_to_anchor=(0.1, 0.9))
plt.tight_layout()
plt.show()

# CIFAR-10 vs CIFAR-100 comparison
print("\nCIFAR-10 vs CIFAR-100 comparison:")
for name in names:
    print(f"{name}: CIFAR-10 acc={results_cifar10[name]['test_accuracy']:.4f}
| "
        f"CIFAR-100 acc={results_cifar100[name]['test_accuracy']:.4f}")

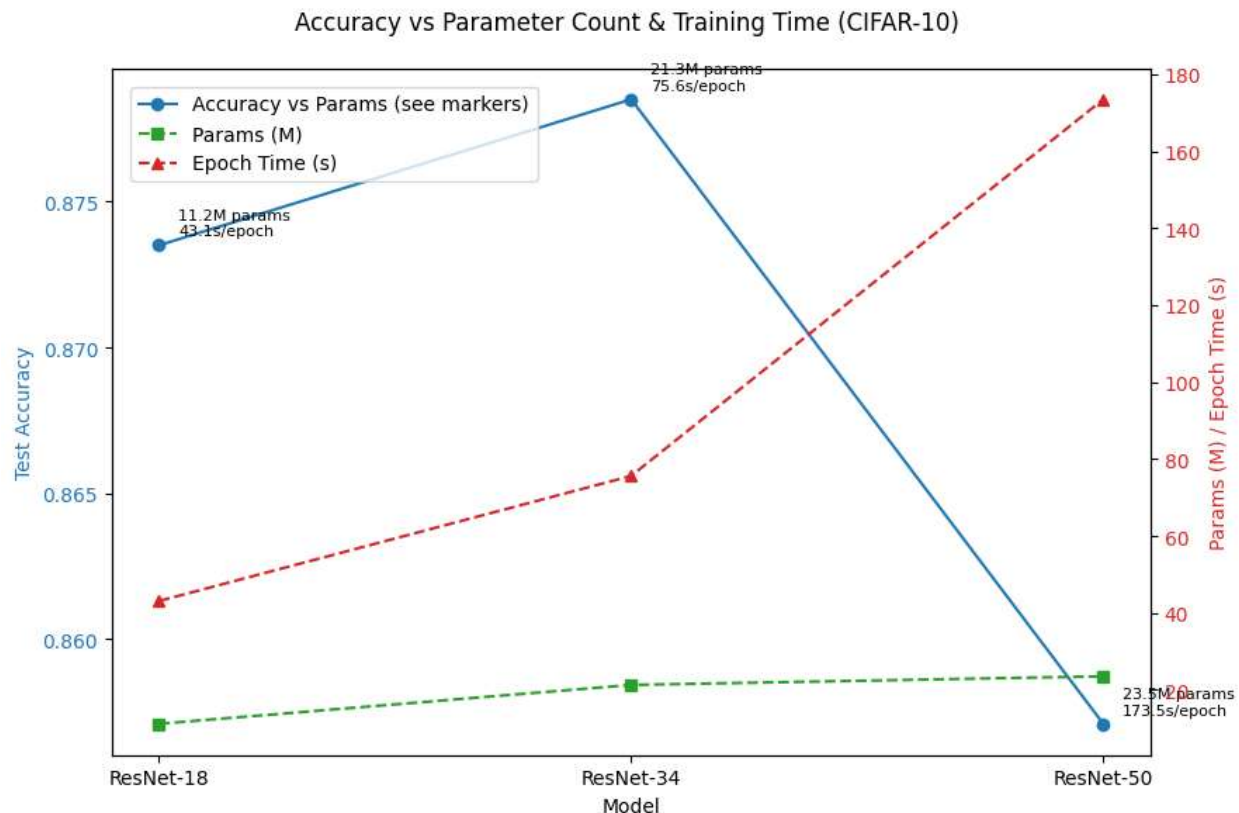
```

This cell visualizes the main results from the CIFAR-10 experiment and makes the comparison between the three ResNet architectures easier to understand. It first extracts the model names, the number of trainable parameters, the final test accuracies, and the average training time per epoch from the `results_cifar10` dictionary. The parameter counts are divided by one million so that they can be displayed in a more readable form, such as 11.2M instead of 11,173,962 parameters.

The figure uses two vertical axes because the values have very different scales. The left axis shows test accuracy in blue, while the right axis shows the number of parameters and the average epoch time. Accuracy is displayed as a blue line with circular markers, parameter count is represented by a green dashed line with square markers, and training time is shown as a red dashed line with triangular markers. Each model point is also annotated with its parameter count and training time, so the figure provides all important information in one place. This allows us to visually inspect the trade-off between model complexity, computational cost, and classification performance.

Finally, the code prints a direct comparison between CIFAR-10 and CIFAR-100 for each architecture. This helps show how model performance changes when moving from the simpler 10-class dataset to the much more difficult 100-class dataset. In general, the accuracy is expected to be lower on CIFAR-100 because the model must distinguish between more visually similar categories with fewer training examples available for each class.

Output:



CIFAR-10 vs CIFAR-100 comparison:

ResNet-18: CIFAR-10 acc=0.8735 | CIFAR-100 acc=0.6064

ResNet-34: CIFAR-10 acc=0.8785 | CIFAR-100 acc=0.5984

ResNet-50: CIFAR-10 acc=0.8571 | CIFAR-100 acc=0.5386

The comparison shows that all three models perform better on CIFAR-10 than on CIFAR-100. This is expected because CIFAR-10 contains only 10 classes, while CIFAR-100 requires the models to distinguish between 100 more detailed and visually similar categories. As a result, the accuracy drops for every architecture when moving to CIFAR-100.

On CIFAR-10, ResNet-34 achieves the highest accuracy at 87.85%, slightly outperforming ResNet-18 at 87.35%. However, on CIFAR-100, ResNet-18 performs best with an accuracy of 60.64%, while ResNet-34 reaches 59.84% and

ResNet-50 reaches only 53.86%. This shows that a deeper architecture does not always produce better results, especially when the training setup is limited to only eight epochs. ResNet-50 may require more training time or better hyperparameter tuning to take advantage of its larger capacity.

The graph also highlights the computational cost of increasing model depth. ResNet-18 has approximately 11.2 million parameters and takes 43.1 seconds per epoch, whereas ResNet-34 has around 21.3 million parameters and requires 75.6 seconds per epoch. ResNet-50 is slightly larger, with about 23.5 million parameters, but its average epoch time increases significantly to 173.5 seconds. Therefore, ResNet-34 provides the best accuracy on CIFAR-10, while ResNet-18 offers the most efficient overall choice because it is faster, smaller, and performs competitively on both datasets.

Cell 8:

```
def make_small_cnn(num_classes=10, use_bn=True, use_skip=True):
    return ResNetCIFAR(BasicBlock, [1, 1, 1, 1], num_classes=num_classes,
                        use_bn=use_bn, use_skip=use_skip)

def interpolate_and_evaluate(model_a, model_b, test_loader, num_classes,
                             use_bn, use_skip, n_points=11):
    state_a = model_a.state_dict()
    state_b = model_b.state_dict()
    alphas = np.linspace(0, 1, n_points)
    accs = []
    for alpha in alphas:
        interp_model = make_small_cnn(num_classes, use_bn, use_skip).to(device)
        interp_state = {}
        for key in state_a:
            interp_state[key] = (1 - alpha) * state_a[key] + alpha * state_b[key]
        interp_model.load_state_dict(interp_state)
        acc = evaluate(interp_model, test_loader)
        accs.append(acc)
        print(f"  alpha={alpha:.2f} -> acc={acc:.4f}")
    return alphas, accs

def run_lmc_experiment(use_bn, use_skip, label, epochs=6):
    print(f"nLMC experiment: {label} (BN={use_bn}, Skip={use_skip})")
    set_seed(1)
```

```

model_a = make_small_cnn(nc10, use_bn, use_skip)
model_a, acc_a, _ = train_model(model_a, train_loader_10, test_loader_10,
epochs=epochs)

set_seed(2)
model_b = make_small_cnn(nc10, use_bn, use_skip)
model_b, acc_b, _ = train_model(model_b, train_loader_10, test_loader_10,
epochs=epochs)

print(f"Model A acc={acc_a:.4f}, Model B acc={acc_b:.4f}")
alphas, accs = interpolate_and_evaluate(model_a, model_b, test_loader_10,
nc10, use_bn, use_skip)
return alphas, accs

configs = [
    (True, True, "BN + Skip"),
    (True, False, "BN, No Skip"),
    (False, True, "No BN, Skip"),
    (False, False, "No BN, No Skip"),
]

lmc_results = {}
for use_bn, use_skip, label in configs:
    alphas, accs = run_lmc_experiment(use_bn, use_skip, label)
    lmc_results[label] = (alphas, accs)

plt.figure(figsize=(9, 6))
for label, (alphas, accs) in lmc_results.items():
    plt.plot(alphas, accs, "o-", label=label)
plt.xlabel("Interpolation coefficient (alpha)")
plt.ylabel("Test Accuracy")
plt.title("Linear Mode Connectivity: effect of BatchNorm & Skip Connections")
plt.legend()
plt.grid(True)
plt.show()

```

This cell sets up the experiment for studying Linear Mode Connectivity in a simple and controlled way. First, `make_small_cnn` creates a smaller version of the ResNet-based model by using only one block in each stage, so it behaves like a lightweight CNN. The reason for using a smaller network here is to make the interpolation experiment easier to run and easier to interpret. After that, `interpolate_and_evaluate` takes two trained models and checks what happens if we move gradually from one model's weights to the other's weights. It does this by creating several points between them using values of α from 0 to 1. For each α , the code builds a new model whose parameters are a weighted mix of the two trained models, then evaluates its test accuracy. This tells us whether there is a smooth high-accuracy path between the two solutions or whether performance drops in the middle.

The `run_lmc_experiment` function automates the whole process. It trains two separate models from different random seeds, so they start from different initializations and hopefully end up in different regions of the loss landscape. Then it prints their final accuracies and runs the interpolation test between them. The code repeats this experiment for four configurations by varying the Batch Normalization setting and the presence of skip connections. This allows us to compare how these architectural settings affect the interpolation behavior. Finally, the results are plotted so we can visually see whether accuracy stays high along the interpolation path. In short, this cell is designed to show whether the model solutions are connected by a simple straight line in weight space, and how batch normalization and skip connections influence that behavior.

Output:

```
LMC experiment: BN + Skip (BN=True, Skip=True)
Epoch 1/6 - time: 27.4s - test_acc: 0.5401
Epoch 2/6 - time: 28.0s - test_acc: 0.6208
Epoch 3/6 - time: 26.8s - test_acc: 0.6900
Epoch 4/6 - time: 27.1s - test_acc: 0.7779
Epoch 5/6 - time: 27.1s - test_acc: 0.8026
Epoch 6/6 - time: 26.9s - test_acc: 0.8222
Epoch 1/6 - time: 27.1s - test_acc: 0.5795
Epoch 2/6 - time: 26.9s - test_acc: 0.6629
Epoch 3/6 - time: 26.9s - test_acc: 0.7291
Epoch 4/6 - time: 27.2s - test_acc: 0.7712
Epoch 5/6 - time: 26.9s - test_acc: 0.7737
Epoch 6/6 - time: 27.0s - test_acc: 0.8273
Model A acc=0.8222, Model B acc=0.8273
alpha=0.00 -> acc=0.8222
alpha=0.10 -> acc=0.6663
alpha=0.20 -> acc=0.2464
```

```
alpha=0.30 -> acc=0.1079
alpha=0.40 -> acc=0.1001
alpha=0.50 -> acc=0.1000
alpha=0.60 -> acc=0.1049
alpha=0.70 -> acc=0.1342
alpha=0.80 -> acc=0.2347
alpha=0.90 -> acc=0.6391
alpha=1.00 -> acc=0.8273
```

LMC experiment: BN, No Skip (BN=True, Skip=False)

```
Epoch 1/6 - time: 24.9s - test_acc: 0.5437
Epoch 2/6 - time: 24.7s - test_acc: 0.6813
Epoch 3/6 - time: 24.7s - test_acc: 0.7370
Epoch 4/6 - time: 24.5s - test_acc: 0.7940
Epoch 5/6 - time: 24.7s - test_acc: 0.8200
Epoch 6/6 - time: 24.6s - test_acc: 0.8437
Epoch 1/6 - time: 24.6s - test_acc: 0.5893
Epoch 2/6 - time: 24.7s - test_acc: 0.5556
Epoch 3/6 - time: 24.7s - test_acc: 0.7396
Epoch 4/6 - time: 24.8s - test_acc: 0.7901
Epoch 5/6 - time: 25.0s - test_acc: 0.7970
Epoch 6/6 - time: 24.8s - test_acc: 0.8411
```

Model A acc=0.8437, Model B acc=0.8411

```
alpha=0.00 -> acc=0.8437
alpha=0.10 -> acc=0.5951
alpha=0.20 -> acc=0.1867
alpha=0.30 -> acc=0.1181
alpha=0.40 -> acc=0.0964
alpha=0.50 -> acc=0.0837
alpha=0.60 -> acc=0.1000
alpha=0.70 -> acc=0.1259
alpha=0.80 -> acc=0.2983
alpha=0.90 -> acc=0.6548
alpha=1.00 -> acc=0.8411
```

LMC experiment: No BN, Skip (BN=False, Skip=True)

```
Epoch 1/6 - time: 27.2s - test_acc: 0.5698
Epoch 2/6 - time: 26.5s - test_acc: 0.6518
```

```
Epoch 3/6 - time: 26.3s - test_acc: 0.7146
Epoch 4/6 - time: 26.6s - test_acc: 0.7685
Epoch 5/6 - time: 26.4s - test_acc: 0.8071
Epoch 6/6 - time: 26.7s - test_acc: 0.8233
Epoch 1/6 - time: 26.4s - test_acc: 0.6017
Epoch 2/6 - time: 26.5s - test_acc: 0.6579
Epoch 3/6 - time: 26.6s - test_acc: 0.6719
Epoch 4/6 - time: 26.6s - test_acc: 0.7549
Epoch 5/6 - time: 26.6s - test_acc: 0.7871
Epoch 6/6 - time: 26.6s - test_acc: 0.8216
Model A acc=0.8233, Model B acc=0.8216
```

```
alpha=0.00 -> acc=0.8233
alpha=0.10 -> acc=0.6156
alpha=0.20 -> acc=0.2091
alpha=0.30 -> acc=0.1692
alpha=0.40 -> acc=0.1231
alpha=0.50 -> acc=0.1015
alpha=0.60 -> acc=0.1006
alpha=0.70 -> acc=0.1091
alpha=0.80 -> acc=0.1755
alpha=0.90 -> acc=0.5930
alpha=1.00 -> acc=0.8216
```

LMC experiment: No BN, No Skip (BN=False, Skip=False)

```
Epoch 1/6 - time: 24.8s - test_acc: 0.5803
Epoch 2/6 - time: 24.3s - test_acc: 0.7063
Epoch 3/6 - time: 24.5s - test_acc: 0.7358
Epoch 4/6 - time: 24.5s - test_acc: 0.8022
Epoch 5/6 - time: 24.7s - test_acc: 0.8201
Epoch 6/6 - time: 24.6s - test_acc: 0.8412
Epoch 1/6 - time: 24.6s - test_acc: 0.5794
Epoch 2/6 - time: 24.6s - test_acc: 0.5436
Epoch 3/6 - time: 24.5s - test_acc: 0.7332
Epoch 4/6 - time: 24.4s - test_acc: 0.7918
Epoch 5/6 - time: 24.6s - test_acc: 0.7965
Epoch 6/6 - time: 24.6s - test_acc: 0.8366
Model A acc=0.8412, Model B acc=0.8366
alpha=0.00 -> acc=0.8412
```


alpha=0.10 -> acc=0.6267

alpha=0.20 -> acc=0.2389

alpha=0.30 -> acc=0.1769

alpha=0.40 -> acc=0.1463

alpha=0.50 -> acc=0.1022

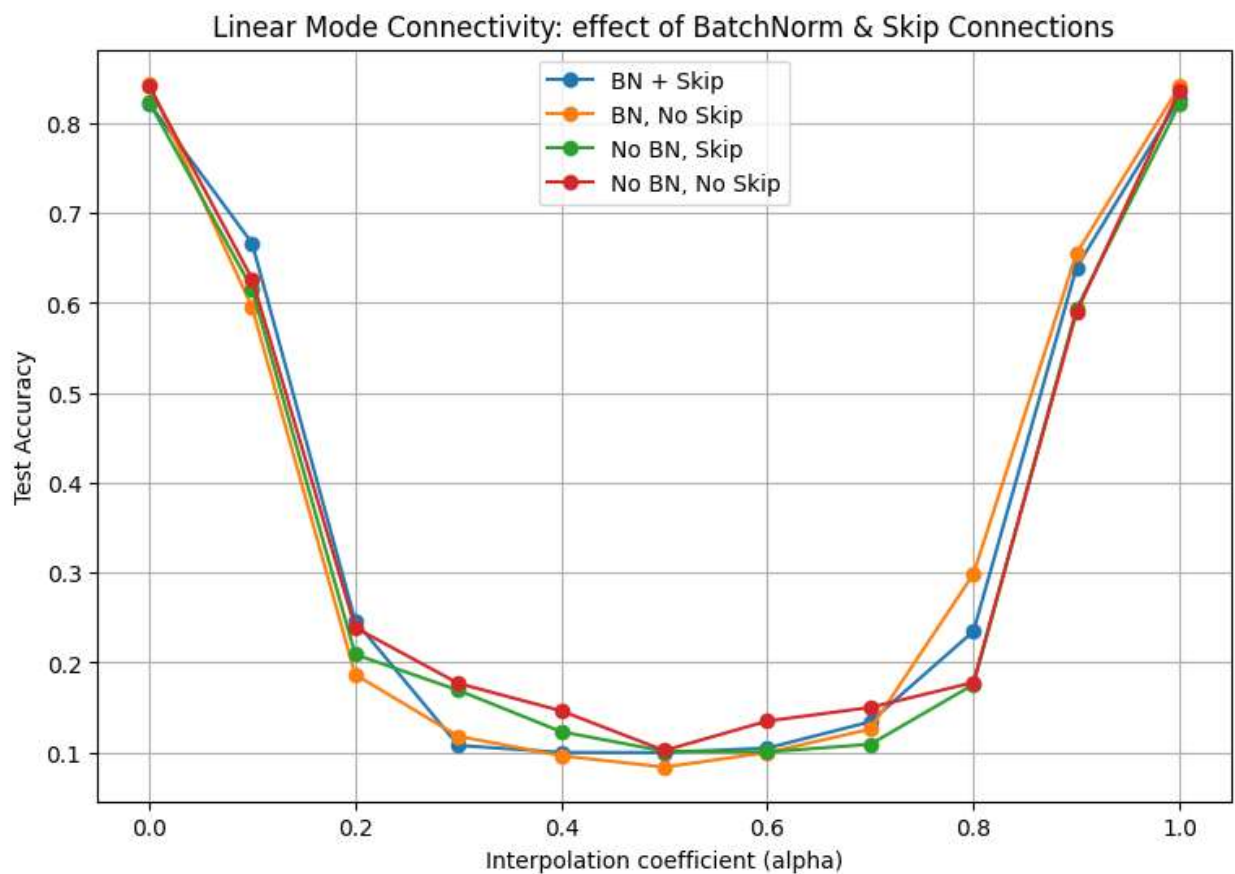
alpha=0.60 -> acc=0.1350

alpha=0.70 -> acc=0.1501

alpha=0.80 -> acc=0.1779

alpha=0.90 -> acc=0.5895

alpha=1.00 -> acc=0.8366



This output shows the Linear Mode Connectivity (LMC) results for four different model settings: BN + Skip, BN + No Skip, No BN + Skip, and No BN + No Skip. For each setting, two models were trained independently with different random seeds, and then the code evaluated the models along the straight path between their weights. The first part of the output shows that both independently trained models in each setting reach a good final test accuracy, around 82% to 84%. This means all four models are able to learn the CIFAR-10 task reasonably well.

However, the most important part is what happens during interpolation. When the code moves from one trained model to the other, the accuracy drops sharply in the middle, especially around $\alpha = 0.4$ to 0.6 , where the accuracy becomes close to 10%, which is almost random guessing for CIFAR-10. This means that although both models work well at the endpoints, the straight line between them passes through a poor region in weight space. In other words, the two solutions are not linearly connected by a high-performance path. The plot also shows that adding Batch Normalization and Skip Connections helps the models train well, but it does not completely remove the sharp drop in accuracy during interpolation. So, this experiment suggests that the loss landscape is still quite non-linear, even when using modern CNN design choices.

A limitation of this experiment is that setting `use_bn=False` disables the initial Batch Normalization layer, while Batch Normalization layers inside the residual blocks remain active. Therefore, the “No BN” configurations should be interpreted as reduced-BN configurations rather than complete removal of Batch Normalization.

Cell 9:

```
best_model_name = max(results_cifar10, key=lambda n:
results_cifar10[n]["test_accuracy"])
best_model = results_cifar10[best_model_name]["model"]
best_model.eval()
print(f"Best model: {best_model_name}")

cifar10_classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog',
'horse', 'ship', 'truck']

mean = torch.tensor((0.4914, 0.4822, 0.4465)).view(1, 3, 1, 1).to(device)
std = torch.tensor((0.2470, 0.2435, 0.2616)).view(1, 3, 1, 1).to(device)

def denormalize(x):
    return (x - std + mean).clamp(0, 1)

def fgsm_attack(model, image, label, epsilon):
    image = image.clone().detach().to(device).requires_grad_(True)
    label = label.to(device)
    output = model(image)
    loss = F.cross_entropy(output, label)
    model.zero_grad()
    loss.backward()
```

```

    perturbed = image + epsilon * image.grad.sign()
    perturbed = perturbed.detach()
    return perturbed

def find_min_epsilon(model, image, label, eps_max=0.3, eps_step=0.005):
    eps = 0.0
    orig_pred = model(image.to(device)).argmax(1).item()
    while eps <= eps_max:
        adv = fgsm_attack(model, image, label, eps)
        pred = model(adv).argmax(1).item()
        if pred != label.item():
            return eps, adv, pred
        eps += eps_step
    return None, adv, pred

correct_samples = []
best_model.to(device)
with torch.no_grad():
    for inputs, labels in test_loader_10:
        inputs, labels = inputs.to(device), labels.to(device)
        preds = best_model(inputs).argmax(1)
        for i in range(inputs.size(0)):
            if preds[i] == labels[i]:
                correct_samples.append((inputs[i:i+1].cpu(),
labels[i:i+1].cpu()))
            if len(correct_samples) >= 10:
                break
        if len(correct_samples) >= 10:
            break

fgsm_results = []
fig, axes = plt.subplots(2, 10, figsize=(24, 5))
for idx, (img, label) in enumerate(correct_samples):
    min_eps, adv_img, adv_pred = find_min_epsilon(best_model, img, label)
    fgsm_results.append({
        "true_label": label.item(),
        "min_epsilon": min_eps,
        "adv_pred": adv_pred

```

```

    })

    orig_display = denormalize(img.to(device)).squeeze().permute(1, 2,
0).cpu().numpy()
    adv_display = denormalize(adv_img).squeeze().permute(1, 2, 0).cpu().numpy()

    axes[0, idx].imshow(orig_display)
    axes[0, idx].set_title(f"{cifar10_classes[label.item()]}", fontsize=9)
    axes[0, idx].axis("off")

    axes[1, idx].imshow(adv_display)
    eps_str = f"{min_eps:.3f}" if min_eps is not None else ">0.3"
    axes[1, idx].set_title(f"eps={eps_str}\n->{cifar10_classes[adv_pred]}",
fontsize=9)
    axes[1, idx].axis("off")

axes[0, 0].set_ylabel("Original", fontsize=10)
axes[1, 0].set_ylabel("Adversarial", fontsize=10)
plt.suptitle("FGSM: Original vs Adversarial Images (minimum epsilon to fool the
model)")
plt.tight_layout()
plt.show()

print("\nFGSM results per image:")
for r in fgsm_results:
    print(r)

```

This cell selects the best-performing model on CIFAR-10 based on the test accuracy stored in `results_cifar10`. The selected model is then switched to evaluation mode using `eval()`, which ensures that layers such as Batch Normalization behave consistently during testing. The CIFAR-10 class names are also defined so that numerical labels can later be displayed as meaningful names such as cat, dog, or car.

The code then defines the mean and standard deviation used to normalize the CIFAR-10 images. Since the images are stored in normalized form, the `denormalize` function reverses this process before displaying them. This allows the original and adversarial images to be shown with normal pixel values between 0 and 1. The `fgsm_attack` function implements the Fast Gradient Sign Method. It enables gradient computation for the input image, calculates the classification loss, and finds the direction in which the image should be changed to increase the model's error. The

image is then modified by adding a small perturbation based on the sign of its gradient. The size of this perturbation is controlled by epsilon.

The `find_min_epsilon` function searches for the smallest perturbation that changes the model's prediction. Since the images are normalized before being fed into the model, epsilon is measured in normalized input space rather than raw pixel space. It first records the model's original prediction and then gradually increases epsilon from 0 to 0.3 in steps of 0.005. After each attack, the new prediction is checked. As soon as the predicted class becomes different from the original prediction, the function returns that epsilon value, the adversarial image, and the incorrect prediction. Therefore, a smaller minimum epsilon indicates that the image is more vulnerable to a small adversarial change, while a larger epsilon suggests greater resistance. If the model is not fooled before epsilon reaches 0.3, the function returns `None`.

Next, the code searches through the CIFAR-10 test set and selects the first ten images that the best model classifies correctly. This is important because the attack should begin with correctly classified examples; otherwise, it would be difficult to determine whether the perturbation actually caused the model to make a mistake. For each selected image, the code finds the minimum epsilon required to fool the model and stores the true label, the epsilon value, and the adversarial prediction in `fgsm_results`.

Finally, the cell creates a figure with two rows. The first row displays the original correctly classified images, while the second row shows their adversarial versions. Each adversarial image is labelled with the minimum epsilon and the new predicted class. The visual difference between the two images is usually very small, even though the model's prediction may change significantly. The printed results provide the same information numerically for each image. Overall, this cell evaluates the model's robustness against FGSM attacks and shows how easily small, carefully designed changes can cause incorrect predictions.

Output:

```
Best model: ResNet-34
```



```
FGSM results per image:
```

```
{'true label': 3, 'min epsilon': 0.025, 'adv pred': 5}
```

```
{'true_label': 8, 'min_epsilon': 0.05999999999999999, 'adv_pred': 1}
{'true_label': 8, 'min_epsilon': 0.02, 'adv_pred': 1}
{'true_label': 6, 'min_epsilon': 0.030000000000000002, 'adv_pred': 2}
{'true_label': 6, 'min_epsilon': 0.02, 'adv_pred': 3}
{'true_label': 1, 'min_epsilon': 0.025, 'adv_pred': 9}
{'true_label': 6, 'min_epsilon': 0.01, 'adv_pred': 2}
{'true_label': 3, 'min_epsilon': 0.025, 'adv_pred': 4}
{'true_label': 1, 'min_epsilon': 0.02, 'adv_pred': 8}
{'true_label': 0, 'min_epsilon': 0.045, 'adv_pred': 2}
```

The output shows that ResNet-34 was selected as the best model according to its test accuracy on CIFAR-10. The cell then tested this model against the Fast Gradient Sign Method (FGSM) attack using ten images that the model initially classified correctly. For each image, the attack gradually increased the value of epsilon until the model changed its prediction. Therefore, the reported min_epsilon represents the smallest perturbation found to fool the model.

The first image was correctly classified as a cat, represented by label 3. With an epsilon of 0.025, the model changed its prediction to dog, label 5. The second and third images were both ships, represented by label 8. The second ship required an epsilon of 0.06 before being misclassified as a car, while the third ship was fooled with a smaller epsilon of 0.02 and was also classified as a car. This difference shows that the model does not have the same level of robustness for every image, even when the images belong to the same class.

The fourth and fifth images were frogs, represented by label 6. The fourth frog was misclassified as a bird with epsilon 0.03, while the fifth frog was classified as a cat with epsilon 0.02. The sixth image was a car, label 1, and the model changed its prediction to truck, label 9, using epsilon 0.025. The seventh image was another frog, but it was the most vulnerable example in the experiment. A very small epsilon of only 0.01 was enough to change the prediction from frog to bird.

The eighth image was a cat and was changed to deer with epsilon 0.025. The ninth image was a car and was misclassified as a ship with epsilon 0.02. Finally, the tenth image was a plane, represented by label 0, and the attack changed its prediction to bird, label 2, with epsilon 0.045. Overall, the required epsilon values range from 0.01 to 0.06, with an average of approximately 0.028. This indicates that the model can be fooled by relatively small input changes.

The figure supports this numerical result. The original images are shown in the first row, and the adversarial versions are shown in the second row. Visually, the two rows appear almost identical, which means that the perturbations are small enough that they are difficult to notice visually in the displayed examples, even though they can still change the model's prediction. However, these small changes are sufficient to produce completely different predictions, such as changing a frog into a bird or a cat into a dog. These results show that high clean accuracy is not sufficient for safe deployment. In real-world settings such as autonomous driving, medical imaging, or surveillance, small but

targeted perturbations can cause harmful misclassifications. Deployment therefore requires an explicit threat model, robustness evaluation under attack, and often a human fallback or rejection mechanism for low-confidence predictions. Although ResNet-34 performs best on the original test data, it remains sensitive to carefully designed perturbations generated from the input gradients.

Cell 10:

```
fixed_eps = 0.03
class_correct = {c: 0 for c in range(nc10)}
class_total = {c: 0 for c in range(nc10)}

best_model.eval()
for inputs, labels in test_loader_10:
    inputs, labels = inputs.to(device), labels.to(device)
    inputs.requires_grad = True
    outputs = best_model(inputs)
    loss = F.cross_entropy(outputs, labels)
    best_model.zero_grad()
    loss.backward()
    adv_inputs = (inputs + fixed_eps * inputs.grad.sign()).detach()
    adv_preds = best_model(adv_inputs).argmax(1)
    for i in range(labels.size(0)):
        c = labels[i].item()
        class_total[c] += 1
        if adv_preds[i] == labels[i]:
            class_correct[c] += 1

print(f"Per-class robust accuracy under FGSM (eps={fixed_eps}):")
class_robust_acc = {}
for c in range(nc10):
    acc = class_correct[c] / class_total[c]
    class_robust_acc[c] = acc
    print(f"  {cifar10_classes[c]}: {acc:.4f}")

plt.figure(figsize=(9, 5))
plt.bar([cifar10_classes[c] for c in range(nc10)], [class_robust_acc[c] for c
in range(nc10)])
```

```

plt.ylabel("Robust Accuracy under FGSM")
plt.title(f"Per-Class Vulnerability to FGSM (eps={fixed_eps})")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

def adversarial_train_model(model, train_loader, test_loader, epochs=8,
lr=0.01, eps=0.03):
    model = model.to(device)
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,
weight_decay=5e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(epochs):
        model.train()
        t0 = time.time()
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            inputs.requires_grad = True
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            model.zero_grad()
            loss.backward()
            adv_inputs = (inputs + eps * inputs.grad.sign()).detach()

            optimizer.zero_grad()
            clean_out = model(inputs.detach())
            adv_out = model(adv_inputs)
            total_loss = 0.5 * criterion(clean_out, labels) + 0.5 *
criterion(adv_out, labels)
            total_loss.backward()
            optimizer.step()
        scheduler.step()
        acc = evaluate(model, test_loader)
        print(f" [AdvTrain] Epoch {epoch+1}/{epochs} - time: {time.time()-
t0:.1f}s - clean_acc: {acc:.4f}")

```



```

        return model

print("\nTraining a robust model with Adversarial Training")
set_seed(SEED)
robust_model = ResNet34(num_classes=nc10).to(device)
robust_model = adversarial_train_model(robust_model, train_loader_10,
test_loader_10, epochs=EPOCHS, eps=fixed_eps)

clean_acc_robust = evaluate(robust_model, test_loader_10)

def eval_robust_accuracy(model, loader, eps):
    model.eval()
    correct, total = 0, 0
    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)
        inputs.requires_grad = True
        outputs = model(inputs)
        loss = F.cross_entropy(outputs, labels)
        model.zero_grad()
        loss.backward()
        adv_inputs = (inputs + eps * inputs.grad.sign()).detach()
        preds = model(adv_inputs).argmax(1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return correct / total

robust_acc_before = eval_robust_accuracy(best_model, test_loader_10,
fixed_eps)
robust_acc_after = eval_robust_accuracy(robust_model, test_loader_10,
fixed_eps)

print("\nRobustness Comparison")
print(f"Original model ->
clean_acc={results_cifar10[best_model_name]['test_accuracy']:.4f}, "
      f"robust_acc(eps={fixed_eps})={robust_acc_before:.4f}")
print(f"Adv-trained model -> clean_acc={clean_acc_robust:.4f}, "
      f"robust_acc(eps={fixed_eps})={robust_acc_after:.4f}")

```

```
print(f"Robustness improvement: {(robust_acc_after -
robust_acc_before)*100:.2f} percentage points")
```

This cell evaluates how resistant the best model is to FGSM attacks with a fixed perturbation size of $\epsilon = 0.03$. Instead of searching for a different minimum epsilon for each image, the code applies the same attack strength to every test image. It then counts the number of correctly classified adversarial examples for each CIFAR-10 class. The dictionaries `class_correct` and `class_total` store these values, allowing the code to calculate robust accuracy separately for planes, cars, birds, cats, and the other classes. Robust accuracy measures the percentage of images that remain correctly classified after the FGSM perturbation has been applied.

After processing the complete test set, the code prints the robust accuracy for each class and displays the results in a bar chart. This chart makes it easier to compare the vulnerability of different categories. A class with a higher robust accuracy is less affected by the attack, while a class with a lower robust accuracy is more vulnerable. This analysis is useful because the model may not have the same level of robustness for all classes. For example, visually similar categories or classes with more complex image patterns may be affected more strongly by a small adversarial perturbation.

The second part of the cell introduces adversarial training. The `adversarial_train_model` function trains a new model using both clean images and adversarial images. For each training batch, the code first performs a forward pass on the original inputs and calculates the loss. It then computes the gradient of this loss with respect to the input images and uses the gradient sign to create adversarial examples with $\epsilon = 0.03$. The model is trained using a combined loss that gives equal importance to the clean examples and the adversarial examples. This is implemented as $0.5 \text{ clean loss} + 0.5 \text{ adversarial loss}$. As a result, the model is encouraged to classify normal images correctly while also learning features that remain useful when the inputs are slightly modified.

The model used for adversarial training is a new ResNet-34, which was selected as the best-performing model on CIFAR-10. It is trained with stochastic gradient descent, momentum, weight decay, and a cosine learning-rate scheduler. After each epoch, the model is evaluated on the test set using clean images, and the clean accuracy is printed. Once training is complete, the code calculates the final clean accuracy of the adversarially trained model.

The function `eval_robust_accuracy` then measures the model's accuracy on adversarial test images. It generates an FGSM attack with the same fixed epsilon for every batch and checks how many adversarial images are still classified correctly. This evaluation is performed for both models: the original best model, which is ResNet-34, and the newly trained adversarially robust model. Finally, the code prints a comparison between their clean accuracy and robust accuracy under the attack. The last value reports the improvement in robustness in percentage points. If adversarial training is effective, the robust model should achieve a higher adversarial accuracy than the original model, although

this improvement may sometimes come with a small reduction in clean accuracy. This reflects the common trade-off between standard performance and resistance to adversarial attacks.

Output:

```
Per-class robust accuracy under FGSM (eps=0.03):
```

```
plane: 0.3320
```

```
car: 0.5440
```

```
bird: 0.2430
```

```
cat: 0.1260
```

```
deer: 0.2350
```

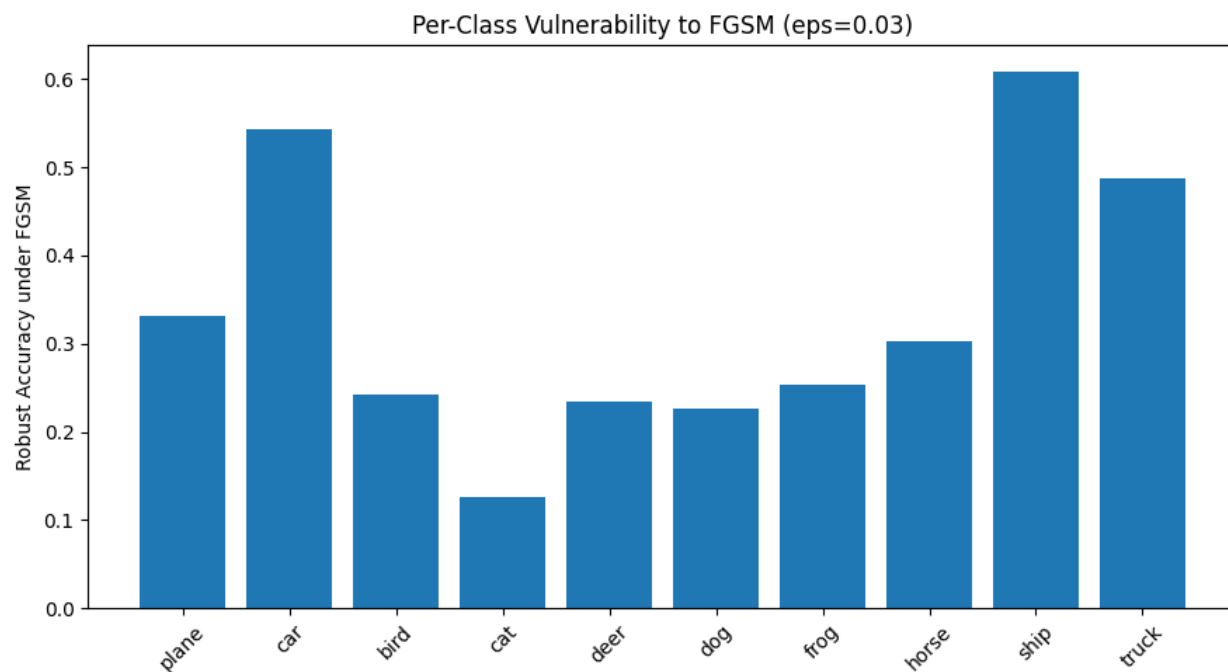
```
dog: 0.2260
```

```
frog: 0.2540
```

```
horse: 0.3030
```

```
ship: 0.6080
```

```
truck: 0.4870
```



```
Training a robust model with Adversarial Training
```

```
[AdvTrain] Epoch 1/8 - time: 135.1s - clean_acc: 0.5184
```

```
[AdvTrain] Epoch 2/8 - time: 133.6s - clean_acc: 0.6203
```

```
[AdvTrain] Epoch 3/8 - time: 134.9s - clean_acc: 0.7054
```

```
[AdvTrain] Epoch 4/8 - time: 135.5s - clean_acc: 0.7481
```

```
[AdvTrain] Epoch 5/8 - time: 134.2s - clean_acc: 0.7744
```

```
[AdvTrain] Epoch 6/8 - time: 134.4s - clean_acc: 0.7958
```

```
[AdvTrain] Epoch 7/8 - time: 134.4s - clean_acc: 0.8105
```

```
[AdvTrain] Epoch 8/8 - time: 135.6s - clean_acc: 0.8277
```

Robustness Comparison

```
Original model -> clean_acc=0.8785, robust_acc(eps=0.03)=0.3358
```

```
Adv-trained model -> clean_acc=0.8277, robust_acc(eps=0.03)=0.6609
```

```
Robustness improvement: 32.51 percentage points
```

The first part of the output reports the per-class robust accuracy of the original best model under an FGSM attack with ($\epsilon = 0.03$). Although the model achieved high clean accuracy on normal CIFAR-10 images, its performance drops considerably after adversarial perturbations are added. The average robust accuracy of the original model is only about 33.6%, meaning that nearly two-thirds of the test images can be misclassified under this relatively small attack.

The results also show that robustness is different across classes. Ships have the highest robust accuracy at 60.8%, followed by cars at 54.4% and trucks at 48.7%. This means the model is relatively more resistant to FGSM perturbations for these categories. These classes often contain clearer global shapes and more consistent visual patterns, such as large vehicles or the horizontal structure of ships. Planes and horses have moderate robustness, with accuracies of 33.2% and 30.3%, respectively.

On the other hand, cats are the most vulnerable class, with a robust accuracy of only 12.6%. Dogs, deer, birds, and frogs are also highly vulnerable, with robust accuracies between approximately 22% and 25%. These animal categories are visually more similar to each other and may contain more varied backgrounds, poses, textures, and shapes. Therefore, a small gradient-based perturbation can more easily push their representations toward another class. The bar chart clearly illustrates this difference: ship, car, and truck have much taller bars, while cat has the smallest bar by a large margin.

The second part of the output shows the training process of the adversarially trained ResNet-34 model. Its clean test accuracy improves steadily from 51.84% in the first epoch to 82.77% in the eighth epoch. This indicates that the model successfully learns the normal CIFAR-10 classification task even though it is trained on both clean and adversarial examples. However, each epoch takes around 134 to 136 seconds, which is much longer than standard training because adversarial training requires an additional gradient calculation to generate adversarial inputs during every training batch.

The final comparison demonstrates the effectiveness of adversarial training. The original ResNet-34 model achieves a higher clean accuracy of 87.85%, but its robust accuracy under FGSM drops to 33.58%. In contrast, the adversarially trained ResNet-34 has a lower clean accuracy of 82.77%, but its robust accuracy increases substantially to 66.09%. This is an improvement of 32.51 percentage points under the same FGSM attack strength. Overall, the experiment shows the trade-off between clean accuracy and adversarial robustness. The original model performs better on ordinary, unmodified images, but it is highly sensitive to carefully designed perturbations. The adversarially trained model sacrifices about five percentage points of clean accuracy, yet it becomes much more reliable when the input images are attacked. Therefore, adversarial training is an effective method for improving robustness, especially in applications where resistance to small malicious changes is important.

Analytical Questions

- The results suggest that ResNet-34 represents the sweet spot among the tested architectures. ResNet-18 achieves a test accuracy of 87.35% with approximately 11.2 million parameters, while ResNet-34 reaches the highest accuracy of 87.85% with 21.3 million parameters. ResNet-50, despite having slightly more parameters at around 23.5 million, performs worse with an accuracy of 85.71%. Therefore, increasing the depth from ResNet-18 to ResNet-34 provides a small but positive improvement in accuracy, whereas increasing the depth further to ResNet-50 does not produce additional benefits. This suggests that ResNet-34 offers the best balance between predictive performance and model complexity. The lower performance of ResNet-50 may be related to overfitting, insufficient training time, or the fact that the model needs more epochs and better hyperparameter tuning to fully benefit from its larger capacity.
- The relationship between model depth and performance changes when the dataset becomes more difficult. On CIFAR-10, which contains only ten classes, ResNet-34 performs slightly better than ResNet-18, showing that additional depth can help the model learn more complex visual patterns. However, on CIFAR-100, the deeper models do not achieve better results. ResNet-18 reaches an accuracy of 60.64%, while ResNet-34 achieves 59.84% and ResNet-50 drops further to 53.86%. This indicates that deeper networks are more sensitive to the difficulty of the dataset and to the limited training budget. Since CIFAR-100 contains one hundred classes with more visually similar categories, the models need to learn much more detailed features. Under a short training schedule, deeper architectures may remain undertrained or may overfit the training data before they can generalize effectively. As a result, increasing depth alone is not sufficient for improving performance on a more complex dataset.
- The Linear Mode Connectivity experiment shows that the accuracy does not remain high along the direct linear path between the two independently trained models. Both endpoint models achieve good accuracy, with values generally above 82%, but the accuracy decreases sharply when the interpolation coefficient moves toward the middle of the path. Around $\alpha = 0.3$ to $\alpha = 0.7$, the accuracy becomes close to 0.1, which is

approximately random guessing for a ten-class classification problem. The accuracy rises again as the interpolation approaches the second model. This behavior means that the two trained models cannot be connected by a simple straight line that maintains good performance. Their parameter configurations are located in different effective regions of the optimization landscape, and the direct path between them passes through a high-loss, low-accuracy area. Within the tested configurations, modifying the Batch Normalization setting and the use of Skip Connections did not eliminate the sharp drop in accuracy during weight interpolation.

- The per-class FGSM results show that some classes are considerably more vulnerable to adversarial perturbations than others. The cat class is the most vulnerable, with a robust accuracy of only 12.6% under an attack with $(\epsilon = 0.03)$. This means that most cat images were misclassified after a relatively small gradient-based perturbation. Other animal classes, including bird, deer, dog, and frog, also show low robust accuracy, with values between approximately 22% and 25%. These classes may be more difficult to separate because their visual features can be similar, and their appearance can vary significantly depending on pose, texture, background, and lighting.
- In contrast, the ship class is the most resistant class, with a robust accuracy of 60.8%. Cars and trucks also show relatively stronger robustness, with robust accuracies of 54.4% and 48.7%. One possible explanation is that these classes often contain clearer and more consistent global structures. However, these results should not be interpreted as meaning that ships or vehicles are inherently secure. They only show that, under this specific model, dataset, attack method, and epsilon value, these classes were less affected than the others. The large differences between classes demonstrate that overall robust accuracy can hide important class-specific weaknesses.

Adversarial examples are important for the real-world deployment of convolutional neural networks because they reveal a gap between normal accuracy and actual reliability. A model may perform very well on clean test images but still make incorrect predictions when an attacker applies a small and carefully designed modification. In practical systems, this can create security and safety risks. For example, an adversarial change could affect a face recognition system, an object detection system, a medical image classifier, or a perception system used in an autonomous vehicle. Even when the perturbation is difficult for humans to notice, it may significantly change the model's output.

For this reason, adversarial robustness should be treated as an important part of the model development and deployment pipeline rather than as an optional final test. The results of this experiment support this point directly: the original model achieved a clean accuracy of 87.85%, but its robust accuracy under FGSM was only 33.58%. After adversarial training, the robust accuracy increased to 66.09%, which represents an improvement of 32.51 percentage points. Although the adversarially trained model had a lower clean accuracy of 82.77%, it was much more reliable under attack. This result illustrates the trade-off between standard accuracy and robustness and shows why real-world systems may need defense techniques such as adversarial training, input preprocessing, stronger evaluation procedures, and continuous monitoring.

Problem 2: Sequence Modeling with RNNs, LSTMs, GRUs, and Transformers

Problem 2 focuses on sequence models and how they can be used for different kinds of data. In this problem, we work with models that process information in a sequence, such as words in a sentence or values in a time series. The main goal is to compare different sequence modeling methods and understand where each one works well. This problem has two main parts. The first part is about sentiment analysis using recurrent neural networks, and the second part is about time-series forecasting using LSTM and Transformer models.

In the first part, the task is to build a binary sentiment classifier using the SST-2 dataset. The goal is to decide whether a sentence has a positive or negative sentiment. Three recurrent architectures are compared: a simple RNN, an LSTM, and a GRU. All three models use the same hidden size of 64 so that the comparison is fair. The input words are represented using 50-dimensional pre-trained GloVe embeddings. During training, we need to record the training and validation loss and accuracy for each model. We also need to compare their final validation accuracy, average training time per epoch, and total number of parameters. This makes it possible to see not only which model is more accurate, but also which one is faster and simpler.

This part also asks an important question about how recurrent models represent a sentence. For sentiment classification, using the final hidden state can be enough because the model only needs to produce one label for the whole sentence. However, this is different from token-level tasks such as part-of-speech tagging, where every word needs its own prediction. For that type of task, the model must use the hidden state from each time step instead of only using the final hidden state. The second part of Problem 2 is about time-series forecasting. In this section, the goal is to compare an LSTM with a Transformer on a real time-series dataset. Both models are used to predict future values from past observations. The task is not limited to predicting only one future point, so the models are tested for multi-step forecasting as well. We also compare different lookback windows, which control how much past information the model receives, and different forecast horizons, which control how far into the future the model tries to predict.

The final goal of this section is to understand how the two architectures behave under different forecasting conditions. We compare their prediction errors, training time, and model size, and we study how their performance changes when the input sequence becomes longer or the forecast horizon becomes larger. We also need to discuss when an LSTM may perform better than a Transformer, when a Transformer may have an advantage, and whether the attention mechanism can help us understand important patterns in a time series. Overall, this problem is designed to show how sequence models behave in both language and forecasting tasks, and how the choice of architecture affects accuracy, efficiency, and the way sequential information is learned.

Explain the code of problem set 2:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
!pip -q install -U datasets gensim statsmodels
```

This cell installs the main libraries needed for Problem 2. The datasets library is used to load the SST-2 dataset for sentiment analysis. The gensim library is used to load the pre-trained GloVe word embeddings, and statsmodels is used later to load the time-series dataset. The -q option keeps the installation output shorter, and -U makes sure the libraries are updated to a recent version.

Output:

```
559.1/559.1 kB 7.5 MB/s eta 0:00:00
27.9/27.9
MB 20.7 MB/s eta 0:00:00
50.1/50.1
MB 12.7 MB/s eta 0:00:00
```

The output shows that the required libraries and their files were downloaded and installed successfully. The progress bars show the size of the downloaded files and the download speed. Since there is no error message, the installation was completed correctly and the environment is ready for the next cells.

Cell 2:

```
import re
import time
import random
import copy
from collections import Counter
import numpy as np
```



```

import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence
from datasets import load_dataset
import gensim.downloader as api
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm

SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
set_seed()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

```

This cell imports all of the libraries that will be used in the rest of Problem 2. The `re` library is used for simple text processing, while `time` is used to measure training time. The `random`, `NumPy`, and `PyTorch` libraries are also used to control randomness and make the experiments more reproducible. `Counter` is used later when building the vocabulary from the SST-2 training sentences.

`Pandas` is used for organizing results into tables, and `Matplotlib` is used to create graphs. `PyTorch` provides the main tools for building and training the neural network models. `Dataset`, `DataLoader`, and `TensorDataset` help prepare data in batches, while `pad_sequence` and `pack_padded_sequence` are useful for working with sentences that have different lengths.

The `load_dataset` function is used to load SST-2, and `gensim.downloader` is used to download the pre-trained GloVe embeddings. `StandardScaler` is used later for normalizing the time-series data, and `statsmodels` is used to load the CO2 dataset.

The seed is set to 42 so that the random parts of the experiment are more consistent between runs. The code then checks whether a GPU is available. If CUDA is available, the models will use the GPU; otherwise, they will run on the CPU.

output:

```
Device: cuda
```

This means that Colab detected a CUDA-enabled GPU and PyTorch will use it for training. This is useful because training the RNN, LSTM, GRU, and later the forecasting models can be much faster on the GPU.

Cell 3:

```
sst2 = load_dataset("stanfordnlp/sst2")
train_data = sst2["train"]
val_data = sst2["validation"]

print(train_data)
print(val_data)
print(train_data[0])
```

This cell loads the SST-2 dataset from the Hugging Face dataset library. SST-2 is used for binary sentiment classification, where each sentence is labeled as either negative or positive. The code keeps the training split in `train_data` and the validation split in `val_data`. The training set is used to train the models, while the validation set is used to check how well the models perform on unseen sentences during the experiment.

The code also prints the structure of both datasets and shows the first training example. Each sample contains an index, the sentence text, and its sentiment label. The label is 0 for negative sentiment and 1 for positive sentiment.

Output:

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md 100% ██████████ 5.27k/5.27k [00:00<00:00, 340kB/s]
data/train-00000-of-00001.parquet: reconstructing file: 100% ██████████ 3.11MB / 3.11MB, 308kB/s
data/train-00000-of-00001.parquet: downloading bytes: ██████████ 3.00MB, 293kB/s
data/validation-00000-of-00001.parquet: reconstructing file: 100% ██████████ 72.8kB / 72.8kB, 7.25kB/s
data/validation-00000-of-00001.parquet: downloading bytes: ██████████ 69.4kB, 6.91kB/s
data/test-00000-of-00001.parquet: reconstructing file: 100% ██████████ 148kB / 148kB, 14.8kB/s
data/test-00000-of-00001.parquet: downloading bytes: ██████████ 148kB, 14.8kB/s
Generating train split: 100% ██████████ 67349/67349 [00:00<00:00, 22585.80 examples/s]
Generating validation split: 100% ██████████ 872/872 [00:00<00:00, 36109.06 examples/s]
Generating test split: 100% ██████████ 1821/1821 [00:00<00:00, 84306.26 examples/s]
Dataset({
  features: ['idx', 'sentence', 'label'],
  num_rows: 67349
})
Dataset({
  features: ['idx', 'sentence', 'label'],
  num_rows: 872
})
['idx': 0, 'sentence': 'hide new secretions from the parental units ', 'label': 0]

```

The output shows that the SST-2 dataset was downloaded successfully. The training split contains 67,349 examples, the validation split contains 872 examples, and the test split contains 1,821 examples. In this experiment, the training and validation splits are used for model development and evaluation.

The first training example is `{'idx': 0, 'sentence': 'hide new secretions from the parental units ', 'label': 0}`. This means that the sentence is classified as a negative sentiment example because its label is 0.

The warning about HF_TOKEN only means that the dataset was downloaded without logging in to the Hugging Face Hub. It does not affect the dataset or the results of this experiment, and the output confirms that all dataset splits were loaded correctly.

Cell 4:

```

def tokenize(text):
    return re.findall(r"[A-Za-z]+(?:'[A-Za-z]+)?|[0-9]+|[\^\w\s]",
text.lower())
counter = Counter()

for sentence in train_data["sentence"]:
    counter.update(tokenize(sentence))
MAX_VOCAB = 30000
MIN_FREQ = 2

words = [
    word for word, count in counter.most_common(MAX_VOCAB - 2)
    if count >= MIN_FREQ

```

```

]
itos = ["<pad>", "<unk>"] + words
stoi = {word: i for i, word in enumerate(itos)}
PAD_IDX = stoi["<pad>"]
UNK_IDX = stoi["<unk>"]
print("Vocabulary size:", len(itos))

```

This cell prepares the text data by creating a vocabulary from the SST-2 training sentences. First, the `tokenize` function converts each sentence to lowercase and splits it into smaller tokens such as words, numbers, and punctuation marks. This gives the model a simple way to process the text word by word.

Next, `Counter` is used to count how many times each token appears in the training set. The vocabulary size is limited to a maximum of 30,000 tokens, and words that appear fewer than two times are removed. This helps avoid keeping very rare words that may not be useful for training.

Two special tokens are also added to the vocabulary. The `<pad>` token is used later to make sentences in the same batch have the same length, and the `<unk>` token is used for words that are not available in the vocabulary. The `itos` list maps indices back to words, while `stoi` maps each word to its numerical index.

Output:

```
Vocabulary size: 13364
```

This means that after tokenizing the training data, removing very rare words, and adding the two special tokens, the final vocabulary contains 13,364 tokens. This vocabulary will be used to convert the SST-2 sentences into numerical sequences before they are passed to the neural network models.

Cell 5:

```

glove = api.load("glove-wiki-gigaword-50")
EMBED_DIM = 50

embedding_matrix = torch.empty(len(itos), EMBED_DIM)
nn.init.normal_(embedding_matrix, mean=0.0, std=0.05)
embedding_matrix[PAD_IDX] = 0
found = 0
for word, idx in stoi.items():

```

```

    if word in glove:
        embedding_matrix[idx] = torch.tensor(glove[word])
        found += 1
coverage = found / len(itos)

print("Embedding shape:", embedding_matrix.shape)
print(f"GloVe coverage: {coverage:.2%}")

```

This cell loads the 50-dimensional pre-trained GloVe word embeddings. GloVe provides vector representations for words based on their meaning and how they appear in real text. Instead of starting with completely random word representations, using pre-trained embeddings gives the sentiment models useful information about the words before training begins.

An embedding matrix is created with one row for each word in our vocabulary and 50 values for each word. At first, the matrix is initialized with small random values. The vector for the <pad> token is set to zero. Then, for every word in the vocabulary, the code checks whether that word exists in the GloVe model. If it exists, the random vector is replaced with the pre-trained GloVe vector. Words that are not found in GloVe keep their randomly initialized vectors.

The code also calculates the GloVe coverage, which shows what percentage of the vocabulary was found in the pre-trained embeddings.

Output:

```

[=====] 100.0% 66.0/66.0MB
downloaded

Embedding shape: torch.Size([13364, 50])
GloVe coverage: 97.98%

```

The output shows that the GloVe model was downloaded successfully. The embedding matrix has the following shape: torch.Size([13364, 50]), this means that the matrix contains one 50-dimensional vector for each of the 13,364 tokens in the vocabulary. And the GloVe coverage is: 97.98%. this is a very high coverage, which means that almost all words in the SST-2 vocabulary were found in the pre-trained GloVe embeddings. Therefore, most words start with useful pre-trained representations instead of random values.

Cell 6:

```

MAX_LEN = 64
BATCH_SIZE = 128

class SentimentDataset(Dataset):
    def __init__(self, dataset):
        self.sentences = dataset["sentence"]
        self.labels = dataset["label"]

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, index):
        tokens = tokenize(self.sentences[index])[:MAX_LEN]

        ids = [stoi.get(token, UNK_IDX) for token in tokens]

        if len(ids) == 0:
            ids = [UNK_IDX]

        return (
            torch.tensor(ids, dtype=torch.long),
            torch.tensor(self.labels[index], dtype=torch.float32)
        )

def collate_batch(batch):
    sequences, labels = zip(*batch)

    lengths = torch.tensor(
        [len(sequence) for sequence in sequences],
        dtype=torch.long
    )
    sequences = pad_sequence(
        sequences,
        batch_first=True,
        padding_value=PAD_IDX
    )

    labels = torch.stack(labels)

    return sequences, lengths, labels

train_dataset = SentimentDataset(train_data)
val_dataset = SentimentDataset(val_data)

```

```

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    collate_fn=collate_batch,
    num_workers=2,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    collate_fn=collate_batch,
    num_workers=2,
    pin_memory=True
)

print("Train samples:", len(train_dataset))
print("Validation samples:", len(val_dataset))

```

This cell prepares the SST-2 data for training the sentiment models. The maximum sentence length is set to 64 tokens, so longer sentences are cut after the first 64 tokens. The batch size is set to 128, which means that 128 samples are processed together during training.

The `SentimentDataset` class converts each sentence into a sequence of token indices using the vocabulary created earlier. If a token is not found in the vocabulary, the `<unk>` index is used. The class also returns the sentiment label as a floating-point value because the classifier will use binary classification loss.

The `collate_batch` function is used because the sentences do not all have the same length. It first records the original length of each sentence and then pads the shorter sequences using the `<pad>` token so that all sentences in the same batch have the same length. These original lengths are kept because they will later be used by the recurrent models to ignore the padded part of each sequence.

Finally, separate `Dataset` and `DataLoader` objects are created for the training and validation sets. The training loader uses shuffling so that the order of the samples changes between epochs, while the validation loader keeps a fixed order because it is only used for evaluation.

Output:

```
Train samples: 67349
```

```
Validation samples: 872
```

This confirms that all 67,349 training samples and 872 validation samples were prepared correctly and are ready to be used by the RNN, LSTM, and GRU models.

Cell 7:

```
class SentimentRNN(nn.Module):
    def __init__(self, model_type, embedding_weights, hidden_size=64):
        super().__init__()
        self.model_type = model_type
        self.embedding = nn.Embedding.from_pretrained(
            embedding_weights,
            freeze=False,
            padding_idx=PAD_IDX
        )

        if model_type == "RNN":
            self.recurrent = nn.RNN(
                input_size=EMBED_DIM,
                hidden_size=hidden_size,
                batch_first=True
            )

        elif model_type == "LSTM":
            self.recurrent = nn.LSTM(
                input_size=EMBED_DIM,
                hidden_size=hidden_size,
                batch_first=True
            )

        elif model_type == "GRU":
            self.recurrent = nn.GRU(
                input_size=EMBED_DIM,
```



```

        hidden_size=hidden_size,
        batch_first=True
    )

    else:
        raise ValueError("model_type must be RNN, LSTM, or GRU")
    self.dropout = nn.Dropout(0.3)
    self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x, lengths):
        x = self.embedding(x)
        x = self.dropout(x)
        packed = pack_padded_sequence(
            x,
            lengths.cpu(),
            batch_first=True,
            enforce_sorted=False
        )

        if self.model_type == "LSTM":
            _, (hidden, _) = self.recurrent(packed)
        else:
            _, hidden = self.recurrent(packed)

        final_hidden = hidden[-1]

        logits = self.fc(self.dropout(final_hidden))

        return logits.squeeze(1)

```

This cell defines the main sentiment classification model that will be used for all three recurrent architectures. Instead of writing three separate models, the same SentimentRNN class can create an RNN, LSTM, or GRU depending on the value of `model_type`. This keeps the experiment simple and makes the comparison between the three architectures more consistent.

First, the model creates an embedding layer using the GloVe embedding matrix from the previous cell. The embeddings are not frozen, so their values can be updated during training. This allows the pre-trained word vectors to adjust to the SST-2 sentiment classification task.

The recurrent layer is then selected based on the model type. If `model_type` is RNN, the model uses a simple recurrent neural network. If it is LSTM, it uses an LSTM layer, and if it is GRU, it uses a GRU layer. All three models use an input size of 50, which matches the GloVe embedding dimension, and a hidden size of 64 as required in the problem.

A dropout layer with a rate of 0.3 is also added to reduce overfitting. After the recurrent layer processes the sentence, the final hidden state is used as the representation of the whole sentence. This final hidden state is passed through a linear layer that produces one output value for binary sentiment classification.

Inside the forward function, the input token indices are first converted into word embeddings. The sequences are then packed using `pack_padded_sequence` so that the recurrent layer can ignore the padded parts of shorter sentences. For the LSTM, both the hidden state and cell state are returned, but only the final hidden state is used for classification. For RNN and GRU, the final hidden state is used directly.

There is no printed output for this cell because it only defines the model architecture. The model will be created and trained in the next cells.

Cell 8:

```
def run_epoch(model, loader, criterion, optimizer=None):
    training = optimizer is not None

    if training:
        model.train()
    else:
        model.eval()

    total_loss = 0
    total_correct = 0
    total_samples = 0
    for sequences, lengths, labels in loader:
        sequences = sequences.to(device)
        lengths = lengths.to(device)
        labels = labels.to(device)
```

```

        if training:
            optimizer.zero_grad()

        with torch.set_grad_enabled(training):
            logits = model(sequences, lengths)
            loss = criterion(logits, labels)

            if training:
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()

        predictions = (logits >= 0).long()

        batch_size = labels.size(0)

        total_loss += loss.item() * batch_size
        total_correct += (predictions == labels.long()).sum().item()
        total_samples += batch_size

    loss = total_loss / total_samples
    accuracy = total_correct / total_samples

    return loss, accuracy
def count_parameters(model):
    return sum(p.numel() for p in model.parameters())

```

This cell defines the functions used to train and evaluate the sentiment models. The `run_epoch` function can be used for both training and validation. It checks whether an optimizer is provided. If an optimizer is available, the model runs in training mode. Otherwise, it runs in evaluation mode.

For each batch, the input sequences, their lengths, and the labels are moved to the selected device. During training, the gradients are cleared before the forward pass. The model then produces the output logits and the loss is calculated using the selected loss function.

When the model is in training mode, the loss is backpropagated and the optimizer updates the model parameters. Gradient clipping is also applied with a maximum value of 1.0. This helps keep the gradients stable during training, which can be useful for recurrent neural networks.

The prediction is considered positive when the output logit is greater than or equal to zero. The function counts the number of correct predictions and calculates the average loss and accuracy over the whole dataset.

At the end, the function returns the average loss and accuracy for one complete epoch. This makes it possible to use the same function for both the training and validation stages.

The `count_parameters` function is also defined in this cell. It calculates the total number of parameters in a model. This value will later be used to compare the size of the RNN, LSTM, and GRU architectures.

There is no printed output for this cell because it only defines helper functions that will be used during model training and evaluation.

Cell 9:

```
EPOCHS = 10

def train_sentiment_model(model_type):
    set_seed()

    model = SentimentRNN(
        model_type=model_type,
        embedding_weights=embedding_matrix,
        hidden_size=64
    ).to(device)

    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.001
    )

    history = {
        "train_loss": [],
        "val_loss": [],
        "train_acc": [],
```

```

        "val_acc": [],
        "epoch_time": []
    }

    print(f"\nTraining {model_type}")
    for epoch in range(EPOCHS):
        start = time.perf_counter()
        train_loss, train_acc = run_epoch(
            model,
            train_loader,
            criterion,
            optimizer
        )

        val_loss, val_acc = run_epoch(
            model,
            val_loader,
            criterion
        )

        elapsed = time.perf_counter() - start

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)
        history["epoch_time"].append(elapsed)

        print(
            f"Epoch {epoch + 1:02d}/{EPOCHS} | "
            f"train loss {train_loss:.4f} | "
            f"train acc {train_acc:.4f} | "
            f"val loss {val_loss:.4f} | "
            f"val acc {val_acc:.4f} | "
            f"time {elapsed:.1f}s"
        )

    return model, history

```

This cell defines the complete training process for each sentiment model. The number of training epochs is set to 10, which is within the maximum limit required in the problem.

The `train_sentiment_model` function receives the model type as input, so the same training process can be used for RNN, LSTM, and GRU. At the beginning, the random seed is set again to make the training conditions more consistent between the three models.

The selected `SentimentRNN` model is then created with a hidden size of 64 and the GloVe embedding matrix from the previous cells. The model is moved to the available device, which is the GPU in this experiment.

For the loss function, `BCEWithLogitsLoss` is used because this is a binary classification problem. The Adam optimizer is used with a learning rate of 0.001 to update the model parameters during training.

A history dictionary is created to save the training loss, validation loss, training accuracy, validation accuracy, and training time for every epoch. These values will later be used to compare the models and create the required plots.

During each epoch, the model is first trained on the training set and then evaluated on the validation set. The time needed for the complete epoch is also measured. After each epoch, all important values are printed, including training loss, training accuracy, validation loss, validation accuracy, and elapsed time.

At the end of the training process, the function returns both the trained model and its complete training history. There is no output from this cell yet because it only defines the training function. The actual training of the RNN, LSTM, and GRU models is performed in the next cell.

Cell 10:

```
sentiment_models = {}
sentiment_history = {}
for model_type in ["RNN", "LSTM", "GRU"]:
    model, history = train_sentiment_model(model_type)
    sentiment_models[model_type] = model
    sentiment_history[model_type] = history
```

This cell trains all three sentiment models one after another. The loop goes through RNN, LSTM, and GRU and calls the training function from the previous cell for each model. After training is finished, the trained model is saved in `sentiment_models` and its training history is saved in `sentiment_history`. These saved values will be used later for plotting the learning curves and creating the final comparison table. Each model is trained for 10 epochs using exactly the same dataset, hidden size, embedding size, optimizer settings, and evaluation process. This makes the

comparison between the three architectures more fair. During every epoch, the code reports the training loss, training accuracy, validation loss, validation accuracy, and the time needed for that epoch.

Output:

Training RNN

Epoch 01/10 | train loss 0.4918 | train acc 0.7686 | val loss 0.4988 | val acc 0.7718 | time 32.2s
Epoch 02/10 | train loss 0.3593 | train acc 0.8491 | val loss 0.4831 | val acc 0.7890 | time 30.7s
Epoch 03/10 | train loss 0.2999 | train acc 0.8783 | val loss 0.4372 | val acc 0.8200 | time 31.3s
Epoch 04/10 | train loss 0.2649 | train acc 0.8956 | val loss 0.5693 | val acc 0.8073 | time 31.1s
Epoch 05/10 | train loss 0.2401 | train acc 0.9075 | val loss 0.5114 | val acc 0.8200 | time 31.8s
Epoch 06/10 | train loss 0.2241 | train acc 0.9135 | val loss 0.5578 | val acc 0.8073 | time 31.0s
Epoch 07/10 | train loss 0.2099 | train acc 0.9190 | val loss 0.5230 | val acc 0.8188 | time 30.6s
Epoch 08/10 | train loss 0.1972 | train acc 0.9264 | val loss 0.4535 | val acc 0.8326 | time 32.2s
Epoch 09/10 | train loss 0.1863 | train acc 0.9296 | val loss 0.5400 | val acc 0.8177 | time 30.9s
Epoch 10/10 | train loss 0.1781 | train acc 0.9328 | val loss 0.5649 | val acc 0.8154 | time 32.3s

Training LSTM

Epoch 01/10 | train loss 0.4603 | train acc 0.7842 | val loss 0.4610 | val acc 0.8016 | time 31.6s
Epoch 02/10 | train loss 0.3094 | train acc 0.8720 | val loss 0.4166 | val acc 0.8337 | time 32.1s
Epoch 03/10 | train loss 0.2514 | train acc 0.8986 | val loss 0.4081 | val acc 0.8429 | time 30.8s
Epoch 04/10 | train loss 0.2186 | train acc 0.9131 | val loss 0.4096 | val acc 0.8303 | time 30.5s
Epoch 05/10 | train loss 0.1967 | train acc 0.9230 | val loss 0.4311 | val acc 0.8314 | time 32.3s
Epoch 06/10 | train loss 0.1757 | train acc 0.9320 | val loss 0.4192 | val acc 0.8486 | time 30.7s
Epoch 07/10 | train loss 0.1628 | train acc 0.9365 | val loss 0.4493 | val acc 0.8463 | time 31.8s
Epoch 08/10 | train loss 0.1492 | train acc 0.9423 | val loss 0.4218 | val acc 0.8509 | time 31.7s
Epoch 09/10 | train loss 0.1396 | train acc 0.9455 | val loss 0.4536 | val acc 0.8475 | time 32.3s

Epoch 10/10 | train loss 0.1318 | train acc 0.9496 | val loss 0.4777 | val acc 0.8509 | time 31.2s

Training GRU

Epoch 01/10 | train loss 0.4510 | train acc 0.7878 | val loss 0.4192 | val acc 0.8073 | time 31.4s

Epoch 02/10 | train loss 0.3044 | train acc 0.8722 | val loss 0.4304 | val acc 0.8165 | time 32.3s

Epoch 03/10 | train loss 0.2503 | train acc 0.8985 | val loss 0.4170 | val acc 0.8372 | time 30.6s

Epoch 04/10 | train loss 0.2174 | train acc 0.9145 | val loss 0.4119 | val acc 0.8280 | time 32.1s

Epoch 05/10 | train loss 0.1940 | train acc 0.9252 | val loss 0.4531 | val acc 0.8280 | time 30.6s

Epoch 06/10 | train loss 0.1763 | train acc 0.9325 | val loss 0.4329 | val acc 0.8326 | time 31.3s

Epoch 07/10 | train loss 0.1626 | train acc 0.9376 | val loss 0.4362 | val acc 0.8452 | time 32.2s

Epoch 08/10 | train loss 0.1533 | train acc 0.9412 | val loss 0.4572 | val acc 0.8314 | time 31.1s

Epoch 09/10 | train loss 0.1403 | train acc 0.9460 | val loss 0.4239 | val acc 0.8452 | time 31.9s

Epoch 10/10 | train loss 0.1313 | train acc 0.9506 | val loss 0.4304 | val acc 0.8521 | time 30.8s

The results show that all three models learned the sentiment classification task successfully. For the simple RNN, the training accuracy increased from 76.86% in the first epoch to 93.28% in the final epoch. Its final validation accuracy was 81.54%. The validation accuracy changed between epochs and reached its highest value of 83.26% at epoch 8. This shows that the RNN learned the training data well, but its performance on the validation set was lower than the other two models.

The LSTM performed better than the simple RNN. Its training accuracy increased from 78.42% to 94.96%, and its final validation accuracy reached 85.09%. The validation accuracy also reached 85.09% at epoch 8 and stayed at the same level in the final epoch. The LSTM was able to keep more useful information from the sentence and gave better validation performance.

The GRU also performed well. Its training accuracy increased from 78.78% in the first epoch to 95.06% in the final epoch. Its final validation accuracy was 85.21%, which was the highest final validation accuracy among the three models. The GRU therefore gave the best final result in this experiment, although its result was very close to the LSTM. The training time was also similar for all three models. Most epochs took about 31 to 32 seconds. Overall, the results show that LSTM and GRU performed better than the simple RNN on SST-2. GRU achieved the highest final validation accuracy, while LSTM produced a very similar result. The gap between training and validation accuracy,

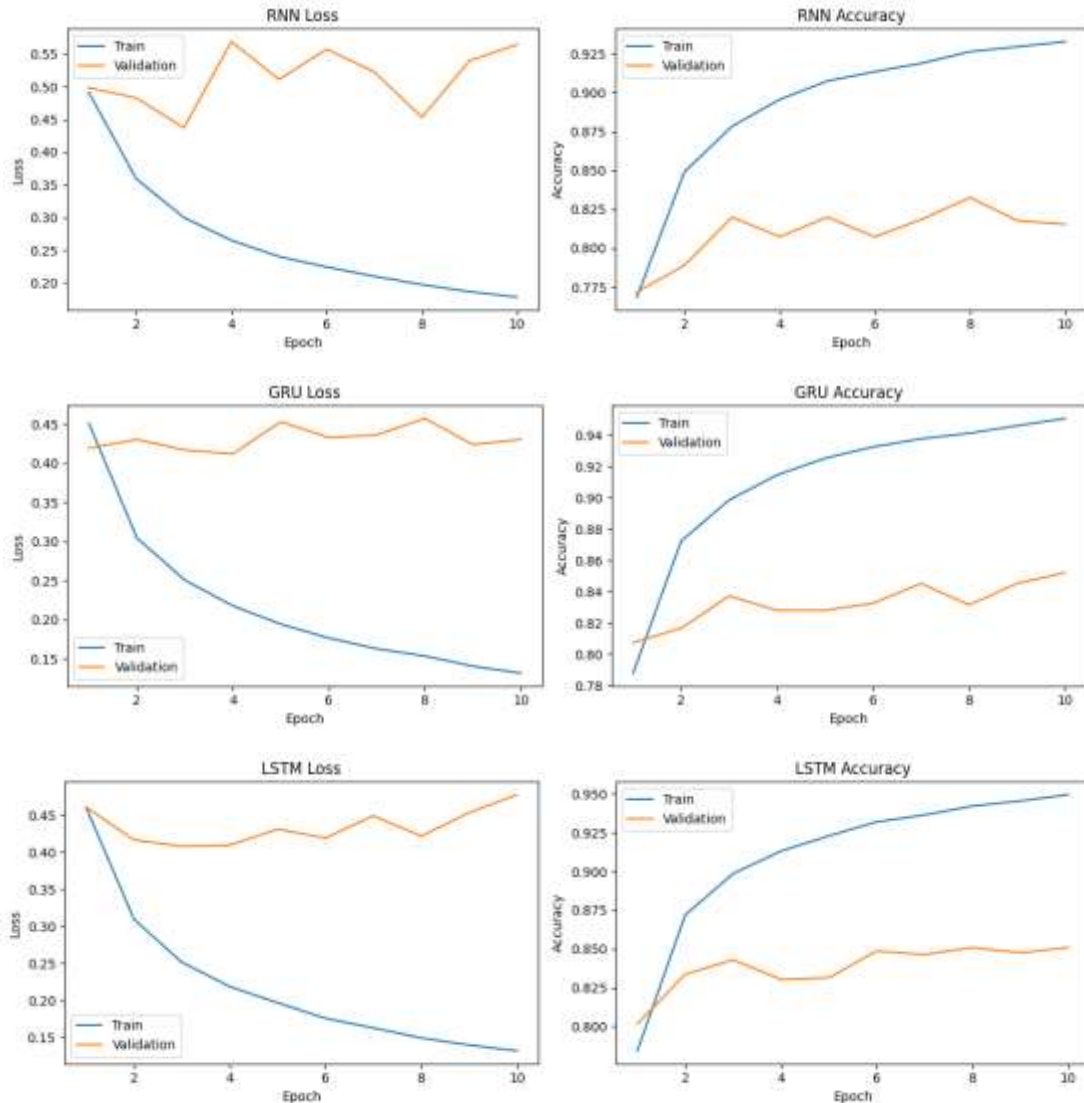
especially in the later epochs, also shows some overfitting, because the training accuracy continues to improve while the validation accuracy improves much more slowly or changes between epochs.

Cell 11:

```
epochs = range(1, EPOCHS + 1)
for model_type in ["RNN", "LSTM", "GRU"]:
    history = sentiment_history[model_type]
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, history["train_loss"], label="Train")
    plt.plot(epochs, history["val_loss"], label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"{model_type} Loss")
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.plot(epochs, history["train_acc"], label="Train")
    plt.plot(epochs, history["val_acc"], label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title(f"{model_type} Accuracy")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

This cell creates the learning curves for all three sentiment models. For each model, it draws two plots. The first plot shows how the training loss and validation loss change across the 10 epochs. The second plot shows how the training accuracy and validation accuracy change across the same epochs. These plots are useful because they help us understand not only the final result, but also the learning behavior of each model during training. For every model, the blue line represents the training values and the orange line represents the validation values. By looking at these curves, we can see whether the model is learning well, whether the validation performance is improving, and whether there are signs of overfitting.

Output:



The output contains three figures, one for each architecture: RNN, LSTM, and GRU. Each figure includes a loss plot and an accuracy plot.

In the RNN plots, the training loss decreases steadily during training, which shows that the model is fitting the training data better over time. However, the validation loss stays much higher and changes a lot from epoch to epoch. The validation accuracy improves at first, but then it becomes unstable and stays clearly below the training accuracy. This suggests that the simple RNN learns the training set, but its generalization is weaker than the other models.

In the LSTM plots, the training loss also decreases smoothly, and the training accuracy keeps rising across the epochs. The validation accuracy is better than the RNN and stays around the mid 80 percent range. The validation loss is still higher than the training loss and also fluctuates, which shows some overfitting, but overall the LSTM gives a stronger and more stable validation performance than the simple RNN.

In the GRU plots, the behavior is similar to the LSTM. The training loss decreases steadily and the training accuracy becomes high by the final epochs. The validation accuracy is slightly better than the LSTM at the end, which matches the numerical results from the previous cell. The validation curves still show some fluctuation, but the GRU gives the best final validation accuracy among the three models.

Overall, the plots confirm the main conclusion of the experiment. All three models learn the sentiment classification task, but LSTM and GRU perform better than the simple RNN. The gap between training and validation curves in all three models also shows that some overfitting appears during later epochs, especially after the models become very accurate on the training set.

Cell 12:

```
comparison_rows = []

for model_type in ["RNN", "LSTM", "GRU"]:
    history = sentiment_history[model_type]
    model = sentiment_models[model_type]

    comparison_rows.append({
        "Architecture": model_type,
        "Final Validation Accuracy": history["val_acc"][-1],
        "Average Time per Epoch (s)": np.mean(history["epoch_time"]),
        "Total Parameters": count_parameters(model)
    })

sentiment_results = pd.DataFrame(comparison_rows)

sentiment_results
```

This cell creates a summary table to compare the three sentiment models. For each architecture, the code takes the final validation accuracy from the last training epoch, calculates the average training time per epoch, and counts the total number of model parameters. These values are stored in a list and then converted into a Pandas DataFrame so that the results can be compared more easily.

The final validation accuracy shows how well each model performs on the validation set after the last epoch. The average time per epoch gives an idea of the training cost of each architecture, while the total number of parameters shows the size and complexity of each model.

Output:

	Architecture	Final Accuracy	Validation	Average Epoch (s)	Time per	Total Parameters
0	RNN	0.815367		31.424780		675689
1	LSTM	0.850917		31.503434		697961
2	GRU	0.852064		31.421086		690537

The table shows that the simple RNN achieved a final validation accuracy of about 81.54%. The LSTM reached about 85.09%, while the GRU achieved the highest final validation accuracy with about 85.21%.

The training time is almost the same for all three models. The RNN takes about 31.42 seconds per epoch, the LSTM takes about 31.50 seconds, and the GRU takes about 31.42 seconds.

The RNN has the smallest number of parameters with 675,689 parameters. The LSTM has the largest model with 697,961 parameters. The GRU has 690,537 parameters, which is fewer than the LSTM but more than the simple RNN.

Overall, the GRU gives the best final validation accuracy while using fewer parameters than the LSTM. The LSTM performs almost as well as the GRU, and both models clearly perform better than the simple RNN. The training times are very similar, so the main differences between the models are their validation accuracy and number of parameters.

Cell 13:

```
display_table = sentiment_results.copy()

display_table["Final Validation Accuracy"] = (
    display_table["Final Validation Accuracy"] * 100
).round(2)

display_table["Average Time per Epoch (s)"] = (
    display_table["Average Time per Epoch (s)"].round(2)
)
```

```
display_table
```

This cell creates a cleaner version of the comparison table from the previous cell. The original results are copied so that the main DataFrame is not changed. The final validation accuracy is converted from decimal form to percentage form by multiplying it by 100, and the values are rounded to two decimal places. The average training time per epoch is also rounded to two decimal places to make the table easier to read.

This step does not change the actual experiment results. It only improves the way the results are displayed so they can be used more clearly in the report.

Output:

	Architecture	Final Accuracy	Validation	Average Epoch (s)	Time per	Total Parameters
0	RNN	81.54		31.42		675689
1	LSTM	85.09		31.50		697961
2	GRU	85.21		31.42		690537

The final table shows the comparison in a simpler format. The RNN reaches a final validation accuracy of 81.54%, the LSTM reaches 85.09%, and the GRU reaches 85.21%. The average training time is almost the same for all three models, around 31.4 to 31.5 seconds per epoch.

The GRU gives the highest final validation accuracy, while the LSTM is very close. The RNN has the lowest validation accuracy but also has the smallest number of parameters. The GRU also uses fewer parameters than the LSTM while giving slightly better final accuracy. This makes the GRU the best overall result in this comparison.

Extension: Final Hidden State for Sentence-Level and Token-Level Tasks

For sentiment classification, the goal is to predict only one label for the whole sentence. Because of this, using the final hidden state is a reasonable choice. After the RNN, LSTM, or GRU processes all words in the sentence, the final hidden state contains information collected from the sequence. This representation can then be passed to a classification layer to predict whether the sentiment is positive or negative.

However, this method is not suitable for token-level tasks such as part-of-speech tagging. In this type of task, every word needs its own prediction. If we use only the final hidden state, the information related to individual words is not directly available for classification. Instead, we should keep the hidden state produced at every time step and apply a classification layer to each hidden state. In this way, the model can produce one label for every token in the input sequence.

Cell 14:

```
co2 = sm.datasets.co2.load_pandas().data["co2"]

co2 = co2.interpolate(limit_direction="both").astype(np.float32)

print(co2.head())
print("Number of observations:", len(co2))

plt.figure(figsize=(12, 4))
plt.plot(co2.index, co2.values)
plt.xlabel("Date")
plt.ylabel("CO2")
plt.title("Mauna Loa Weekly CO2")
plt.show()
```

This cell starts the time-series forecasting part of Problem 2. It loads the Mauna Loa weekly CO2 dataset from statsmodels. This dataset contains weekly measurements of atmospheric CO2 over many years, so it is useful for comparing sequence models on a real time-series problem.

The code then fills any missing values using interpolation. This is important because the forecasting models need a continuous sequence of numerical values. After that, the data is converted to float32, which is a common format for neural network training.

The first few values of the dataset are printed, together with the total number of observations. Finally, the complete time series is plotted so that we can visually inspect its main pattern before preparing it for training.

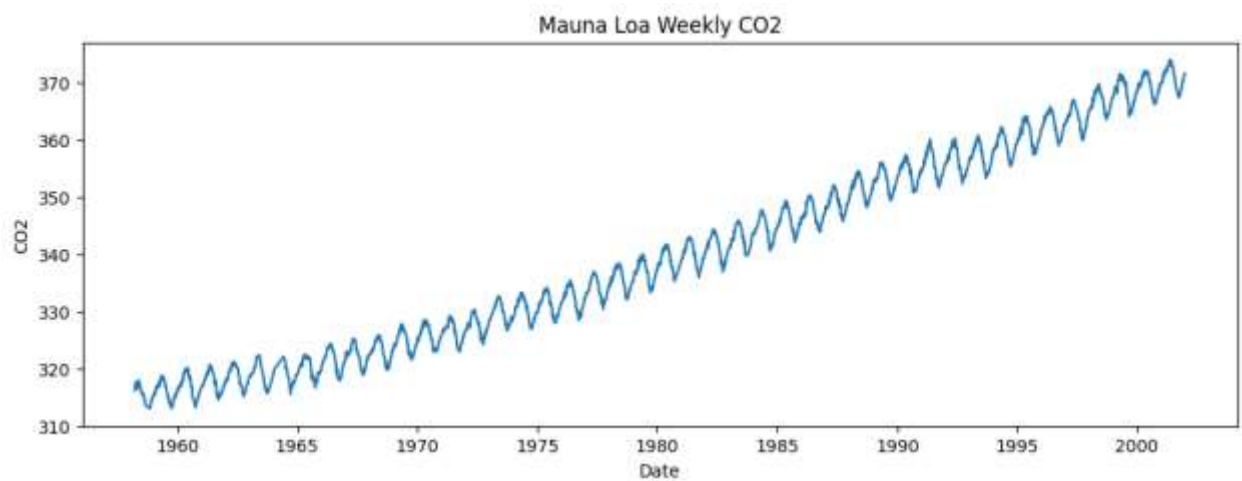
Output:

```
1958-03-29    316.100006
```

```

1958-04-05    317.299988
1958-04-12    317.600006
1958-04-19    317.500000
1958-04-26    316.399994
Freq: W-SAT, Name: co2, dtype: float32
Number of observations: 2284

```



The output shows the first observations from the dataset and confirms that the series contains 2,284 weekly measurements. The first values are around 316 to 317 CO₂ units, starting in 1958.

The plot shows two clear patterns in the data. First, there is a long-term upward trend, because the CO₂ level increases over the years. Second, there is a repeating seasonal pattern, visible as regular up-and-down movements throughout the series.

These characteristics make the dataset suitable for time-series forecasting. The models need to learn both the repeated seasonal behavior and the longer-term increase in CO₂ levels in order to predict future values.

Cell 15:

```

values = co2.values.reshape(-1, 1)

n = len(values)

train_end = int(n * 0.70)
val_end = int(n * 0.85)

scaler = StandardScaler()

```

```
scaler.fit(values[:train_end])

scaled_values = scaler.transform(values).flatten()

train_values = scaled_values[:train_end]
val_values = scaled_values[train_end:val_end]
test_values = scaled_values[val_end:]

print("Train:", len(train_values))
print("Validation:", len(val_values))
print("Test:", len(test_values))
```

This cell prepares the CO2 time-series data for training, validation, and testing. First, the CO2 values are reshaped into a two-dimensional format because StandardScaler expects the input in this form.

The dataset is then divided in chronological order. The first 70% of the observations are used for training, the next 15% are used for validation, and the final 15% are used for testing. Since this is time-series data, the order of the observations is kept and the data is not shuffled.

Next, StandardScaler is fitted only on the training part of the data. This is important because information from the validation and test sets should not be used during training. After fitting the scaler, the same transformation is applied to the full series. This makes the values easier for the neural network models to learn by putting them on a similar scale.

Finally, the scaled data is separated into training, validation, and test sets.

Output:

```
Train: 1598
Validation: 343
Test: 343
```

This confirms that the 2,284 observations were divided correctly. The training set contains 1,598 values, while the validation and test sets each contain 343 values. This split will be used in the next steps to create input sequences for the LSTM and Transformer forecasting models.

Cell 16:


```

def make_sequences(values, lookback, horizon):
    x = []
    y = []

    for i in range(len(values) - lookback - horizon + 1):
        x.append(values[i:i + lookback])
        y.append(values[i + lookback:i + lookback + horizon])

    x = np.array(x, dtype=np.float32)
    y = np.array(y, dtype=np.float32)

    x = x[..., None]

    return torch.tensor(x), torch.tensor(y)

def make_time_loaders(lookback, horizon, batch_size=64):
    x_train, y_train = make_sequences(
        train_values,
        lookback,
        horizon
    )

    x_val, y_val = make_sequences(
        val_values,
        lookback,
        horizon
    )

    x_test, y_test = make_sequences(
        test_values,
        lookback,
        horizon
    )

    train_loader = DataLoader(
        TensorDataset(x_train, y_train),
        batch_size=batch_size,

```

```

        shuffle=True
    )

    val_loader = DataLoader(
        TensorDataset(x_val, y_val),
        batch_size=batch_size,
        shuffle=False
    )

    test_loader = DataLoader(
        TensorDataset(x_test, y_test),
        batch_size=batch_size,
        shuffle=False
    )

    return train_loader, val_loader, test_loader

```

This cell converts the time-series data into input and output sequences that can be used by the forecasting models. The `make_sequences` function receives the data together with a lookback value and a forecast horizon.

The lookback determines how many previous time steps are given to the model as input. The horizon determines how many future values the model must predict. For example, if the lookback is 24 and the horizon is 4, the model receives 24 past CO2 values and tries to predict the next 4 values.

Inside the function, the input sequences are stored in `x` and the future target values are stored in `y`. The sequences are then converted to NumPy arrays using `float32`. An extra dimension is added to `x` because each time step has one feature, which is the CO2 value. Finally, both `x` and `y` are converted into PyTorch tensors.

The `make_time_loaders` function uses `make_sequences` separately for the training, validation, and test sets. It then creates `DataLoader` objects with a batch size of 64. The training loader uses `shuffle=True` so that the training samples are presented in a different order during training. The validation and test loaders are not shuffled because they are only used for evaluation.

At the end, the function returns the three `DataLoaders`. These loaders can be created again with different lookback windows and forecast horizons, which allows us to compare the LSTM and Transformer under different forecasting conditions.

There is no printed output for this cell because it only defines the functions used to prepare the time-series sequences and `DataLoaders`.

Cell 17:

```
class LSTMForecaster(nn.Module):
    def __init__(self, horizon, hidden_size=64):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=hidden_size,
            batch_first=True
        )

        self.fc = nn.Linear(hidden_size, horizon)

    def forward(self, x):
        _, (hidden, _) = self.lstm(x)

        return self.fc(hidden[-1])
```

This cell defines the LSTM model that will be used for time-series forecasting. The model receives a sequence of past CO2 values and predicts one or more future values depending on the selected forecast horizon.

The LSTM layer uses one input feature because each time step contains only one CO2 value. The hidden size is set to 64, which means that the LSTM keeps a 64-dimensional hidden representation while processing the input sequence. The model also uses `batch_first=True`, so the input shape is organized as batch size, sequence length, and number of features.

In the forward function, the complete input sequence is passed through the LSTM. The final hidden state is then used as a summary of the past observations. This hidden state is passed to a linear layer.

The output size of the linear layer is equal to the forecast horizon. This allows the same model to be used for different forecasting tasks. For example, if the horizon is 1, the model predicts one future CO2 value. If the horizon is 4 or 12, it predicts the next 4 or 12 values at the same time.

There is no printed output for this cell because it only defines the LSTM forecasting architecture.

Cell 18:

```

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()

        position = torch.arange(max_len).unsqueeze(1)

        div_term = torch.exp(
            torch.arange(0, d_model, 2)
            * (-np.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, d_model)

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

```

This cell defines the positional encoding used by the Transformer model. Unlike an LSTM, a Transformer does not automatically process the sequence step by step, so it needs extra information to understand the order of the time steps.

The PositionalEncoding class creates a fixed position representation using sine and cosine functions. Each position in the input sequence gets a different pattern, and this pattern is added to the input features before they are passed to the Transformer.

The variable `d_model` represents the size of the Transformer representation, while `max_len` sets the maximum sequence length that can be supported. The positional values are created once and stored inside the model with `register_buffer`, so they move with the model to the correct device but are not trained as normal parameters.

In the forward function, the positional encoding is added to the input sequence. This gives the Transformer information about where each time step appears in the sequence.

There is no printed output for this cell because it only defines the positional encoding module that will be used in the Transformer forecasting model.

Cell 19:

```
class TransformerForecaster(nn.Module):
    def __init__(
        self,
        horizon,
        d_model=32,
        nhead=4,
        num_layers=2
    ):
        super().__init__()

        self.input_projection = nn.Linear(1, d_model)

        self.position = PositionalEncoding(d_model)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=64,
            dropout=0.1,
            activation="gelu",
            batch_first=True
        )

        self.encoder = nn.TransformerEncoder(
            encoder_layer,
            num_layers=num_layers
        )

        self.fc = nn.Linear(d_model, horizon)

    def forward(self, x):
        x = self.input_projection(x)
        x = self.position(x)
        x = self.encoder(x)

        return self.fc(x[:, -1])
```

This cell defines the Transformer model used for time-series forecasting. The goal of this model is the same as the LSTM model: it receives a sequence of past CO2 values and predicts future values based on the selected forecast horizon.

At the beginning, the input has only one feature for each time step, which is the CO2 value. The input_projection layer converts this single value into a 32-dimensional representation so that it can be processed by the Transformer. After that, the positional encoding from the previous cell is added to keep information about the order of the time steps.

The Transformer encoder layer uses a model dimension of 32 and 4 attention heads. It also contains a feedforward layer with size 64, dropout of 0.1, and the GELU activation function. Two Transformer encoder layers are stacked together in this model. These layers use self-attention to learn relationships between different positions in the input sequence.

In the forward function, the input is first projected to the Transformer dimension, then positional information is added, and the sequence is passed through the Transformer encoder. After the encoder finishes processing the sequence, the representation from the last time step is selected.

This final representation is passed through a linear layer. The output size of this layer is equal to the forecast horizon, so the model can predict one or several future CO2 values at the same time.

There is no printed output for this cell because it only defines the Transformer forecasting architecture.

Cell 20:

```
def evaluate_forecaster(model, loader, criterion):
    model.eval()

    total_loss = 0
    total_samples = 0

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)
            predictions = model(x)
            loss = criterion(predictions, y)
            total_loss += loss.item() * x.size(0)
            total_samples += x.size(0)
```

```

        return total_loss / total_samples
def train_forecaster(model, train_loader, val_loader, epochs=10):
    model = model.to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=0.001,
        weight_decay=1e-4
    )
    best_val_loss = float("inf")
    best_state = None
    epoch_times = []

    for epoch in range(epochs):
        model.train()
        start = time.perf_counter()
        for x, y in train_loader:
            x = x.to(device)
            y = y.to(device)
            optimizer.zero_grad()
            predictions = model(x)
            loss = criterion(predictions, y)
            loss.backward()
            optimizer.step()

        val_loss = evaluate_forecaster(
            model,
            val_loader,
            criterion
        )

        elapsed = time.perf_counter() - start
        epoch_times.append(elapsed)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = copy.deepcopy(model.state_dict())

```

```
model.load_state_dict(best_state)

return model, np.mean(epoch_times)
```

This cell defines the functions used to train and evaluate the time-series forecasting models. The `evaluate_forecaster` function is used during validation. It puts the model in evaluation mode and disables gradient calculation because the model parameters are not updated during this step.

For each validation batch, the input and target values are moved to the selected device. The model makes predictions and the loss is calculated using the selected criterion. The function adds the loss from all batches and finally returns the average validation loss over the full validation set.

The `train_forecaster` function is used to train either the LSTM or Transformer model. First, the model is moved to the GPU. Mean Squared Error is used as the loss function because this is a regression problem and the goal is to predict continuous CO2 values.

The AdamW optimizer is used with a learning rate of 0.001 and a small weight decay value. During each epoch, the model goes through the training data, calculates predictions and loss, performs backpropagation, and updates its parameters.

After every training epoch, the model is evaluated on the validation set. The code keeps track of the model state that gives the lowest validation loss. This is useful because the best model may appear before the final epoch. At the end of training, the best saved model is loaded again.

The training time for each epoch is also measured, and the function returns the trained model together with the average training time per epoch.

There is no printed output for this cell because it only defines the training and evaluation functions that will be used in the following experiments.

Cell 21:

```
def get_predictions(model, loader):
    model.eval()

    predictions = []
    targets = []
```



```

with torch.no_grad():
    for x, y in loader:
        x = x.to(device)

        pred = model(x)

        predictions.append(pred.cpu().numpy())
        targets.append(y.numpy())

predictions = np.concatenate(predictions)
targets = np.concatenate(targets)

return predictions, targets

def inverse_scale(x):
    shape = x.shape

    x = scaler.inverse_transform(
        x.reshape(-1, 1)
    )

    return x.reshape(shape)

```

This cell defines two helper functions that are used after the forecasting models are trained.

The `get_predictions` function is used to collect the model predictions and the real target values from a `DataLoader`. The model is first placed in evaluation mode, and gradient calculation is disabled because no training is happening in this step. For each batch, the input data is moved to the selected device and passed through the model. The predicted values are moved back to the CPU and converted to NumPy arrays. The real target values are also stored.

After all batches are processed, the predictions and targets are combined into complete arrays. These arrays are then returned so that we can calculate evaluation metrics such as MAE and RMSE.

The `inverse_scale` function converts the normalized values back to their original CO2 scale. Before training, the time-series data was standardized using `StandardScaler`. Because of this, the raw model outputs are also in the scaled form. This function reshapes the data, applies the inverse transformation, and then restores the original shape.

Using the original CO2 scale makes the final errors and plots easier to understand and compare.

There is no printed output for this cell because it only defines helper functions that will be used in the next experiment cells.

Cell 22:

```
LOOKBACKS = [24, 52]
HORIZONS = [1, 4, 12]
TS_EPOCHS = 10
forecast_results = []
forecast_outputs = {}

for lookback in LOOKBACKS:
    for horizon in HORIZONS:
        print(
            f"\nLookback={lookback}, Horizon={horizon}"
        )

        train_ts_loader, val_ts_loader, test_ts_loader = (
            make_time_loaders(
                lookback,
                horizon
            )
        )

        for model_name in ["LSTM", "Transformer"]:
            set_seed()
            if model_name == "LSTM":
                model = LSTMForecaster(
                    horizon=horizon,
                    hidden_size=64
                )
            else:
                model = TransformerForecaster(
                    horizon=horizon
                )
            model, avg_time = train_forecaster(
                model,
```

```

        train_ts_loader,
        val_ts_loader,
        epochs=TS_EPOCHS
    )

    pred_scaled, target_scaled = get_predictions(
        model,
        test_ts_loader
    )
    predictions = inverse_scale(pred_scaled)
    targets = inverse_scale(target_scaled)

    mae = np.mean(
        np.abs(predictions - targets)
    )
    rmse = np.sqrt(
        np.mean((predictions - targets) ** 2)
    )
    params = count_parameters(model)
    forecast_results.append({
        "Model": model_name,
        "Lookback": lookback,
        "Horizon": horizon,
        "MAE": mae,
        "RMSE": rmse,
        "Average Time per Epoch": avg_time,
        "Parameters": params
    })

    forecast_outputs[
        (model_name, lookback, horizon)
    ] = (predictions, targets)

    print(
        f"{model_name}: "
        f"MAE={mae:.3f}, "
        f"RMSE={rmse:.3f}"
    )

```

This cell runs the main time-series forecasting experiments for both the LSTM and Transformer models. The goal is to compare the two architectures under different input lengths and different forecasting horizons.

Two lookback values are used, 24 and 52. The lookback controls how many past CO2 values are given to the model as input. Three forecast horizons are also used: 1, 4, and 12. This means that the models are tested for predicting the next 1, 4, or 12 future values.

For every combination of lookback and horizon, the code first creates new training, validation, and test DataLoaders. Then both the LSTM and Transformer are trained using the same data split and the same number of epochs, which is 10.

After training, predictions are collected from the test set. Since the data was standardized before training, both the predictions and targets are converted back to the original CO2 scale. Two evaluation metrics are then calculated. MAE measures the average absolute prediction error, while RMSE gives more weight to larger errors.

The code also records the average training time per epoch and the total number of parameters for each model. All results are saved in `forecast_results`, while the actual predictions and targets are saved in `forecast_outputs` for later analysis and plotting.

This experiment produces six different forecasting settings for each architecture, because there are two lookback values and three forecast horizons. This makes it possible to study how input length and prediction distance affect the performance of LSTM and Transformer.

The printed output from this cell reports the MAE and RMSE for every model and every forecasting setting. These values will be organized and compared more clearly in the next cells.

Output:

```
Lookback=24, Horizon=1
```

```
LSTM: MAE=6.106, RMSE=6.501
```

```
Transformer: MAE=9.510, RMSE=10.001
```

```
Lookback=24, Horizon=4
```

```
LSTM: MAE=8.214, RMSE=8.674
```

```
Transformer: MAE=13.775, RMSE=14.254
```

```
Lookback=24, Horizon=12
```

```
LSTM: MAE=10.828, RMSE=11.262
```

```
Transformer: MAE=15.498, RMSE=15.939
```

```
Lookback=52, Horizon=1
```

```
LSTM: MAE=5.857, RMSE=6.197
```

```
Transformer: MAE=10.193, RMSE=10.629
```

```
Lookback=52, Horizon=4
```

```
LSTM: MAE=6.382, RMSE=6.808
```

```
Transformer: MAE=14.272, RMSE=14.772
```

```
Lookback=52, Horizon=12
```

```
LSTM: MAE=12.261, RMSE=12.677
```

```
Transformer: MAE=15.766, RMSE=16.208
```

The output shows the test performance of both the LSTM and Transformer for all combinations of lookback window and forecast horizon. Lower MAE and RMSE values mean better forecasting performance.

For lookback 24 and horizon 1, the LSTM reaches an MAE of 6.106 and an RMSE of 6.501, while the Transformer has a higher MAE of 9.510 and RMSE of 10.001. This means the LSTM performs better for short-term forecasting in this setting.

When the forecast horizon increases to 4, the error becomes larger for both models. The LSTM reaches an RMSE of 8.674, while the Transformer reaches 14.254. For horizon 12, the error increases again, with an RMSE of 11.262 for the LSTM and 15.939 for the Transformer. This shows that predicting further into the future is more difficult.

The same general pattern appears with lookback 52. For horizon 1, the LSTM gives an RMSE of 6.197, which is the lowest RMSE among all tested configurations. For horizon 4, the LSTM also performs well with an RMSE of 6.808, while the Transformer reaches 14.772. For horizon 12, the LSTM has an RMSE of 12.677 and the Transformer has an RMSE of 16.208.

Overall, the LSTM performs better than the Transformer in all six forecasting settings. The results also show that increasing the forecast horizon usually increases the prediction error. A longer lookback of 52 helps the LSTM for short and medium forecast horizons, especially for horizons 1 and 4, but it does not improve the result for horizon 12.

These results suggest that the LSTM is more suitable for this relatively small and simple univariate CO2 dataset. The Transformer does not show an advantage in this experiment, and its errors remain higher across all tested lookback windows and forecast horizons.

Cell 23:

```

forecast_df = pd.DataFrame(forecast_results)
forecast_df = forecast_df.sort_values(
    ["Lookback", "Horizon", "Model"]
).reset_index(drop=True)

forecast_df

```

This cell organizes all of the forecasting results into a single table. The `forecast_results` list from the previous experiment is converted into a Pandas DataFrame. The table is then sorted by lookback value, forecast horizon, and model name so that the LSTM and Transformer results can be compared more easily for each setting.

The table includes the model name, lookback window, forecast horizon, MAE, RMSE, average training time per epoch, and total number of parameters. This gives a complete summary of both prediction performance and computational cost.

Output:

Model	Lookback	Horizon	MAE	RMSE	Average per Epoch	Time	Parameters
0	LSTM	24	1	6.106191	6.501007	0.083744	17217
1	Transformer	24	1	9.509612	10.001097	0.168440	17185
2	LSTM	24	4	8.214398	8.674197	0.066599	17412
3	Transformer	24	4	13.775141	14.254384	0.154568	17284
4	LSTM	24	12	10.827605	11.261613	0.065275	17932
5	Transformer	24	12	15.498055	15.939251	0.156471	17548
6	LSTM	52	1	5.856747	6.197041	0.065351	17217
7	Transformer	52	1	10.192596	10.628975	0.162152	17185
8	LSTM	52	4	6.382191	6.807568	0.091928	17412

Model	Lookback	Horizon	MAE	RMSE	Average per Epoch	Time	Parameters
9	Transformer	52	4	14.271742	14.771534	0.189662	17284
10	LSTM	52	12	12.260808	12.676767	0.065730	17932
11	Transformer	52	12	15.766046	16.208118	0.153485	17548

The results show that the LSTM has lower MAE and RMSE than the Transformer in every tested configuration. For example, with lookback 24 and horizon 1, the LSTM has an RMSE of 6.501, while the Transformer has an RMSE of 10.001. With lookback 52 and horizon 1, the LSTM improves slightly and reaches an RMSE of 6.197, which is the best result in the whole table.

The LSTM also remains better for longer forecast horizons. For lookback 24 and horizon 12, its RMSE is 11.262 compared with 15.939 for the Transformer. For lookback 52 and horizon 12, the LSTM has an RMSE of 12.677, while the Transformer reaches 16.208.

The training time per epoch is also lower for the LSTM in most cases. The LSTM takes roughly 0.07 to 0.09 seconds per epoch, while the Transformer usually takes around 0.15 to 0.19 seconds. This shows that the Transformer is slower in this experiment.

The parameter counts of the two models are quite similar. Depending on the forecast horizon, the LSTM has between 17,217 and 17,932 parameters, while the Transformer has between 17,185 and 17,548 parameters. This means that the performance difference is not mainly caused by a large difference in model size.

Overall, this table confirms that the LSTM is both more accurate and generally faster than the Transformer on this CO2 forecasting task.

Cell 24:

```
comparison = forecast_df.pivot_table(
    index=["Lookback", "Horizon"],
    columns="Model",
    values="RMSE"
)

comparison["Winner"] = np.where(
```

```

comparison["LSTM"] < comparison["Transformer"],
    "LSTM",
    "Transformer"
)

comparison

```

This cell creates a simpler comparison table using only the RMSE values of the LSTM and Transformer models. The purpose is to make it easier to see which model performs better for each combination of lookback window and forecast horizon.

The `pivot_table` function reorganizes the previous results so that the LSTM and Transformer RMSE values appear next to each other. The table is indexed by the lookback and horizon values.

The code then compares the RMSE of the two models. Since a lower RMSE means better forecasting performance, the model with the smaller value is selected as the winner for each configuration. The result is stored in a new column called Winner.

Output:

Model	LSTM	Transformer	Winner
Lookback	Horizon		
24	1	6.501007	10.001097 LSTM
	4	8.674197	14.254384 LSTM
	12	11.261613	15.939251 LSTM
52	1	6.197041	10.628975 LSTM
	4	6.807568	14.771534 LSTM
	12	12.676767	16.208118 LSTM

The table clearly shows that the LSTM performs better than the Transformer in every tested configuration. For lookback 24, the LSTM gives lower RMSE values for horizons 1, 4, and 12. The same result is also seen with lookback 52.

For lookback 24, the LSTM RMSE values are 6.501, 8.674, and 11.262 for horizons 1, 4, and 12, while the Transformer gives higher values of 10.001, 14.254, and 15.939. and for lookback 52, the LSTM reaches RMSE values of 6.197, 6.808, and 12.677, while the Transformer reaches 10.629, 14.772, and 16.208.

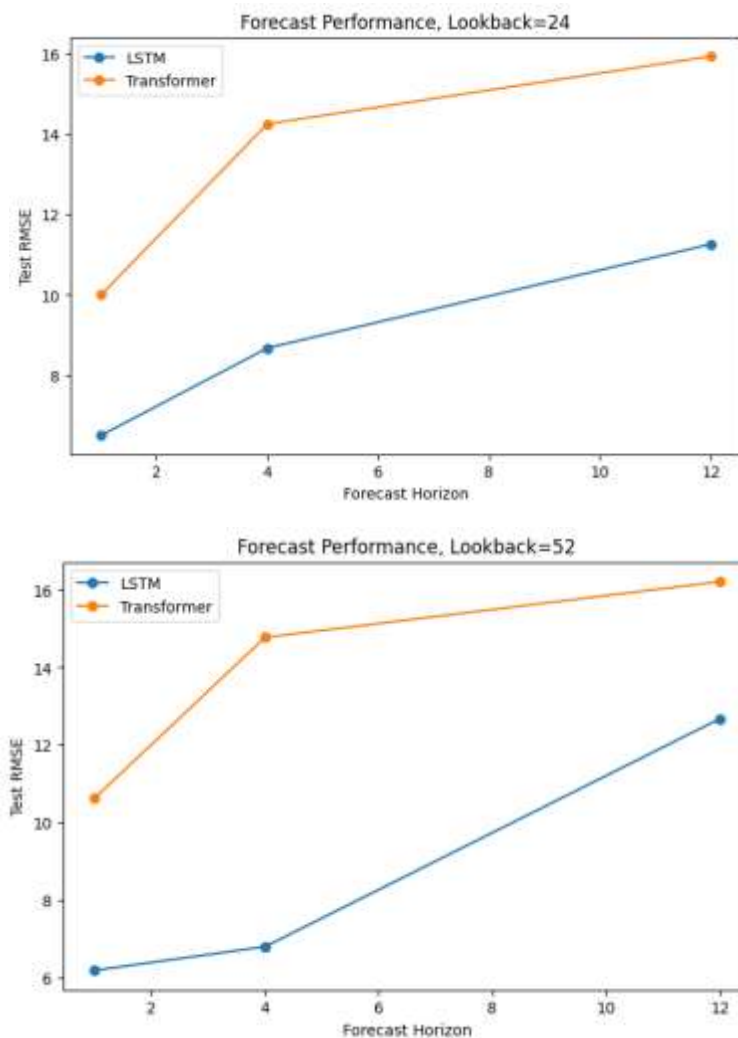
Therefore, the Winner column shows LSTM for all six experiments. The best overall result is also achieved by the LSTM with lookback 52 and horizon 1, where the RMSE is 6.197. This comparison makes the main result of the forecasting experiment very clear. On this CO2 dataset, the LSTM gives more accurate forecasts than the Transformer for both tested input lengths and all three forecast horizons.

Cell 25:

```
for lookback in LOOKBACKS:
    plt.figure(figsize=(8, 5))
    subset = forecast_df[
        forecast_df["Lookback"] == lookback
    ]
    for model_name in ["LSTM", "Transformer"]:
        part = subset[
            subset["Model"] == model_name
        ]
        plt.plot(
            part["Horizon"],
            part["RMSE"],
            marker="o",
            label=model_name
        )
    plt.xlabel("Forecast Horizon")
    plt.ylabel("Test RMSE")
    plt.title(f"Forecast Performance, Lookback={lookback}")
    plt.legend()
    plt.show()
```

This cell plots the forecasting performance of the two models for each lookback value. For each plot, the x-axis shows the forecast horizon and the y-axis shows the test RMSE. One line is drawn for the LSTM and the other line is drawn for the Transformer. These plots make it easier to see how the forecasting error changes as the prediction horizon becomes larger. The code first selects the results for one lookback value, either 24 or 52. Then it separates the rows for the LSTM and the Transformer and plots their RMSE values across the three forecast horizons. This gives one figure for lookback 24 and one figure for lookback 52.

Output:



The two plots show a clear pattern. In both figures, the RMSE increases as the forecast horizon becomes larger. This means that predicting further into the future is more difficult for both models.

For lookback 24, the LSTM starts with the lowest error at horizon 1 and then the error increases at horizons 4 and 12. The Transformer also shows increasing error, but its RMSE is much higher than the LSTM at every horizon. This means that the LSTM gives better forecasts than the Transformer for all tested prediction lengths when the input window is 24.

The same pattern appears for lookback 52. The LSTM again has lower RMSE than the Transformer for horizons 1, 4, and 12. The error still increases as the horizon becomes larger, especially from horizon 4 to horizon 12. The Transformer remains worse than the LSTM across the whole plot.

Overall, these graphs visually confirm the numerical results from the previous tables. The LSTM performs better than the Transformer in every setting, and both models become less accurate when they are asked to predict further into the future.

Cell 26:

```
print("Average RMSE by lookback")
display(
    forecast_df.groupby(
        ["Model", "Lookback"]
    )["RMSE"].mean().round(3)
)
print("\nAverage RMSE by forecast horizon")
display(
    forecast_df.groupby(
        ["Model", "Horizon"]
    )["RMSE"].mean().round(3)
)
```

This cell summarizes the forecasting results in two different ways. First, it calculates the average RMSE for each model at each lookback value. This helps us understand whether using a longer input sequence improves the forecasting performance. Second, it calculates the average RMSE for each model at each forecast horizon. This shows how the prediction error changes when the model has to predict further into the future. For the lookback comparison, the results from all three forecast horizons are averaged together. For the horizon comparison, the results from both lookback values are averaged together. This gives a simple overall view of how input length and forecast distance affect the two models.

Output:

```
Average RMSE by lookback
```

RMSE		
Model	Lookback	
LSTM	24	8.812
	52	8.560
Transformer	24	13.398
	52	13.870

```
dtype: float32
```

```
Average RMSE by forecast horizon
```

RMSE		
Model	Horizon	
LSTM	1	6.349000
	4	7.741000
	12	11.969000
Transformer	1	10.315000
	4	14.513000
	12	16.073999

```
dtype: float32
```

For the LSTM, the average RMSE is 8.812 with lookback 24 and 8.560 with lookback 52. This shows that using a longer input window gives a small improvement for the LSTM overall.

For the Transformer, the average RMSE is 13.398 with lookback 24 and 13.870 with lookback 52. In this case, increasing the lookback does not improve the result and actually makes the average error slightly larger. The second part of the output shows the effect of the forecast horizon. For the LSTM, the average RMSE increases from 6.349 at horizon 1 to 7.741 at horizon 4 and 11.969 at horizon 12. The same pattern appears for the Transformer, where the RMSE increases from 10.315 at horizon 1 to 14.513 at horizon 4 and 16.074 at horizon 12.

These results show that longer forecast horizons make the prediction task more difficult for both models. They also show that the LSTM benefits slightly from a longer lookback window, while the Transformer does not gain the same advantage in this experiment.

Cell 27:

```
best_row = forecast_df.loc[
    forecast_df["RMSE"].idxmin()
]
best_model_name = best_row["Model"]
best_lookback = int(best_row["Lookback"])
best_horizon = int(best_row["Horizon"])
print("Best configuration")
print("Model:", best_model_name)
print("Lookback:", best_lookback)
print("Horizon:", best_horizon)
print("RMSE:", round(best_row["RMSE"], 3))
```

This cell finds the best forecasting configuration among all of the LSTM and Transformer experiments. The code searches the results table for the row with the lowest RMSE value, because a smaller RMSE means better forecasting performance. After finding the best row, the code saves the model name, lookback window, forecast horizon, and RMSE value. These values are then printed so that the best overall configuration can be clearly identified.

Output:

```
Best configuration
Model: LSTM
Lookback: 52
Horizon: 1
RMSE: 6.197
```

This means that the best forecasting result in the experiment was produced by the LSTM model. It used the previous 52 weekly CO2 observations to predict the next value and achieved an RMSE of 6.197. This result agrees with the previous comparisons, where the LSTM performed better than the Transformer in all tested settings. It also shows that, for one-step forecasting, using a longer lookback window of 52 gave the best result in this experiment.

Cell 28:

```
key = (
    best_model_name,
    best_lookback,
    best_horizon
)
predictions, targets = forecast_outputs[key]
n_show = min(150, len(predictions))
plt.figure(figsize=(12, 4))
plt.plot(
    targets[:n_show, 0],
    label="Actual"
)

plt.plot(
    predictions[:n_show, 0],
    label="Forecast"
)

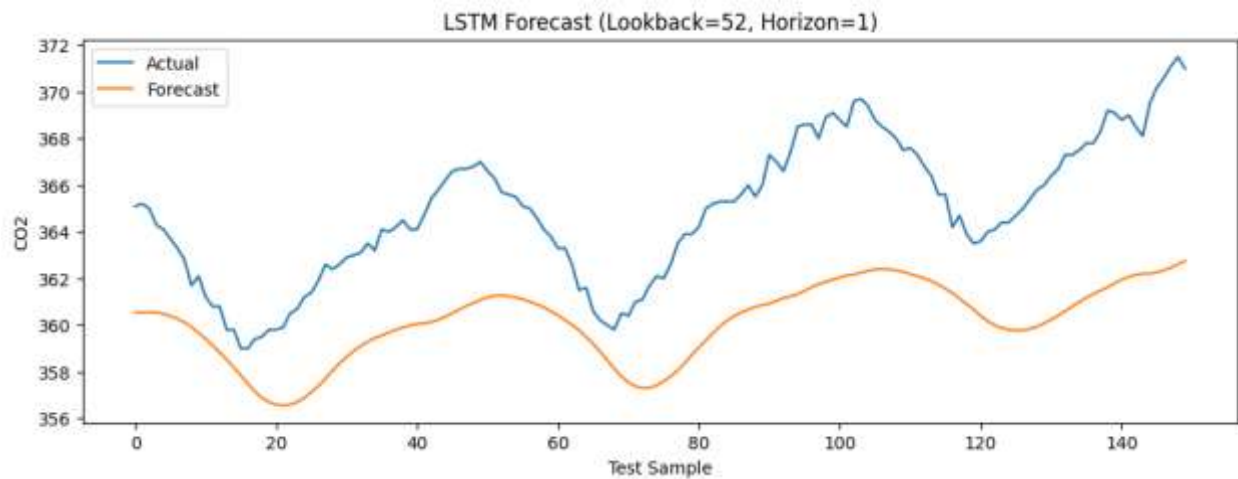
plt.xlabel("Test Sample")
plt.ylabel("CO2")
plt.title(
    f"{best_model_name} Forecast "
    f"(Lookback={best_lookback}, Horizon={best_horizon}) "
)

plt.legend()
plt.show()
```

This cell visualizes the predictions of the best forecasting configuration found in the previous cell. The best model is the LSTM with a lookback of 52 and a forecast horizon of 1. First, the code uses the best model name, lookback, and horizon to find the saved predictions and actual target values. Then, up to 150 samples from the test set are selected

for visualization. The actual CO2 values and the predicted values are plotted on the same graph. This makes it possible to visually compare the behavior of the forecast with the real time-series values.

Output:



The plot shows the actual CO2 values in blue and the LSTM predictions in orange. The forecast follows the general seasonal up-and-down pattern of the real data. This means that the LSTM has learned an important repeating pattern in the CO2 time series.

However, the predicted values are generally lower than the actual values, especially in the later part of the test set. The real CO2 values continue to increase over time, while the predictions increase more slowly. This shows that the model captures the seasonal pattern better than the long-term upward trend.

Even with this difference, this configuration has the lowest RMSE among all tested LSTM and Transformer settings. Therefore, the LSTM with lookback 52 and horizon 1 is selected as the best forecasting configuration in this experiment.

Analysis Questions

1. Under what conditions does Transformer outperform LSTM? When does LSTM prevail?

In our experiment, the LSTM performed better than the Transformer in every tested configuration. This can be explained by the characteristics of the CO2 dataset. The dataset is relatively small, with only 2,284 weekly observations, and it contains only one main variable. LSTM models are often effective for this type of small and structured time-series data because they can learn sequential and seasonal patterns with fewer training requirements. Transformers can become more useful when the dataset is much larger, when the input sequences

are very long, or when there are many variables and complex relationships between distant time steps. The attention mechanism allows a Transformer to directly compare different positions in a sequence, which can be useful for long-range dependencies. However, Transformers usually need more data to learn these relationships well. In this experiment, the LSTM had lower RMSE than the Transformer for all six combinations of lookback and forecast horizon. For example, with lookback 52 and horizon 1, the LSTM achieved an RMSE of 6.197, while the Transformer achieved 10.629. Therefore, the LSTM was more suitable for this dataset and experiment.

2. Does the attention mechanism provide interpretable insights for time-series patterns?

The attention mechanism can provide some useful information about which parts of an input sequence are important for a prediction. For example, attention weights may show that the model gives more importance to some previous time steps than others. In a seasonal time series, this may help us see whether the model focuses on earlier values that have a similar seasonal position. However, attention weights should not always be treated as a complete explanation of the model. A high attention value shows that a time step received more attention, but it does not prove that this time step was the direct cause of the prediction. In our current implementation, the Transformer uses self-attention internally, but the attention weights were not directly extracted or plotted. Therefore, we cannot make a specific interpretation of individual attention weights from this experiment. We can only say that attention gives the Transformer a way to learn relationships between different positions in the input sequence.

3. How do input length and forecasting horizon affect the relative performance of each architecture?

The results show that the forecast horizon has a clear effect on both models. As the forecast horizon becomes larger, the prediction error generally increases. This is expected because predicting values further into the future is more difficult.

For the LSTM, the average RMSE increased from 6.349 for horizon 1 to 7.741 for horizon 4 and 11.969 for horizon 12. The Transformer showed the same general behavior, with average RMSE increasing from 10.315 for horizon 1 to 14.513 for horizon 4 and 16.074 for horizon 12. This shows that both models lose accuracy when they have to predict a longer future sequence. The lookback length had a different effect on the two models. For the LSTM, increasing the lookback from 24 to 52 slightly improved the average RMSE from 8.812 to 8.560. This suggests that the LSTM was able to use the additional past information. For the Transformer, increasing the lookback from 24 to 52 did not improve the result. Its average RMSE increased from 13.398 to 13.870. Overall, the LSTM benefited slightly from a longer input sequence, especially for short and medium forecast horizons. The Transformer did not show the same improvement. The results also clearly show that longer forecasting horizons make the task more difficult for both architectures.

Problem 3: Autoencoders and VAEs – Latent Space Analysis

Problem 3 is about learning compressed representations of data and understanding what happens inside the latent space of different models. The main idea is to take high-dimensional input data, compress it into a smaller representation, and then reconstruct the original data from that compressed form. In this problem, we compare several methods for dimensionality reduction and reconstruction, and then study how different autoencoder and VAE architectures behave.

The first part of the problem uses the CIFAR-10 dataset. In this section, three different methods are compared: Singular Value Decomposition, a Linear Autoencoder, and a Non-linear Autoencoder. The goal is to see how well each method can compress and reconstruct the images. To make the comparison fair, the three methods use the same latent dimension. Their reconstruction quality is measured using Mean Squared Error, so a lower MSE means that the reconstructed images are closer to the original images. We also visualize the learned latent representations using t-SNE to see how the different CIFAR-10 classes are organized in the compressed space.

The second part of the problem focuses on Variational Autoencoders using the Fashion-MNIST dataset. Three VAE architectures are tested: Base, Deeper, and Wider. All of them use a latent dimension of 20, but their encoder and decoder structures are different. The goal is to study how changing the depth or width of the network affects reconstruction quality and the ability of the model to generate new samples. For each architecture, we calculate the reconstruction loss, examine reconstructed images, and generate new images from random latent vectors.

Another important part of the VAE experiment is the structure of the latent space. Because a VAE is designed to learn a more organized and continuous latent representation, we also examine whether it is possible to move smoothly between two points in the latent space. By interpolating between the latent representations of two Fashion-MNIST classes and decoding the intermediate points, we can visually study whether the generated images change gradually from one type of object to another.

The final part of Problem 3 compares a VAE with a standard Autoencoder. We take the best VAE architecture from the previous experiment and create a normal Autoencoder with a similar structure. In the standard Autoencoder, the reparameterization step is removed and the model is trained only with reconstruction loss. We then compare the two models based on their final loss, reconstruction quality, and ability to generate new samples from random points in the latent space.

Overall, this problem helps us understand the difference between simple linear compression, deterministic Autoencoders, and Variational Autoencoders. It also shows that reconstruction quality is not the only important goal. The structure of the latent space and the ability to generate meaningful new samples are also important when evaluating generative models. In the following sections, each part of the implementation and its results are explained step by step.

Explain the code of problem set 3:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
import time
import random
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Subset, random_split
import torchvision
import torchvision.transforms as transforms
from sklearn.manifold import TSNE
```

This cell imports the main libraries that will be used throughout Problem 3. The time library is used to measure training time, while random and copy are used for reproducibility and saving model states during training.

NumPy is used for numerical operations, Pandas is used to organize experiment results into tables, and Matplotlib is used to create plots and visualize images and latent spaces.

PyTorch provides the main tools for building and training the Autoencoder and VAE models. The nn and functional modules are used to define neural network layers and loss functions. DataLoader is used to load data in batches, Subset is used to select a smaller part of a dataset when needed, and random_split is used to divide a dataset into training and validation sets.

Torchvision is used to load the CIFAR-10 and Fashion-MNIST datasets and to convert images into tensors.

Finally, TSNE from scikit-learn is imported for visualizing high-dimensional latent representations in two dimensions. This will later help us compare how SVD, Linear Autoencoder, and Non-linear Autoencoder organize the CIFAR-10 data in their latent spaces.

There is no printed output for this cell because it only imports the libraries needed for the rest of the implementation.

Cell 2:

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def count_parameters(model):
    return sum(p.numel() for p in model.parameters())

print("Device:", device)
```

This cell sets the random seed and prepares the device that will be used for the experiments. The seed is set to 42 for Python, NumPy, and PyTorch. This helps make the random parts of the experiment more consistent between different runs.

The code then checks whether CUDA is available. If a GPU is available, PyTorch uses it. Otherwise, the code uses the CPU.

The `count_parameters` function is also defined in this cell. It calculates the total number of parameters in a model. This will later help us compare the size of different Autoencoder and VAE architectures.

Output:

```
Device: cpu
```

This means that PyTorch did not detect an available CUDA GPU, so the models are currently running on the CPU. The code is still correct and can run normally, but training may take longer than it would on a GPU.

Cell 3:

```
transform_cifar = transforms.ToTensor()

cifar_train = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform_cifar
)

cifar_test = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform_cifar
)

TASK_A_TRAIN_SIZE = 20000
TASK_A_TEST_SIZE = 5000
BATCH_SIZE_A = 256
generator = torch.Generator().manual_seed(SEED)
train_indices = torch.randperm(
    len(cifar_train),
    generator=generator
)[:TASK_A_TRAIN_SIZE]

test_indices = torch.randperm(
    len(cifar_test),
    generator=generator
)[:TASK_A_TEST_SIZE]

cifar_train_subset = Subset(cifar_train, train_indices)
cifar_test_subset = Subset(cifar_test, test_indices)
```

```

train_loader_a = DataLoader(
    cifar_train_subset,
    batch_size=BATCH_SIZE_A,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

test_loader_a = DataLoader(
    cifar_test_subset,
    batch_size=BATCH_SIZE_A,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

print("Task A train samples:", len(cifar_train_subset))
print("Task A test samples:", len(cifar_test_subset))

```

This cell prepares the CIFAR-10 data for Task A. First, the images are converted into PyTorch tensors using ToTensor. The CIFAR-10 training and test datasets are then downloaded and loaded.

To keep the experiment faster and more manageable, the code uses a fixed subset of the full dataset. It selects 20,000 samples from the training set and 5,000 samples from the test set. The same random seed is used when selecting these samples, so the subset can be reproduced in another run.

The selected samples are wrapped with Subset and then passed to DataLoader objects. The batch size is set to 256. The training loader uses shuffling because the Autoencoder models will be trained on these samples, while the test loader keeps a fixed order for evaluation.

Output:

```

100%|██████████| 170M/170M [04:43<00:00, 602kB/s]
Task A train samples: 20000
Task A test samples: 5000

```

The first line shows that the CIFAR-10 dataset was downloaded successfully. And the final confirms that Task A uses 20,000 training images and 5,000 test images. These same subsets will be used for the SVD, Linear Autoencoder, and Non-linear Autoencoder comparison so that the reconstruction results are based on the same data.

Cell 4:

```
def dataset_to_matrix(loader):
    images = []
    labels = []

    for x, y in loader:
        images.append(x.view(x.size(0), -1))
        labels.append(y)

    images = torch.cat(images)
    labels = torch.cat(labels)

    return images, labels

svd_train_loader = DataLoader(
    cifar_train_subset,
    batch_size=512,
    shuffle=False
)

svd_test_loader = DataLoader(
    cifar_test_subset,
    batch_size=512,
    shuffle=False
)

train_matrix, train_labels_a = dataset_to_matrix(svd_train_loader)
test_matrix, test_labels_a = dataset_to_matrix(svd_test_loader)

print("Train matrix:", train_matrix.shape)
print("Test matrix:", test_matrix.shape)
```

This cell converts the CIFAR-10 images into matrix form so they can be used with SVD. Each CIFAR-10 image has a size of 32 by 32 pixels with 3 color channels. The `dataset_to_matrix` function flattens every image into a one-dimensional vector.

Since each image contains $32 \times 32 \times 3$ values, every image becomes a vector with 3072 values. The function also keeps the class labels so they can be used later when visualizing the latent representations.

Separate DataLoaders are created for the training and test subsets with a batch size of 512. Shuffling is disabled because the goal here is only to convert the complete subsets into matrices.

The flattened image batches are combined using `torch.cat` to create one large training matrix and one large test matrix.

Output:

```
Train matrix: torch.Size([20000, 3072])
Test matrix: torch.Size([5000, 3072])
```

This means that the training matrix contains 20,000 images, with 3072 features for each image. The test matrix contains 5,000 images with the same 3072 features. These matrices are now ready to be used for the SVD dimensionality reduction and reconstruction experiment.

Cell 5:

```
LATENT_A = 64

train_matrix_gpu = train_matrix.to(device)
test_matrix_gpu = test_matrix.to(device)

svd_mean = train_matrix_gpu.mean(dim=0, keepdim=True)

train_centered = train_matrix_gpu - svd_mean
test_centered = test_matrix_gpu - svd_mean

_, _, svd_v = torch.svd_lowrank(
    train_centered,
    q=LATENT_A,
    niter=2
```

```

)

svd_latent = test_centered @ svd_v
svd_reconstruction = svd_latent @ svd_v.T + svd_mean

svd_mse = F.mse_loss(
    svd_reconstruction,
    test_matrix_gpu
).item()

svd_latent_cpu = svd_latent.cpu().numpy()

print("SVD latent shape:", svd_latent.shape)
print(f"SVD test MSE: {svd_mse:.6f}")

```

This cell applies Singular Value Decomposition to the CIFAR-10 data and creates a lower-dimensional representation of the images. The latent dimension is set to 64, which means that each original image with 3072 values is compressed into only 64 values.

First, the training and test matrices are moved to the selected device. Then the mean of the training data is calculated and subtracted from both the training and test matrices. Centering the data before SVD helps the method focus on the main variations in the images instead of the overall average pixel values.

The `torch.svd_lowrank` function is then used to find a low-rank representation of the training data. Only 64 dimensions are kept. The test images are projected into this 64-dimensional latent space using the learned SVD directions.

After creating the latent representation, the code reconstructs the test images back into the original 3072-dimensional space. The reconstruction quality is measured using Mean Squared Error. A lower MSE means that the reconstructed images are closer to the original images.

The latent representation is also moved back to the CPU and converted to a NumPy array because it will later be used for t-SNE visualization.

Output:

```

SVD latent shape: torch.Size([5000, 64])
SVD test MSE: 0.008719

```


The latent shape shows that all 5,000 test images were successfully compressed from 3072 dimensions to 64 dimensions.

The test MSE is 0.008719. This gives us the reconstruction error of SVD and will later be compared with the Linear Autoencoder and Non-linear Autoencoder to see which method gives the best reconstruction quality.

Cell 6:

```
class LinearAutoencoder(nn.Module):
    def __init__(self, latent_dim=64):
        super().__init__()

        self.encoder = nn.Linear(3072, latent_dim)
        self.decoder = nn.Linear(latent_dim, 3072)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        z = self.encoder(x)
        reconstruction = self.decoder(z)

        return reconstruction

    def encode(self, x):
        x = x.view(x.size(0), -1)

        return self.encoder(x)
```

This cell defines the Linear Autoencoder used in Task A. The model has two main parts: an encoder and a decoder. The encoder receives the flattened CIFAR-10 image with 3072 input values and compresses it into a 64-dimensional latent representation. Because the encoder uses only a linear layer and no activation function, this model is a purely linear Autoencoder. The decoder performs the opposite operation. It takes the 64-dimensional latent vector and maps it back to 3072 values, which represent the reconstructed image.

In the forward function, each image is first flattened into a one-dimensional vector. The encoder then creates the latent representation z , and the decoder uses this representation to reconstruct the original image. The encode function is also defined separately because later we need direct access to the 64-dimensional latent representations for t-SNE visualization.

There is no printed output for this cell because it only defines the Linear Autoencoder architecture.

Cell 7:

```
class NonlinearAutoencoder(nn.Module):
    def __init__(self, latent_dim=64):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Linear(3072, 512),
            nn.ReLU(),
            nn.Linear(512, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 512),
            nn.ReLU(),
            nn.Linear(512, 3072),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        z = self.encoder(x)

        return self.decoder(z)

    def encode(self, x):
        x = x.view(x.size(0), -1)

        return self.encoder(x)
```

This cell defines the Non-linear Autoencoder used in Task A. Unlike the Linear Autoencoder from the previous cell, this model uses several hidden layers and ReLU activation functions. These non-linear layers allow the model to learn more complex relationships in the CIFAR-10 images.

The encoder gradually reduces the input size from 3072 values to 512, then to 128, and finally to the 64-dimensional latent space. This creates a compressed representation of each image.

The decoder performs the reverse process. It takes the 64-dimensional latent representation and expands it back to 128, then 512, and finally 3072 values. ReLU is used between the hidden layers, and a Sigmoid activation is used at the final output because the CIFAR-10 pixel values are between 0 and 1 after using ToTensor. In the forward function, the input image is first flattened. The encoder then produces the latent representation, and the decoder reconstructs the original image from this compressed representation.

The encode function is also provided separately so that the 64-dimensional latent vectors can later be extracted and visualized with t-SNE.

There is no printed output for this cell because it only defines the Non-linear Autoencoder architecture.

Cell 8:

```
def evaluate_autoencoder(model, loader):
    model.eval()

    total_squared_error = 0
    total_values = 0

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            target = x.view(x.size(0), -1)

            reconstruction = model(x)

            total_squared_error += F.mse_loss(
                reconstruction,
                target,
                reduction="sum"
            ).item()
```

```

        total_values += target.numel()

    return total_squared_error / total_values

def train_autoencoder(model, train_loader, test_loader, epochs=12):
    model = model.to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.001
    )

    history = []

    for epoch in range(epochs):
        model.train()

        total_loss = 0
        total_values = 0

        for x, _ in train_loader:
            x = x.to(device)
            target = x.view(x.size(0), -1)

            optimizer.zero_grad()

            reconstruction = model(x)

            loss = F.mse_loss(
                reconstruction,
                target
            )

            loss.backward()
            optimizer.step()

            total_loss += F.mse_loss(

```

```

        reconstruction.detach(),
        target,
        reduction="sum"
    ).item()

    total_values += target.numel()

train_mse = total_loss / total_values
test_mse = evaluate_autoencoder(model, test_loader)

history.append((train_mse, test_mse))

print(
    f"Epoch {epoch + 1:02d}/{epochs} | "
    f"train MSE {train_mse:.6f} | "
    f"test MSE {test_mse:.6f}"
)

return model, history

```

This cell defines the functions used to train and evaluate the Linear and Non-linear Autoencoders.

The `evaluate_autoencoder` function measures how well a trained Autoencoder can reconstruct the test images. The model is first switched to evaluation mode and gradient calculation is disabled because the model is not being trained in this step. For each batch, the images are moved to the selected device and flattened into vectors with 3072 values. The model reconstructs these vectors, and the squared reconstruction error is calculated.

The total squared error is divided by the total number of pixel values in the dataset. This gives the average Mean Squared Error for the whole test set. A lower value means that the reconstructed images are closer to the original images.

The `train_autoencoder` function is used to train either the Linear Autoencoder or the Non-linear Autoencoder. The Adam optimizer is used with a learning rate of 0.001. During each epoch, the model processes the training batches, creates reconstructions, calculates the MSE loss, performs backpropagation, and updates its parameters.

After each epoch, the code calculates both the training MSE and the test MSE. These values are saved in the history list and printed so that we can see how the reconstruction quality changes during training.

The function trains the model for 12 epochs and finally returns the trained model together with its training history.

There is no output from this cell yet because it only defines the training and evaluation functions. The actual Autoencoder training is performed in the next cell.

Cell 9:

```
set_seed()

linear_ae = LinearAutoencoder(
    latent_dim=LATENT_A
).to(device)

linear_ae, linear_history = train_autoencoder(
    linear_ae,
    train_loader_a,
    test_loader_a,
    epochs=12
)

set_seed()

nonlinear_ae = NonlinearAutoencoder(
    latent_dim=LATENT_A
).to(device)

nonlinear_ae, nonlinear_history = train_autoencoder(
    nonlinear_ae,
    train_loader_a,
    test_loader_a,
    epochs=12
)
```

This cell trains both Autoencoder models from Task A. First, the random seed is reset so that the experiment starts from a controlled random state. The Linear Autoencoder is created with a latent dimension of 64 and then trained for 12 epochs using the training and test DataLoaders defined earlier.

After the Linear Autoencoder finishes training, the seed is reset again and the Non-linear Autoencoder is created with the same latent dimension of 64. It is also trained for 12 epochs using the same data. Using the same latent dimension and the same training data makes the comparison between the two models more fair.

During training, the code prints the training MSE and test MSE after every epoch. These values show how well each model reconstructs the CIFAR-10 images and how the reconstruction quality changes during training.

Output:

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:1118:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
    super().__init__(loader)
Epoch 01/12 | train MSE 0.145323 | test MSE 0.047369
Epoch 02/12 | train MSE 0.045910 | test MSE 0.045079
Epoch 03/12 | train MSE 0.042570 | test MSE 0.040244
Epoch 04/12 | train MSE 0.037296 | test MSE 0.035347
Epoch 05/12 | train MSE 0.033360 | test MSE 0.032255
Epoch 06/12 | train MSE 0.030756 | test MSE 0.029880
Epoch 07/12 | train MSE 0.028670 | test MSE 0.027678
Epoch 08/12 | train MSE 0.026567 | test MSE 0.025690
Epoch 09/12 | train MSE 0.024527 | test MSE 0.023965
Epoch 10/12 | train MSE 0.022958 | test MSE 0.022980
Epoch 11/12 | train MSE 0.022112 | test MSE 0.021665
Epoch 12/12 | train MSE 0.020908 | test MSE 0.021000
Epoch 01/12 | train MSE 0.044295 | test MSE 0.033282
Epoch 02/12 | train MSE 0.030468 | test MSE 0.027348
Epoch 03/12 | train MSE 0.025471 | test MSE 0.024915
Epoch 04/12 | train MSE 0.023724 | test MSE 0.023737
Epoch 05/12 | train MSE 0.022714 | test MSE 0.023398
Epoch 06/12 | train MSE 0.021525 | test MSE 0.021626
Epoch 07/12 | train MSE 0.020587 | test MSE 0.020628
Epoch 08/12 | train MSE 0.019982 | test MSE 0.019751
Epoch 09/12 | train MSE 0.019240 | test MSE 0.021098
Epoch 10/12 | train MSE 0.019049 | test MSE 0.018938
Epoch 11/12 | train MSE 0.018310 | test MSE 0.019434
Epoch 12/12 | train MSE 0.018092 | test MSE 0.018087
```

The warning about `pin_memory` appears because the code is running on the CPU and no CUDA accelerator is available. This warning does not affect the correctness of the experiment. It only means that the `pin_memory` option does not provide any speed benefit in this run.

For the Linear Autoencoder, the training MSE decreases from 0.145323 in the first epoch to 0.020908 in the final epoch. The test MSE also improves from 0.047369 to 0.021000. This shows that the Linear Autoencoder successfully learns a compressed representation of the CIFAR-10 images and improves its reconstruction quality during training.

The Non-linear Autoencoder performs better. Its training MSE decreases from 0.044295 to 0.018092, while the final test MSE reaches 0.018087. The test error generally decreases across the epochs, although there are small changes in some epochs.

Overall, both Autoencoders learn to reconstruct the images, but the Non-linear Autoencoder achieves a lower final test MSE than the Linear Autoencoder. The final test MSE values are approximately 0.021000 for the Linear Autoencoder and 0.018087 for the Non-linear Autoencoder. This suggests that the additional hidden layers and non-linear activation functions help the model learn a better representation of the CIFAR-10 images.

Cell 10:

```
linear_mse = evaluate_autoencoder(
    linear_ae,
    test_loader_a
)

nonlinear_mse = evaluate_autoencoder(
    nonlinear_ae,
    test_loader_a
)

task_a_results = pd.DataFrame({
    "Method": [
        "SVD",
        "Linear Autoencoder",
        "Non-linear Autoencoder"
    ],
    "Latent Dimension": [
```



```

        LATENT_A,
        LATENT_A,
        LATENT_A
    ],
    "Test MSE": [
        svd_mse,
        linear_mse,
        nonlinear_mse
    ]
})

task_a_results

```

This cell evaluates the final reconstruction performance of the three methods used in Task A: SVD, Linear Autoencoder, and Non-linear Autoencoder.

First, the trained Linear Autoencoder and Non-linear Autoencoder are evaluated again on the test dataset using the `evaluate_autoencoder` function. Their final test MSE values are then combined with the SVD test MSE calculated earlier.

The results are stored in a Pandas DataFrame so that the three methods can be compared clearly. All three methods use the same latent dimension of 64, which makes the comparison more consistent.

Output:

	Method	Latent Dimension	Test MSE
0	SVD	64	0.008719
1	Linear Autoencoder	64	0.021000
2	Non-linear Autoencoder	64	0.018087

The results show that SVD achieves the lowest reconstruction error with a test MSE of 0.008719. Therefore, among these three methods, SVD gives the best reconstruction quality in this experiment.

The Non-linear Autoencoder performs better than the Linear Autoencoder, with a test MSE of 0.018087 compared with 0.021000. This shows that adding hidden layers and non-linear activation functions improves the Autoencoder reconstruction.

However, even though the Non-linear Autoencoder is more flexible, it does not outperform SVD in terms of reconstruction MSE in this experiment. This can happen because SVD directly finds a strong linear low-rank representation of the data, while the Autoencoders must learn their representations through iterative training.

Cell 11:

```
def collect_latent(model, loader):
    model.eval()

    latent_values = []
    labels = []

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)

            z = model.encode(x)

            latent_values.append(z.cpu())
            labels.append(y)

    latent_values = torch.cat(latent_values).numpy()
    labels = torch.cat(labels).numpy()
    return latent_values, labels

linear_latent, latent_labels = collect_latent(
    linear_ae,
    test_loader_a
)
nonlinear_latent, _ = collect_latent(
    nonlinear_ae,
    test_loader_a
)

print("SVD:", svd_latent_cpu.shape)
```

```
print("Linear AE:", linear_latent.shape)
print("Non-linear AE:", nonlinear_latent.shape)
```

This cell collects the latent representations produced by the trained Autoencoder models. The purpose is to prepare the compressed features for the t-SNE visualization in the next step.

The `collect_latent` function first switches the model to evaluation mode and disables gradient calculation because no training is needed. For each batch of test images, the `encode` function of the model is used to obtain the latent vector `z`.

These latent vectors are moved back to the CPU and stored together with their class labels. After processing all batches, the latent vectors and labels are combined and converted to NumPy arrays.

The function is used for both the Linear Autoencoder and the Non-linear Autoencoder. The SVD latent representation was already calculated earlier, so the code prints the shapes of all three representations for comparison.

Output:

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:1118:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
  super().__init__(loader)
SVD: (5000, 64)
Linear AE: (5000, 64)
Non-linear AE: (5000, 64)
```

This confirms that all three methods produced a 64-dimensional latent representation for each of the 5,000 CIFAR-10 test images. Therefore, the three representations have the same size and can be compared fairly using t-SNE.

The warning about `pin_memory` appears again because the code is running on the CPU. It does not affect the latent representations or the correctness of the results.

Cell 12:

```
TSNE_SAMPLES = 2000
```

```

representations = {
    "SVD": svd_latent_cpu[:TSNE_SAMPLES],
    "Linear Autoencoder": linear_latent[:TSNE_SAMPLES],
    "Non-linear Autoencoder": nonlinear_latent[:TSNE_SAMPLES]
}
tsne_labels = latent_labels[:TSNE_SAMPLES]
tsne_results = {}
for name, representation in representations.items():
    print("Running t-SNE for", name)
    tsne = TSNE(
        n_components=2,
        perplexity=30,
        learning_rate="auto",
        init="pca",
        max_iter=1000,
        random_state=SEED
    )
    tsne_results[name] = tsne.fit_transform(
        representation
    )

```

This cell prepares the latent representations for visualization using t-SNE. Since applying t-SNE to all 5,000 test samples can take more time, the code uses the first 2,000 samples from each method.

The latent representations from SVD, Linear Autoencoder, and Non-linear Autoencoder are stored in a dictionary. The corresponding class labels for the same 2,000 CIFAR-10 samples are also selected.

For each method, t-SNE reduces the 64-dimensional latent representation into two dimensions. The number of output dimensions is set to 2 so that the latent space can be plotted on a normal 2D graph.

The perplexity is set to 30, which controls how t-SNE considers local neighborhoods in the data. The learning rate is selected automatically, and PCA initialization is used as the starting point. The algorithm runs for up to 1000 iterations, and the random seed is fixed to make the result more reproducible.

The two-dimensional t-SNE results are saved in the `tsne_results` dictionary and will be plotted in the next cell.

Output:

```
Running t-SNE for SVD
Running t-SNE for Linear Autoencoder
Running t-SNE for Non-linear Autoencoder
```

This output confirms that t-SNE was successfully applied to all three latent representations. No numerical result is printed in this cell because the main output is the transformed two-dimensional data, which will be visualized in the next step.

Cell 13:

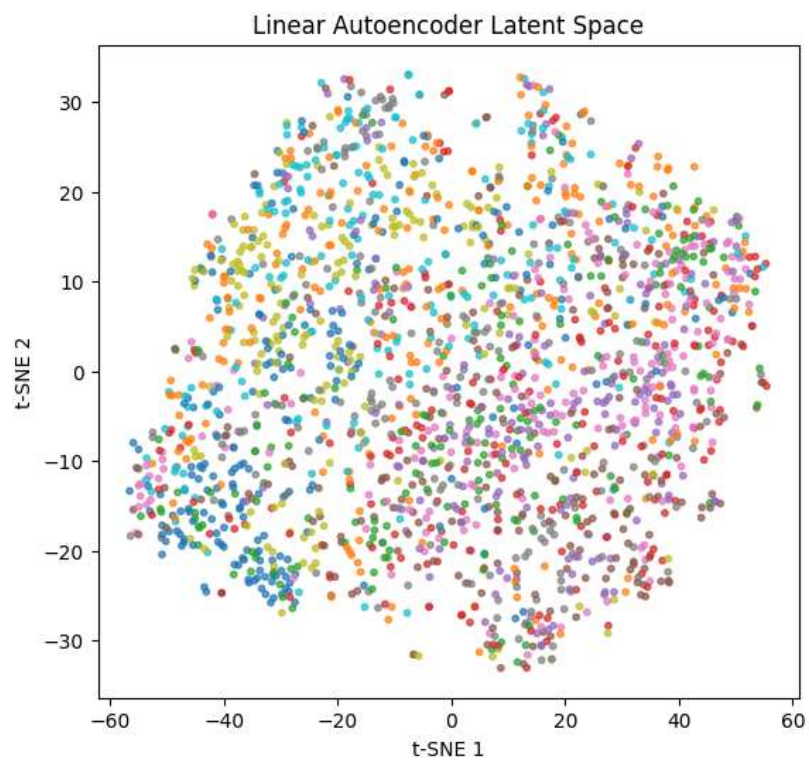
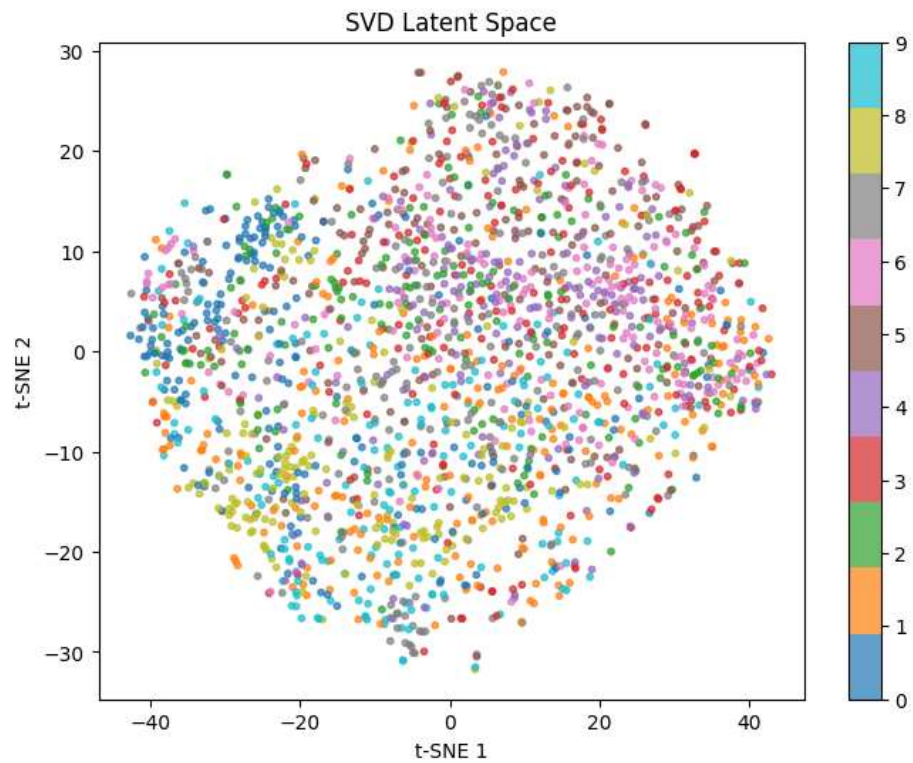
```
for name, points in tsne_results.items():
    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(
        points[:, 0],
        points[:, 1],
        c=tsne_labels,
        s=10,
        alpha=0.7,
        cmap="tab10"
    )
    plt.colorbar(scatter)
    plt.title(f"{name} Latent Space")
    plt.xlabel("t-SNE 1")
    plt.ylabel("t-SNE 2")
    plt.show()
```

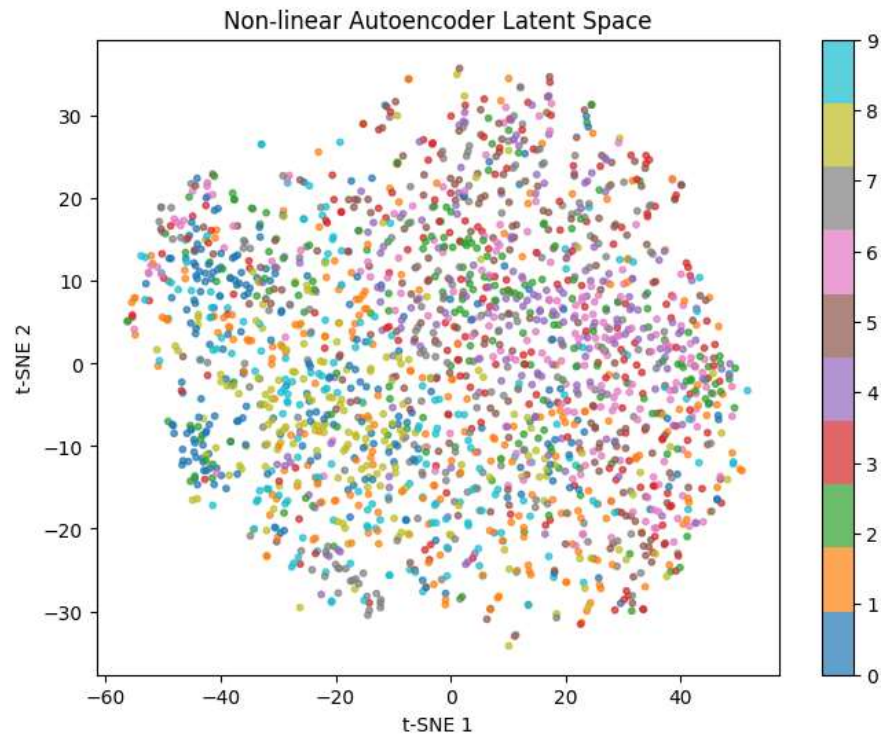
This cell visualizes the t-SNE results for the three latent representations. For each method, the code creates a separate scatter plot using the two-dimensional points produced by t-SNE in the previous cell.

Each point in the plot represents one CIFAR-10 test image. The x-axis and y-axis are the two t-SNE dimensions. The color of each point shows the class label, using the CIFAR-10 labels from 0 to 9. A colorbar is added so that the class values can be read more easily.

The point size is kept small and a little transparency is used so that the distribution of the latent space can be seen more clearly. The title of each plot shows which method produced that latent representation.

Output:





This cell produces three plots, one for SVD, one for the Linear Autoencoder, and one for the Non-linear Autoencoder. These plots help us visually compare how the three methods organize the CIFAR-10 images in latent space.

From the plots, we can see that all three methods create some local groupings, but the classes are still mixed in many areas. This is normal for CIFAR-10 because it is a more complex dataset with natural color images, and many classes share visual similarities.

The SVD latent space shows some broad structure, but many class points are mixed together. The Linear Autoencoder also shows a spread latent space, with some small local groups but still a large amount of overlap between classes. The Non-linear Autoencoder appears to create slightly more meaningful local structure in some regions, but the separation is still not very clean.

So, the general conclusion from these plots is that none of the three methods produces perfectly separated clusters for CIFAR-10, but they do capture some structure in the data. The t-SNE plots are useful here because they give a qualitative view of how the compressed representations differ, while the MSE values from the previous cells give the quantitative comparison of reconstruction quality.

Cell 14:

```
transform_fashion = transforms.ToTensor()
```

```

fashion_full_train = torchvision.datasets.FashionMNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform_fashion
)

fashion_test = torchvision.datasets.FashionMNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform_fashion
)

train_size = 50000
val_size = 10000

fashion_train, fashion_val = random_split(
    fashion_full_train,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(SEED)
)

BATCH_SIZE_B = 256

fashion_train_loader = DataLoader(
    fashion_train,
    batch_size=BATCH_SIZE_B,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

fashion_val_loader = DataLoader(
    fashion_val,
    batch_size=BATCH_SIZE_B,
    shuffle=False,
    num_workers=2,

```



```

        pin_memory=True
    )

fashion_test_loader = DataLoader(
    fashion_test,
    batch_size=BATCH_SIZE_B,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

print("Train:", len(fashion_train))
print("Validation:", len(fashion_val))
print("Test:", len(fashion_test))

```

This cell prepares the Fashion-MNIST dataset for Task B, where the Variational Autoencoder architectures will be trained and compared.

First, ToTensor is used to convert the Fashion-MNIST images into PyTorch tensors. The training and test datasets are then downloaded and loaded.

The original Fashion-MNIST training set contains 60,000 images. In this experiment, it is divided into two parts: 50,000 images for training and 10,000 images for validation. The validation set is used to check the model performance during training and to select the best model state. A fixed random seed is used during the split so that the same training and validation samples can be reproduced.

The original Fashion-MNIST test set contains 10,000 images and is kept separate for the final evaluation.

The batch size is set to 256. The training DataLoader uses shuffling so that the model sees the training samples in a different order during each epoch. The validation and test DataLoaders do not use shuffling because they are only used for evaluation.

Output:

```

100%|██████████| 26.4M/26.4M [00:01<00:00, 18.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 270kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.03MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 11.1MB/s]

```

```
Train: 50000
Validation: 10000
Test: 10000
```

The first lines show that the Fashion-MNIST files were downloaded successfully. The final output confirms that 50,000 images will be used for training, 10,000 for validation, and 10,000 for final testing. These datasets will be used to train and compare the Base, Deeper, and Wider VAE architectures.

Cell 15:

```
LATENT_B = 20

vae_configs = {
    "Base": {
        "encoder": [400, 100],
        "decoder": [100, 400]
    },
    "Deeper": {
        "encoder": [400, 100, 50],
        "decoder": [50, 100, 400]
    },
    "Wider": {
        "encoder": [600, 200],
        "decoder": [200, 600]
    }
}

class VAE(nn.Module):
    def __init__(
        self,
        encoder_dims,
        decoder_dims,
        latent_dim=20
    ):
        super().__init__()
```

```

encoder_layers = []
previous = 784

for size in encoder_dims:
    encoder_layers.append(
        nn.Linear(previous, size)
    )
    encoder_layers.append(nn.ReLU())
    previous = size

self.encoder_net = nn.Sequential(
    *encoder_layers
)

self.fc_mu = nn.Linear(
    previous,
    latent_dim
)

self.fc_logvar = nn.Linear(
    previous,
    latent_dim
)

decoder_layers = []
previous = latent_dim

for size in decoder_dims:
    decoder_layers.append(
        nn.Linear(previous, size)
    )
    decoder_layers.append(nn.ReLU())
    previous = size

decoder_layers.append(
    nn.Linear(previous, 784)
)

decoder_layers.append(nn.Sigmoid())

```

```

        self.decoder = nn.Sequential(
            *decoder_layers
        )

    def encode(self, x):
        x = x.view(x.size(0), -1)

        h = self.encoder_net(x)

        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)

        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)

        return mu + eps * std

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)

        z = self.reparameterize(
            mu,
            logvar
        )

        reconstruction = self.decode(z)

        return reconstruction, mu, logvar

```

This cell defines the Variational Autoencoder architectures that will be used in Task B. The latent dimension is set to 20 for all models so that the three architectures can be compared under the same latent space size.

Three configurations are defined: Base, Deeper, and Wider.

The Base architecture uses encoder hidden layers of 400 and 100 units, followed by the 20-dimensional latent representation. Its decoder uses the reverse structure, going from 20 to 100, then 400, and finally back to 784 output values.

The Deeper architecture adds another hidden layer. Its encoder uses 400, 100, and 50 hidden units before reaching the 20-dimensional latent space. The decoder follows the reverse structure using 50, 100, and 400 units.

The Wider architecture uses larger hidden layers. Its encoder uses 600 and 200 hidden units before the 20-dimensional latent representation, while the decoder uses 200 and 600 hidden units.

The VAE class is then defined so that all three architectures can be created using the same general model structure.

In the encoder, the input Fashion-MNIST image is first flattened from 28×28 pixels into a vector of 784 values. The encoder hidden layers process this vector using Linear layers followed by ReLU activation functions.

Unlike a standard Autoencoder, the VAE does not directly produce one latent vector. Instead, it produces two vectors: μ and $\log\text{var}$. The μ vector represents the mean of the latent distribution, while $\log\text{var}$ represents the logarithm of its variance.

The reparameterize function implements the reparameterization trick. It first converts $\log\text{var}$ into a standard deviation and generates a random noise vector. The latent vector z is then calculated from the mean, standard deviation, and random noise. This allows the model to sample from the latent distribution while still allowing gradient-based training.

The decoder takes the sampled latent vector and transforms it back through the decoder hidden layers. The final layer contains 784 outputs, corresponding to the 784 pixels of a Fashion-MNIST image. A Sigmoid activation is used at the output because the image pixel values are between 0 and 1.

Finally, the forward function connects the complete VAE process. It encodes the input into μ and $\log\text{var}$, samples the latent vector using the reparameterization trick, and then decodes that latent vector to create the reconstructed image.

There is no printed output for this cell because it only defines the three VAE configurations and the general VAE model architecture.

Cell 16:

```

def vae_loss(reconstruction, target, mu, logvar):
    reconstruction_loss = F.mse_loss(
        reconstruction,
        target,
        reduction="sum"
    )

    kl_loss = -0.5 * torch.sum(
        1
        + logvar
        - mu.pow(2)
        - logvar.exp()
    )

    total_loss = reconstruction_loss + kl_loss

    return (
        total_loss,
        reconstruction_loss,
        kl_loss
    )

def evaluate_vae(model, loader):
    model.eval()

    total_loss = 0
    total_reconstruction = 0
    total_kl = 0
    total_samples = 0

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            target = x.view(x.size(0), -1)

            reconstruction, mu, logvar = model(x)

```

```

        loss, recon_loss, kl_loss = vae_loss(
            reconstruction,
            target,
            mu,
            logvar
        )

        total_loss += loss.item()
        total_reconstruction += recon_loss.item()
        total_kl += kl_loss.item()
        total_samples += x.size(0)

    return (
        total_loss / total_samples,
        total_reconstruction / total_samples,
        total_kl / total_samples
    )

```

This cell defines the loss function and the evaluation function for the Variational Autoencoder.

The `vae_loss` function calculates two different parts of the VAE loss. The first part is the reconstruction loss, which measures how different the reconstructed image is from the original image. Mean Squared Error is used for this part, and the errors are summed over all pixels in the batch.

The second part is the KL divergence loss. This term measures how different the learned latent distribution is from a standard normal distribution. In other words, it encourages the latent space to have a more regular and organized structure. This is one of the main differences between a VAE and a standard Autoencoder.

The total VAE loss is calculated as the sum of the reconstruction loss and the KL loss:

Total Loss = Reconstruction Loss + KL Loss

The function returns all three values separately so that we can later examine both reconstruction quality and latent-space regularization.

The `evaluate_vae` function is used to evaluate a trained VAE on a validation or test dataset. The model is switched to evaluation mode, and gradient calculation is disabled because no parameter updates are needed.

For each batch, the input images are flattened into vectors of 784 values. The VAE produces the reconstructed images together with mu and logvar. The previously defined `vae_loss` function is then used to calculate the total loss, reconstruction loss, and KL loss.

The losses are accumulated over the complete dataset and finally divided by the total number of samples. Therefore, the function returns the average total loss, average reconstruction loss, and average KL loss per image.

There is no printed output for this cell because it only defines the VAE loss and evaluation functions. These functions will be used during the training and comparison of the three VAE architectures.

Cell 17:

```
def train_vae(model, train_loader, val_loader, epochs=15):
    model = model.to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.001
    )

    best_val_loss = float("inf")
    best_state = None

    history = []

    for epoch in range(epochs):
        model.train()

        total_loss = 0
        total_samples = 0

        for x, _ in train_loader:
            x = x.to(device)
            target = x.view(x.size(0), -1)

            optimizer.zero_grad()

            reconstruction, mu, logvar = model(x)
```



```

        loss, _, _ = vae_loss(
            reconstruction,
            target,
            mu,
            logvar
        )

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        total_samples += x.size(0)

    train_loss = total_loss / total_samples

    val_loss, val_recon, val_kl = evaluate_vae(
        model,
        val_loader
    )

    history.append({
        "train_loss": train_loss,
        "val_loss": val_loss,
        "val_reconstruction": val_recon,
        "val_kl": val_kl
    })

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = copy.deepcopy(
            model.state_dict()
        )

    print(
        f"Epoch {epoch + 1:02d}/{epochs} | "
        f"train {train_loss:.4f} | "
        f"val {val_loss:.4f} | "

```

```

        f"recon {val_recon:.4f} | "
        f"KL {val_kl:.4f}"
    )

    model.load_state_dict(best_state)

    return model, history

```

This cell defines the training function for the Variational Autoencoder models.

The `train_vae` function receives a VAE model, the training `DataLoader`, the validation `DataLoader`, and the number of epochs. The default number of training epochs is set to 15.

First, the model is moved to the selected device. The Adam optimizer is used with a learning rate of 0.001.

The variables `best_val_loss` and `best_state` are used to keep the best version of the model during training. The history list stores the training and validation results from each epoch.

For every epoch, the model is placed in training mode. Each training batch is moved to the device and flattened into vectors of 784 values. The optimizer gradients are cleared, and the VAE produces the reconstructed image together with `mu` and `logvar`.

The `vae_loss` function is then used to calculate the total VAE loss. This total loss includes both the reconstruction loss and the KL divergence loss. Backpropagation is performed, and the optimizer updates the model parameters.

After all training batches are processed, the average training loss per sample is calculated.

The model is then evaluated on the validation set using the `evaluate_vae` function. This gives the validation total loss, reconstruction loss, and KL loss. These values are stored in the history list.

The code also checks whether the current validation loss is better than the best validation loss seen so far. If it is, a copy of the current model parameters is saved. This means that the final model is not simply taken from the last epoch, but from the epoch that produced the lowest validation loss.

After every epoch, the code prints the training loss, validation loss, reconstruction loss, and KL loss. These values allow us to follow the learning process and see how the two parts of the VAE objective change during training.

At the end of training, the best saved model state is loaded back into the model. The function then returns the best trained model and its complete training history.

There is no output in this cell because it only defines the VAE training function. The actual training of the Base, Deeper, and Wider VAE models is performed in the next cell.

Cell 18:

```
VAE_EPOCHS = 15

vae_models = {}
vae_histories = {}
vae_results = []

for name, config in vae_configs.items():
    print(f"\nTraining {name} VAE")

    set_seed()

    model = VAE(
        encoder_dims=config["encoder"],
        decoder_dims=config["decoder"],
        latent_dim=LATENT_B
    )

    start = time.perf_counter()

    model, history = train_vae(
        model,
        fashion_train_loader,
        fashion_val_loader,
        epochs=VAE_EPOCHS
    )

    elapsed = time.perf_counter() - start

    test_total, test_recon, test_kl = evaluate_vae(
        model,
        fashion_test_loader
    )
```

```

vae_models[name] = model
vae_histories[name] = history

vae_results.append({
    "Architecture": name,
    "Test Total Loss": test_total,
    "Test Reconstruction Loss": test_recon,
    "Test KL Loss": test_kl,
    "Parameters": count_parameters(model),
    "Training Time (s)": elapsed
})

print(
    f"{name}: "
    f"total={test_total:.4f}, "
    f"recon={test_recon:.4f}, "
    f"KL={test_kl:.4f}"
)

```

This cell trains and evaluates all three VAE architectures: Base, Deeper, and Wider.

First, the number of training epochs is set to 15. Three containers are also created to store the trained models, their training histories, and their final results.

The code then goes through each architecture defined earlier. Before training each model, the random seed is reset to make the experiment more consistent. A VAE is created using the encoder and decoder dimensions of the current architecture.

The training time is measured using `time.perf_counter`. Each VAE is trained using the `train_vae` function on the Fashion-MNIST training set, while the validation set is used to select the best model state.

After training, the model is evaluated on the test set. Three values are calculated: total loss, reconstruction loss, and KL loss. The number of model parameters and the total training time are also stored. Finally, all results are saved so that the three architectures can be compared later.

The warning about `pin_memory` appears because the experiment is running on the CPU. It does not affect the correctness of the training.

For the Base VAE, the training loss decreases from 52.4324 in the first epoch to 24.3798 in the final epoch. The validation loss also decreases from 38.9862 to 24.6107. The final test results are:

Base: total = 24.5898, recon = 17.4422, KL = 7.1477

This shows that the Base architecture learns successfully and produces a relatively low reconstruction loss.

For the Deeper VAE, the training loss decreases from 57.7895 to 25.1758. The final test results are:

Deeper: total = 25.3807, recon = 19.2622, KL = 6.1185

The Deeper model has a slightly lower KL loss than the Base model, but its reconstruction loss and total loss are higher. This means that adding more hidden layers did not improve reconstruction quality in this experiment.

For the Wider VAE, the training loss decreases from 47.9905 to 24.0386. Its final test results are:

Wider: total = 24.2627, recon = 16.6335, KL = 7.6293

The Wider architecture achieves the lowest total test loss and the lowest reconstruction loss among the three models. Its reconstruction loss of 16.6335 is better than 17.4422 for the Base model and 19.2622 for the Deeper model.

At this stage, the Wider VAE gives the best numerical reconstruction performance. The Deeper model has the lowest KL loss, but its reconstruction quality is weaker. The final choice of the best architecture should also consider the reconstructed images, generated samples, and latent-space traversal, which will be examined in the next cells.

Output:

```
Training Base VAE
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:1118:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
  super().__init__(loader)
Epoch 01/15 | train 52.4324 | val 38.9862 | recon 32.5731 | KL 6.4131
Epoch 02/15 | train 34.4394 | val 31.4016 | recon 25.0334 | KL 6.3682
Epoch 03/15 | train 29.8042 | val 28.9498 | recon 22.3194 | KL 6.6303
Epoch 04/15 | train 28.2262 | val 28.2464 | recon 22.1997 | KL 6.0466
Epoch 05/15 | train 27.3473 | val 27.0427 | recon 20.4812 | KL 6.5616
Epoch 06/15 | train 26.7147 | val 26.4562 | recon 19.9644 | KL 6.4918
Epoch 07/15 | train 26.1419 | val 25.9345 | recon 19.1709 | KL 6.7636
Epoch 08/15 | train 25.7421 | val 25.8056 | recon 18.9328 | KL 6.8728
```

```
Epoch 09/15 | train 25.3863 | val 25.4666 | recon 18.4830 | KL 6.9836
Epoch 10/15 | train 25.1154 | val 25.1863 | recon 18.2698 | KL 6.9165
Epoch 11/15 | train 24.9347 | val 25.0054 | recon 17.5707 | KL 7.4347
Epoch 12/15 | train 24.7822 | val 24.8077 | recon 17.7766 | KL 7.0311
Epoch 13/15 | train 24.5911 | val 25.2504 | recon 17.5731 | KL 7.6773
Epoch 14/15 | train 24.4602 | val 24.6835 | recon 17.5947 | KL 7.0888
Epoch 15/15 | train 24.3798 | val 24.6107 | recon 17.4378 | KL 7.1729
Base: total=24.5898, recon=17.4422, KL=7.1477
```

Training Deeper VAE

```
Epoch 01/15 | train 57.7895 | val 41.6234 | recon 36.0883 | KL 5.5351
Epoch 02/15 | train 36.9800 | val 33.1866 | recon 27.7580 | KL 5.4286
Epoch 03/15 | train 31.2774 | val 30.3666 | recon 25.0391 | KL 5.3275
Epoch 04/15 | train 28.9724 | val 28.3866 | recon 22.8135 | KL 5.5731
Epoch 05/15 | train 27.6817 | val 27.7197 | recon 22.1581 | KL 5.5616
Epoch 06/15 | train 27.0255 | val 27.1720 | recon 21.3941 | KL 5.7780
Epoch 07/15 | train 26.5661 | val 26.5822 | recon 20.9302 | KL 5.6520
Epoch 08/15 | train 26.3401 | val 26.5073 | recon 20.3798 | KL 6.1275
Epoch 09/15 | train 26.0803 | val 26.1938 | recon 20.2813 | KL 5.9125
Epoch 10/15 | train 25.8834 | val 26.0301 | recon 19.8587 | KL 6.1714
Epoch 11/15 | train 25.6777 | val 25.7661 | recon 19.9431 | KL 5.8231
Epoch 12/15 | train 25.5433 | val 25.6684 | recon 19.5150 | KL 6.1534
Epoch 13/15 | train 25.3988 | val 25.5121 | recon 19.3072 | KL 6.2048
Epoch 14/15 | train 25.2748 | val 25.4128 | recon 19.2625 | KL 6.1503
Epoch 15/15 | train 25.1758 | val 25.4860 | recon 19.4822 | KL 6.0038
Deeper: total=25.3807, recon=19.2622, KL=6.1185
```

Training Wider VAE

```
Epoch 01/15 | train 47.9905 | val 34.9981 | recon 29.2166 | KL 5.7814
Epoch 02/15 | train 31.3551 | val 29.0535 | recon 22.3844 | KL 6.6691
Epoch 03/15 | train 27.8085 | val 27.9039 | recon 20.9141 | KL 6.9898
Epoch 04/15 | train 26.6280 | val 26.6043 | recon 20.0618 | KL 6.5425
Epoch 05/15 | train 25.9928 | val 25.9672 | recon 19.3344 | KL 6.6328
Epoch 06/15 | train 25.5391 | val 25.5346 | recon 18.7682 | KL 6.7665
Epoch 07/15 | train 25.2051 | val 25.2936 | recon 18.2650 | KL 7.0286
Epoch 08/15 | train 24.9505 | val 25.1009 | recon 18.2852 | KL 6.8157
Epoch 09/15 | train 24.7879 | val 24.8419 | recon 17.4471 | KL 7.3947
Epoch 10/15 | train 24.5847 | val 24.6659 | recon 17.1803 | KL 7.4856
```

```
Epoch 11/15 | train 24.4650 | val 24.8391 | recon 17.8647 | KL 6.9744
Epoch 12/15 | train 24.3413 | val 24.4547 | recon 17.2145 | KL 7.2402
Epoch 13/15 | train 24.2484 | val 24.4573 | recon 17.4608 | KL 6.9965
Epoch 14/15 | train 24.1364 | val 24.2550 | recon 16.6080 | KL 7.6471
Epoch 15/15 | train 24.0386 | val 24.2783 | recon 16.6726 | KL 7.6057
Wider: total=24.2627, recon=16.6335, KL=7.6293
```

The output shows the training process for all three VAE architectures over 15 epochs. For every epoch, the training loss, validation loss, reconstruction loss, and KL loss are printed.

A warning also appears:

UserWarning: 'pin_memory' argument is set as true but no accelerator is found

This happens because the code is running on the CPU. It does not affect the correctness of the experiment.

The final test results are:

Base: total = 24.5898, recon = 17.4422, KL = 7.1477

Deeper: total = 25.3807, recon = 19.2622, KL = 6.1185

Wider: total = 24.2627, recon = 16.6335, KL = 7.6293

The Wider VAE has the lowest total test loss, 24.2627, and also the lowest reconstruction loss, 16.6335. Therefore, it gives the best numerical reconstruction performance among the three architectures.

The Base VAE is the second-best model with a total loss of 24.5898 and a reconstruction loss of 17.4422.

The Deeper VAE has the highest total loss and reconstruction loss, although it has the lowest KL loss. This shows that making the network deeper did not improve the reconstruction performance in this experiment.

At this point, the Wider architecture appears to be the strongest model based on numerical results. However, the reconstructed images, generated samples, and latent-space traversal should also be examined before making the final conclusion about the best VAE architecture.

Cell 19:

```
vae_results_df = pd.DataFrame(
    vae_results
)
```

```
vae_results_df
```

This cell converts the results collected from the three VAE experiments into a Pandas DataFrame. This makes it easier to compare the Base, Deeper, and Wider architectures in one table.

The table contains the test total loss, reconstruction loss, KL loss, total number of trainable parameters, and training time for each model. These values help us compare not only reconstruction performance, but also model complexity and computational cost.

Output:

Architecture	Test Total Loss	Test Reconstructio n Loss	Test KL Loss	Parameter s	Trainin g Time (s)	
0	Base	24.589814	17.442151	7.147663	715024	188.172120
1	Deeper	25.380681	19.262209	6.118472	722124	197.579951
2	Wider	24.262745	16.633475	7.629270	1195224	232.895777

The Wider VAE achieves the lowest total test loss, 24.262745, and also the lowest reconstruction loss, 16.633475. This means that it gives the best reconstruction performance among the three architectures.

The Base VAE gives the second-best result, with a reconstruction loss of 17.442151. It also has the fewest parameters and the shortest training time, so it is the simplest and fastest architecture in this comparison.

The Deeper VAE has the highest reconstruction loss and total loss. Although its KL loss is the lowest, adding another hidden layer does not improve the overall performance in this experiment.

The Wider model contains 1,195,224 parameters, which is much larger than the Base and Deeper models, and it also requires the longest training time. Therefore, its better reconstruction quality comes with a higher computational cost.

Based only on the numerical results, the Wider VAE currently gives the best performance. However, reconstruction and generation plots should also be considered before deciding which architecture provides the best overall balance between reconstruction quality and generative capability.

Cell 20:

```
def plot_reconstruction(
    model,
    loader,
    title,
    n=10,
    is_vae=True
):
    model.eval()

    x, _ = next(iter(loader))
    x = x[:n].to(device)

    with torch.no_grad():
        if is_vae:
            mu, _ = model.encode(x)
            reconstruction = model.decode(mu)
        else:
            reconstruction = model(x)

    reconstruction = reconstruction.view(
        -1,
        1,
        28,
        28
    )

    x = x.cpu()
    reconstruction = reconstruction.cpu()

    plt.figure(figsize=(15, 3))
```

```

for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(
        x[i, 0],
        cmap="gray"
    )
    plt.axis("off")

    plt.subplot(2, n, n + i + 1)
    plt.imshow(
        reconstruction[i, 0],
        cmap="gray"
    )
    plt.axis("off")

plt.suptitle(title)
plt.tight_layout()
plt.show()

for name, model in vae_models.items():
    plot_reconstruction(
        model,
        fashion_test_loader,
        f"{name} VAE Reconstruction"
    )

```

This cell defines a function to visually compare original Fashion-MNIST images with their reconstructions from each trained VAE model.

The function `plot_reconstruction` takes a model, a data loader, a title, and the number of images to display. It first puts the model in evaluation mode. Then it takes one batch from the loader and keeps only the first 10 images.

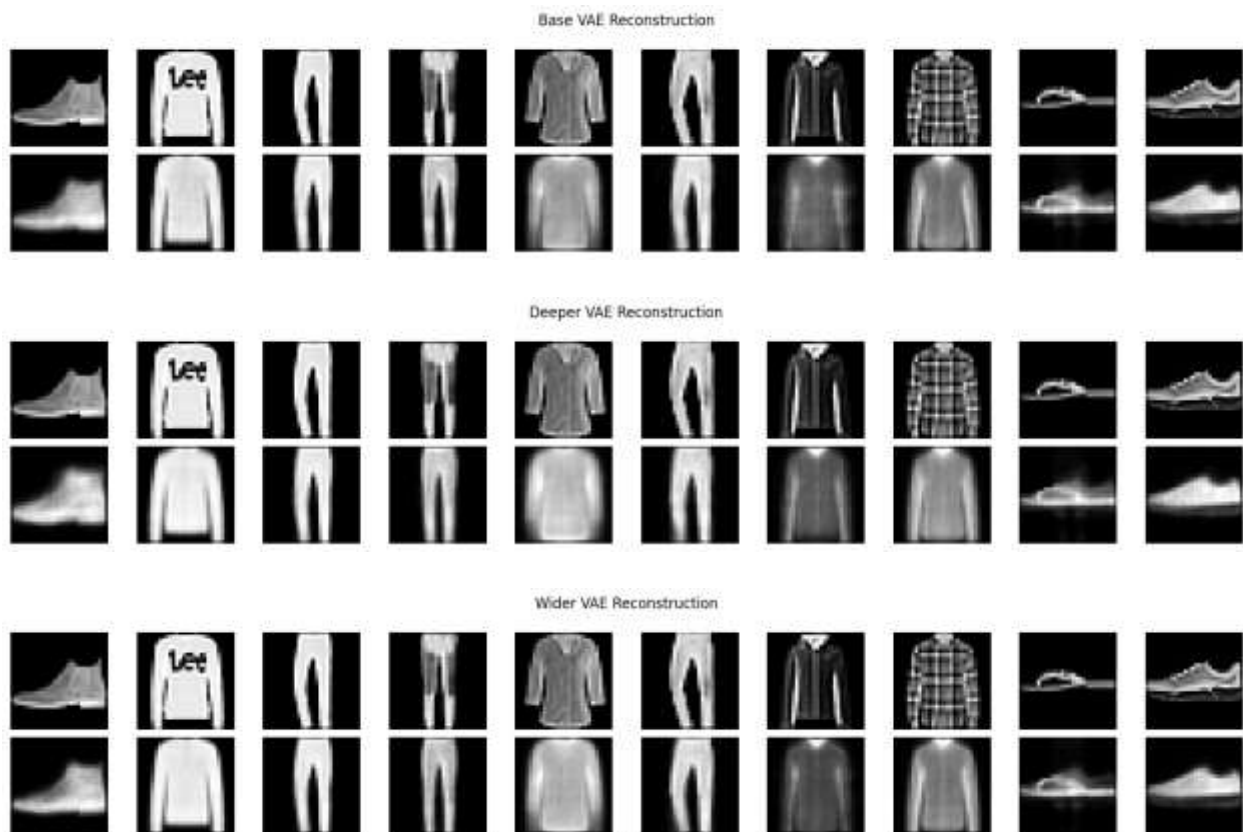
Because this is a VAE, the code uses the encoder to get the latent mean μ and then sends this mean vector to the decoder. This gives a stable reconstruction for each image. The reconstructed outputs are then reshaped back into 28 by 28 images.

After that, the function creates one figure with two rows. The first row shows the original test images, and the second row shows the corresponding reconstructed images. This makes it easy to compare how well the model preserves the main shape and structure of each clothing item.

At the end of the cell, the function is called for all three trained VAE models: Base, Deeper, and Wider. So this cell produces three reconstruction plots.

Output:

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:1118:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
super().__init__(loader)
```



The warning about `pin_memory` appears again because the code is running on the CPU. It does not affect the correctness of the results.

The three output plots show original images in the top row and reconstructed images in the bottom row for the Base VAE, Deeper VAE, and Wider VAE.

From these plots, we can see that all three models are able to reconstruct the general shape of the clothing items. Shoes, trousers, shirts, and coats are still recognizable in the reconstructed row. However, the reconstructed images are smoother and blurrier than the originals, which is normal for VAEs.

The Base VAE gives reasonable reconstructions and preserves the main object shapes well. The Deeper VAE also reconstructs the images correctly, but some outputs look a bit more blurred. The Wider VAE generally gives the clearest reconstructions among the three and preserves the object structure slightly better. This matches the numerical results from the previous cells, where the Wider VAE had the lowest reconstruction loss.

So, this cell gives a qualitative comparison, and the visual results support the earlier conclusion that the Wider VAE provides the best reconstruction quality overall.

Cell 21:

```
def plot_generation(
    model,
    title,
    n=10
):
    model.eval()

    z = torch.randn(
        n,
        LATENT_B,
        device=device
    )

    with torch.no_grad():
        samples = model.decode(z)

    samples = samples.view(
        -1,
        1,
        28,
        28
    ).cpu()

    plt.figure(figsize=(15, 2))
```

```

    for i in range(n):
        plt.subplot(1, n, i + 1)
        plt.imshow(
            samples[i, 0],
            cmap="gray"
        )
        plt.axis("off")

    plt.suptitle(title)
    plt.tight_layout()
    plt.show()

for name, model in vae_models.items():
    plot_generation(
        model,
        f"{name} VAE Generated Samples"
    )

```

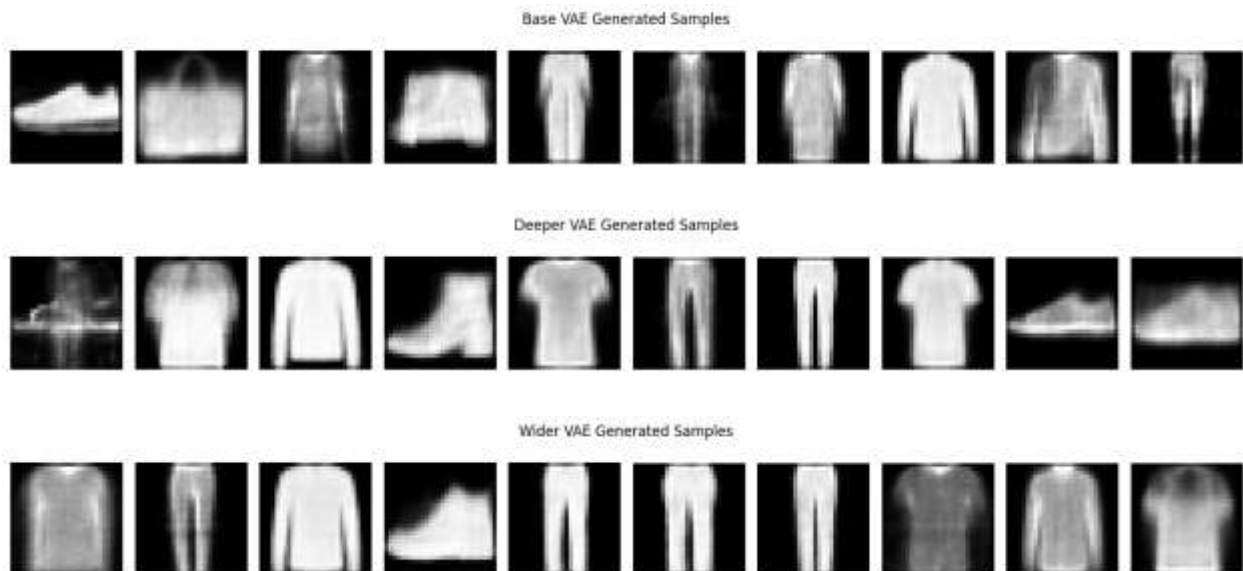
This cell is used to check the generative ability of each trained VAE. Unlike the reconstruction step, where the model receives a real image as input, here the model creates completely new samples from random latent vectors.

The function `plot_generation` first puts the model in evaluation mode. Then it creates `n` random latent vectors using a standard normal distribution. Each vector has dimension 20, which is the latent size used for the VAE models.

These random latent vectors are sent directly to the decoder. The decoder then transforms them into image samples. After that, the generated outputs are reshaped into 28 by 28 grayscale images so they can be displayed.

The function creates one row of images and shows the generated samples. At the end of the cell, this function is called for all three trained VAE models, so we get three plots: one for the Base VAE, one for the Deeper VAE, and one for the Wider VAE.

Output:



This cell produces three plots of generated Fashion-MNIST-like samples.

The generated images show that all three VAE models have learned a meaningful latent space. When random latent points are sampled, the decoder is able to produce recognizable clothing-like shapes instead of pure noise.

The Base VAE generates understandable samples, and many of them clearly look like shoes, trousers, shirts, or coats. The Deeper VAE also produces valid clothing shapes, although some samples look blurrier or less stable. The Wider VAE generally produces the clearest and most realistic generated samples among the three models. Many of its outputs have more consistent structure and are easier to recognize as Fashion-MNIST items.

This result is important because it shows the main advantage of the VAE. The model does not only reconstruct existing images, but it can also generate new images by sampling from the latent space. From these plots, the Wider VAE again looks like the strongest model overall, because it combines better reconstruction quality with better-looking generated samples.

Cell 22:

```
def get_class_image(dataset, class_id):
    for image, label in dataset:
        if label == class_id:
            return image.unsqueeze(0)

def plot_latent_traversal(
    model,
```

```

dataset,
class_a=0,
class_b=9,
steps=10,
title=""
):
    model.eval()

    image_a = get_class_image(
        dataset,
        class_a
    ).to(device)

    image_b = get_class_image(
        dataset,
        class_b
    ).to(device)

    with torch.no_grad():
        mu_a, _ = model.encode(image_a)
        mu_b, _ = model.encode(image_b)

        alphas = torch.linspace(
            0,
            1,
            steps,
            device=device
        )

        latent_points = []

        for alpha in alphas:
            z = (
                (1 - alpha) * mu_a
                + alpha * mu_b
            )

            latent_points.append(z)

```

```

latent_points = torch.cat(
    latent_points,
    dim=0
)

images = model.decode(
    latent_points
)

images = images.view(
    -1,
    1,
    28,
    28
).cpu()

plt.figure(figsize=(15, 2))

for i in range(steps):
    plt.subplot(1, steps, i + 1)
    plt.imshow(
        images[i, 0],
        cmap="gray"
    )
    plt.axis("off")

plt.suptitle(title)
plt.tight_layout()
plt.show()

```

This cell defines the functions used to examine the structure of the VAE latent space by moving smoothly between two different Fashion-MNIST classes.

The `get_class_image` function searches the dataset and returns the first image that belongs to a selected class. The image is given an extra batch dimension so that it can be passed directly into the VAE.

The `plot_latent_traversal` function then takes two classes, which are class 0 and class 9 by default. In Fashion-MNIST, class 0 represents a T-shirt/top and class 9 represents an ankle boot.

The two selected images are passed through the encoder, and the mean latent vectors `mu_a` and `mu_b` are obtained. Instead of sampling random latent vectors, the code uses these mean vectors as stable representations of the two images.

Next, the code creates several values between 0 and 1 using `torch.linspace`. These values are used to linearly interpolate between the two latent vectors. In other words, the code starts from the latent representation of the first class and gradually moves toward the latent representation of the second class.

Each intermediate latent point is then passed through the decoder to generate an image. The generated images are reshaped into 28 by 28 format and displayed in one row.

The purpose of this experiment is to check whether the VAE latent space is smooth and continuous. If the latent space is well organized, the generated images should gradually change from one class into the other instead of changing suddenly.

There is no output from this cell yet because it only defines the functions for latent-space traversal. The actual traversal plots are produced in the next cell.

Cell 23:

```
for name, model in vae_models.items():
    plot_latent_traversal(
        model,
        fashion_test,
        class_a=0,
        class_b=9,
        steps=10,
        title=f"{name} VAE Latent Traversal"
    )
```

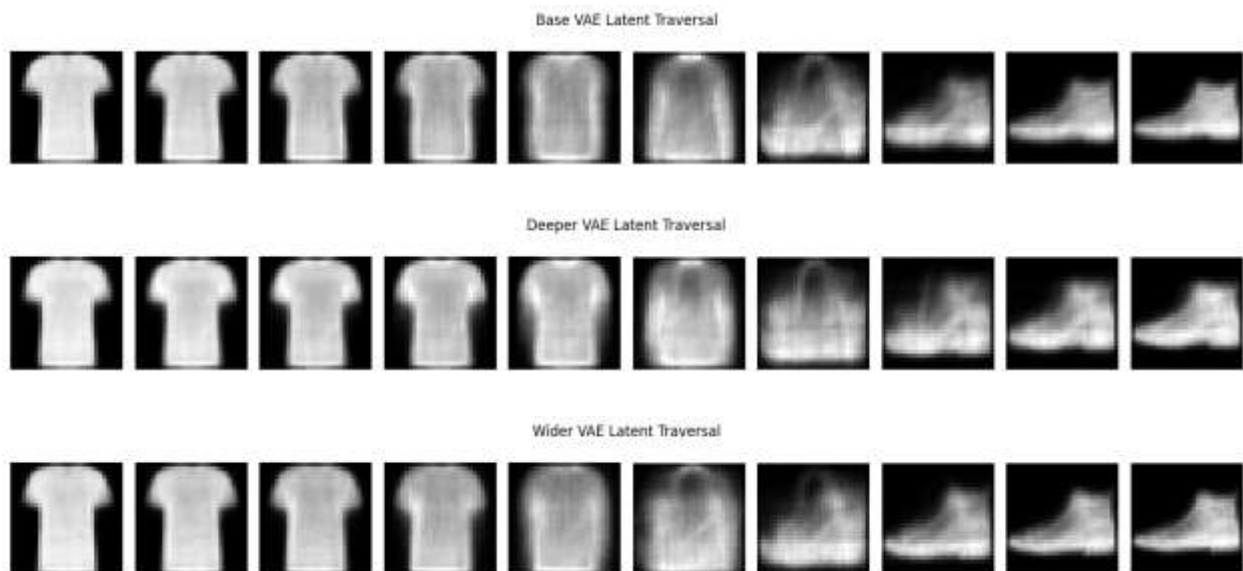
This cell runs the latent traversal function for all trained VAE models.

The loop goes through each model in `vae_models` and calls `plot_latent_traversal`. For every architecture, it uses class 0 as the starting point and class 9 as the ending point. In Fashion-MNIST, this means the interpolation starts from a T-shirt or top and gradually moves toward an ankle boot.

The number of interpolation steps is set to 10, so each plot contains 10 images. The first image is close to class 0, the last image is close to class 9, and the images in between are intermediate latent points between the two classes.

The goal of this cell is to visually check whether the latent space is smooth and meaningful. If the VAE has learned a good latent representation, the transition from one class to the other should happen gradually, not randomly or abruptly.

Output:



This cell produces three latent traversal plots, one for the Base VAE, one for the Deeper VAE, and one for the Wider VAE.

In all three plots, the images start with a clear T-shirt or top shape on the left side and gradually change into a boot shape on the right side. This shows that the latent space is continuous and that the decoder can generate smooth intermediate images between two different classes.

The Base VAE shows a smooth transition, and the main shape changes in a natural way from clothing to footwear. The Deeper VAE also shows the same general behavior, although some middle images look a bit blurrier. The Wider VAE gives a very smooth and stable transition, and the structure of the objects remains clear through most of the path.

These plots are important because they show that the latent space is organized in a meaningful way. The VAE does not only memorize training images. It learns a continuous representation where moving in latent space creates gradual visual changes. This is one of the main strengths of VAEs.

Overall, all three models show smooth traversal, but the Wider VAE again appears slightly stronger because the transition looks more stable and visually cleaner.

Cell 24:

```
best_vae_name = None
best_vae_val_loss = float("inf")

for name in vae_models:
    history = vae_histories[name]

    best_loss = min(
        row["val_loss"]
        for row in history
    )

    if best_loss < best_vae_val_loss:
        best_vae_val_loss = best_loss
        best_vae_name = name

best_vae = vae_models[
    best_vae_name
]

best_config = vae_configs[
    best_vae_name
]

print("Best VAE architecture:", best_vae_name)
print(
    "Best validation loss:",
    round(best_vae_val_loss, 4)
)
```

This cell selects the best VAE architecture based on the validation loss obtained during training.

First, `best_vae_name` is initialized as `None` and `best_vae_val_loss` is set to infinity. The code then checks the training history of each VAE model.

For every architecture, it finds the minimum validation loss reached during the 15 training epochs. If this value is lower than the best validation loss found so far, the architecture name and its loss are saved.

After checking all three models, the code selects the corresponding trained model and its architecture configuration. These are stored in `best_vae` and `best_config` so they can be used in the next part of the problem.

Output:

```
Best VAE architecture: Wider  
Best validation loss: 24.255
```

This result shows that the Wider VAE achieved the lowest validation loss among the three architectures, with a value of 24.255.

This is consistent with the previous test results, where the Wider VAE also had the lowest total test loss and the lowest reconstruction loss. Its reconstruction and latent traversal plots were also strong. Therefore, the Wider architecture is selected as the best VAE for the next part of the experiment, where it will be compared with a standard Autoencoder.

Cell 25:

```
class StandardAutoencoder(nn.Module):  
    def __init__(  
        self,  
        encoder_dims,  
        decoder_dims,  
        latent_dim=20  
    ):  
        super().__init__()   
  
        encoder_layers = []  
        previous = 784  
  
        for size in encoder_dims:  
            encoder_layers.append(  

```

```

        nn.Linear(previous, size)
    )
    encoder_layers.append(nn.ReLU())
    previous = size

    encoder_layers.append(
        nn.Linear(previous, latent_dim)
    )

    self.encoder = nn.Sequential(
        *encoder_layers
    )

    decoder_layers = []
    previous = latent_dim

    for size in decoder_dims:
        decoder_layers.append(
            nn.Linear(previous, size)
        )
        decoder_layers.append(nn.ReLU())
        previous = size

    decoder_layers.append(
        nn.Linear(previous, 784)
    )
    decoder_layers.append(nn.Sigmoid())

    self.decoder = nn.Sequential(
        *decoder_layers
    )

    def encode(self, x):
        x = x.view(x.size(0), -1)

        return self.encoder(x)

    def decode(self, z):

```

```

        return self.decoder(z)

    def forward(self, x):
        z = self.encode(x)

        return self.decode(z)

```

This cell defines the Standard Autoencoder that will be compared with the best VAE architecture in Task C.

The Standard Autoencoder uses the same general encoder and decoder structure as the selected VAE, but it removes the variational part. In other words, there is no separate mu and logvar output and there is no reparameterization step.

The encoder receives a Fashion-MNIST image, flattens it from 28×28 pixels into 784 values, and then passes it through the hidden layers defined by `encoder_dims`. After the hidden layers, a final Linear layer creates the 20-dimensional latent representation.

The decoder performs the reverse process. It starts from the 20-dimensional latent vector and passes it through the hidden layers defined by `decoder_dims`. The final layer produces 784 output values, and a Sigmoid activation is used so that the reconstructed pixel values stay between 0 and 1.

The `encode` function returns the deterministic latent vector directly. The `decode` function reconstructs an image from a latent vector.

In the `forward` function, the input image is first encoded into the latent representation `z`, and this latent vector is then passed to the decoder to produce the reconstruction.

The main difference from the VAE is that this Autoencoder uses only one fixed latent representation for each input. It does not learn a probability distribution in the latent space and does not use random sampling.

There is no printed output for this cell because it only defines the Standard Autoencoder architecture.

Cell 26:

```

def evaluate_standard_ae(model, loader):
    model.eval()

    total_loss = 0
    total_samples = 0

```

```

with torch.no_grad():
    for x, _ in loader:
        x = x.to(device)
        target = x.view(x.size(0), -1)

        reconstruction = model(x)

        loss = F.mse_loss(
            reconstruction,
            target,
            reduction="sum"
        )

        total_loss += loss.item()
        total_samples += x.size(0)

    return total_loss / total_samples

def train_standard_ae(
    model,
    train_loader,
    val_loader,
    epochs=15
):
    model = model.to(device)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.001
    )

    best_val_loss = float("inf")
    best_state = None

    history = []

```

```

for epoch in range(epochs):
    model.train()

    total_loss = 0
    total_samples = 0

    for x, _ in train_loader:
        x = x.to(device)
        target = x.view(x.size(0), -1)

        optimizer.zero_grad()

        reconstruction = model(x)

        loss = F.mse_loss(
            reconstruction,
            target,
            reduction="sum"
        )

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        total_samples += x.size(0)

    train_loss = (
        total_loss / total_samples
    )

    val_loss = evaluate_standard_ae(
        model,
        val_loader
    )

    history.append({
        "train_loss": train_loss,
        "val_loss": val_loss
    })

```



```

    })

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = copy.deepcopy(
            model.state_dict()
        )

    print(
        f"Epoch {epoch + 1:02d}/{epochs} | "
        f"train MSE {train_loss:.4f} | "
        f"val MSE {val_loss:.4f}"
    )

    model.load_state_dict(best_state)

    return model, history

```

This cell defines the evaluation and training functions for the Standard Autoencoder.

The `evaluate_standard_ae` function measures the reconstruction loss of the Autoencoder on a validation or test dataset. The model is first placed in evaluation mode, and gradient calculation is disabled because no training is needed during evaluation.

For each batch, the Fashion-MNIST images are moved to the selected device and flattened into vectors of 784 values. The Autoencoder reconstructs the input, and the reconstruction error is calculated using Mean Squared Error with `reduction="sum"`. The total loss is then divided by the number of samples to obtain the average reconstruction loss per image.

The `train_standard_ae` function is used to train the Standard Autoencoder for 15 epochs. The Adam optimizer is used with a learning rate of 0.001.

During each epoch, the model is placed in training mode. For every batch, the input images are flattened, passed through the Autoencoder, and reconstructed. The reconstruction loss is calculated using only MSE. This is an important difference from the VAE, because there is no KL divergence term in the Standard Autoencoder loss.

After calculating the loss, backpropagation is performed and the optimizer updates the model parameters.

At the end of each epoch, the average training reconstruction loss and validation reconstruction loss are calculated. These values are stored in the history list and printed.

The code also keeps track of the best validation loss. Whenever a lower validation loss is obtained, the current model parameters are saved. After training is complete, the best saved model state is loaded again.

Therefore, the final returned model is the version that achieved the lowest validation reconstruction loss during training.

There is no output from this cell because it only defines the training and evaluation functions. The actual Standard Autoencoder training is performed in the next cell.

Cell 27:

```
set_seed()

standard_ae = StandardAutoencoder(
    encoder_dims=best_config["encoder"],
    decoder_dims=best_config["decoder"],
    latent_dim=LATENT_B
).to(device)

standard_ae, standard_ae_history = train_standard_ae(
    standard_ae,
    fashion_train_loader,
    fashion_val_loader,
    epochs=15
)
```

This cell creates and trains the Standard Autoencoder that will be compared with the best VAE model.

First, the random seed is reset to make the experiment more reproducible. The Standard Autoencoder is then created using the same encoder and decoder dimensions as the best VAE architecture selected earlier. Since the Wider VAE was selected as the best model, the Standard Autoencoder also uses the Wider architecture. The latent dimension is kept at 20 so that the comparison between the two models is fair.

The model is then trained for 15 epochs using the `train_standard_ae` function. The same Fashion-MNIST training and validation datasets are used.

Unlike the VAE, this model is trained only with reconstruction loss. There is no KL divergence term and no random sampling in the latent space.

Output:

```
Epoch 01/15 | train MSE 35.1275 | val MSE 19.6029
Epoch 02/15 | train MSE 17.3040 | val MSE 16.2803
Epoch 03/15 | train MSE 14.8395 | val MSE 14.0547
Epoch 04/15 | train MSE 13.2731 | val MSE 12.8627
Epoch 05/15 | train MSE 12.2148 | val MSE 12.3718
Epoch 06/15 | train MSE 11.4582 | val MSE 11.3582
Epoch 07/15 | train MSE 10.8501 | val MSE 10.7858
Epoch 08/15 | train MSE 10.4031 | val MSE 10.5500
Epoch 09/15 | train MSE 10.0763 | val MSE 11.0615
Epoch 10/15 | train MSE 9.7973 | val MSE 9.8911
Epoch 11/15 | train MSE 9.4956 | val MSE 9.6812
Epoch 12/15 | train MSE 9.2508 | val MSE 9.4449
Epoch 13/15 | train MSE 9.0322 | val MSE 9.1503
Epoch 14/15 | train MSE 8.8229 | val MSE 9.0204
Epoch 15/15 | train MSE 8.6306 | val MSE 8.8204
```

The training loss decreases from 35.1275 in the first epoch to 8.6306 in the final epoch. The validation loss also decreases from 19.6029 to 8.8204.

This shows that the Standard Autoencoder learns the reconstruction task successfully and continues to improve during training. There is a small increase in validation loss at epoch 9, but after that the loss decreases again.

The final validation reconstruction loss of 8.8204 is much lower than the reconstruction loss obtained by the VAE models. This is expected because the Standard Autoencoder is optimized only for reconstruction, while the VAE must also regularize its latent space using the KL divergence term.

So, at this stage, the Standard Autoencoder gives better reconstruction performance, but its ability to generate meaningful new samples still needs to be tested.

Cell 28:

```
ae_test_mse = evaluate_standard_ae(
    standard_ae,
```

```

        fashion_test_loader
    )

vae_test_total, vae_test_recon, vae_test_kl = (
    evaluate_vae(
        best_vae,
        fashion_test_loader
    )
)

comparison_c = pd.DataFrame({
    "Model": [
        f"{best_vae_name} VAE",
        "Standard Autoencoder"
    ],
    "Reconstruction Loss": [
        vae_test_recon,
        ae_test_mse
    ],
    "KL Loss": [
        vae_test_kl,
        0.0
    ],
    "Total Loss": [
        vae_test_total,
        ae_test_mse
    ]
})

comparison_c

```

This cell performs the final numerical comparison between the best VAE model and the Standard Autoencoder.

First, the Standard Autoencoder is evaluated on the Fashion-MNIST test set using the `evaluate_standard_ae` function. Its final reconstruction loss is stored in `ae_test_mse`.

Then, the selected best VAE, which is the Wider VAE, is evaluated on the same test set. For the VAE, three values are obtained: reconstruction loss, KL loss, and total loss.

Finally, these results are organized into a Pandas DataFrame so that the two models can be compared directly.

Output:

Model	Reconstruction Loss	KL Loss	Total Loss	
0	Wider VAE	16.592694	7.62927	24.221964
1	Standard Autoencoder	8.805940	0.00000	8.805940

The Standard Autoencoder has a much lower reconstruction loss, 8.805940, compared with 16.592694 for the Wider VAE. This means that the Standard Autoencoder reconstructs the input images more accurately.

The Standard Autoencoder has no KL loss because it does not use a probabilistic latent distribution. Therefore, its total loss is equal to its reconstruction loss.

The Wider VAE has an additional KL loss of 7.629270. This increases its total loss to 24.221964. However, this extra term is important because it forces the VAE latent space to follow a more regular and continuous distribution.

So, based only on reconstruction error, the Standard Autoencoder performs better. However, the VAE is designed not only for reconstruction but also for learning a structured latent space that allows meaningful sampling, interpolation, and generation of new images. The next cells visually compare these properties.

Cell 29:

```
plot_reconstruction(
    best_vae,
    fashion_test_loader,
    f"{best_vae_name} VAE Reconstruction",
    is_vae=True
)

plot_reconstruction(
    standard_ae,
    fashion_test_loader,
    "Standard Autoencoder Reconstruction",
```

```

    is_vae=False
)

```

This cell visually compares the reconstruction quality of the best VAE model and the Standard Autoencoder.

The `plot_reconstruction` function is first applied to the Wider VAE, which was selected as the best VAE architecture. The parameter `is_vae` is set to `True`, so the reconstruction is created from the mean latent representation produced by the VAE encoder.

The same function is then applied to the Standard Autoencoder. In this case, `is_vae` is set to `False` because the Standard Autoencoder directly produces the reconstruction without using `mu`, `logvar`, or the reparameterization process.

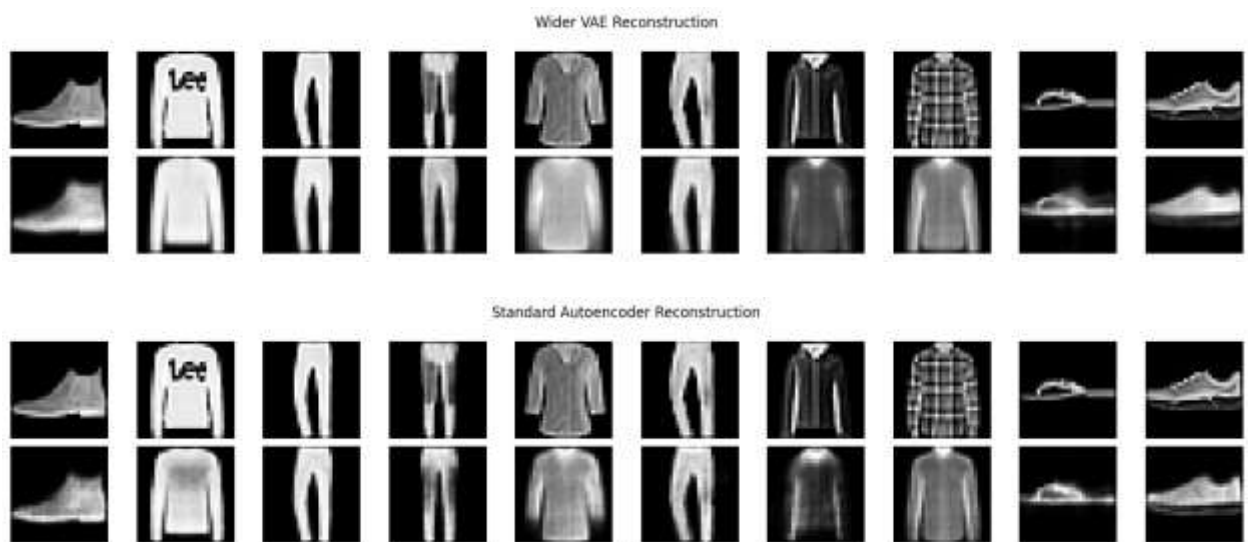
For both models, the first row contains the original Fashion-MNIST images and the second row contains their reconstructed versions. This allows us to compare the visual reconstruction quality directly.

Output:

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:1118:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,
then device pinned memory won't be used.
  super().__init__(loader)

```



The warning about `pin_memory` appears because the experiment is running on the CPU. It does not affect the results.

This cell produces two reconstruction plots: one for the Wider VAE and one for the Standard Autoencoder.

In both models, the reconstructed images preserve the main shape and class of the original objects. Shoes, trousers, shirts, and coats are still clearly recognizable.

However, the Standard Autoencoder reconstructions generally look slightly sharper and preserve more details from the original images. The Wider VAE reconstructions are smoother and more blurred in comparison.

This visual result agrees with the numerical results from the previous cell. The Standard Autoencoder achieved a reconstruction loss of 8.805940, while the Wider VAE had a higher reconstruction loss of 16.592694.

Therefore, the Standard Autoencoder provides better reconstruction quality. This is expected because it is trained only to minimize reconstruction error, while the VAE must also organize and regularize its latent space for generation and sampling.

Cell 30:

```
plot_generation(
    best_vae,
    f"{best_vae_name} VAE Random Generation"
)

z = torch.randn(
    10,
    LATENT_B,
    device=device
)

standard_ae.eval()

with torch.no_grad():
    ae_samples = standard_ae.decode(z)

ae_samples = ae_samples.view(
    -1,
    1,
    28,
    28
).cpu()
```

```
plt.figure(figsize=(15, 2))

for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(
        ae_samples[i, 0],
        cmap="gray"
    )
    plt.axis("off")

plt.suptitle(
    "Standard Autoencoder Random Generation"
)

plt.tight_layout()
plt.show()
```

This cell compares the generative ability of the best VAE model with the Standard Autoencoder.

First, the code uses `plot_generation` on the best VAE model, which is the Wider VAE. This function samples random latent vectors from a standard normal distribution and sends them to the decoder. Because the VAE is trained with latent space regularization, these random latent points usually produce meaningful and realistic Fashion-MNIST-like images.

After that, the code tries the same idea with the Standard Autoencoder. It creates random latent vectors z and directly passes them to the decoder of the standard autoencoder. Then the generated outputs are reshaped into image form and displayed.

Output:





This cell produces two plots. The first plot shows random samples generated by the Wider VAE. The second plot shows random samples generated by the Standard Autoencoder.

The Wider VAE outputs look much more meaningful. Most generated images resemble real clothing items such as shoes, dresses, coats, or trousers. They are still a bit blurry, but they have recognizable shapes and structure.

In contrast, the Standard Autoencoder outputs look noisy, distorted, and much less realistic. Most of them do not look like clean Fashion-MNIST items. This happens because a standard autoencoder is not trained to organize its latent space in a smooth probabilistic way. It can reconstruct known inputs well, but random points in its latent space usually do not correspond to valid images.

So this cell shows an important difference between the two models. The Standard Autoencoder is better for reconstruction, but the VAE is clearly better for random generation.

Analysis Questions

1. Which method yields the best reconstruction quality (MSE)?

In our experiment, SVD gives the best reconstruction quality because it has the lowest test MSE. The test MSE for SVD is 0.008719, while the Linear Autoencoder has an MSE of 0.021000 and the Non-linear Autoencoder has an MSE of 0.018087. Since a lower MSE means that the reconstructed images are closer to the original images, SVD performs best in this comparison. The Non-linear Autoencoder performs better than the Linear Autoencoder, which shows that the hidden layers and non-linear activation functions help the model learn a better representation. However, in this experiment, SVD still gives the lowest reconstruction error among the three methods.

2. Visualize the learned latent space using t-SNE or UMAP. What can we observe from the visualization?

We used t-SNE to visualize the latent representations of SVD, the Linear Autoencoder, and the Non-linear Autoencoder. The original latent representations have 64 dimensions, and t-SNE reduces them to two dimensions so that they can be shown in a plot. The points are colored according to their CIFAR-10 class labels. From the plots, we can see that all three methods learn some structure in the data, but the classes are not completely separated. There is still a large amount of overlap between different classes. This is reasonable because CIFAR-10 contains natural images, and some classes can have similar visual features.

3. How do the representations differ qualitatively?

The three latent representations show some differences in their structure. The SVD representation shows broad groups in some parts of the latent space, but many classes are mixed together. The Linear Autoencoder also creates some local groups, but there is still a large amount of overlap between the classes. The Non-linear Autoencoder appears to create slightly clearer local groups in some regions. This suggests that the non-linear layers help the Autoencoder learn more complex features from the images. However, none of the three methods creates clearly separated clusters for all CIFAR-10 classes. Therefore, the main difference is that the Non-linear Autoencoder shows slightly more meaningful local structure, while SVD gives the best reconstruction MSE.

4. Which VAE architecture provides the best balance of reconstruction quality and generative capability? Explain why.

Based on our results, the Wider VAE provides the best overall balance between reconstruction quality and generative capability. It has the lowest test reconstruction loss among the three VAE architectures, with a reconstruction loss of about 16.63. The Base VAE has a reconstruction loss of about 17.44, while the Deeper VAE has a higher value of about 19.26. The generated images from the Wider VAE also look more consistent and recognizable than the samples from the other two models. In the latent-space traversal experiment, the Wider VAE also produces a smooth transition between the two selected Fashion-MNIST classes. However, the Wider model has more parameters and requires more training time. Therefore, it has a higher computational cost, but its better reconstruction quality, generated samples, and smooth latent space make it the best overall VAE architecture in this experiment.

5. Why might the variational approach be preferred despite potentially higher reconstruction loss?

A Variational Autoencoder may be preferred even when its reconstruction loss is higher because it learns a more organized and continuous latent space. In our experiment, the Standard Autoencoder has a much lower reconstruction loss of about 8.81, while the Wider VAE has a reconstruction loss of about 16.59. This means that the Standard Autoencoder reconstructs the original images more accurately. However, the Standard Autoencoder is mainly trained for reconstruction, and its latent space is not forced to follow a regular distribution. Because of this, random latent vectors often produce noisy or unclear images. The VAE uses the KL divergence term to regularize its latent space. As a result, random points in the latent space can produce meaningful Fashion-MNIST-like images, and interpolation between different latent points gives smooth changes between images. Therefore, the VAE can be more useful when generation, sampling, and a smooth latent representation are important, even if its reconstruction error is higher.

Problem 4: Generative Adversarial Networks

Problem 4 is about Generative Adversarial Networks, or GANs, which are models designed to generate new data that looks similar to real data. Unlike the models in the previous problems, a GAN is made of two neural networks that are trained at the same time. The first network is called the Generator, and its job is to create fake images from random input values. The second network is called the Discriminator, and its job is to decide whether an image is real or generated. During training, these two networks compete with each other. The Generator tries to produce more realistic images, while the Discriminator tries to become better at detecting fake images.

In the first part of this problem, I implemented a Deep Convolutional GAN, or DCGAN, using the CIFAR-10 dataset. CIFAR-10 contains small color images from ten different classes, which makes it a useful dataset for testing image generation models. The Generator uses transposed convolutional layers to gradually transform a random latent vector into a 32×32 RGB image. Batch Normalization and ReLU activation are also used inside the Generator. The Discriminator uses normal convolutional layers together with LeakyReLU and Dropout to classify real and generated images.

The DCGAN was trained for 100 epochs, as required in the problem. During training, I saved generated samples at epochs 10, 50, and 100 so that I could visually observe how the image quality changed over time. I also recorded the Generator and Discriminator losses and monitored their behavior during training. In addition to visual inspection, I used the FID score to measure the quality of generated images. A lower FID value means that the generated image distribution is closer to the real image distribution, so it gives us another way to evaluate the model instead of only looking at the images.

One important challenge when training GANs is that the Generator and Discriminator must improve together. If one network becomes much stronger than the other, training can become unstable. Another common problem is mode collapse. In mode collapse, the Generator learns to produce only a small number of similar outputs instead of generating a wide variety of images. To study this problem, I measured the diversity of the generated DCGAN samples using different simple statistics. Then I applied Feature Matching as a method for reducing mode collapse. After this intervention, I compared the generated samples and their diversity measurements before and after Feature Matching.

The second main part of this problem focuses on WGAN-GP, which is another type of GAN designed to make training more stable. Instead of using the normal binary cross-entropy objective of DCGAN, WGAN-GP uses the Wasserstein loss. It also adds a Gradient Penalty to encourage the Critic to satisfy the Lipschitz constraint. In this implementation, the Discriminator is therefore replaced by a Critic that produces a score instead of a probability. I trained the WGAN-GP on the same CIFAR-10 dataset so that its behavior could be compared with the DCGAN under similar conditions.

Training GANs was one of the more challenging parts of this problem because the losses do not behave as simply as classification or reconstruction losses. A lower loss does not always mean that the generated images are better, so I

needed to look at several types of results together. I used generated samples, loss curves, Gradient Penalty values, training time, and FID scores to understand what was happening during training. I also compared the convergence speed of DCGAN and WGAN-GP by looking at how their FID scores changed with training time.

Overall, the goal of this problem is not only to generate images, but also to understand why GAN training can be difficult and how different methods can improve it. By comparing DCGAN, Feature Matching, and WGAN-GP, we can study image quality, diversity, training stability, and convergence from different points of view. In the following sections, I explain the implementation step by step and analyze the results produced by each part of the experiment.

Explain the code of problem set 4:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
!pip -q install torchmetrics torch-fidelity
!pip -q install "torchmetrics[image]" torch-fidelity
```

This cell installs the extra libraries that are needed later for evaluating the generated images. The torchmetrics library provides useful evaluation metrics for deep learning experiments, and torch-fidelity is used as a dependency for image quality metrics such as FID.

The second installation command adds the image-related components of torchmetrics. This is important because the FID metric belongs to the image evaluation part of the library. I used both commands to make sure all required packages were available in the Colab environment before running the rest of the notebook.

Output:

```
983.4/983.4 kB 26.5 MB/s eta 0:00:00
```

```
85.6/85.6 kB 9.8 MB/s eta 0:00:00
```

The output shows that the packages were downloaded and installed successfully. The progress bars reached 100%, so the required libraries were ready to use in the next cells.

Cell 2:

```
import os
import time
import random
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import make_grid, save_image
from torchmetrics.image.fid import FrechetInceptionDistance
from google.colab import drive
```

This cell imports all the libraries needed for Problem 4. The `os` library is used for working with folders and file paths, while `time` is used to measure how long the training takes. The `random` and `copy` libraries are useful for reproducibility and for creating copies of trained models. NumPy and Pandas are used for numerical operations and organizing the experiment results, and Matplotlib is used to draw graphs and display the results.

PyTorch is the main library used to build and train the neural networks. The `nn` module provides the layers needed for the Generator, Discriminator, and Critic, while `torch.nn.functional` provides some additional functions used later in the notebook. `DataLoader` is used to load the dataset in batches during training.

Torchvision is used for the CIFAR-10 dataset and image processing. The `transforms` module is used to prepare and normalize the images. The `make_grid` and `save_image` functions are used to organize generated images into a grid and save them for later analysis.

The `FrechetInceptionDistance` class is imported from `TorchMetrics` to calculate the FID score. This metric is used later to compare the generated images with real images. Finally, the Google Drive library is imported so that important results, images, and model files can be saved in Google Drive during the experiment.

This cell does not produce any visible output. It only prepares the libraries and tools that are needed in the next cells.

Cell 3:

```
drive.mount("/content/drive")
SEED = 42
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed()
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
if device.type != "cuda":
    raise RuntimeError(
        "GPU is not active. Enable GPU in Colab before continuing."
    )
torch.backends.cudnn.benchmark = True
OUTPUT_DIR = (
    "/content/drive/MyDrive/dl_pset4_results"
)
os.makedirs(
    OUTPUT_DIR,
    exist_ok=True
)
print("Device:", device)
print("GPU:", torch.cuda.get_device_name(0))
print("Output folder:", OUTPUT_DIR)
```

This cell prepares the environment before training starts. First, Google Drive is mounted so that the important results can be saved directly in Drive. This is useful because training GANs can take some time, and saving the results helps prevent losing them if the Colab session is disconnected.

A fixed random seed is also defined and applied to Python, NumPy, and PyTorch. Using the same seed helps make the experiment more reproducible because the random operations start from the same initial state each time.

Next, the code checks whether a GPU is available. If CUDA is available, the GPU is selected as the main device for training. If no GPU is active, the code stops and shows an error message. This check was important because GAN training on CPU would be much slower. The cuDNN benchmark option is also enabled to improve the speed of convolution operations when the input sizes stay mostly the same.

Finally, a folder called `dl_pset4_results` is created inside Google Drive. This folder is used later to save generated images, model checkpoints, training histories, and evaluation results.

Output:

```
Mounted at /content/drive
Device: cuda
GPU: Tesla T4
Output folder: /content/drive/MyDrive/dl_pset4_results
```

The output shows that Google Drive was mounted successfully. It also confirms that the code is using CUDA with a Tesla T4 GPU. The last line shows the folder where the experiment results will be saved.

Cell 4:

```
BATCH_SIZE = 256
LATENT_DIM = 100
DCGAN_EPOCHS = 100
FM_EPOCHS = 5
WGAN_EPOCHS = 30
DCGAN_CHECKPOINTS = [10, 50, 100]
WGAN_CHECKPOINTS = [10, 20, 30]
FID_SAMPLES = 2000
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
    )
])
```

```

train_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform
)
test_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True,
    drop_last=True
)
fid_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True
)
print("Training samples:", len(train_dataset))
print("Test samples:", len(test_dataset))

```

This cell defines the main settings for the experiments and prepares the CIFAR-10 dataset. The batch size is set to 256, which means that 256 images are processed together in each training step. The latent dimension is set to 100, so the Generator receives a random vector with 100 values as its input.

The DCGAN is trained for 100 epochs because this is required in the problem. Feature Matching is applied for 5 additional epochs, while WGAN-GP is trained for 30 epochs. During training, checkpoints are saved at epochs 10, 50,

and 100 for DCGAN, and at epochs 10, 20, and 30 for WGAN-GP. These checkpoints are later used to compare how the generated images improve over time. The FID evaluation uses 2000 generated samples so that image quality can be measured without making the evaluation unnecessarily slow.

The transform section converts the CIFAR-10 images into PyTorch tensors and normalizes each RGB channel using a mean and standard deviation of 0.5. This changes the pixel values from the normal image range into a range that is more suitable for GAN training, especially because the Generator uses a Tanh activation in its final layer.

After that, the CIFAR-10 training and test datasets are downloaded and loaded. The training set is used for training the GAN models, while the test set is used later for FID evaluation. The training DataLoader shuffles the images and uses `drop_last=True` so that every training batch has the same size. The FID DataLoader does not shuffle the test data because it is only used for evaluation.

Output:

```
100%|██████████| 170M/170M [23:14<00:00, 122kB/s]
Training samples: 50000
Test samples: 10000
```

The output shows that the CIFAR-10 dataset was downloaded successfully. It also confirms that the training set contains 50,000 images and the test set contains 10,000 images. The long download time shown in the output only belongs to downloading the dataset for the first time and is not part of the normal training time.

Cell 5:

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(
                latent_dim,
                256,
                4,
                1,
                0,
                bias=False
            ),
            nn.BatchNorm2d(256),
```

```

        nn.ReLU(True),

        nn.ConvTranspose2d(
            256,
            128,
            4,
            2,
            1,
            bias=False
        ),
        nn.BatchNorm2d(128),
        nn.ReLU(True),
        nn.ConvTranspose2d(
            128,
            64,
            4,
            2,
            1,
            bias=False
        ),
        nn.BatchNorm2d(64),
        nn.ReLU(True),

        nn.ConvTranspose2d(
            64,
            3,
            4,
            2,
            1,
            bias=False
        ),
        nn.Tanh()
    )

    def forward(self, z):
        return self.net(z)
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()

```

```

self.features = nn.Sequential(
    nn.Conv2d(
        3,
        64,
        4,
        2,
        1,
        bias=False
    ),
    nn.LeakyReLU(0.2, inplace=True),
    nn.Dropout2d(0.25),
    nn.Conv2d(
        64,
        128,
        4,
        2,
        1,
        bias=False
    ),
    nn.BatchNorm2d(128),
    nn.LeakyReLU(0.2, inplace=True),
    nn.Dropout2d(0.25),

    nn.Conv2d(
        128,
        256,
        4,
        2,
        1,
        bias=False
    ),
    nn.BatchNorm2d(256),
    nn.LeakyReLU(0.2, inplace=True),
    nn.Dropout2d(0.25)
)
self.output = nn.Conv2d(
    256,
    1,

```

```

        4,
        1,
        0,
        bias=False
    )

    def forward(
        self,
        x,
        return_features=False
    ):
        features = self.features(x)

        output = self.output(
            features
        ).view(-1)

        if return_features:
            pooled = features.mean(
                dim=(2, 3)
            )

            return output, pooled

        return output

class Critic(nn.Module):
    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Conv2d(
                3,
                64,
                4,
                2,
                1
            ),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(

```

```

        64,
        128,
        4,
        2,
        1
    ),
    nn.LeakyReLU(0.2, inplace=True),

    nn.Conv2d(
        128,
        256,
        4,
        2,
        1
    ),
    nn.LeakyReLU(0.2, inplace=True),

    nn.Conv2d(
        256,
        1,
        4,
        1,
        0
    )
)

def forward(self, x):
    return self.net(x).view(-1)
def initialize_weights(model):
    for module in model.modules():
        if isinstance(
            module,
            (nn.Conv2d, nn.ConvTranspose2d)
        ):
            nn.init.normal_(
                module.weight,
                0.0,
                0.02
            )

```

```

        if module.bias is not None:
            nn.init.zeros_(
                module.bias
            )
    elif isinstance(
        module,
        nn.BatchNorm2d
    ):
        nn.init.normal_(
            module.weight,
            1.0,
            0.02
        )
        nn.init.zeros_(
            module.bias
        )
def count_parameters(model):
    return sum(
        p.numel()
        for p in model.parameters()
        if p.requires_grad
    )

```

This cell defines the three main neural networks used in Problem 4: the Generator, the Discriminator, and the Critic. It also defines two helper functions for weight initialization and counting model parameters.

The Generator takes a random latent vector as input and gradually converts it into a 32×32 RGB image. It uses transposed convolutional layers to increase the spatial size step by step. The number of channels decreases from 256 to 128 and then to 64 before producing the final 3-channel image. Batch Normalization is used after the hidden transposed convolution layers to make training more stable, and ReLU is used as the activation function. The last layer uses Tanh so that the generated pixel values match the normalized range used for CIFAR-10.

The Discriminator receives either a real CIFAR-10 image or an image generated by the Generator. It uses normal convolutional layers to reduce the spatial size of the image while increasing the number of feature channels. LeakyReLU is used instead of normal ReLU because it allows a small gradient for negative values, which can help GAN training. Dropout is also added to reduce overfitting and to make the Discriminator less likely to become too strong compared with the Generator. Batch Normalization is used in the deeper layers to improve training stability.

The Discriminator has an optional `return_features` argument. Normally, it only returns the final prediction score. However, when `return_features=True`, it also returns an average representation from the last feature maps. This extra output is needed later for Feature Matching, where the Generator tries to make the features of generated images more similar to the features of real images.

The Critic is used later in the WGAN-GP experiment. Its structure is similar to the Discriminator, but it does not use Batch Normalization, Dropout, or a final sigmoid function. Instead of predicting a probability between real and fake, the Critic gives a continuous score to each image. This is needed because WGAN-GP uses the Wasserstein objective instead of binary classification.

The `initialize_weights` function applies a common GAN initialization method. Convolutional and transposed convolutional weights are initialized from a normal distribution with a small standard deviation, while Batch Normalization weights are initialized around 1. This gives the networks a controlled starting point before training.

Finally, the `count_parameters` function calculates the total number of trainable parameters in a model. This is useful later when checking the size of the Generator, Discriminator, and Critic.

This cell does not produce any visible output because it only defines the model architectures and helper functions that will be used in the next cells.

Cell 6:

```
fixed_noise = torch.randn(
    64,
    LATENT_DIM,
    1,
    1,
    device=device
)
comparison_noise = torch.randn(
    256,
    LATENT_DIM,
    1,
    1,
    device=device
)
def copy_state(model):
    return {
```

```

        key: value.detach().cpu().clone()
    for key, value
        in model.state_dict().items()
    }
def set_requires_grad(
    model,
    value
):
    for parameter in model.parameters():
        parameter.requires_grad_(value)
def show_samples(
    samples,
    title
):
    grid = make_grid(
        samples,
        nrow=8,
        normalize=True,
        value_range=(-1, 1)
    )
    plt.figure(figsize=(8, 8))
    plt.imshow(
        grid.permute(1, 2, 0)
    )
    plt.axis("off")
    plt.title(title)
    plt.show()
def save_samples(
    samples,
    filename
):
    save_image(
        samples,
        os.path.join(
            OUTPUT_DIR,
            filename
        ),
        nrow=8,

```



```

        normalize=True,
        value_range=(-1, 1)
    )
def generate_samples(
    generator,
    noise
):
    generator.eval()
    with torch.no_grad():
        samples = generator(
            noise
        ).cpu()
    return samples

```

And this cell prepares several helper variables and functions that are used later for generating, comparing, displaying, and saving images.

First, `fixed_noise` is created with 64 random latent vectors. These vectors stay the same during the experiment, so they can be used to generate images at different training epochs. Using the same noise makes the visual comparison fair because any change in the generated images comes from the model learning, not from using different random inputs.

The `comparison_noise` variable contains 256 random latent vectors. It is used later when comparing sample diversity before and after Feature Matching. Using the same set of random inputs for both models makes this comparison more consistent.

The `copy_state` function creates a copy of the model parameters and moves them to the CPU. This is useful for saving model checkpoints at important epochs such as 10, 50, and 100. It allows us to keep the model state from a specific point in training and use it later without changing the current model.

The `set_requires_grad` function controls whether a model should calculate gradients. During GAN training, sometimes the Discriminator or Critic needs to be temporarily frozen while the Generator is updated. This function makes that easier by changing the `requires_grad` setting for all model parameters.

The `show_samples` function displays generated images in an 8 by 8 grid. The images are normalized back into a visible range and shown using Matplotlib. This function is used later to visually compare the quality of generated samples at different stages of training.

The `save_samples` function does almost the same thing, but instead of only displaying the images, it saves them inside the output folder in Google Drive. This is useful for keeping important generated samples even if the Colab session is closed.

Finally, the `generate_samples` function puts the Generator in evaluation mode and creates images from the given noise vectors. Gradient calculation is disabled because it is not needed when we are only generating samples for evaluation or visualization. The generated images are then moved to the CPU so they can be displayed or saved more easily.

This cell does not produce any visible output. It only prepares the helper tools that are used in the training and evaluation sections later.

Cell 7:

```
def train_dcgan(
    generator,
    discriminator,
    loader,
    epochs
):
    generator = generator.to(device)
    discriminator = discriminator.to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer_g = torch.optim.Adam(
        generator.parameters(),
        lr=0.0002,
        betas=(0.5, 0.999)
    )
    optimizer_d = torch.optim.Adam(
        discriminator.parameters(),
        lr=0.0002,
        betas=(0.5, 0.999)
    )
    scaler = torch.amp.GradScaler(
        "cuda"
    )
    history = {
        "generator_loss": [],
```

```

        "discriminator_loss": [],
        "d_real": [],
        "d_fake": [],
        "epoch_time": [],
        "cumulative_time": []
    }
    snapshots = {}
    checkpoints = {}
    total_start = time.perf_counter()
    for epoch in range(
        1,
        epochs + 1
    ):
        generator.train()
        discriminator.train()
        g_losses = []
        d_losses = []
        real_scores = []
        fake_scores = []
        epoch_start = time.perf_counter()
        for real, _ in loader:
            real = real.to(
                device,
                non_blocking=True
            )
            batch_size = real.size(0)
            ones = torch.ones(
                batch_size,
                device=device
            )
            zeros = torch.zeros(
                batch_size,
                device=device
            )
            optimizer_d.zero_grad(
                set_to_none=True
            )
            noise = torch.randn(

```

```

        batch_size,
        LATENT_DIM,
        1,
        1,
        device=device
    )
    with torch.amp.autocast(
        "cuda"
    ):
        fake = generator(noise)
        real_output = discriminator(
            real
        )
        fake_output = discriminator(
            fake.detach()
        )
        loss_real = criterion(
            real_output,
            ones
        )
        loss_fake = criterion(
            fake_output,
            zeros
        )

        loss_d = (
            loss_real
            + loss_fake
        )
    scaler.scale(
        loss_d
    ).backward()
    scaler.step(
        optimizer_d
    )
    set_requires_grad(
        discriminator,
        False

```

```

    )
    optimizer_g.zero_grad(
        set_to_none=True
    )
    with torch.amp.autocast(
        "cuda"
    ):
        fake_output_g = (
            discriminator(fake)
        )
        loss_g = criterion(
            fake_output_g,
            ones
        )
    scaler.scale(
        loss_g
    ).backward()

    scaler.step(
        optimizer_g
    )
    scaler.update()
    set_requires_grad(
        discriminator,
        True
    )
    g_losses.append(
        loss_g.item()
    )
    d_losses.append(
        loss_d.item()
    )
    real_scores.append(
        torch.sigmoid(
            real_output
        ).mean().item()
    )

```

```

        fake_scores.append(
            torch.sigmoid(
                fake_output
            ).mean().item()
        )
    elapsed = (
        time.perf_counter()
        - epoch_start
    )
    cumulative = (
        time.perf_counter()
        - total_start
    )
    history[
        "generator_loss"
    ].append(
        np.mean(g_losses)
    )
    history[
        "discriminator_loss"
    ].append(
        np.mean(d_losses)
    )
    history[
        "d_real"
    ].append(
        np.mean(real_scores)
    )
    history[
        "d_fake"
    ].append(
        np.mean(fake_scores)
    )
    history[
        "epoch_time"
    ].append(elapsed)
    history[
        "cumulative_time"

```

```

].append(cumulative)
if epoch in DCGAN_CHECKPOINTS:
    samples = generate_samples(
        generator,
        fixed_noise
    )
    snapshots[epoch] = samples
    state = copy_state(
        generator
    )
    checkpoints[epoch] = state

    torch.save(
        state,
        os.path.join(
            OUTPUT_DIR,
            f"drgan_epoch_{epoch}.pt"
        )
    )
    save_samples(
        samples,
        f"drgan_epoch_{epoch}.png"
    )
    print(
        f"Epoch {epoch:03d}/{epochs} | "
        f"G {history['generator_loss'][-1]:.4f} | "
        f"D {history['discriminator_loss'][-1]:.4f} | "
        f"time {elapsed:.1f}s"
    )
total_time = (
    time.perf_counter()
    - total_start
)
return (
    generator,
    discriminator,
    history,
    snapshots,

```

```
        checkpoints,  
        total_time  
    )
```

This cell defines the main training function for the DCGAN model. It receives the Generator, the Discriminator, the training DataLoader, and the number of epochs. The first step is to move both networks to the selected device so that training can run on the GPU.

The loss function used here is BCEWithLogitsLoss. This is a common choice for the original GAN setup because the Discriminator is trained as a binary classifier. Real images are given the target value 1, while generated images are given the target value 0. The Generator tries to make the Discriminator predict 1 for fake images, which means it is trying to produce more realistic samples.

Two separate Adam optimizers are used, one for the Generator and one for the Discriminator. Both use a learning rate of 0.0002 and beta values of 0.5 and 0.999. These settings are commonly used for DCGAN training because they usually provide more stable updates than using the default Adam settings.

The code also uses mixed precision training with GradScaler and autocast. This helps reduce GPU memory usage and can make training faster on the Tesla T4 GPU. This was useful in this experiment because the model had to run for 100 epochs.

Before training starts, several lists are created inside the history dictionary. These lists store the Generator loss, Discriminator loss, average Discriminator score for real images, average score for fake images, time for each epoch, and total cumulative training time. The snapshots and checkpoints dictionaries are also created to save important intermediate results.

Inside each epoch, both models are switched to training mode. For every batch, real CIFAR-10 images are moved to the GPU. Two target tensors are created: ones for real images and zeros for generated images.

The Discriminator is updated first. Random noise is given to the Generator to produce fake images. The Discriminator then processes both real and fake images. The loss for real images and the loss for fake images are calculated separately and then added together. The fake images are detached at this point because the Generator should not be updated while the Discriminator is being trained.

After the Discriminator update, its parameters are temporarily frozen. The Generator is then updated using the same generated images. This time, the fake images are given the target value 1 because the Generator wants the Discriminator to believe that they are real. After the Generator update is finished, the Discriminator parameters are enabled again for the next batch.

During training, the code stores the losses and the average Discriminator scores for real and fake images. These values are useful later for checking whether the two networks are learning together or whether one of them becomes much stronger than the other. This is important because GAN training can become unstable when the balance between the Generator and Discriminator is lost.

At the end of each epoch, the average values are added to the history dictionary. The training time for the epoch and the total training time are also recorded.

At epochs 10, 50, and 100, the code generates images using the fixed noise created earlier. These samples are saved so that the visual quality can be compared at different stages of training. The Generator weights are also copied and saved as checkpoints in Google Drive. This makes it possible to use the models from these specific epochs later for FID evaluation without training them again.

Finally, the function prints the Generator loss, Discriminator loss, and training time after every epoch. When all epochs are finished, it returns the trained models, training history, saved samples, checkpoints, and total training time. This cell itself too does not start the training yet. It only defines the complete DCGAN training procedure. The actual training is performed in the next cell.

Cell 8:

```
set_seed()

dcgan_generator = Generator(
    LATENT_DIM
)
dcgan_discriminator = Discriminator()
initialize_weights(
    dcgan_generator
)
initialize_weights(
    dcgan_discriminator
)
print(
    "Generator parameters:",
    count_parameters(
        dcgan_generator
    )
)
```

```

print(
    "Discriminator parameters:",
    count_parameters(
        dcgan_discriminator
    )
)
(
    dcgan_generator,
    dcgan_discriminator,
    dcgan_history,
    dcgan_snapshots,
    dcgan_checkpoints,
    dcgan_total_time
) = train_dcgan(
    dcgan_generator,
    dcgan_discriminator,
    train_loader,
    DCGAN_EPOCHS
)
pd.DataFrame(
    dcgan_history
).to_csv(
    os.path.join(
        OUTPUT_DIR,
        "dcgan_history.csv"
    ),
    index=False
)
print(
    "DCGAN total training time:",
    round(
        dcgan_total_time / 60,
        2
    ),
    "minutes"
)

```

This cell actually starts the DCGAN training. First, the random seed is set again so that the experiment starts from a more reproducible state. Then a new Generator and Discriminator are created using the architectures defined in the previous cells. After creating the models, the `initialize_weights` function is applied to both of them so that their weights start from the initialization method selected for this experiment.

The code then counts the number of trainable parameters in both models. The Generator has 1,068,928 trainable parameters, while the Discriminator has 663,296 parameters. This shows that the Generator is slightly larger than the Discriminator in this implementation.

After that, the `train_dcgan` function from the previous cell is called. The Generator and Discriminator are trained on the CIFAR-10 training data for 100 epochs. During each epoch, the code records the Generator loss, Discriminator loss, training time, and other stability information. It also saves the important checkpoints and generated samples at epochs 10, 50, and 100.

Output:

```
Generator parameters: 1068928
Discriminator parameters: 663296
Epoch 001/100 | G 4.9586 | D 0.3585 | time 14.7s
Epoch 002/100 | G 4.8704 | D 0.5804 | time 12.8s
Epoch 003/100 | G 3.8128 | D 0.5424 | time 13.3s
Epoch 004/100 | G 3.9287 | D 0.4393 | time 13.2s
Epoch 005/100 | G 4.3367 | D 0.4203 | time 13.3s
Epoch 006/100 | G 3.9645 | D 0.4777 | time 13.4s
Epoch 007/100 | G 3.3394 | D 0.6611 | time 13.3s
Epoch 008/100 | G 3.1970 | D 0.6644 | time 13.3s
Epoch 009/100 | G 2.9954 | D 0.6740 | time 13.5s
Epoch 010/100 | G 2.7802 | D 0.7259 | time 12.9s
Epoch 011/100 | G 2.7728 | D 0.7545 | time 12.5s
Epoch 012/100 | G 2.8930 | D 0.6855 | time 12.8s
Epoch 013/100 | G 3.0441 | D 0.6654 | time 13.2s
Epoch 014/100 | G 2.6827 | D 0.7118 | time 13.5s
Epoch 015/100 | G 2.5052 | D 0.8541 | time 13.3s
Epoch 016/100 | G 2.2548 | D 0.8930 | time 13.3s
Epoch 017/100 | G 2.1548 | D 0.8967 | time 13.4s
Epoch 018/100 | G 2.1117 | D 0.9019 | time 13.3s
Epoch 019/100 | G 2.0827 | D 0.9552 | time 13.2s
Epoch 020/100 | G 1.9660 | D 1.0087 | time 13.2s
```

Epoch 021/100		G 1.9063		D 0.9894		time 13.4s
Epoch 022/100		G 1.8646		D 1.0368		time 12.7s
Epoch 023/100		G 1.8557		D 1.0080		time 12.6s
Epoch 024/100		G 1.8060		D 1.0007		time 13.2s
Epoch 025/100		G 1.8290		D 1.0202		time 13.4s
Epoch 026/100		G 1.8074		D 1.0047		time 13.4s
Epoch 027/100		G 1.7975		D 1.0018		time 13.2s
Epoch 028/100		G 1.7562		D 1.0102		time 13.2s
Epoch 029/100		G 1.6882		D 1.0842		time 13.0s
Epoch 030/100		G 1.6821		D 1.0554		time 13.0s
Epoch 031/100		G 1.6547		D 1.0475		time 13.1s
Epoch 032/100		G 1.6722		D 1.0853		time 12.7s
Epoch 033/100		G 1.6115		D 1.0766		time 12.2s
Epoch 034/100		G 1.5613		D 1.0596		time 12.4s
Epoch 035/100		G 1.6083		D 1.0762		time 13.1s
Epoch 036/100		G 1.5669		D 1.1048		time 13.1s
Epoch 037/100		G 1.5222		D 1.0934		time 13.1s
Epoch 038/100		G 1.5182		D 1.0942		time 13.1s
Epoch 039/100		G 1.4948		D 1.0664		time 13.0s
Epoch 040/100		G 1.5212		D 1.1046		time 12.9s
Epoch 041/100		G 1.5134		D 1.0732		time 13.0s
Epoch 042/100		G 1.4773		D 1.1044		time 12.3s
Epoch 043/100		G 1.4293		D 1.1271		time 12.3s
Epoch 044/100		G 1.4159		D 1.1270		time 12.9s
Epoch 045/100		G 1.4016		D 1.1398		time 13.0s
Epoch 046/100		G 1.3948		D 1.1693		time 12.9s
Epoch 047/100		G 1.3307		D 1.1654		time 12.9s
Epoch 048/100		G 1.3366		D 1.1512		time 12.9s
Epoch 049/100		G 1.3384		D 1.1824		time 13.1s
Epoch 050/100		G 1.3085		D 1.1593		time 12.5s
Epoch 051/100		G 1.3208		D 1.1797		time 12.2s
Epoch 052/100		G 1.2690		D 1.1721		time 12.4s
Epoch 053/100		G 1.2766		D 1.1970		time 12.9s
Epoch 054/100		G 1.2623		D 1.1886		time 13.0s
Epoch 055/100		G 1.2399		D 1.1900		time 12.9s
Epoch 056/100		G 1.2316		D 1.2041		time 12.9s
Epoch 057/100		G 1.2087		D 1.1909		time 13.1s
Epoch 058/100		G 1.2524		D 1.1944		time 13.0s

Epoch 059/100		G 1.2327		D 1.1935		time 13.0s
Epoch 060/100		G 1.2295		D 1.2052		time 12.5s
Epoch 061/100		G 1.1977		D 1.1935		time 12.3s
Epoch 062/100		G 1.1930		D 1.2024		time 12.7s
Epoch 063/100		G 1.2123		D 1.2083		time 12.9s
Epoch 064/100		G 1.1847		D 1.2144		time 13.2s
Epoch 065/100		G 1.1830		D 1.2058		time 13.0s
Epoch 066/100		G 1.1759		D 1.1917		time 13.0s
Epoch 067/100		G 1.1637		D 1.2058		time 13.0s
Epoch 068/100		G 1.1680		D 1.2076		time 12.8s
Epoch 069/100		G 1.1822		D 1.2091		time 12.3s
Epoch 070/100		G 1.1511		D 1.2087		time 12.0s
Epoch 071/100		G 1.1785		D 1.2210		time 13.0s
Epoch 072/100		G 1.1693		D 1.2040		time 12.9s
Epoch 073/100		G 1.1319		D 1.2196		time 13.1s
Epoch 074/100		G 1.1488		D 1.2065		time 13.0s
Epoch 075/100		G 1.1379		D 1.2155		time 13.0s
Epoch 076/100		G 1.1471		D 1.2194		time 12.9s
Epoch 077/100		G 1.1481		D 1.2302		time 12.8s
Epoch 078/100		G 1.1180		D 1.2225		time 12.9s
Epoch 079/100		G 1.1218		D 1.2319		time 12.5s
Epoch 080/100		G 1.1257		D 1.2218		time 12.9s
Epoch 081/100		G 1.1030		D 1.2243		time 13.1s
Epoch 082/100		G 1.1121		D 1.2380		time 13.2s
Epoch 083/100		G 1.1101		D 1.2159		time 13.2s
Epoch 084/100		G 1.1233		D 1.2339		time 13.2s
Epoch 085/100		G 1.1324		D 1.2317		time 13.2s
Epoch 086/100		G 1.0955		D 1.2194		time 13.3s
Epoch 087/100		G 1.0986		D 1.2356		time 13.2s
Epoch 088/100		G 1.0985		D 1.2156		time 13.2s
Epoch 089/100		G 1.1064		D 1.2112		time 12.3s
Epoch 090/100		G 1.1215		D 1.2418		time 12.5s
Epoch 091/100		G 1.0833		D 1.2284		time 13.1s
Epoch 092/100		G 1.0943		D 1.2318		time 13.4s
Epoch 093/100		G 1.0951		D 1.2514		time 13.4s
Epoch 094/100		G 1.0823		D 1.2315		time 13.4s
Epoch 095/100		G 1.0586		D 1.2316		time 13.4s
Epoch 096/100		G 1.0636		D 1.2390		time 13.4s

```
Epoch 097/100 | G 1.0755 | D 1.2330 | time 13.5s
Epoch 098/100 | G 1.0943 | D 1.2404 | time 13.2s
Epoch 099/100 | G 1.0834 | D 1.2452 | time 13.1s
Epoch 100/100 | G 1.0503 | D 1.2500 | time 12.6s
DCGAN total training time: 21.68 minutes
```

The training output shows how the two losses changed during the 100 epochs. At the beginning, the Generator loss was quite high. For example, it started at 4.9586 in epoch 1. This means that the Generator initially had difficulty producing images that could fool the Discriminator. The Discriminator loss at the same time was only 0.3585, which suggests that the Discriminator was much better at separating real and fake images in the early stage of training.

As training continued, the Generator loss gradually became lower. Around epoch 10, the Generator loss had decreased to 2.7802. Later, around epoch 50, it reached 1.3085, and at epoch 100 it was 1.0503. This gradual decrease suggests that the

Generator was learning to create images that were more difficult for the Discriminator to reject.

The Discriminator loss followed a different pattern. It started at a low value, but gradually increased during training. For example, it was 0.7259 at epoch 10, 1.1593 at epoch 50, and 1.2500 at epoch 100. In GAN training, this behavior is not necessarily a bad result. At the beginning, the Discriminator can easily recognize the poor generated images. Later, as the Generator improves, the discrimination task becomes more difficult, so the Discriminator loss can increase.

The losses also show small changes from one epoch to another instead of moving smoothly in one direction. This is normal for GAN training because the Generator and Discriminator are learning against each other at the same time. When one model improves, it changes the learning problem for the other model. Because of this competition, GAN loss curves usually contain more fluctuations than the loss curves of normal classification models.

The training time was quite consistent across epochs. Most epochs took around 12 to 14 seconds on the Tesla T4 GPU. This shows that using the GPU and mixed precision training made the 100-epoch experiment practical. The complete DCGAN training took 21.68 minutes.

After training, the history dictionary is converted to a Pandas DataFrame and saved as `dcgan_history.csv` in the output folder. This file keeps all the recorded losses and timing information, so these results can be used later for plotting and analysis without training the model again.

Overall, this output shows that the complete 100-epoch DCGAN training finished successfully. The Generator became much more competitive during training, while the losses of both models gradually moved toward a more balanced range. However, loss values alone are not enough to decide whether the generated images are good. For this reason, the next cells also examine the generated samples, training curves, diversity, and FID scores.

Cell 9:

```
for epoch in DCGAN_CHECKPOINTS:

    show_samples(
        dcgan_snapshots[epoch],
        f"DCGAN Generated Samples - Epoch {epoch}"
    )
epochs = np.arange(
    1,
    DCGAN_EPOCHS + 1
)
plt.figure(figsize=(9, 5))
plt.plot(
    epochs,
    dcgan_history[
        "generator_loss"
    ],
    label="Generator"
)
plt.plot(
    epochs,
    dcgan_history[
        "discriminator_loss"
    ],
    label="Discriminator"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("DCGAN Loss Curves")
plt.legend()
plt.show()
plt.figure(figsize=(9, 5))
plt.plot(
    epochs,
    dcgan_history["d_real"],
    label="D(real)"
)
```

```

)
plt.plot(
    epochs,
    dcgan_history["d_fake"],
    label="D(fake) "
)
plt.xlabel("Epoch")
plt.ylabel("Mean Probability")
plt.title("DCGAN Training Stability")
plt.legend()
plt.show()

```

This cell is used to visualize the main DCGAN results after training. In the first part, the code loops through the saved DCGAN checkpoints at epochs 10, 50, and 100, and displays the generated samples for each one. Because the same fixed noise was used when these samples were saved, the images can be compared fairly across training time. This makes it easier to see how the Generator improves during training.

In the second part, the code creates an array of epoch numbers from 1 to 100. Then it plots the Generator loss and the Discriminator loss on the same graph. This helps show how the two models changed during the full DCGAN training process.

In the last part, the code plots the average Discriminator score for real images and fake images. These values are stored as D(real) and D(fake). This graph is useful for checking training stability, because it shows how the Discriminator responded to real and generated samples across epochs.

Overall, this cell does not train the model again. It only uses the saved training history and saved generated samples to visualize the results in a clearer way.

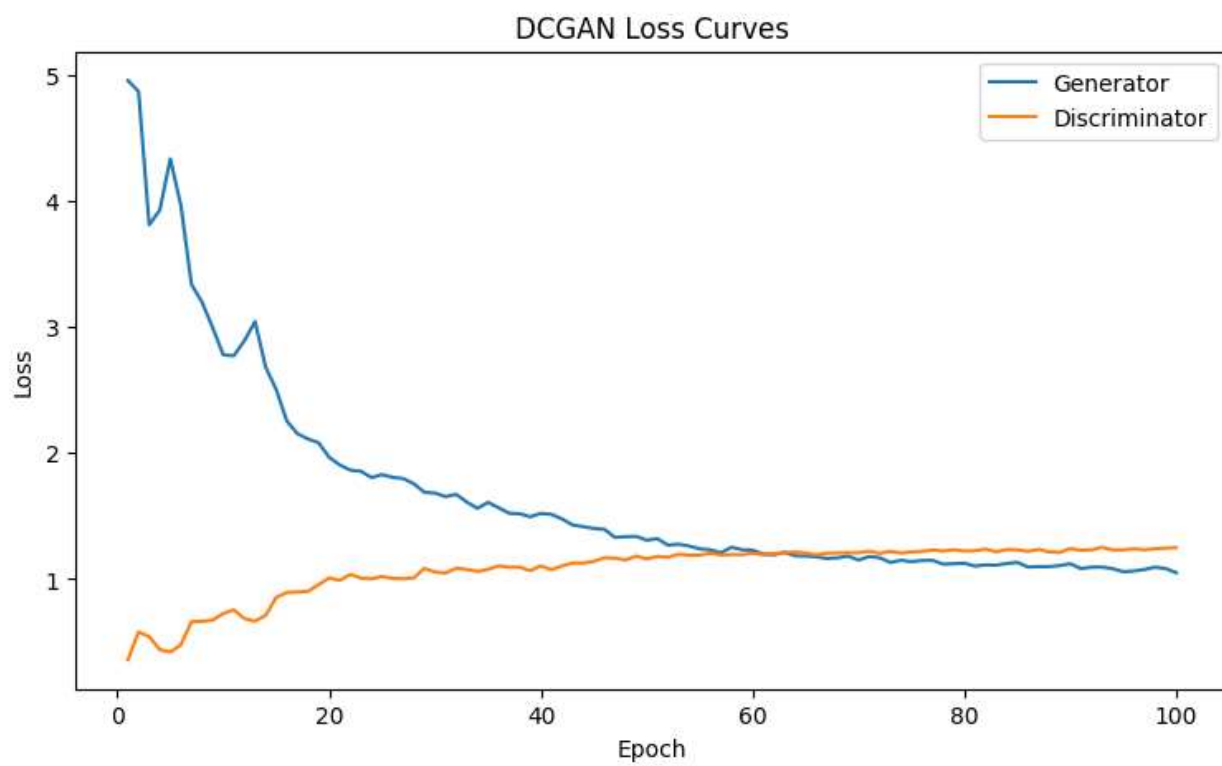
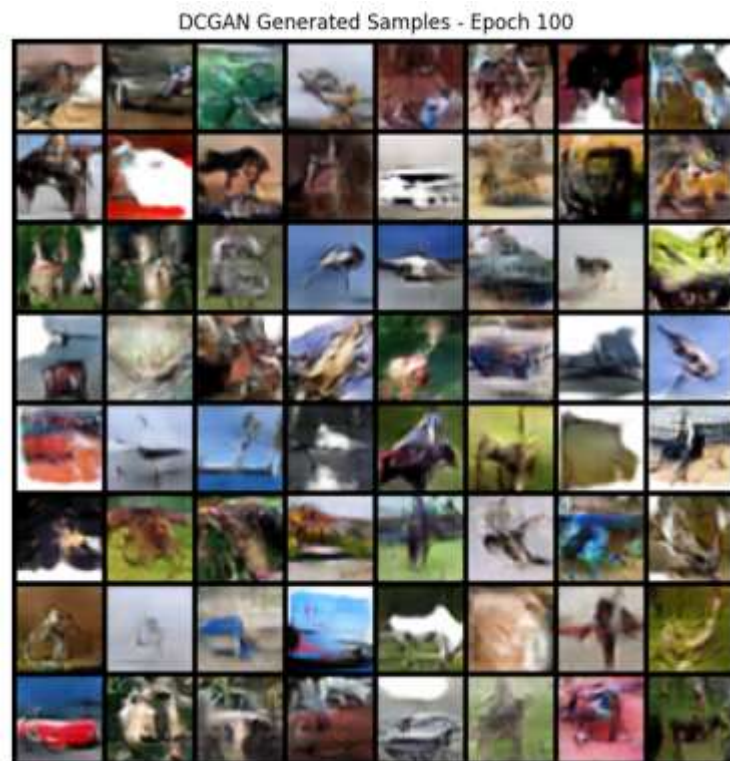
Output:

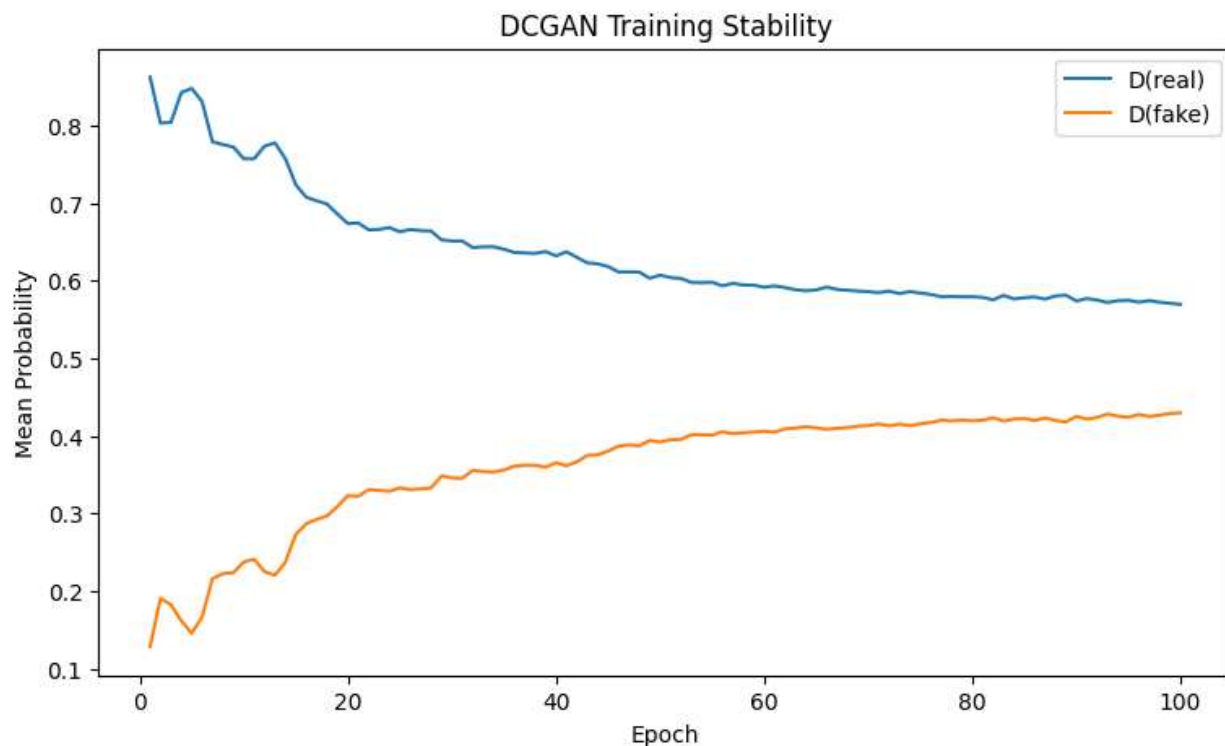
DCGAN Generated Samples - Epoch 10



DCGAN Generated Samples - Epoch 50







The first three outputs show the generated DCGAN samples at epochs 10, 50, and 100. In the epoch 10 image, the generated samples are still very blurry and unclear. Some images already contain rough color patterns and object-like shapes, but most of them do not yet look like meaningful CIFAR-10 images. This is normal in the early stage of GAN training because the Generator is still learning the basic structure of the data.

In the epoch 50 image, the generated samples become much better. Compared with epoch 10, the images now have clearer object shapes, stronger color structure, and more visual variety. Some samples begin to look like animals, vehicles, or natural scenes. Even though many images are still imperfect, the overall visual quality is much better than before. This suggests that the Generator learned important patterns from the training data during the middle stage of training.

In the epoch 100 image, the generated samples look more realistic and more stable than in the earlier epochs. The shapes are more organized, many images show more recognizable objects, and the colors are more natural. The samples are still not perfect, which is expected because CIFAR-10 image generation is a difficult task, but the overall result is clearly better than epoch 10 and also somewhat better than epoch 50. This shows that the Generator continued to improve across the full 100 epochs.

The fourth output is the DCGAN loss curve. In this graph, the Generator loss starts at a high value and gradually decreases over time, while the Discriminator loss starts lower and gradually increases. Around the beginning of training, the Generator loss is much higher than the Discriminator loss, which means the Generator was weak at first

and the Discriminator could easily detect fake images. As training continues, the two curves move closer together. This is a good sign because it suggests that the Generator became stronger and the competition between the two networks became more balanced. The graph also contains small fluctuations, which is normal in GAN training.

The fifth output is the training stability graph. In this figure, $D(\text{real})$ starts at a high value and slowly decreases, while $D(\text{fake})$ starts at a low value and gradually increases. At the beginning, the gap between the two is large, which means the Discriminator clearly separates real and fake images. As training continues, the gap becomes smaller. By the end of training, $D(\text{real})$ is around 0.57 and $D(\text{fake})$ is around 0.43. This shows that the Generator improved enough to make the fake images harder to distinguish from real ones. At the same time, the two curves do not become identical, so the Discriminator still keeps some ability to separate real and fake samples. This behavior suggests that the training remained reasonably stable.

Overall, the outputs of this cell show that the DCGAN training was successful. The generated images improved clearly from epoch 10 to epoch 100, the loss curves moved toward a more balanced range, and the stability graph shows that the Generator gradually became more competitive against the Discriminator. These results provide both visual and numerical evidence that the model learned meaningful patterns from the CIFAR-10 dataset.

Cell 10:

```
def diversity_metrics(samples):
    samples = samples[:256]
    pixel_std = (
        samples
        .std(dim=0)
        .mean()
        .item()
    )
    small = F.interpolate(
        samples,
        size=(8, 8),
        mode="bilinear",
        align_corners=False
    )
    flat = small.flatten(1)
    pairwise_distance = (
        torch.pdist(
            flat,
```

```

        p=2
    )
    .mean()
    .item()
)
normalized = F.normalize(
    flat,
    dim=1
)
similarities = (
    normalized
    @ normalized.T
)
mask = torch.triu(
    torch.ones(
        similarities.shape,
        dtype=torch.bool
    ),
    diagonal=1
)
mean_similarity = (
    similarities[
        mask
    ]
    .mean()
    .item()
)
return {
    "Pixel Std":
        pixel_std,
    "Mean Pairwise Distance":
        pairwise_distance,
    "Mean Cosine Similarity":
        mean_similarity
}

```

This cell defines a function called `diversity_metrics`, which is used to measure how different the generated images are from each other. This is important for checking mode collapse. If a GAN produces many very similar images, it may be a sign that the Generator has learned only a small number of patterns instead of learning the full variety of the dataset.

The function first uses up to 256 generated samples. The first measurement is Pixel Std. It calculates the standard deviation of pixel values across the generated images and then takes the average. If this value is higher, it usually means that the images contain more variation in their colors and visual patterns. A very low value could mean that many generated images are too similar.

The images are then resized to 8×8 before calculating the next measurements. This makes the calculation faster while still keeping the main visual information. After resizing, each image is flattened into a single vector.

The Mean Pairwise Distance calculates the average Euclidean distance between the generated images. A larger distance means that the samples are generally more different from each other, while a smaller value means that the generated samples are more similar. This gives us another simple way to measure diversity.

The final measurement is Mean Cosine Similarity. First, the image vectors are normalized, and then the similarity between every pair of images is calculated. The code only uses each pair once by selecting the upper part of the similarity matrix. A higher cosine similarity means that the generated images are more similar to each other, while a lower value usually indicates greater diversity.

Finally, the function returns all three measurements in a dictionary. Later, these values are used to compare the DCGAN samples before and after Feature Matching. Using several diversity measurements is helpful because mode collapse cannot always be identified reliably from only one number.

This cell does not produce any visible output because it only defines the `diversity_metrics` function. The function is called in the next cells when the generated samples are analyzed.

Cell 11:

```
before_fm_samples = (
    generate_samples(
        dcgan_generator,
        comparison_noise
    )
)
show_samples(
    before_fm_samples[:64],
```

```

    "DCGAN Before Feature Matching"
)
before_fm_metrics = (
    diversity_metrics(
        before_fm_samples
    )
)
pd.DataFrame([
    {
        "Model":
            "Before Feature Matching",
        **before_fm_metrics
    }
])

```

This cell checks the diversity of the DCGAN samples before applying Feature Matching. First, the trained DCGAN Generator uses `comparison_noise` to create 256 generated images. The same noise will also be used after Feature Matching, which makes the comparison more fair because both versions of the Generator are tested with the same inputs.

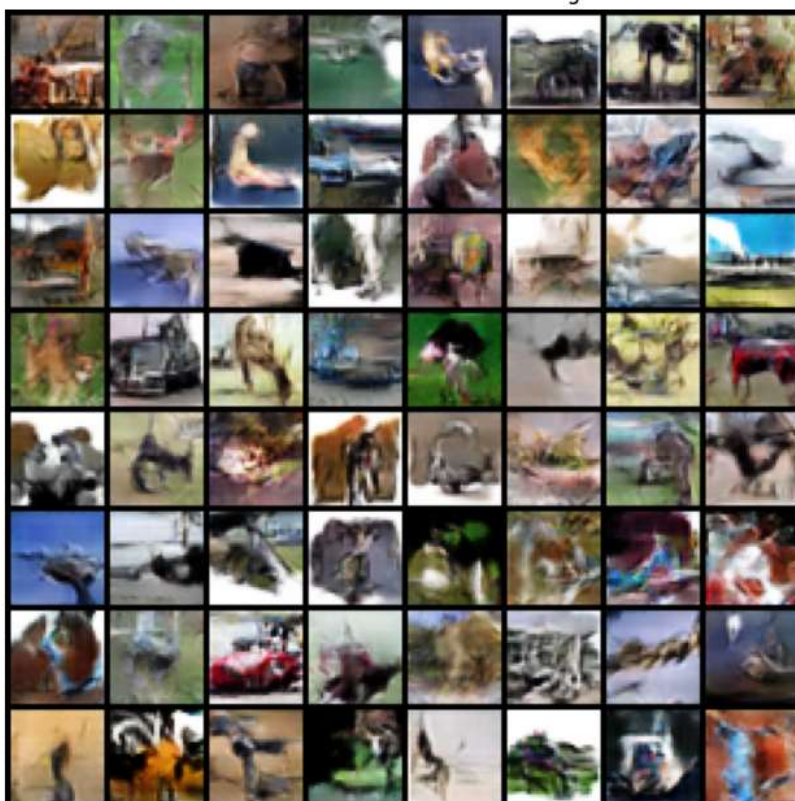
The first 64 generated images are displayed in a grid called "DCGAN Before Feature Matching". This gives a visual view of the sample quality and also helps us look for possible signs of mode collapse. If many generated images looked almost the same, it could mean that the Generator was producing only a limited number of patterns.

After displaying the images, the `diversity_metrics` function is applied to all 256 samples. It calculates three values: Pixel Std, Mean Pairwise Distance, and Mean Cosine Similarity. These values give a numerical baseline for the diversity of the DCGAN before Feature Matching.

Finally, the values are placed in a Pandas DataFrame under the name "Before Feature Matching". This table will later be compared with the results after Feature Matching to see whether the intervention improves the variety of generated samples.

Output:

DCGAN Before Feature Matching



Model	Pixel Std	Mean Pairwise Distance	Mean Cosine Similarity
0	Before Feature Matching	0.50012	9.193496 0.072287

The first output shows 64 samples generated by the trained DCGAN before Feature Matching. Visually, the images are not all identical. There is a noticeable variety in colors, object shapes, backgrounds, and general image structure. Some samples look like animals, vehicles, or natural scenes, while others are still blurry or difficult to recognize. This means that the model is not showing a very strong form of mode collapse, because it is producing several different types of outputs instead of repeating the same image many times.

The numerical results support this visual observation. The Pixel Std is 0.50012, which shows that there is a reasonable amount of variation in pixel values across the generated images. The Mean Pairwise Distance is 9.193496, which also suggests that the samples are not extremely close to each other in the image space.

The Mean Cosine Similarity is 0.072287, which is relatively low. A low cosine similarity means that, on average, the generated image vectors are not highly similar to each other. This is another sign that the Generator is producing a useful amount of diversity.

Overall, the output suggests that the DCGAN already has some diversity before Feature Matching and does not suffer from severe mode collapse. However, there are still some visually similar structures and repeated patterns, so Feature Matching is applied in the next cells to see whether the sample diversity can be improved further.

Cell 12:

```
def train_feature_matching(
    generator,
    discriminator,
    loader,
    epochs=5,
    feature_weight=10.0
):
    generator = generator.to(device)
    discriminator = discriminator.to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer_g = torch.optim.Adam(
        generator.parameters(),
        lr=0.0001,
        betas=(0.5, 0.999)
    )
    optimizer_d = torch.optim.Adam(
        discriminator.parameters(),
        lr=0.0001,
        betas=(0.5, 0.999)
    )
    history = {
        "generator_loss": [],
        "discriminator_loss": [],
        "feature_loss": []
    }
    for epoch in range(
        1,
```

```

        epochs + 1
    ):
        g_losses = []
        d_losses = []
        feature_losses = []
        for real, _ in loader:
            real = real.to(
                device,
                non_blocking=True
            )
            batch_size = real.size(0)
            ones = torch.ones(
                batch_size,
                device=device
            )
            zeros = torch.zeros(
                batch_size,
                device=device
            )
            discriminator.train()
            set_requires_grad(
                discriminator,
                True
            )
            optimizer_d.zero_grad(
                set_to_none=True
            )
            noise = torch.randn(
                batch_size,
                LATENT_DIM,
                1,
                1,
                device=device
            )
            fake = generator(
                noise
            )
            real_output = discriminator(

```

```

        real
    )
    fake_output = discriminator(
        fake.detach()
    )
    loss_d = (
        criterion(
            real_output,
            ones
        )
        +
        criterion(
            fake_output,
            zeros
        )
    )

    loss_d.backward()
    optimizer_d.step()
    discriminator.eval()
    set_requires_grad(
        discriminator,
        False
    )
    optimizer_g.zero_grad(
        set_to_none=True
    )
    with torch.no_grad():

        _, real_features = (
            discriminator(
                real,
                return_features=True
            )
        )

    noise = torch.randn(
        batch_size,
        LATENT_DIM,

```

```

        1,
        1,
        device=device
    )
    fake = generator(
        noise
    )
    (
        fake_output,
        fake_features
    ) = discriminator(
        fake,
        return_features=True
    )
    adversarial_loss = (
        criterion(
            fake_output,
            ones
        )
    )
    feature_loss = F.mse_loss(
        fake_features.mean(
            dim=0
        ),
        real_features.mean(
            dim=0
        )
    )
    loss_g = (
        adversarial_loss
        +
        feature_weight
        * feature_loss
    )
    loss_g.backward()
    optimizer_g.step()
    set_requires_grad(
        discriminator,

```

```

        True
    )
    g_losses.append(
        loss_g.item()
    )
    d_losses.append(
        loss_d.item()
    )
    feature_losses.append(
        feature_loss.item()
    )
    history[
        "generator_loss"
    ].append(
        np.mean(g_losses)
    )
    history[
        "discriminator_loss"
    ].append(
        np.mean(d_losses)
    )
    history[
        "feature_loss"
    ].append(
        np.mean(
            feature_losses
        )
    )
)
print(
    f"Epoch {epoch:02d}/{epochs} | "
    f"G {history['generator_loss'][-1]:.4f} | "
    f"D {history['discriminator_loss'][-1]:.4f} | "
    f"FM {history['feature_loss'][-1]:.4f}"
)
return (
    generator,
    discriminator,
    history

```

)

This cell defines the training function for Feature Matching. The goal of this part is to reduce possible mode collapse by changing the way the Generator is trained. Instead of only trying to fool the Discriminator, the Generator also tries to make the internal features of fake images similar to the internal features of real images.

At the beginning, the Generator and Discriminator are moved to the selected device. The same binary cross-entropy loss is still used for the adversarial part of training. Two Adam optimizers are created with a smaller learning rate of 0.0001 because this stage starts from an already trained DCGAN and only fine-tunes it for a few more epochs.

The history dictionary stores three values during training: Generator loss, Discriminator loss, and Feature Matching loss. These values are recorded for every epoch so that the behavior of the models can be checked later.

For each batch, the Discriminator is trained first. Real CIFAR-10 images and newly generated fake images are passed through the Discriminator. The real images use target value 1 and the fake images use target value 0. The two losses are added together and used to update the Discriminator.

After the Discriminator update, its parameters are temporarily frozen. The Generator is then trained. First, the Discriminator extracts internal features from the real images. These real features are calculated without gradients because they are only used as a target.

Next, a new batch of fake images is generated and passed through the Discriminator. This time, the code gets both the final Discriminator output and the internal fake features. The normal adversarial loss is still calculated because the Generator should continue trying to fool the Discriminator.

The Feature Matching loss is then calculated using the mean internal features of the real and fake images. Mean Squared Error is used to measure the difference between these two feature representations. If this difference becomes smaller, it means that the generated images are producing internal patterns that are more similar to real images.

The final Generator loss is a combination of the normal adversarial loss and the Feature Matching loss. The feature loss is multiplied by `feature_weight`, which is set to 10.0. This gives the Feature Matching objective enough importance during the update.

At the end of each epoch, the average Generator loss, Discriminator loss, and Feature Matching loss are stored and printed. The function finally returns the updated Generator, updated Discriminator, and the training history.

This cell does not produce any output yet because it only defines the Feature Matching training procedure. The function is actually called in the next cell.

Cell 13:

```
fm_generator = copy.deepcopy(
    dcgan_generator
)
fm_discriminator = copy.deepcopy(
    dcgan_discriminator
)

(
    fm_generator,
    fm_discriminator,
    fm_history
) = train_feature_matching(
    fm_generator,
    fm_discriminator,
    train_loader,
    epochs=FM_EPOCHS,
    feature_weight=10.0
)
after_fm_samples = (
    generate_samples(
        fm_generator,
        comparison_noise
    )
)
show_samples(
    before_fm_samples[:64],
    "Before Feature Matching"
)
show_samples(
    after_fm_samples[:64],
    "After Feature Matching"
)
after_fm_metrics = (
    diversity_metrics(
        after_fm_samples
    )
)
```

```

)
diversity_comparison = pd.DataFrame([
    {
        "Model":
            "Before Feature Matching",
        **before_fm_metrics
    },
    {
        "Model":
            "After Feature Matching",
        **after_fm_metrics
    }
])
diversity_comparison

```

This cell applies Feature Matching to the trained DCGAN and then compares the generated samples before and after this extra training step. First, the code creates deep copies of the trained DCGAN Generator and Discriminator. This is important because it keeps the original DCGAN unchanged, while the copied models are used for the Feature Matching experiment.

After that, the copied Generator and Discriminator are passed to the `train_feature_matching` function. The models are trained for 5 epochs using the same training data. In this stage, the Generator is not only trained to fool the Discriminator, but also trained to make the internal features of fake images similar to the internal features of real images.

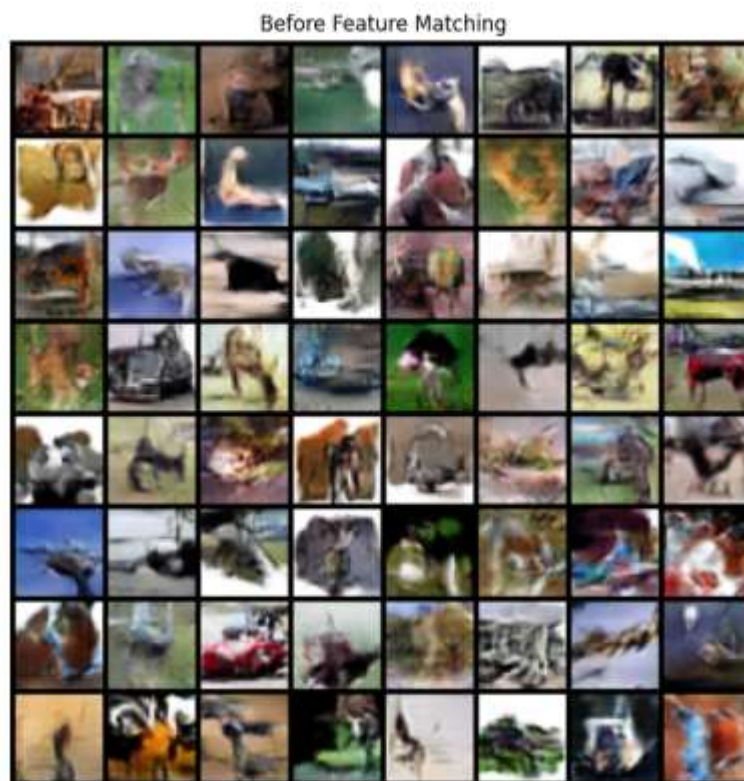
When the Feature Matching training is finished, the updated Generator is used to generate a new set of samples with the same `comparison_noise` that was used before. This makes the comparison fair because both the original DCGAN and the Feature Matching version are tested with the same latent inputs.

Then the code displays two image grids. The first grid shows the samples before Feature Matching, and the second grid shows the samples after Feature Matching. This visual comparison helps us see whether the extra training changed the sample quality or the visual diversity.

Next, the code calculates the diversity metrics for the samples after Feature Matching. These values are then combined with the previous values from before Feature Matching. Finally, both results are stored in a Pandas DataFrame called `diversity_comparison` so that the numerical differences can be seen clearly in one table.

Output:

```
Epoch 01/5 | G 1.0087 | D 0.6165 | FM 0.0060  
Epoch 02/5 | G 1.0133 | D 0.4379 | FM 0.0064  
Epoch 03/5 | G 1.0230 | D 0.3497 | FM 0.0054  
Epoch 04/5 | G 1.0029 | D 0.2016 | FM 0.0069  
Epoch 05/5 | G 0.9265 | D 0.0753 | FM 0.0086
```





Model	Pixel Std	Mean Pairwise Distance	Mean Cosine Similarity	
0	Before Feature Matching	0.500120	9.193496	0.072287
1	After Feature Matching	0.379152	6.798036	0.054089

The first part of the output shows the training log for the 5 Feature Matching epochs. The Generator loss starts at 1.0087 and ends at 0.9265. This small decrease suggests that the Generator continued learning during this fine-tuning stage. The Discriminator loss decreases much more strongly, from 0.6165 in the first epoch to 0.0753 in the fifth epoch. This means the Discriminator became very confident during this stage. The Feature Matching loss stays small during all 5 epochs, with values between about 0.0054 and 0.0086. This shows that the feature-based objective was active during training, although it did not decrease in a fully smooth way.

The next two outputs are the image grids before and after Feature Matching. These two figures are useful for direct visual comparison. In this kind of comparison, we mainly look for repeated patterns, very similar images, or a visible

loss of variety. We also look at whether the generated objects become more organized or more stable after Feature Matching.

The final table gives the numerical comparison between the diversity before and after Feature Matching. Before Feature Matching, the Pixel Std was 0.500120, the Mean Pairwise Distance was 9.193496, and the Mean Cosine Similarity was 0.072287. After Feature Matching, the Pixel Std became 0.379152, the Mean Pairwise Distance became 6.798036, and the Mean Cosine Similarity became 0.054089.

These results are mixed and are important to explain honestly. The Pixel Std decreased after Feature Matching, which means the overall pixel-level variation became smaller. The Mean Pairwise Distance also decreased, which suggests that the generated images became closer to each other on average. From these two metrics, it does not look like diversity improved in this run. However, the Mean Cosine Similarity also decreased, from 0.072287 to 0.054089. A lower cosine similarity means the generated image vectors are less aligned with each other, which can be interpreted as a positive sign for diversity.

So, in this specific experiment, Feature Matching did not produce a clear overall increase in diversity based on all metrics. Two metrics suggest lower variation after Feature Matching, while one metric suggests a small improvement. This means the effect of Feature Matching in this run was limited and not fully consistent. This is not necessarily a problem, because GAN methods do not always improve every metric in every run. The important point is that the experiment was performed correctly and the results give a realistic basis for later analysis.

Overall, this cell shows that Feature Matching was successfully applied and evaluated, but in this particular run its effect on diversity was not clearly better than the original DCGAN. This is a meaningful result and should be discussed honestly in the final analysis section.

Cell 14:

```
diversity_comparison.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "diversity_comparison.csv"
    ),
    index=False
)
save_samples(
    before_fm_samples[:64],
    "before_feature_matching.png"
)
```

```

save_samples(
    after_fm_samples[:64],
    "after_feature_matching.png"
)
torch.save(
    copy_state(
        fm_generator
    ),
    os.path.join(
        OUTPUT_DIR,
        "feature_matching_generator.pt"
    )
)
del fm_discriminator
torch.cuda.empty_cache()

```

This cell saves the important results from the Feature Matching experiment and also clears some GPU memory before moving to the next part of the problem.

First, the diversity_comparison table is saved as a CSV file called diversity_comparison.csv inside the output folder in Google Drive. This file contains the diversity values before and after Feature Matching, so the results can be used later without running the experiment again.

Next, the generated image grids from before and after Feature Matching are saved as PNG files. The first file is saved as before_feature_matching.png and the second one is saved as after_feature_matching.png. Saving these images is useful because they can later be used directly in the report for visual comparison.

The trained Feature Matching Generator is also saved as feature_matching_generator.pt. Before saving, the copy_state function creates a copy of the model parameters. This makes it possible to keep the trained Generator and load it again later if needed, without repeating the training process.

After saving the important files, the fm_discriminator variable is deleted because it is no longer needed in the next experiments. Finally, torch.cuda.empty_cache() is used to release unused GPU memory. This helps make more memory available before starting the WGAN-GP part.

This cell does not produce any visible output. Its main purpose is to save the results of the Feature Matching experiment safely and prepare the GPU memory for the next section.

Cell 15:

```
def gradient_penalty(
    critic,
    real,
    fake
):
    batch_size = real.size(0)

    alpha = torch.rand(
        batch_size,
        1,
        1,
        1,
        device=device
    )
    interpolated = (
        alpha * real
        +
        (1 - alpha) * fake
    )
    interpolated.requires_grad_(
        True
    )
    scores = critic(
        interpolated
    )
    gradients = torch.autograd.grad(
        outputs=scores,
        inputs=interpolated,
        grad_outputs=torch.ones_like(
            scores
        ),
        create_graph=True,
        retain_graph=True
    )[0]

    gradients = gradients.reshape(
```

```

        batch_size,
        -1
    )
    return (
        (
            gradients.norm(
                2,
                dim=1
            )
            - 1
        ) ** 2
    ).mean()
def train_wgan_gp(
    generator,
    critic,
    loader,
    epochs,
    lambda_gp=10.0,
    n_critic=1
):
    generator = generator.to(device)
    critic = critic.to(device)
    optimizer_g = torch.optim.Adam(
        generator.parameters(),
        lr=0.0001,
        betas=(0.0, 0.9)
    )
    optimizer_c = torch.optim.Adam(
        critic.parameters(),
        lr=0.0001,
        betas=(0.0, 0.9)
    )
    history = {
        "generator_loss": [],
        "critic_loss": [],
        "gradient_penalty": [],
        "wasserstein_gap": [],
        "epoch_time": [],

```

```

        "cumulative_time": []
    }
    snapshots = {}
    checkpoints = {}
    total_start = time.perf_counter()
    for epoch in range(
        1,
        epochs + 1
    ):
        generator.train()
        critic.train()
        g_losses = []
        c_losses = []
        gp_values = []
        gaps = []
        epoch_start = time.perf_counter()
        for real, _ in loader:
            real = real.to(
                device,
                non_blocking=True
            )
            batch_size = real.size(0)
            for _ in range(n_critic):
                noise = torch.randn(
                    batch_size,
                    LATENT_DIM,
                    1,
                    1,
                    device=device
                )
                with torch.no_grad():
                    fake = generator(
                        noise
                    )
                optimizer_c.zero_grad(
                    set_to_none=True
                )
            real_score = (

```

```

        critic(real).mean()
    )
    fake_score = (
        critic(fake).mean()
    )
    gp = gradient_penalty(
        critic,
        real,
        fake
    )
    loss_c = (
        fake_score
        - real_score
        + lambda_gp * gp
    )
    loss_c.backward()
    optimizer_c.step()

optimizer_g.zero_grad(
    set_to_none=True
)
noise = torch.randn(
    batch_size,
    LATENT_DIM,
    1,
    1,
    device=device
)
fake = generator(noise)
loss_g = (
    -critic(fake).mean()
)
loss_g.backward()
optimizer_g.step()
g_losses.append(
    loss_g.item()
)
c_losses.append(

```



```

        loss_c.item()
    )
    gp_values.append(
        gp.item()
    )
    gaps.append(
        (
            real_score
            - fake_score
        ).item()
    )
elapsed = (
    time.perf_counter()
    - epoch_start
)
cumulative = (
    time.perf_counter()
    - total_start
)
history[
    "generator_loss"
].append(
    np.mean(g_losses)
)
history[
    "critic_loss"
].append(
    np.mean(c_losses)
)
history[
    "gradient_penalty"
].append(
    np.mean(gp_values)
)
history[
    "wasserstein_gap"
].append(
    np.mean(gaps)
)

```

```

    )
    history[
        "epoch_time"
    ].append(elapsed)
    history[
        "cumulative_time"
    ].append(cumulative)
    if epoch in WGAN_CHECKPOINTS:
        samples = generate_samples(
            generator,
            fixed_noise
        )
        snapshots[epoch] = samples
        state = copy_state(
            generator
        )
        checkpoints[epoch] = state
        torch.save(
            state,
            os.path.join(
                OUTPUT_DIR,
                f"wgan_gp_epoch_{epoch}.pt"
            )
        )
        save_samples(
            samples,
            f"wgan_gp_epoch_{epoch}.png"
        )
    print(
        f"Epoch {epoch:03d}/{epochs} | "
        f"G {history['generator_loss'][-1]:.4f} | "
        f"C {history['critic_loss'][-1]:.4f} | "
        f"GP {history['gradient_penalty'][-1]:.4f} | "
        f"time {elapsed:.1f}s"
    )
total_time = (
    time.perf_counter()
    - total_start

```

```

    )
    return (
        generator,
        critic,
        history,
        snapshots,
        checkpoints,
        total_time
    )

```

This cell defines the main functions needed for training WGAN-GP. It contains two parts: the Gradient Penalty calculation and the complete WGAN-GP training procedure. The purpose of WGAN-GP is to make GAN training more stable by replacing the normal binary classification loss with the Wasserstein objective and adding a Gradient Penalty.

The first function, `gradient_penalty`, calculates the Gradient Penalty between real and generated images. First, a random value called `alpha` is created for each image in the batch. This value is used to create an interpolated image somewhere between a real image and a fake image.

The interpolated images are then passed through the Critic. The code calculates the gradient of the Critic output with respect to these interpolated images. After that, each gradient is flattened into a vector and its L2 norm is calculated.

The Gradient Penalty measures how far this gradient norm is from 1. The expression essentially calculates the average squared difference between the gradient norm and 1. This encourages the Critic to behave more smoothly and helps enforce the Lipschitz constraint required by WGAN. In simple words, it prevents the Critic from changing its output too sharply and helps make training more stable.

The second function, `train_wgan_gp`, defines the complete WGAN-GP training process. The Generator and Critic are first moved to the GPU. Two Adam optimizers are created with a learning rate of 0.0001 and beta values of 0.0 and 0.9. These settings are commonly used for WGAN-GP training.

The history dictionary stores several values during training. These include the Generator loss, Critic loss, Gradient Penalty, Wasserstein gap, training time for each epoch, and total cumulative training time. The snapshots and checkpoints dictionaries are also created to save generated samples and model states at important epochs.

During each training batch, the Critic is updated first. Random noise is passed through the Generator to create fake images. In this step, `torch.no_grad()` is used because the Generator should not be updated while the Critic is being trained.

The Critic then gives scores to both real and fake images. Unlike the DCGAN Discriminator, the Critic does not try to predict probabilities such as 0 or 1. Instead, it gives continuous scores. The goal is for real images to receive higher scores than fake images.

The Critic loss is calculated from the difference between the fake and real scores together with the Gradient Penalty. The Gradient Penalty is multiplied by `lambda_gp`, which is set to 10.0. This penalty helps keep the Critic gradients close to the required range and is an important part of WGAN-GP.

After the Critic is updated, the Generator is trained. A new batch of random noise is used to create fake images, and these images are passed through the Critic. The Generator loss is the negative average Critic score for fake images. Therefore, the Generator tries to increase the Critic score of its generated images, which means it tries to make them look more like real samples.

The parameter `n_critic` controls how many times the Critic is updated for each Generator update. In this implementation, it is set to 1, so both networks are updated once per batch. This keeps the experiment faster while still using the WGAN-GP loss and Gradient Penalty.

The code also records the Wasserstein gap, which is the difference between the average real score and fake score. This value is useful for observing how well the Critic separates real and generated data during training.

At the end of each epoch, the average losses, Gradient Penalty, Wasserstein gap, and training times are stored. At epochs 10, 20, and 30, generated samples and Generator checkpoints are saved. These checkpoints are later used for image visualization and FID evaluation.

Finally, the function prints the Generator loss, Critic loss, Gradient Penalty, and training time for every epoch. After training finishes, it returns the trained Generator and Critic together with the training history, saved samples, checkpoints, and total training time.

This cell does not produce any visible output because it only defines the Gradient Penalty function and the WGAN-GP training function. The actual WGAN-GP training starts in the next cell.

Cell 16:

```
set_seed()
wgan_generator = Generator(
    LATENT_DIM
)
wgan_critic = Critic()
initialize_weights(
```

```

        wgan_generator
    )
    initialize_weights(
        wgan_critic
    )
    print(
        "WGAN Generator parameters:",
        count_parameters(
            wgan_generator
        )
    )
    print(
        "Critic parameters:",
        count_parameters(
            wgan_critic
        )
    )
    (
        wgan_generator,
        wgan_critic,
        wgan_history,
        wgan_snapshots,
        wgan_checkpoints,
        wgan_total_time
    ) = train_wgan_gp(
        wgan_generator,
        wgan_critic,
        train_loader,
        epochs=WGAN_EPOCHS,
        lambda_gp=10.0,
        n_critic=1
    )
    pd.DataFrame(
        wgan_history
    ).to_csv(
        os.path.join(
            OUTPUT_DIR,
            "wgan_gp_history.csv"
        )
    )

```

```

    ),
    index=False
)
print(
    "WGAN-GP total training time:",
    round(
        wgan_total_time / 60,
        2
    ),
    "minutes"
)

```

This cell creates and trains the WGAN-GP model. First, the random seed is set again so that the experiment starts from a reproducible state. Then a new Generator and Critic are created using the architectures defined earlier.

The `initialize_weights` function is applied to both models before training. After that, the code prints the number of trainable parameters. The WGAN Generator has 1,068,928 parameters, which is the same Generator architecture used in DCGAN. The Critic has 662,977 trainable parameters.

Next, the `train_wgan_gp` function is called. The model is trained on the CIFAR-10 training set for 30 epochs. The Gradient Penalty weight is set to 10.0, and the Critic is updated once for each Generator update because `n_critic` is set to 1.

During training, the code records the Generator loss, Critic loss, Gradient Penalty, Wasserstein gap, and training time. It also saves generated samples and model checkpoints at epochs 10, 20, and 30.

After training is finished, the complete WGAN-GP history is converted to a Pandas DataFrame and saved as `wgan_gp_history.csv`. Finally, the total training time is printed.

Output:

```

WGAN Generator parameters: 1068928
Critic parameters: 662977
Epoch 001/30 | G -8.7358 | C -1.5362 | GP 0.1961 | time 17.8s
Epoch 002/30 | G -5.7519 | C 1.5655 | GP 0.0653 | time 17.3s
Epoch 003/30 | G -4.2079 | C 0.5853 | GP 0.0399 | time 17.2s
Epoch 004/30 | G -1.1628 | C 1.0101 | GP 0.0617 | time 17.6s
Epoch 005/30 | G 1.6915 | C 1.9200 | GP 0.0613 | time 16.9s

```

```

Epoch 006/30 | G 5.2532 | C 2.1405 | GP 0.0609 | time 17.0s
Epoch 007/30 | G 8.8325 | C 1.7255 | GP 0.0469 | time 17.1s
Epoch 008/30 | G 12.4313 | C 1.1672 | GP 0.0342 | time 16.8s
Epoch 009/30 | G 13.9978 | C 0.7747 | GP 0.0212 | time 17.7s
Epoch 010/30 | G 14.9002 | C 0.4922 | GP 0.0154 | time 17.4s
Epoch 011/30 | G 15.4727 | C 0.2912 | GP 0.0133 | time 16.8s
Epoch 012/30 | G 15.0897 | C 0.4096 | GP 0.0111 | time 16.9s
Epoch 013/30 | G 17.3251 | C 0.3747 | GP 0.0091 | time 17.0s
Epoch 014/30 | G 19.1494 | C 0.1638 | GP 0.0089 | time 16.9s
Epoch 015/30 | G 19.1840 | C 0.0228 | GP 0.0082 | time 17.6s
Epoch 016/30 | G 19.2441 | C -0.0436 | GP 0.0069 | time 17.1s
Epoch 017/30 | G 18.2860 | C -0.0281 | GP 0.0061 | time 16.8s
Epoch 018/30 | G 17.8051 | C -0.0724 | GP 0.0065 | time 16.9s
Epoch 019/30 | G 16.6600 | C -0.1428 | GP 0.0071 | time 16.8s
Epoch 020/30 | G 16.3444 | C -0.2672 | GP 0.0071 | time 17.0s
Epoch 021/30 | G 16.3279 | C -0.2846 | GP 0.0057 | time 17.6s
Epoch 022/30 | G 17.0149 | C -0.3758 | GP 0.0056 | time 17.0s
Epoch 023/30 | G 17.3944 | C -0.3868 | GP 0.0051 | time 16.7s
Epoch 024/30 | G 16.8369 | C -0.4426 | GP 0.0051 | time 16.7s
Epoch 025/30 | G 15.3099 | C -0.5218 | GP 0.0051 | time 16.9s
Epoch 026/30 | G 14.3422 | C -0.5180 | GP 0.0051 | time 16.9s
Epoch 027/30 | G 14.3844 | C -0.5724 | GP 0.0045 | time 17.6s
Epoch 028/30 | G 14.2402 | C -0.6159 | GP 0.0053 | time 16.9s
Epoch 029/30 | G 14.0154 | C -0.5881 | GP 0.0049 | time 16.8s
Epoch 030/30 | G 13.1196 | C -0.5320 | GP 0.0047 | time 16.9s
WGAN-GP total training time: 8.55 minutes

```

The output first shows the model sizes. The WGAN Generator has 1,068,928 trainable parameters, while the Critic has 662,977 parameters. This means the Generator has the same size as the DCGAN Generator, while the Critic is slightly smaller than the DCGAN Discriminator. The WGAN-GP training was completed for all 30 epochs. The Generator loss behaves very differently from the DCGAN Generator loss. It starts at -8.7358, becomes positive after a few epochs, reaches values around 19 in the middle of training, and then decreases to 13.1196 at epoch 30. These values should not be interpreted in the same way as normal classification loss. In WGAN-GP, the Generator loss is based directly on the Critic score, so negative or large positive values are possible. The important point is to observe the training behavior together with the Critic loss, Gradient Penalty, generated samples, and FID score.

The Critic loss also changes during training. It starts at -1.5362, becomes positive in some early epochs, and later becomes negative again. By epoch 30, the Critic loss is -0.5320. These changes are expected because the Critic is

constantly adapting to the improving Generator. One of the clearest results in this output is the behavior of the Gradient Penalty. It starts at 0.1961 in the first epoch and quickly becomes much smaller. By epoch 10 it is 0.0154, and by epoch 30 it reaches 0.0047. This shows that during training, the gradient behavior of the Critic moved closer to the constraint encouraged by the Gradient Penalty. The training time is also very consistent. Most epochs take around 17 seconds, and the complete 30-epoch WGAN-GP training takes 8.55 minutes. This is much shorter than the 21.68 minutes used for the 100-epoch DCGAN training, although the two models were trained for different numbers of epochs. Overall, the output shows that WGAN-GP completed successfully and that the Gradient Penalty became small and stable during training. However, these loss values alone are not enough to decide whether WGAN-GP produced better images than DCGAN. The next cells use generated samples, stability plots, and FID scores to make this comparison more clearly.

Cell 17:

```
for epoch in WGAN_CHECKPOINTS:

    show_samples(
        wgan_snapshots[epoch],
        f"WGAN-GP Generated Samples - Epoch {epoch}"
    )
wgan_epochs = np.arange(
    1,
    WGAN_EPOCHS + 1
)
plt.figure(figsize=(9, 5))
plt.plot(
    wgan_epochs,
    wgan_history[
        "generator_loss"
    ],
    label="Generator"
)
plt.plot(
    wgan_epochs,
    wgan_history[
        "critic_loss"
    ],
    label="Critic"
```



```

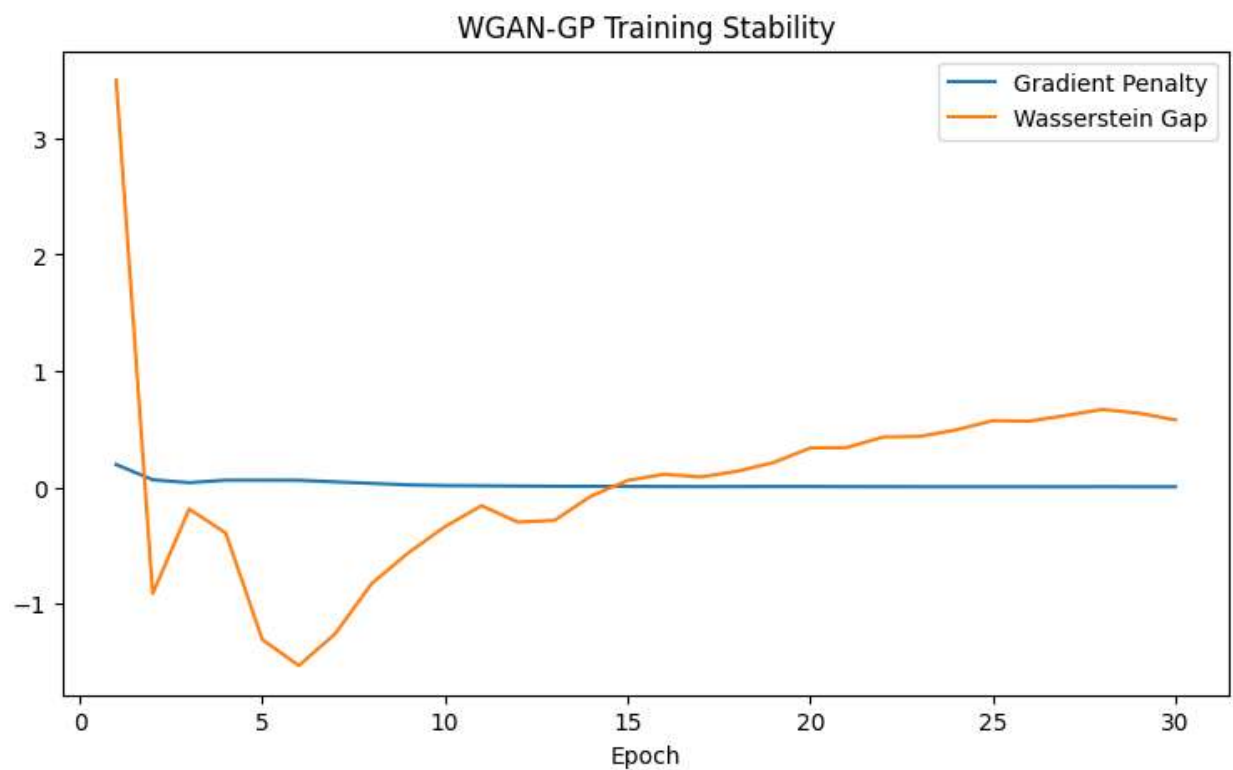
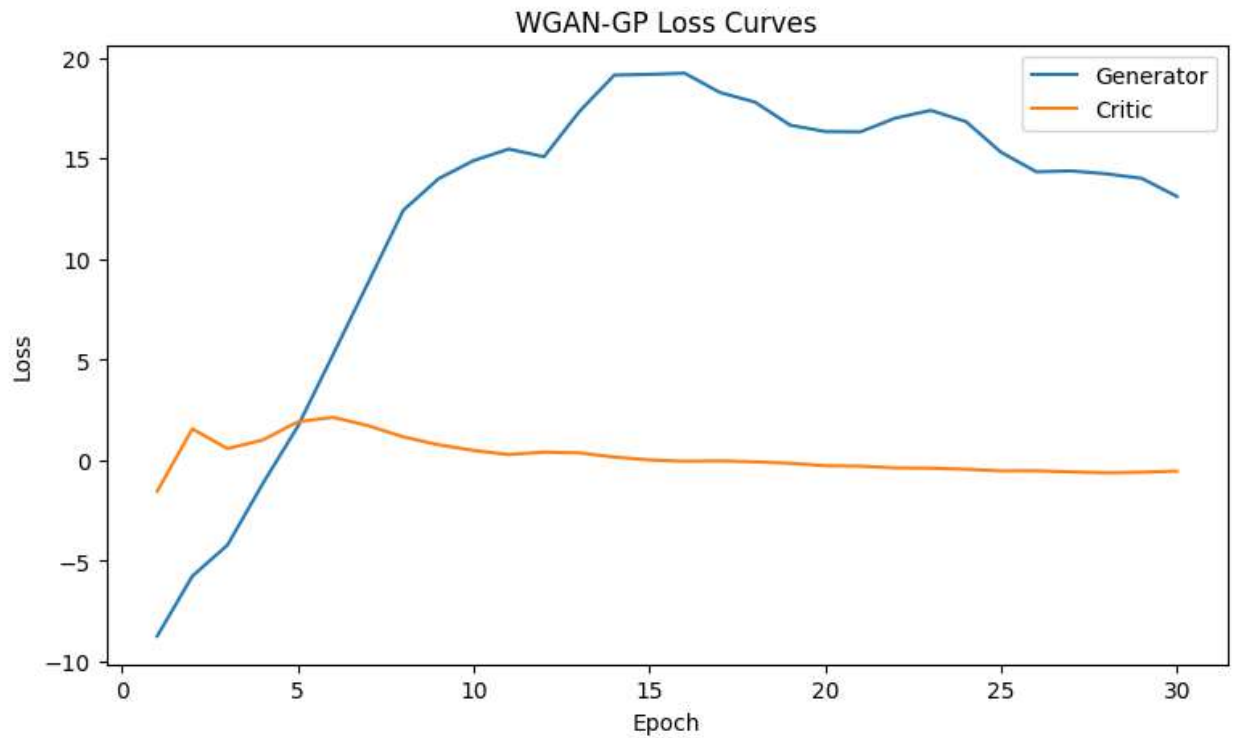
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("WGAN-GP Loss Curves")
plt.legend()
plt.show()
plt.figure(figsize=(9, 5))
plt.plot(
    wgan_epochs,
    wgan_history[
        "gradient_penalty"
    ],
    label="Gradient Penalty"
)
plt.plot(
    wgan_epochs,
    wgan_history[
        "wasserstein_gap"
    ],
    label="Wasserstein Gap"
)
plt.xlabel("Epoch")
plt.title("WGAN-GP Training Stability")
plt.legend()
plt.show()

```

This cell visualizes the main results of the WGAN-GP training. In the first part, the code goes through the saved checkpoints at epochs 10, 20, and 30 and displays the generated samples for each one. Because these samples were generated using the same fixed noise, it is possible to compare them fairly across training time and see how the Generator improved. In the second part, the code creates an array of epoch numbers from 1 to 30. Then it plots the Generator loss and the Critic loss on the same graph. This helps show how the two models changed during WGAN-GP training. In the last part, the code plots the Gradient Penalty and the Wasserstein gap. These two values are important in WGAN-GP because they help us understand training stability. The Gradient Penalty shows how strongly the model is enforcing the gradient constraint, and the Wasserstein gap shows the difference between the Critic scores for real and fake images. This cell does not train the model again. It only uses the saved samples and training history to present the WGAN-GP results in a more clear and visual way.

Output:





The first three outputs show the generated WGAN-GP samples at epochs 10, 20, and 30. In the epoch 10 image, the samples are still very blurry and many of them look repetitive. Some images seem to contain similar dark circular

patterns, and the visual quality is still quite weak. This suggests that at this stage the Generator had not yet learned good image structure.

In the epoch 20 image, the results become more diverse than epoch 10, but they are still blurry and not very clear. Some color regions and rough object-like structures appear, but many samples are still hard to recognize. This shows that the model improved compared with epoch 10, but the image quality was still limited.

In the epoch 30 image, the generated samples look clearly better than in the earlier WGAN-GP checkpoints. The images show more variety, more stable color patterns, and more meaningful structures. Some samples begin to look like animals, vehicles, or natural scenes. Even though many images are still not sharp, the overall quality at epoch 30 is better than at epochs 10 and 20.

The fourth output is the WGAN-GP loss curve. In this graph, the Generator loss starts with a negative value, then increases quickly, reaches its highest region around the middle of training, and later decreases slightly. The Critic loss starts below zero, becomes positive in the early part of training, and then slowly moves back toward negative values. These curves are different from DCGAN losses, and this is normal because WGAN-GP uses a different training objective. The important point is not that the losses should decrease in the same way as classification losses, but that they change in a controlled and stable manner.

The fifth output is the WGAN-GP training stability graph. The Gradient Penalty starts at a higher value and quickly decreases to a value close to zero. This is a good sign because it shows that the gradient constraint is being enforced more effectively as training continues. The Wasserstein gap changes more strongly during training. It starts high, becomes negative in the early epochs, and then gradually increases again. By the end of training, it becomes positive. This shows that the relationship between the Critic scores for real and fake samples changed during training, which is expected as the Generator improved.

Overall, the outputs show that the WGAN-GP training was completed successfully and that the generated images improved from epoch 10 to epoch 30. The plots also show that the training remained controlled, especially because the Gradient Penalty quickly became small and stable. However, based on visual quality alone, the WGAN-GP samples in this run still seem weaker than the best DCGAN samples. This is an important observation and will be discussed later in the final comparison section.

Cell 18:

```
def to_fid_range(images):  
    return (  
        (images + 1) / 2  
    ).clamp(0, 1)
```

```

def prepare_real_fid(
    metric,
    loader,
    n_samples
):
    count = 0
    with torch.no_grad():
        for real, _ in loader:
            if count >= n_samples:
                break
            remaining = (
                n_samples - count
            )
            real = real[
                :remaining
            ].to(
                device,
                non_blocking=True
            )
            metric.update(
                to_fid_range(real),
                real=True
            )

            count += real.size(0)

def fid_for_generator(
    generator,
    metric,
    n_samples
):
    metric.reset()
    generator.eval()
    count = 0
    with torch.no_grad():
        while count < n_samples:
            current = min(
                BATCH_SIZE,
                n_samples - count
            )

```

```

    )
    noise = torch.randn(
        current,
        LATENT_DIM,
        1,
        1,
        device=device
    )
    fake = generator(
        noise
    )
    metric.update(
        to_fid_range(fake),
        real=False
    )
    count += current
return metric.compute().item()
def generator_from_state(state):
    model = Generator(
        LATENT_DIM
    ).to(device)

    model.load_state_dict(
        state
    )
    return model

```

This cell defines the helper functions that are needed to calculate the FID score for the trained GAN models. FID, or Frechet Inception Distance, is used to compare the distribution of generated images with the distribution of real images. A lower FID score generally means that the generated images are closer to the real dataset.

The first function, `to_fid_range`, changes the image values from the range used during GAN training to the range expected by the FID metric. During training, the CIFAR-10 images and the Generator outputs are approximately in the range from -1 to 1. This function transforms them into the range from 0 to 1 and also uses `clamp` to make sure that no value goes outside this range.

The second function, `prepare_real_fid`, prepares the real CIFAR-10 images for the FID calculation. It reads real images from the evaluation DataLoader until the required number of samples is collected. The images are moved to the

GPU, converted to the correct value range, and then added to the FID metric as real samples. The code uses `torch.no_grad()` because no model training or gradient calculation is needed during this evaluation step.

The third function, `fid_for_generator`, calculates the FID score for a specific Generator. First, the metric is reset so that the generated-image information from a previous evaluation does not affect the new result. The Generator is then changed to evaluation mode.

Random latent vectors are generated in batches and passed through the Generator to create fake images. These generated images are converted to the correct value range and added to the FID metric as fake samples. This process continues until the requested number of generated samples has been collected. Finally, `metric.compute()` calculates the FID value and returns it as a normal Python number.

Using batches instead of generating all images at once is useful because it reduces GPU memory usage. It also allows the same function to evaluate different Generator checkpoints in a simple way.

The final function, `generator_from_state`, creates a new Generator and loads a previously saved checkpoint into it. This is important because checkpoints from different epochs were saved during DCGAN and WGAN-GP training. By loading each checkpoint into a new Generator, the FID score can be calculated at different stages of training without training the model again.

Overall, this cell prepares the complete FID evaluation process. It allows real images to be prepared once, different Generator checkpoints to be loaded, and generated images to be compared with the real CIFAR-10 images using the same evaluation method.

This cell does not produce any visible output because it only defines the functions that will be used for FID calculation in the next cell.

Cell 19:

```
fid_metric = FrechetInceptionDistance(  
    feature=2048,  
    normalize=True,  
    reset_real_features=False  
) .to(device)  
prepare_real_fid(  
    fid_metric,  
    fid_loader,  
    FID_SAMPLES  
)
```

```

fid_rows = []
for epoch in DCGAN_CHECKPOINTS:
    model = generator_from_state(
        dcgan_checkpoints[epoch]
    )
    score = fid_for_generator(
        model,
        fid_metric,
        FID_SAMPLES
    )
    fid_rows.append({
        "Model": "DCGAN",
        "Epoch": epoch,
        "Training Minutes":
            dcgan_history[
                "cumulative_time"
            ][epoch - 1] / 60,
        "FID": score
    })
    del model
    torch.cuda.empty_cache()
for epoch in WGAN_CHECKPOINTS:
    model = generator_from_state(
        wgan_checkpoints[epoch]
    )
    score = fid_for_generator(
        model,
        fid_metric,
        FID_SAMPLES
    )
    fid_rows.append({
        "Model": "WGAN-GP",
        "Epoch": epoch,
        "Training Minutes":
            wgan_history[
                "cumulative_time"
            ][epoch - 1] / 60,
        "FID": score
    })

```



```

    })
    del model
    torch.cuda.empty_cache()
fid_results = pd.DataFrame(
    fid_rows
)
fid_results

```

This cell performs the actual FID evaluation for the saved DCGAN and WGAN-GP checkpoints. First, the FrechetInceptionDistance metric is created with 2048 features. The option `normalize=True` is used because the images are provided in the range from 0 to 1. The option `reset_real_features=False` keeps the real-image features in memory, so the same real CIFAR-10 samples can be reused when evaluating several Generator checkpoints.

Next, the `prepare_real_fid` function is called to add 2000 real CIFAR-10 images to the FID metric. After that, an empty list called `fid_rows` is created to store the results.

The code then evaluates the DCGAN checkpoints from epochs 10, 50, and 100. For each checkpoint, a Generator is created and the saved model state is loaded. The FID score is calculated using 2000 generated images, and the model name, epoch number, cumulative training time, and FID score are saved in `fid_rows`. After each evaluation, the temporary model is deleted and unused GPU memory is cleared.

The same process is then repeated for the WGAN-GP checkpoints at epochs 10, 20, and 30. This allows both models to be compared using exactly the same FID evaluation method.

Finally, all stored results are converted into a Pandas DataFrame. This table makes it possible to compare image quality at different stages of training and also relate the FID score to the amount of training time.

Output:

```

Downloading: "https://github.com/toshas/torch-
fidelity/releases/download/v0.2.0/weights-inception-2015-12-05-6726825d.pth"
to /root/.cache/torch/hub/checkpoints/weights-inception-2015-12-05-
6726825d.pth
100%|██████████| 91.2M/91.2M [00:00<00:00, 152MB/s]

```

	Model	Epoch	Training Minutes	FID
0	DCGAN	10	2.228030	151.802063
1	DCGAN	50	10.894854	85.564095
2	DCGAN	100	21.677964	68.649940
3	WGAN-GP	10	2.882213	320.537903
4	WGAN-GP	20	5.714775	210.290939
5	WGAN-GP	30	8.552351	164.105850

The first part of the output shows that the pretrained Inception weights needed for the FID calculation were downloaded successfully. These weights are used by the FID metric to extract image features. This download happens only because the required model was not already available in the Colab environment.

The table shows a clear improvement in the DCGAN results during training. At epoch 10, the DCGAN FID score is 151.802063. At epoch 50, it decreases to 85.564095, and at epoch 100 it reaches 68.649940. Since a lower FID score generally means that the generated images are closer to the real image distribution, this result shows that the quality of the DCGAN samples improved strongly during training.

The WGAN-GP results also improve over time. Its FID score starts at 320.537903 at epoch 10, decreases to 210.290939 at epoch 20, and reaches 164.105850 at epoch 30. This shows that WGAN-GP was learning and producing better samples as training continued.

However, in this experiment, DCGAN achieved much lower FID scores than WGAN-GP. The final DCGAN FID is 68.649940, while the final WGAN-GP FID is 164.105850. This means that, based on this evaluation, the final DCGAN samples were closer to the real CIFAR-10 distribution.

The training-time values also show an important difference. DCGAN reached an FID of 151.802063 after about 2.23 minutes and improved to 68.649940 after about 21.68 minutes. WGAN-GP reached an FID of 164.105850 after about 8.55 minutes. So WGAN-GP improved during its shorter training period, but it did not reach the same sample quality as DCGAN in this run.

Overall, the table shows that both models improved as training continued, but DCGAN performed better according to FID in this particular experiment. This result is useful because it shows that WGAN-GP is not automatically better in every short experiment. Its main advantage is related to the training objective and stability, while the final sample quality also depends on the architecture, number of epochs, hyperparameters, and training setup.

Cell 20:

```
fid_results.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "fid_results.csv"
    ),
    index=False
)
plt.figure(figsize=(9, 5))
for model_name in [
    "DCGAN",
    "WGAN-GP"
]:
    data = fid_results[
        fid_results[
            "Model"
        ] == model_name
    ]
    plt.plot(
        data[
            "Training Minutes"
        ],
        data["FID"],
        marker="o",
        label=model_name
    )
plt.xlabel("Training Time (minutes)")
plt.ylabel("FID")
plt.title("Convergence Speed")
plt.legend()
plt.show()
```

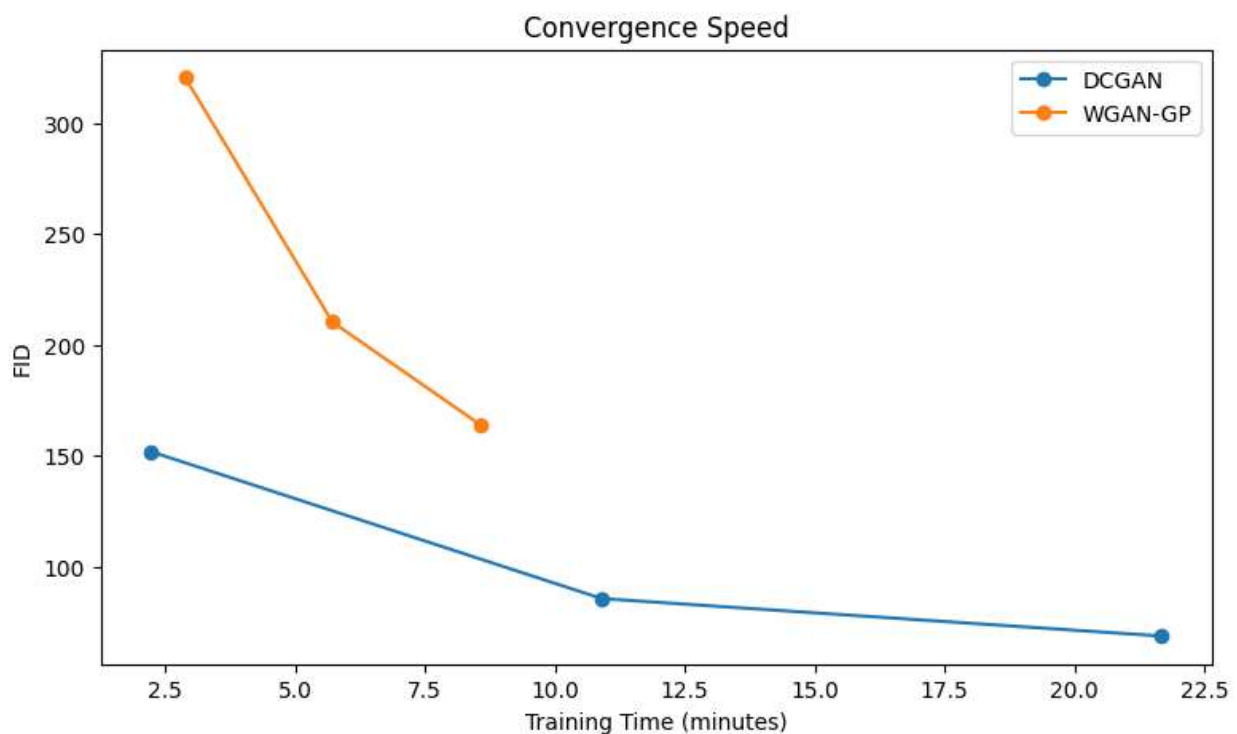
This cell saves the FID results and creates a graph to compare the convergence speed of DCGAN and WGAN-GP. First, the `fid_results` table from the previous cell is saved as a CSV file called `fid_results.csv` inside the output folder. This keeps the FID scores and training times available for later analysis.

After that, the code creates a new figure and loops over the two models, DCGAN and WGAN-GP. For each model, it selects the corresponding rows from the FID results table and plots FID score against cumulative training time.

The x-axis shows training time in minutes, while the y-axis shows FID score. Since a lower FID score is better, a model that reaches lower FID values in less time can be considered to have faster practical convergence in terms of sample quality.

This graph is useful because the two models were trained for different numbers of epochs. Instead of comparing only epoch numbers, the plot compares them using actual training time, which gives a more fair view of how quickly the generated image quality improves.

Output:



The convergence graph shows that the FID score decreases for both DCGAN and WGAN-GP as training time increases. This means that both models improve their generated image quality during training.

For DCGAN, the FID starts at about 151.8 after around 2.23 minutes, decreases to about 85.6 after 10.89 minutes, and reaches about 68.6 after 21.68 minutes. The downward trend is clear and consistent, showing that DCGAN continued improving throughout the full training process.

WGAN-GP also shows a clear downward trend. Its FID decreases from about 320.5 after 2.88 minutes to about 210.3 after 5.71 minutes, and finally to about 164.1 after 8.55 minutes. This shows that WGAN-GP was also learning and improving over time.

However, the DCGAN curve stays below the WGAN-GP curve in the measured points. Since lower FID is better, this means that DCGAN produced better sample quality at the evaluated training times in this experiment. Even the DCGAN result at around 2.23 minutes is slightly better than the final WGAN-GP result after about 8.55 minutes.

This result suggests that DCGAN converged faster in terms of FID in this specific setup. WGAN-GP showed improvement and stable progress, but it did not reach the same image quality within the 30 epochs used here.

Overall, this graph gives a clear visual comparison of convergence speed. In this experiment, DCGAN achieved lower FID values more quickly, while WGAN-GP improved steadily but remained behind in sample quality during the available training time.

Cell 21:

```
def relative_loss_variation(
    values,
    last_n=10
):
    values = np.array(
        values[-last_n:]
    )
    changes = np.abs(
        np.diff(values)
    ).mean()
    scale = (
        np.abs(values).mean()
        + 1e-8
    )
    return changes / scale
dcgan_final_fid = (
    fid_results[
```

```

        (
            fid_results["Model"]
            == "DCGAN"
        )
        &
        (
            fid_results["Epoch"]
            == 100
        )
    ]["FID"].iloc[0]
)
wgan_final_fid = (
    fid_results[
        (
            fid_results["Model"]
            == "WGAN-GP"
        )
        &
        (
            fid_results["Epoch"]
            == WGAN_EPOCHS
        )
    ]["FID"].iloc[0]
)
comparison_results = pd.DataFrame([
    {
        "Model": "DCGAN",
        "Epochs": DCGAN_EPOCHS,
        "Final FID":
            dcgan_final_fid,
        "Average Epoch Time":
            np.mean(
                dcgan_history[
                    "epoch_time"
                ]
            ),
        "Total Training Minutes":
            dcgan_total_time / 60,
    }
])

```

```

    "Generator Loss Variation":
        relative_loss_variation(
            dcgan_history[
                "generator_loss"
            ]
        ),
    "Adversary Loss Variation":
        relative_loss_variation(
            dcgan_history[
                "discriminator_loss"
            ]
        )
},
{
    "Model": "WGAN-GP",
    "Epochs": WGAN_EPOCHS,
    "Final FID":
        wgan_final_fid,
    "Average Epoch Time":
        np.mean(
            wgan_history[
                "epoch_time"
            ]
        ),
    "Total Training Minutes":
        wgan_total_time / 60,
    "Generator Loss Variation":
        relative_loss_variation(
            wgan_history[
                "generator_loss"
            ]
        ),
    "Adversary Loss Variation":
        relative_loss_variation(
            wgan_history[
                "critic_loss"
            ]
        )
}

```

```

    }
])
comparison_results

```

This cell creates a final numerical comparison between DCGAN and WGAN-GP. The goal is to summarize the most important results of the two models in one table, including sample quality, training time, and the amount of change in their losses near the end of training. First, the `relative_loss_variation` function is defined. This function looks at the last 10 recorded loss values of a model. It calculates how much the loss changes from one epoch to the next and then divides this value by the average size of the loss. This makes the result relative to the scale of the loss values. A smaller value means that the loss changed less during the final epochs, while a larger value means that the loss had more variation. The code then extracts the final FID score for each model. For DCGAN, the FID value from epoch 100 is selected. For WGAN-GP, the FID value from its final epoch, which is epoch 30, is selected.

After that, a Pandas DataFrame called `comparison_results` is created. For each model, the table stores the number of training epochs, final FID score, average time required for one epoch, total training time, Generator loss variation, and the variation of the opposing network loss. For DCGAN, the opposing network is the Discriminator, while for WGAN-GP it is the Critic.

This table gives a compact summary of the two experiments and makes it easier to compare their final behavior. However, the two models were trained for different numbers of epochs, so the total training time should not be interpreted alone. The average epoch time and FID values are also important when comparing the models.

Output:

Model	Epochs	Final FID	Average Epoch Time	Total Training Minutes	Generator Loss Variation	Adversary Loss Variation	
0	DCGAN	100	68.64994	13.000161	21.679097	0.013170	0.006608
1	WGAN-GP	30	164.10585	17.100573	8.553563	0.039407	0.097123

The final comparison table shows several differences between DCGAN and WGAN-GP in this experiment.

DCGAN was trained for 100 epochs and achieved a final FID score of 68.64994. WGAN-GP was trained for 30 epochs and achieved a final FID score of 164.10585. Since a lower FID score is better, DCGAN produced better final sample quality according to this metric.

The average training time for one DCGAN epoch was about 13.00 seconds, while one WGAN-GP epoch took about 17.10 seconds. This makes sense because WGAN-GP has the additional Gradient Penalty calculation, which requires extra gradient operations during every training batch. The total DCGAN training time was about 21.68 minutes, while WGAN-GP required about 8.55 minutes. However, this does not mean that WGAN-GP was computationally faster overall. It was trained for only 30 epochs compared with 100 epochs for DCGAN. In fact, the average epoch time shows that each WGAN-GP epoch was slightly slower.

The Generator Loss Variation for DCGAN is 0.013170, while the value for WGAN-GP is 0.039407. Based on this specific measurement, the DCGAN Generator changed less during its final 10 epochs. A similar difference can be seen in the opposing network losses. The DCGAN Discriminator Loss Variation is only 0.006608, while the WGAN-GP Critic Loss Variation is 0.097123. Therefore, according to this simple relative variation metric, the final DCGAN losses were more stable than the WGAN-GP losses in this particular experiment. This result is interesting because WGAN-GP is generally designed to improve GAN training stability. However, this experiment does not show lower loss variation for WGAN-GP. One possible reason is that WGAN-GP was trained for only 30 epochs, while DCGAN had 100 epochs and had more time to reach a stable later stage. Also, the loss functions of DCGAN and WGAN-GP have different meanings and different numerical scales, so their loss values should not be compared directly.

Overall, the final table shows that DCGAN performed better in this specific experiment in terms of FID score and final relative loss variation. WGAN-GP still showed successful learning and a controlled Gradient Penalty, but it did not outperform DCGAN under the training settings used here.

Cell 22:

```
comparison_results.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "final_comparison.csv"
    ),
    index=False
)
final_dcgan_samples = (
    generate_samples(
        dcgan_generator,
```

```

        fixed_noise
    )
)
final_wgan_samples = (
    generate_samples(
        wgan_generator,
        fixed_noise
    )
)
show_samples(
    final_dcgan_samples,
    "DCGAN Final Samples"
)
show_samples(
    final_wgan_samples,
    "WGAN-GP Final Samples"
)
save_samples(
    final_dcgan_samples,
    "dcgan_final_samples.png"
)
save_samples(
    final_wgan_samples,
    "wgan_gp_final_samples.png"
)
print(
    "All important results saved in:"
)
print(
    OUTPUT_DIR
)

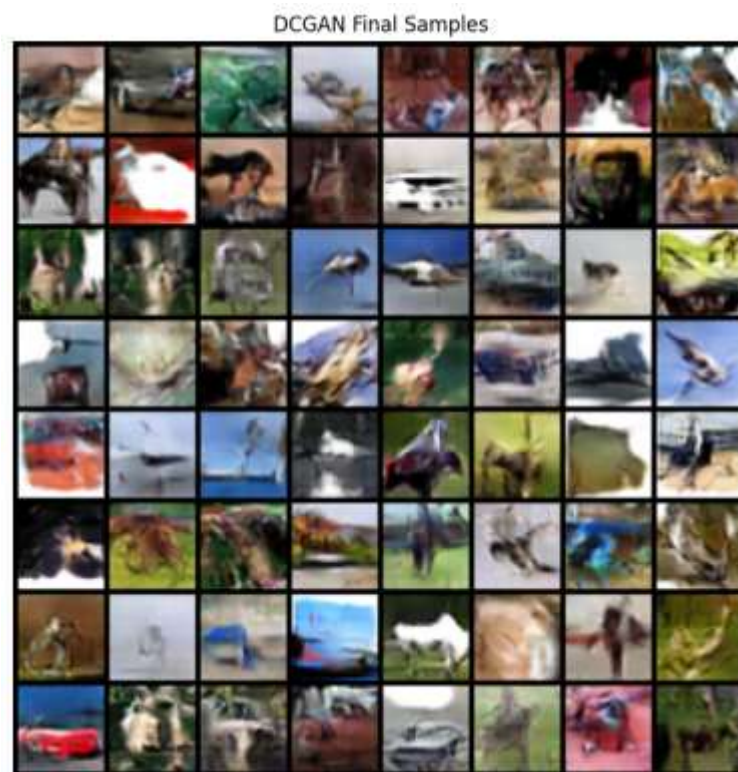
```

This cell is the final step of Problem 4. Its main purpose is to save the final comparison results, generate the final sample images from both trained models, display them, and save them in the output folder. First, the `comparison_results` table from the previous cell is saved as a CSV file called `final_comparison.csv`. This file contains the main summary of the comparison between DCGAN and WGAN-GP, including the final FID score, training time, and loss variation values. After that, the code generates final samples from both trained models using the same

fixed_noise. The DCGAN Generator produces one final set of samples, and the WGAN-GP Generator produces another final set. Using the same fixed noise makes the final comparison fair, because both models are tested with the same latent inputs. Then the code displays the final generated images for both models. This helps compare their final visual quality directly. After showing the images, the same sample grids are also saved as `dcgan_final_samples.png` and `wgan_gp_final_samples.png` inside the output folder.

At the end, the code prints the path of the output folder. This makes it clear where the important results were intended to be stored.

Output:





```
All important results saved in:  
/content/drive/MyDrive/dl_pset4_results
```

This cell produces two final image grids and one text output. The two image grids show the final generated samples from DCGAN and WGAN-GP.

The DCGAN final samples look more clear, more structured, and more visually meaningful. Many images contain shapes that look closer to CIFAR-10 categories such as animals, vehicles, and natural scenes. The colors also look more natural, and the overall diversity is good. Even though the images are still not perfect, the DCGAN output looks stronger and more recognizable. And the WGAN-GP final samples also show some variety, but they are more blurry and less clear than the DCGAN samples. Some images have reasonable color structure, but many samples are still difficult to recognize. Compared with DCGAN, the WGAN-GP results in this experiment look weaker in final visual quality. This visual comparison is consistent with the earlier FID results. DCGAN had a much lower final FID score than WGAN-GP, and the final images also support that result. In this run, DCGAN gave the better final output.

The printed message shows the output folder path as `/content/drive/MyDrive/dl_pset4_results`. This means the notebook was designed to save all important results in that folder. Because there was a saving issue in Colab during this problem, this printed path is still useful because it shows the intended location for storing the final files. Overall, this cell successfully completes the experiment by showing the final model outputs and preparing the final results for documentation and comparison.

Analysis Questions

1. What architectural or training modifications most effectively prevent mode collapse?

Mode collapse happens when the Generator produces only a small number of similar outputs instead of learning the full diversity of the real dataset. In our DCGAN experiment, there was no strong mode collapse at the end of training. The generated samples before Feature Matching already showed different colors, objects, and backgrounds. The diversity measurements also supported this observation. Before Feature Matching, the Pixel Std was 0.500120, the Mean Pairwise Distance was 9.193496, and the Mean Cosine Similarity was 0.072287. These values show that the generated images were not all identical or extremely similar. In this experiment, Feature Matching was used as the method for reducing possible mode collapse. Instead of training the Generator only to fool the Discriminator, Feature Matching also encourages the internal features of generated images to become similar to the features of real images. The idea is that the Generator should learn more general characteristics of the real data instead of finding only a few outputs that can fool the Discriminator.

However, the results after Feature Matching were mixed. The Pixel Std decreased from 0.500120 to 0.379152, and the Mean Pairwise Distance decreased from 9.193496 to 6.798036. These two results suggest that the overall variation between generated images became smaller. On the other hand, the Mean Cosine Similarity decreased from 0.072287 to 0.054089, which means that the directions of the generated image representations became slightly less similar. Therefore, Feature Matching did not clearly improve all diversity measurements in this particular experiment. This shows that there is no single method that always solves mode collapse. Feature Matching can help because it gives the Generator a richer training target, but its effect depends on the model and training settings. Other useful modifications include keeping a good balance between the Generator and Discriminator, using suitable learning rates, adding regularization, and avoiding a Discriminator that becomes too strong too early. Methods such as minibatch discrimination can also help because they allow the Discriminator to compare samples inside the same batch and detect when the Generator produces too many similar images.

Overall, the most useful approach is to combine stable training settings with methods that directly encourage diversity. In our experiment, Feature Matching was successfully implemented, but the original DCGAN already had reasonable diversity, so the improvement was not clear in every metric. This is an important result because it shows that a method designed to reduce mode collapse does not automatically improve every experiment.

2. Why does WGAN-GP tend to be more stable than DCGAN? Explain the theoretical motivation behind the Wasserstein loss and gradient penalty.

DCGAN and WGAN-GP use different ideas for training the Generator and the network that evaluates generated images. In DCGAN, the Discriminator performs a binary classification task and tries to separate real images from fake

images. The Generator then tries to make its fake images look real to the Discriminator. This approach can become unstable when the Discriminator becomes too strong, because the Generator may receive weak or unhelpful gradients.

WGAN uses the Wasserstein loss instead of the normal binary classification loss. The Critic does not directly predict whether an image is real or fake. Instead, it gives continuous scores to real and generated images. The training objective tries to increase the difference between the scores of real and fake data. The main idea is that the Wasserstein distance gives a smoother way to measure how different the generated distribution is from the real distribution. This can provide more useful gradients to the Generator, especially when the generated images are still far from the real data. Because of this, WGAN training can continue to give useful information to the Generator even when the generated samples are poor. This is one reason why WGAN is theoretically expected to have more stable training behavior than a standard GAN.

For the Wasserstein idea to work correctly, the Critic must satisfy a Lipschitz constraint. WGAN-GP handles this by adding a Gradient Penalty. The Gradient Penalty checks the gradient of the Critic on points between real and fake images and encourages the gradient norm to stay close to 1. This prevents the Critic from changing too sharply and helps keep its behavior smooth. In our experiment, the Gradient Penalty showed a clear stable trend. It started at 0.1961 and decreased quickly during training. At epoch 10 it was 0.0154, and by epoch 30 it reached 0.0047. This shows that the gradient constraint became much better satisfied as training continued. The WGAN-GP generated images also improved from epoch 10 to epoch 30, and its FID decreased from 320.537903 to 210.290939 and finally to 164.105850. However, WGAN-GP did not outperform DCGAN in our final results. DCGAN achieved a much better final FID of 68.649940, while the final WGAN-GP FID was 164.105850. The final loss-variation measurements also showed smaller values for DCGAN in this experiment. The DCGAN Generator Loss Variation was 0.013170, compared with 0.039407 for WGAN-GP. The opposing network loss variation was also smaller for DCGAN.

This result does not mean that the theoretical idea of WGAN-GP is wrong. The two models were trained under different training budgets. DCGAN was trained for 100 epochs, while WGAN-GP was trained for only 30 epochs. Also, the losses in DCGAN and WGAN-GP have different meanings and numerical scales, so their raw loss behavior cannot be compared directly. WGAN-GP also used only one Critic update for each Generator update in this experiment, mainly to keep the training time reasonable.

Therefore, WGAN-GP tends to be more stable in theory because the Wasserstein loss gives a smoother and more meaningful training signal, while the Gradient Penalty controls the Critic and enforces the required Lipschitz constraint. In our actual experiment, these mechanisms worked correctly, especially as shown by the decreasing Gradient Penalty, but the shorter WGAN-GP training did not produce better final image quality than the 100-epoch DCGAN.

Problem 5: Transformer Architecture – Depth and Understanding

Problem 5 focuses on understanding how Transformers work with sequential information even though they do not use recurrence like RNNs or LSTMs. Instead of processing the input one step at a time, a Transformer uses the attention mechanism to directly compare different positions in the input sequence. This allows the model to learn relationships between tokens even when they are far from each other. In this problem, the main goal is not only to train a Transformer, but also to look inside the model and understand what different attention heads and deeper layers are learning.

For the main experiment, I implemented a small GPT-style Transformer for character-level text modeling. The model contains 4 Transformer layers, 4 attention heads, and an embedding dimension of 128, exactly as required in the problem. I used the Tiny Shakespeare text dataset because it is small enough to train quickly in Google Colab while still providing a real sequential language modeling task. The text is converted into character tokens, and the model learns to predict the next character based on the previous characters. After training, I also generated new text from the model to check whether it had learned some basic patterns from the dataset.

One important part of this problem is the attention mechanism. I extracted the attention weights from the Transformer and visualized them as heatmaps. I compared the attention patterns of different heads and different layers to see whether some heads focus on nearby tokens while others attend to more distant positions. I also calculated simple statistics such as mean attention distance, attention entropy, self-attention, and attention to the previous token. These measurements help make the comparison between attention heads more clear instead of relying only on visual inspection of the heatmaps.

Another experiment studies the effect of context length. I trained smaller versions of the same Transformer using context lengths of 32, 64, and 128. For each context size, I measured validation loss, perplexity, training time per step, GPU memory usage, and number of parameters. The purpose of this experiment is to understand whether giving the model more previous tokens always improves performance. It also shows the computational cost of longer context lengths, because self-attention becomes more expensive when the sequence becomes longer.

The last part of the problem asks about spatial locality and specifically mentions a token that comes from a top-left image patch. Since the main GPT experiment uses text and does not contain image patches, I created a second small Transformer experiment using the Digits image dataset. Each 8×8 image is divided into small patches, and these patches are treated as tokens. A small Patch Transformer with the same general depth and attention settings is then trained to classify the digit images. This model reached a test accuracy of about 95.56%, so its internal representations could be studied more meaningfully.

Using this Patch Transformer, I measured how strongly tokens in deeper layers still correspond to their original image patches. I also examined the top-left patch separately. To study the role of positional information, I compared the same measurements with positional encoding enabled and disabled. This experiment is useful because it shows that

deeper Transformer tokens combine information from different locations, but they can still keep some connection to their original position. It also helps us study whether positional encoding is the only reason for this locality or whether other parts of the architecture, such as residual connections and patch content, also contribute.

One challenge in this problem was that the first parts are based on a text Transformer, while the final experiment refers to image patches. Instead of answering that part only theoretically, I used a small image-based Transformer so that the spatial locality question could also be tested experimentally. This made it possible to study attention patterns, context length, deeper-layer locality, and positional encoding in a practical way while keeping the total training time reasonable.

Overall, this problem helps us understand Transformers from several different points of view. The language modeling experiment shows how a GPT-style Transformer learns sequential patterns, the attention analysis shows how different heads behave, the context-length experiment shows the trade-off between performance and computational cost, and the patch experiment shows how positional and spatial information changes across deeper layers. Together, these experiments provide a clearer practical understanding of how Transformer models process and preserve information.

Explain the code of problem set 5:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
import time
import random
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.utils.data import TensorDataset
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
```



```

SEED = 42
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
set_seed()
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
if device.type != "cuda":
    raise RuntimeError(
        "GPU is not active. Enable GPU in Colab."
    )
torch.set_float32_matmul_precision(
    "high"
)
print("Device:", device)
print(
    "GPU:",
    torch.cuda.get_device_name(0)
)

```

This cell prepares the main libraries and settings that are needed for Problem 5. First, it imports general Python libraries such as time, random, and urllib.request. These libraries are later used for measuring training time, controlling randomness, and downloading the text dataset.

NumPy and Pandas are imported for working with numerical values and tables. Matplotlib is used for creating graphs and attention heatmaps. PyTorch is imported for building and training the Transformer models. The DataLoader and TensorDataset classes are also imported because they are needed later for preparing the image dataset used in the spatial locality experiment.

The load_digits function and train_test_split are imported from scikit-learn. These are used later in the second part of the problem, where a small image Transformer is trained on the Digits dataset to study image patches and positional information.

The random seed is set to 42. The `set_seed` function applies the same seed to Python, NumPy, and PyTorch. This helps make the experiments more reproducible, because the random operations start from the same initial condition each time the notebook is run.

After that, the code checks whether CUDA is available and selects the GPU as the main device. If a GPU is not active, the code stops and shows an error message. This is useful because the Transformer experiments are faster when they run on the GPU.

The code also sets the matrix multiplication precision to high. This allows PyTorch to use an efficient numerical setting for matrix operations on the GPU, which is useful for Transformer models because they perform many matrix multiplications.

Finally, the selected device and GPU name are printed so that we can confirm that the notebook is using the correct hardware.

Output:

```
Device: cuda
GPU: Tesla T4
```

The output shows that CUDA is active, so the notebook is using the GPU instead of the CPU. The detected GPU is a Tesla T4. This means that the environment is correctly prepared for the Transformer experiments and the following training steps can run much faster than they would on the CPU.

Cell 2:

```
url = (
    "https://raw.githubusercontent.com/"
    "karpathy/char-rnn/master/"
    "data/tinyshakespeare/input.txt"
)
text = urllib.request.urlopen(
    url
).read().decode(
    "utf-8"
)
chars = sorted(
    list(set(text))
```

```

)
vocab_size = len(chars)
stoi = {
    ch: i
    for i, ch in enumerate(chars)
}
itos = {
    i: ch
    for i, ch in enumerate(chars)
}
def encode(s):

    return [
        stoi[c]
        for c in s
    ]
def decode(values):

    return "".join(
        itos[i]
        for i in values
    )
data = torch.tensor(
    encode(text),
    dtype=torch.long
)
split = int(
    0.9 * len(data)
)
train_data = data[:split]

val_data = data[split:]
print(
    "Text length:",
    len(text)
)
print(
    "Vocabulary size:",

```

```

vocab_size
)
print(
    "Training characters:",
    len(train_data)
)
print(
    "Validation characters:",
    len(val_data)
)

```

This cell downloads and prepares the text dataset that will be used to train the GPT-style Transformer. The dataset is Tiny Shakespeare, which contains text from Shakespeare's plays. It is a small text dataset, so it is suitable for a simple language modeling experiment and can be trained quickly in Google Colab.

The text is downloaded directly from the given URL and decoded into normal UTF-8 text. After that, the code finds all unique characters that appear in the dataset. These characters form the vocabulary of the model.

The variable `vocab_size` stores the total number of unique characters. Two dictionaries are then created. The `stoi` dictionary converts each character into a numerical index, while the `itos` dictionary performs the opposite operation and converts an index back into its original character.

The `encode` function changes a text string into a list of numerical token IDs. The `decode` function does the reverse and converts a list of token IDs back into readable text. These two functions are useful because neural networks work with numbers instead of raw characters.

After encoding the full text, the data is converted into a PyTorch tensor with integer values. Then 90% of the text is used as training data and the remaining 10% is used as validation data. This allows the model to learn from one part of the text and later be evaluated on another part that was not directly used for training.

Finally, the code prints the total text length, vocabulary size, number of training characters, and number of validation characters.

Output:

```

Text length: 1115394
Vocabulary size: 65
Training characters: 1003854

```

```
Validation characters: 111540
```

The output shows that the Tiny Shakespeare dataset contains 1,115,394 characters in total. The vocabulary contains 65 unique characters, which includes letters, punctuation marks, spaces, and other symbols used in the text.

The training set contains 1,003,854 characters, while the validation set contains 111,540 characters. This confirms that the dataset was divided approximately into 90% training data and 10% validation data.

Overall, this output shows that the text dataset was downloaded and prepared correctly and is ready to be used for training the character-level Transformer model.

Cell 3:

```
BATCH_SIZE = 64
BLOCK_SIZE = 128
EMBED_DIM = 128
N_HEADS = 4
N_LAYERS = 4
DROPOUT = 0.1
TRAIN_STEPS = 600
EVAL_INTERVAL = 100
EVAL_ITERS = 15

def get_batch(
    split_name,
    block_size=BLOCK_SIZE,
    batch_size=BATCH_SIZE
):
    source = (
        train_data
        if split_name == "train"
        else val_data
    )
    positions = torch.randint(
        0,
        len(source)
        - block_size
        - 1,
        (batch_size,)
```

```

    )
    x = torch.stack([
        source[
            i:i + block_size
        ]
        for i in positions
    ])
    y = torch.stack([
        source[
            i + 1:
            i + block_size + 1
        ]
        for i in positions
    ])
    return (
        x.to(device),
        y.to(device)
    )
x, y = get_batch(
    "train"
)
print(
    "Input shape:",
    x.shape
)
print(
    "Target shape:",
    y.shape
)

```

This cell defines the main settings used for training the GPT-style Transformer. The batch size is set to 64, which means that the model processes 64 text sequences at the same time during training.

The block size is set to 128. This means that each input sequence contains 128 characters. The embedding dimension is also set to 128, while the Transformer uses 4 attention heads and 4 layers. These values match the main architecture required in the problem.

The dropout value is set to 0.1 to add a small amount of regularization during training. The model is trained for 600 steps. Every 100 steps, the training and validation losses are evaluated. The evaluation is repeated for 15 mini-batches so that the reported loss is more stable and not based on only one batch.

The `get_batch` function creates training or validation batches. First, it selects either `train_data` or `val_data` depending on the value of `split_name`. Then it chooses random starting positions from the selected text.

For every starting position, `x` contains a sequence of 128 characters. The target sequence `y` contains the same part of the text shifted one character forward. This means that for every position in `x`, the model learns to predict the next character in `y`.

At the end of the cell, one training batch is created and the shapes of `x` and `y` are printed. This is a simple check to make sure that the batching process works correctly before training starts.

Output:

```
Input shape: torch.Size([64, 128])  
Target shape: torch.Size([64, 128])
```

The output shows that both the input and target tensors have shape `[64, 128]`.

The first number, 64, is the batch size. This means that one batch contains 64 different text sequences.

The second number, 128, is the context length. Each sequence contains 128 character tokens.

The input and target shapes are the same because the model predicts one next character for every input position. The target sequence is simply shifted by one position compared with the input sequence.

This confirms that the data batching process is working correctly and the text is ready to be passed into the Transformer model.

Cell 4:

```
class CausalSelfAttention(nn.Module):  
    def __init__(  
        self,  
        dim,  
        heads,  
        max_context,  
        dropout
```

```

):
    super().__init__()
    self.heads = heads
    self.head_dim = (
        dim // heads
    )
    self.qkv = nn.Linear(
        dim,
        3 * dim
    )
    self.projection = nn.Linear(
        dim,
        dim
    )
    self.attention_dropout = (
        nn.Dropout(dropout)
    )
    self.output_dropout = (
        nn.Dropout(dropout)
    )
    mask = torch.tril(
        torch.ones(
            max_context,
            max_context
        )
    )
    self.register_buffer(
        "mask",
        mask.view(
            1,
            1,
            max_context,
            max_context
        )
    )

    def forward(
        self,
        x,

```



```

        return_attention=False
    ):
        batch_size, length, dim = (
            x.shape
        )
        qkv = self.qkv(x)
        qkv = qkv.view(
            batch_size,
            length,
            3,
            self.heads,
            self.head_dim
        )
        qkv = qkv.permute(
            2,
            0,
            3,
            1,
            4
        )
        q = qkv[0]
        k = qkv[1]
        v = qkv[2]
        scores = (
            q
            @ k.transpose(-2, -1)
        )
        scores = scores / (
            self.head_dim ** 0.5
        )
        scores = scores.masked_fill(
            self.mask[
                :,
                :,
                :length,
                :length
            ] == 0,
            float("-inf")

```

```

    )
    weights = F.softmax(
        scores,
        dim=-1
    )
    weights = (
        self.attention_dropout(
            weights
        )
    )
    output = weights @ v
    output = output.transpose(
        1,
        2
    ).contiguous()
    output = output.view(
        batch_size,
        length,
        dim
    )
    output = self.projection(
        output
    )
    output = self.output_dropout(
        output
    )
    if return_attention:
        return output, weights
    return output, None
class FeedForward(nn.Module):
    def __init__(
        self,
        dim,
        dropout
    ):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(

```

```

        dim,
        4 * dim
    ),
    nn.GELU(),
    nn.Linear(
        4 * dim,
        dim
    ),
    nn.Dropout(
        dropout
    )
)

def forward(self, x):
    return self.net(x)

class TransformerBlock(nn.Module):
    def __init__(
        self,
        dim,
        heads,
        max_context,
        dropout
    ):
        super().__init__()

        self.norm1 = nn.LayerNorm(
            dim
        )

        self.norm2 = nn.LayerNorm(
            dim
        )

        self.attention = (
            CausalSelfAttention(
                dim,
                heads,
                max_context,
                dropout
            )
        )

```

```

        self.feed_forward = (
            FeedForward(
                dim,
                dropout
            )
        )
    def forward(
        self,
        x,
        return_attention=False
    ):
        attention_output, weights = (
            self.attention(
                self.norm1(x),
                return_attention
            )
        )
        x = x + attention_output
        x = x + self.feed_forward(
            self.norm2(x)
        )
        return x, weights
class SmallGPT(nn.Module):
    def __init__(
        self,
        vocab_size,
        max_context,
        dim=128,
        heads=4,
        layers=4,
        dropout=0.1
    ):
        super().__init__()

        self.max_context = (
            max_context
        )

        self.token_embedding = (

```

```

        nn.Embedding(
            vocab_size,
            dim
        )
    )
    self.position_embedding = (
        nn.Embedding(
            max_context,
            dim
        )
    )
    self.blocks = nn.ModuleList([
        TransformerBlock(
            dim,
            heads,
            max_context,
            dropout
        )
        for _ in range(layers)
    ])
    self.final_norm = (
        nn.LayerNorm(dim)
    )
    self.output = nn.Linear(
        dim,
        vocab_size
    )
def forward(
    self,
    tokens,
    targets=None,
    return_attention=False
):
    batch_size, length = (
        tokens.shape
    )
    positions = torch.arange(
        length,

```

```

        device=tokens.device
    )
    x = (
        self.token_embedding(
            tokens
        )
        +
        self.position_embedding(
            positions
        )
    )
    attentions = []
    hidden_states = []
    for block in self.blocks:
        x, weights = block(
            x,
            return_attention
        )
        hidden_states.append(x)
        if return_attention:
            attentions.append(
                weights
            )
    x = self.final_norm(x)
    logits = self.output(x)
    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.reshape(
                -1,
                logits.size(-1)
            ),
            targets.reshape(-1)
        )
    return (
        logits,
        loss,
        attentions,

```

```

        hidden_states
    )
def count_parameters(model):
    return sum(
        p.numel()
        for p in model.parameters()
        if p.requires_grad
    )

```

This cell defines the main architecture of the GPT-style Transformer. It includes the causal self-attention mechanism, the feed-forward network, the Transformer block, and finally the complete SmallGPT model.

The first class is `CausalSelfAttention`. This part is responsible for the attention mechanism. It receives the input representation and creates Query, Key, and Value vectors using one linear layer. Because the model uses 4 attention heads, the embedding dimension is divided between the heads so that each head can learn a different attention pattern.

The attention scores are calculated by multiplying the Query vectors by the transposed Key vectors. The scores are then divided by the square root of the head dimension. This scaling helps keep the values stable during training.

A causal mask is also created. This mask prevents each token from looking at future tokens. In other words, when the model predicts the next character, it is only allowed to use the current and previous characters. This is necessary for GPT-style language modeling.

After applying the mask, softmax converts the attention scores into attention weights. These weights show how much importance each token gives to earlier positions in the sequence. The weighted Values are then combined, projected back to the original embedding dimension, and passed through dropout.

The `return_attention` option is important for this problem because it allows the attention weights to be returned. These weights are later used to create the attention heatmaps and study the behavior of different attention heads.

The `FeedForward` class defines the second main part of a Transformer block. It first increases the representation size from `dim` to four times `dim`, applies the GELU activation function, and then reduces it back to the original dimension. Dropout is also used at the end. This part allows each token representation to be processed further after the attention operation.

The `TransformerBlock` class combines self-attention and the feed-forward network. Layer Normalization is applied before both parts. Residual connections are also used by adding the original representation back to the output of

the attention and feed-forward layers. These connections help information move through deeper layers and make training more stable.

The SmallGPT class combines all these parts into the complete language model. First, each character token is converted into a 128-dimensional vector using token embedding. A positional embedding is then added so that the model knows the position of each character in the sequence.

The model contains four Transformer blocks. During the forward pass, the input passes through these blocks one after another. The hidden representation after each layer is saved in `hidden_states`. The attention weights can also be saved when `return_attention` is enabled. These values are useful later for studying the deeper layers and attention patterns.

After the four Transformer blocks, Layer Normalization is applied and a final Linear layer produces scores for all characters in the vocabulary. If target values are provided, Cross-Entropy Loss is calculated between the predicted character scores and the real next characters.

Finally, the `count_parameters` function calculates the total number of trainable parameters in the model. This will be used in the next cell to report the size of the Transformer.

This cell does not produce any visible output because it only defines the model architecture and helper function.

Cell 5:

```
set_seed()
gpt_model = SmallGPT(
    vocab_size=vocab_size,
    max_context=BLOCK_SIZE,
    dim=EMBED_DIM,
    heads=N_HEADS,
    layers=N_LAYERS,
    dropout=DROPOUT
).to(device)
print(
    "Layers:",
    N_LAYERS
)
print(
    "Attention heads:",
    N_HEADS
```



```

)
print(
    "Embedding dimension:",
    EMBED_DIM
)
print(
    "Context length:",
    BLOCK_SIZE
)
print(
    "Trainable parameters:",
    count_parameters(
        gpt_model
    )
)
)

```

This cell creates the actual GPT model using the SmallGPT architecture defined in the previous cell.

First, the random seed is set again using `set_seed()`. This helps keep the model initialization reproducible.

Then the SmallGPT model is created with the settings that were already defined earlier. The vocabulary size is taken from the Tiny Shakespeare dataset. The maximum context length is 128, the embedding dimension is 128, the model uses 4 attention heads, and it contains 4 Transformer layers. The dropout value is 0.1.

After the model is created, it is moved to the GPU using `.to(device)`. This allows the training process to use CUDA and run faster.

The cell then prints the main architecture settings. It also uses the `count_parameters` function to calculate the total number of trainable parameters in the model. This gives us a better idea of the size and complexity of the Transformer.

Output:

```

Layers: 4
Attention heads: 4
Embedding dimension: 128
Context length: 128
Trainable parameters: 826433

```

This cell creates the actual GPT model using the SmallGPT architecture defined in the previous cell.

First, the random seed is set again using `set_seed()`. This helps keep the model initialization reproducible.

Then the SmallGPT model is created with the settings that were already defined earlier. The vocabulary size is taken from the Tiny Shakespeare dataset. The maximum context length is 128, the embedding dimension is 128, the model uses 4 attention heads, and it contains 4 Transformer layers. The dropout value is 0.1.

After the model is created, it is moved to the GPU using `.to(device)`. This allows the training process to use CUDA and run faster.

The cell then prints the main architecture settings. It also uses the `count_parameters` function to calculate the total number of trainable parameters in the model. This gives us a better idea of the size and complexity of the Transformer.

The output confirms that the model has been created with the correct architecture.

It contains 4 Transformer layers and 4 attention heads, which matches the required model structure. The embedding dimension is 128, so every character token is represented by a vector with 128 values.

The context length is also 128. This means that during training, the model can use up to 128 previous character positions when learning to predict the next character.

The model has 826,433 trainable parameters. These parameters include the token embeddings, positional embeddings, attention layers, feed-forward layers, normalization layers, and the final output layer.

Overall, the output confirms that the GPT-style Transformer was created successfully and is ready for training.

Cell 6:

```
@torch.no_grad()
def estimate_loss(
    model,
    block_size,
    eval_iters=EVAL_ITERS,
    batch_size=BATCH_SIZE
):
    model.eval()
```

```

results = {}
for split_name in [
    "train",
    "val"
]:
    losses = []
    for _ in range(
        eval_iters
    ):
        x, y = get_batch(
            split_name,
            block_size,
            batch_size
        )
        _, loss, _, _ = model(
            x,
            y
        )
        losses.append(
            loss.item()
        )
        results[split_name] = (
            np.mean(losses)
        )
    model.train()
    return results
def train_language_model(
    model,
    block_size,
    steps,
    batch_size,
    eval_interval=100
):
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=3e-4,
        weight_decay=0.01
    )

```

```

rows = []
start = time.perf_counter()
for step in range(
    1,
    steps + 1
):
    model.train()
    x, y = get_batch(
        "train",
        block_size,
        batch_size
    )
    optimizer.zero_grad(
        set_to_none=True
    )
    _, loss, _, _ = model(
        x,
        y
    )
    loss.backward()
    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        1.0
    )
    optimizer.step()
    if (
        step == 1
        or step % eval_interval == 0
        or step == steps
    ):
        losses = estimate_loss(
            model,
            block_size,
            eval_iters=EVAL_ITERS,
            batch_size=batch_size
        )
        rows.append({
            "Step": step,

```

```

        "Train Loss":
            losses["train"],
        "Validation Loss":
            losses["val"]
    })
    print(
        f"Step {step:03d}/{steps} | "
        f"Train {losses['train']:.4f} | "
        f"Val {losses['val']:.4f}"
    )
total_time = (
    time.perf_counter()
    - start
)
return (
    model,
    pd.DataFrame(rows),
    total_time
)

```

This cell defines the functions used to evaluate and train the GPT language model.

The first function is `estimate_loss`. Its purpose is to measure how well the model is performing on both the training data and the validation data. The `@torch.no_grad()` command is used because gradient calculations are not needed during evaluation. This makes the evaluation faster and also reduces memory usage.

Inside the function, the model is first switched to evaluation mode using `model.eval()`. Then the function checks both the training and validation sets. For each one, it creates several batches and calculates the loss. In this notebook, the evaluation is repeated for 15 batches. The losses from these batches are averaged, which gives a more stable value than checking only one batch.

After evaluation, the model is switched back to training mode using `model.train()`. The function then returns the average training loss and validation loss.

The second function is `train_language_model`. This function performs the main training process for the Transformer.

AdamW is used as the optimizer with a learning rate of 0.0003. Weight decay is also applied with a value of 0.01. This optimizer is commonly used for Transformer models because it works well with this type of architecture.

For every training step, the model is switched to training mode and a new batch is taken from the training data. The old gradients are cleared, then the model performs a forward pass and calculates the loss.

The loss is then used for backpropagation with `loss.backward()`. Gradient clipping is also applied with a maximum value of 1.0. This helps prevent very large gradients and makes the training process more stable.

After that, `optimizer.step()` updates the model parameters.

At the first step, every 100 steps, and at the final step, the `estimate_loss` function is called. The training and validation losses are saved in a table and also printed on the screen. This makes it possible to follow the learning process and see whether the model is improving.

The total training time is also measured using `time.perf_counter()`. At the end, the function returns the trained model, a Pandas DataFrame containing the recorded losses, and the total training time.

This cell does not produce an output by itself because it only defines the training and evaluation functions. The actual training starts in the next cell.

Cell 7:

```
set_seed()
(
    gpt_model,
    gpt_history,
    gpt_training_time
) = train_language_model(
    gpt_model,
    block_size=BLOCK_SIZE,
    steps=TRAIN_STEPS,
    batch_size=BATCH_SIZE,
    eval_interval=EVAL_INTERVAL
)
print(
    "Training time:",
    round(
        gpt_training_time / 60,
        2
    ),
    "minutes"
```

```
)  
gpt_history
```

This cell starts the actual training of the GPT-style Transformer.

First, the random seed is set again so that the training process starts in a reproducible way. Then the `train_language_model` function from the previous cell is called.

The model is trained with a context length of 128, a batch size of 64, and a total of 600 training steps. The evaluation interval is 100 steps, so the code checks the training loss and validation loss regularly during training.

The function returns three values. The first value is the trained GPT model. The second value is `gpt_history`, which stores the recorded training and validation losses in a table. The third value is `gpt_training_time`, which stores the total training time.

After training, the total time is converted from seconds to minutes and printed. Finally, the history table is displayed so that the change in loss can be seen more clearly.

Output:

```
Step 001/600 | Train 4.1834 | Val 4.1768  
Step 100/600 | Train 2.6232 | Val 2.6185  
Step 200/600 | Train 2.5108 | Val 2.5140  
Step 300/600 | Train 2.4454 | Val 2.4642  
Step 400/600 | Train 2.3902 | Val 2.4014  
Step 500/600 | Train 2.3191 | Val 2.3271  
Step 600/600 | Train 2.2289 | Val 2.2562  
Training time: 0.37 minutes
```

	Step	Train Loss	Validation Loss
0	1	4.183380	4.176838
1	100	2.623177	2.618524
2	200	2.510778	2.513997
3	300	2.445434	2.464204
4	400	2.390172	2.401416

	Step	Train Loss	Validation Loss
5	500	2.319062	2.327103
6	600	2.228904	2.256198

The output shows that the model learned successfully during training.

At the beginning, the training loss was 4.1834 and the validation loss was 4.1768. These values are relatively high because the model had just started training and its parameters were still close to their initial random values.

As training continued, both losses decreased steadily. At step 100, the validation loss had already dropped to about 2.62. At step 300, it reached about 2.46, and at the final step it reached 2.2562.

The final training loss was 2.2289 and the final validation loss was 2.2562. These two values are close to each other, which is a good sign because it means the model is learning useful patterns from the training text without showing a large difference between training and validation performance.

The total training time was only 0.37 minutes, which is about 22 seconds. This was possible because the model is relatively small, the dataset is simple, and the training was performed on a Tesla T4 GPU.

Overall, the continuous decrease in both training and validation loss shows that the GPT model learned the character patterns in the Tiny Shakespeare dataset successfully.

Cell 8:

```
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    gpt_history["Step"],
    gpt_history["Train Loss"],
    marker="o",
    label="Training"
)
plt.plot(
    gpt_history["Step"],
    gpt_history["Validation Loss"],
    marker="o",
    label="Validation"
)
```



```

        "Validation Loss"
    ],
    marker="o",
    label="Validation"
)
plt.xlabel("Training Step")
plt.ylabel("Loss")
plt.title(
    "GPT Training and Validation Loss"
)
plt.legend()
plt.show()

```

This cell creates a graph to show how the training loss and validation loss changed during the GPT training process.

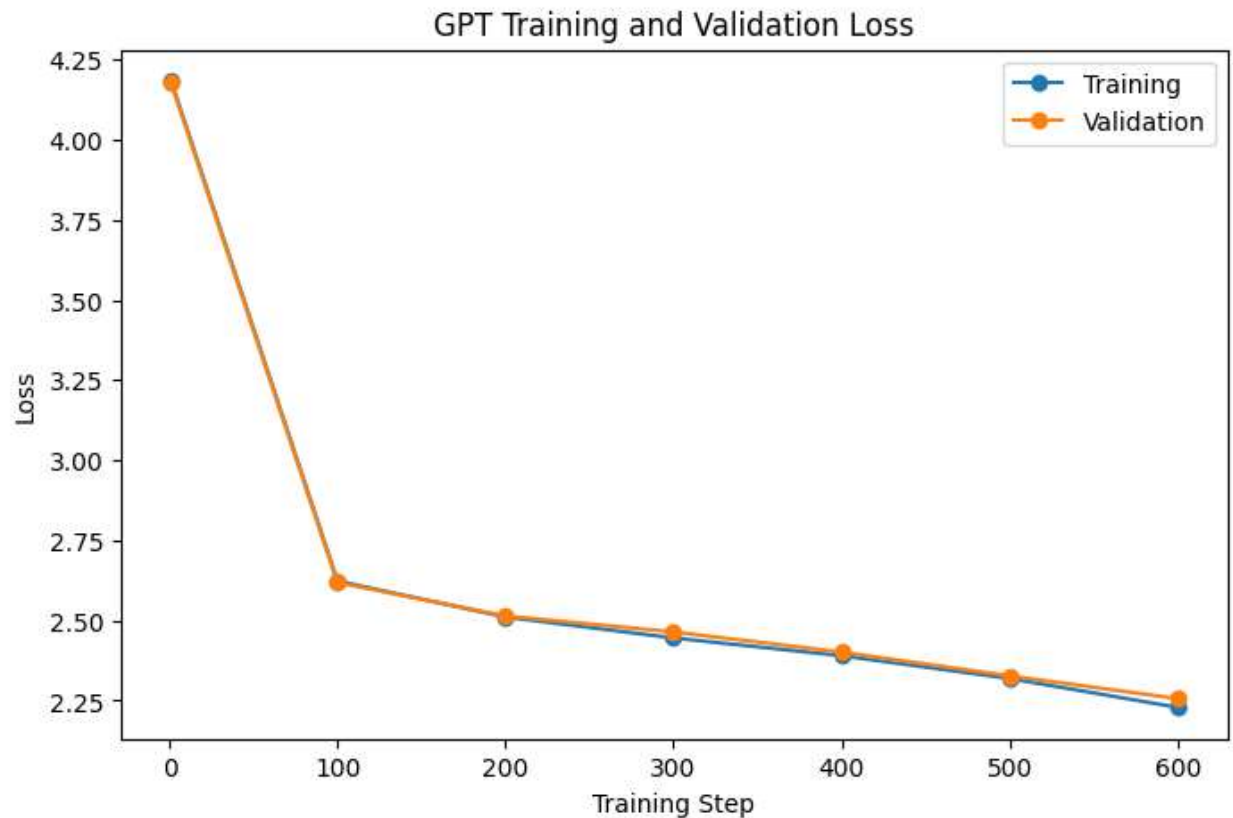
First, a figure with size 8 by 5 is created. Then the training loss values from `gpt_history` are plotted against the training steps. Circular markers are added so that the recorded evaluation points can be seen clearly.

The validation loss is plotted on the same graph. This makes it easy to compare the model performance on the training data and the validation data at each recorded step.

The x axis represents the training step and the y axis represents the loss value. A title and legend are also added so that the two curves can be identified easily.

The main purpose of this graph is to check whether the model is learning and whether there is a large difference between training and validation performance.

Output:



The graph shows a clear decrease in both training and validation loss during the training process.

At the beginning, both losses are around 4.18. After the first 100 steps, they drop quickly to around 2.62. After that, the losses continue to decrease more gradually until the end of training.

At step 600, the training loss is about 2.23 and the validation loss is about 2.26.

The two curves stay very close to each other during the whole training process. This is a good sign because it shows that the model is improving on both the training and validation data at almost the same rate.

There is no large gap between the two curves, so there is no strong sign of overfitting in this experiment.

Overall, the graph confirms the numerical results from the previous cell and shows that the GPT model learned steadily and remained stable during training.

Cell 9:

```
@torch.no_grad()
def generate_text(
    model,
```

```

start_tokens,
new_tokens=400,
temperature=0.8
):
    model.eval()
    tokens = start_tokens
    for _ in range(
        new_tokens
    ):
        current = tokens[
            :,
            -model.max_context:
        ]

        logits, _, _, _ = model(
            current
        )
        logits = (
            logits[:, -1, :]
            / temperature
        )
        probabilities = F.softmax(
            logits,
            dim=-1
        )
        next_token = (
            torch.multinomial(
                probabilities,
                1
            )
        )

        tokens = torch.cat(
            [
                tokens,
                next_token
            ],
            dim=1

```

```

    )
    return tokens
start_text = "ROMEO:"
start_tokens = torch.tensor(
    [
        encode(
            start_text
        )
    ],
    dtype=torch.long,
    device=device
)
generated = generate_text(
    gpt_model,
    start_tokens,
    new_tokens=400
)
print(
    decode(
        generated[0]
        .cpu()
        .tolist()
    )
)

```

This cell tests whether the trained GPT model can generate new text.

The `generate_text` function receives the trained model and a starting sequence of tokens. It then generates new characters one by one. The model is first switched to evaluation mode because no further training is needed during text generation.

At each step, the function keeps only the most recent characters that fit inside the model context length. The model then predicts scores for the next possible character.

The logits from the last position are divided by the temperature value. In this experiment, the temperature is 0.8. This controls how random the generated text is. A lower temperature usually makes the output more predictable, while a higher temperature allows more variation.

The logits are converted into probabilities using softmax. Then `torch.multinomial` selects the next character based on these probabilities. The selected token is added to the sequence, and the same process continues until 400 new characters are generated.

The generation starts with the text ROMEO:. This gives the model a short Shakespeare-style beginning and allows us to see what kind of text it produces after training.

Output:

ROMEO:

NI, markeat your thathembe ble on to verin soneer spe ase.

SANIIO: no ghen sid the fo treak will?

For, de whitist al danco ps I panose med thengiss d?

Conond sicosts; Cand he inort whipre ke het ism browid

An ameroure sthing mere toror ap sbey teid ware;

I palitee me nd le my yoford then reime avel t y watheattin oul f f dyomo ise
sthe ces brane t?

Turd te de lleeed be Loowoen the sshere ore

The generated text is not fully correct English, but it shows that the model learned some important patterns from the Tiny Shakespeare dataset.

For example, the output contains punctuation, spaces, capital letters, short word-like structures, and dialogue-style text such as ROMEO: and SANIIO:. These patterns are similar to the structure of Shakespeare plays.

The model also produces sequences that look similar to English words, even when they are not real words. This shows that it learned local character patterns and some basic structure of the dataset.

At the same time, the sentences are still not grammatically correct and many generated words are meaningless. This is expected because the model is small and was trained for only 600 steps at the character level.

Overall, the output shows that the Transformer learned more than random character generation. It captured some writing patterns and structure from the training text, but it would need a larger model or longer training to generate more natural and meaningful language.

Cell 10:

```
x_attention, _ = get_batch(  
    "val",
```

```

        block_size=32,
        batch_size=1
    )
gpt_model.eval()
with torch.no_grad():
    (
        _,
        _,
        attention_maps,
        -
    ) = gpt_model(
        x_attention,
        return_attention=True
    )
labels = [
    decode([int(value)])
    for value
    in x_attention[0].cpu()
]
labels = [
    "\\n"
    if value == "\\n"
    else "_"
    if value == " "
    else value
    for value in labels
]
def show_attention_layer(
    attention,
    layer_number
):
    fig, axes = plt.subplots(
        1,
        N_HEADS,
        figsize=(18, 4)
    )
    for head in range(
        N_HEADS

```

```

):

    image = axes[
        head
    ].imshow(
        attention[
            0,
            head
        ].cpu(),
        aspect="auto"
    )
    axes[head].set_title(
        f"Head {head + 1}"
    )
    axes[head].set_xlabel(
        "Key Position"
    )
    axes[head].set_ylabel(
        "Query Position"
    )
    fig.colorbar(
        image,
        ax=axes[head]
    )
    fig.suptitle(
        f"Attention Heatmaps - Layer {layer_number}"
    )
    plt.tight_layout()
    plt.show()
show_attention_layer(
    attention_maps[0],
    1
)
show_attention_layer(
    attention_maps[-1],
    4
)

```

This cell visualizes the attention patterns learned by the Transformer.

First, one validation sequence with a length of 32 characters is selected. The batch size is set to 1 because only one sequence is needed for visualization.

The GPT model is switched to evaluation mode, and the forward pass is performed with `return_attention=True`. This makes the model return the attention weights from all Transformer layers.

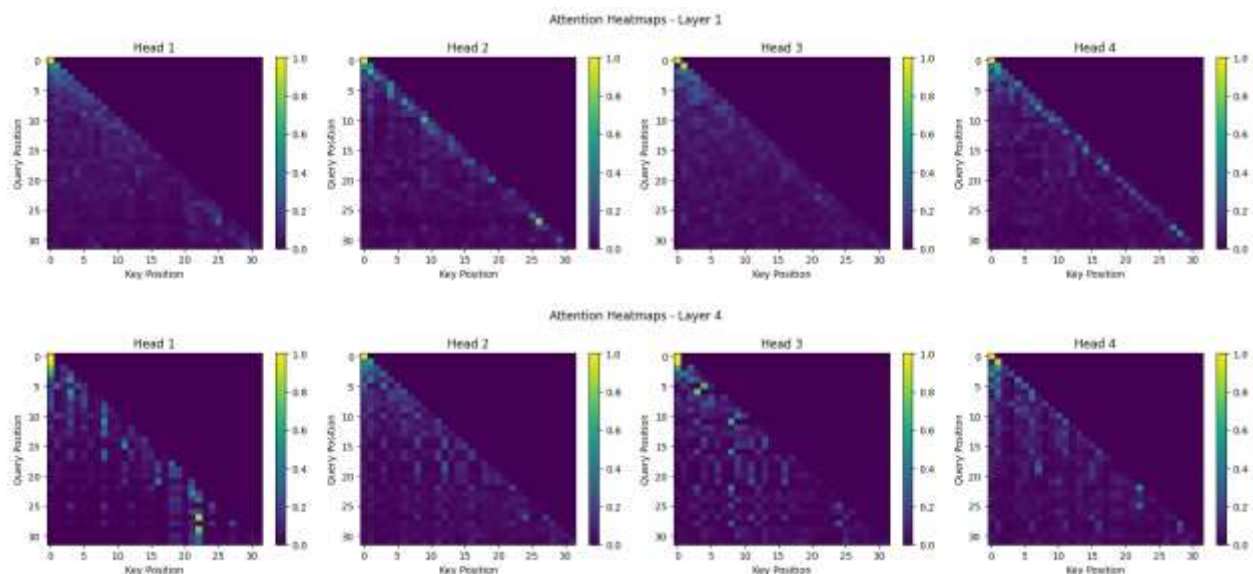
The code also decodes the token IDs back into characters. Spaces and newline characters are changed into simple visible symbols so that the sequence is easier to understand if needed.

The `show_attention_layer` function is then defined. This function creates one heatmap for each attention head in a selected Transformer layer. Since the model has 4 heads, each figure contains 4 heatmaps.

In every heatmap, the horizontal axis represents Key Position and the vertical axis represents Query Position. The color shows how much attention one position gives to another position. Brighter colors mean stronger attention, while darker colors mean weaker attention.

Finally, the function is used for Layer 1 and Layer 4. This allows us to compare the attention behavior in an early layer and a deeper layer of the Transformer.

Output:



The output contains two figures. As you see, the first figure is titled Attention Heatmaps Layer 1 and shows the attention patterns of the 4 heads in the first Transformer layer.

The second figure is titled Attention Heatmaps Layer 4 and shows the attention patterns of the 4 heads in the final Transformer layer.

The heatmaps have a triangular shape because the model uses causal attention. A character can only attend to itself and the characters before it. It cannot look at future positions. This is why the upper right part of each heatmap remains dark.

In Layer 1, the attention patterns are relatively simple. Many of the brighter values are close to the diagonal. This means that several heads give more attention to nearby or recent characters. However, the exact pattern is not the same for all four heads.

In Layer 4, the attention patterns become more varied. Some heads still give attention to nearby positions, while others show stronger attention to specific earlier positions that are farther away from the current token. The brighter regions are more separated and different between the heads.

This difference suggests that the attention heads are not learning exactly the same behavior. Some heads appear to focus more on local information, while others use a wider part of the previous context.

Overall, these figures show that the Transformer learns different attention patterns across its heads and layers. The first layer mainly shows simpler and more local relationships, while the deeper layer shows more complex and selective attention to earlier positions in the sequence.

Cell 11:

```
x_stats, _ = get_batch(
    "val",
    block_size=64,
    batch_size=16
)
gpt_model.eval()
with torch.no_grad():
    (
        _,
        _,
        attention_stats,
        _
    ) = gpt_model(
        x_stats,
        return_attention=True
```

```

    )
rows = []
for layer_index, attention in enumerate(
    attention_stats
):
    length = attention.size(-1)
    positions = torch.arange(
        length,
        device=device
    )
    distance = (
        positions[:, None]
        - positions[None, :]
    ).abs().float()

    for head in range(
        N_HEADS
    ):
        weights = attention[
            :,
            head
        ]
        mean_distance = (
            weights
            * distance
        ).sum(
            dim=-1
        ).mean().item()

        entropy = (
            -(
                weights.clamp_min(
                    1e-9
                )
                *
                weights.clamp_min(
                    1e-9
                ).log()
            )

```

```

        )
        .sum(dim=-1)
        .mean()
        .item()
    )
    self_attention = (
        torch.diagonal(
            weights,
            offset=0,
            dim1=1,
            dim2=2
        )
        .mean()
        .item()
    )
    previous_attention = (
        torch.diagonal(
            weights,
            offset=-1,
            dim1=1,
            dim2=2
        )
        .mean()
        .item()
    )
    rows.append({
        "Layer":
            layer_index + 1,
        "Head":
            head + 1,
        "Mean Attention Distance":
            mean_distance,
        "Attention Entropy":
            entropy,
        "Self Attention":
            self_attention,
        "Previous Token Attention":
            previous_attention
    })

```

```

    })
head_statistics = pd.DataFrame(
    rows
)
head_statistics

```

This cell gives a more detailed numerical analysis of the attention heads.

First, the code takes 16 validation sequences, each with a length of 64 characters. The trained GPT model is then used in evaluation mode and the attention weights from all Transformer layers are collected.

After that, the code studies every attention head separately. Four different measurements are calculated for each head.

Mean Attention Distance shows how far, on average, a head looks from the current token. A larger value means the head often attends to positions that are farther away, while a smaller value means it focuses more on nearby tokens.

Attention Entropy shows how spread out the attention is. A higher entropy means the attention is distributed across many positions. A lower entropy means the head focuses more strongly on a smaller number of positions.

Self Attention measures how much a token attends to itself.

Previous Token Attention measures how much attention is given to the token immediately before the current token.

The results from all 4 layers and all 4 heads are stored in a Pandas DataFrame. Since there are 4 layers and 4 heads in each layer, the table contains 16 rows.

Output:

Layer	Head	Mean Attention Distance	Attention Entropy	Self Attention	Previous Token Attention	
0	1	1	14.594135	3.038259	0.082191	0.084707
1	1	2	13.276671	2.832626	0.082544	0.187888
2	1	3	14.593260	3.044423	0.073296	0.067107
3	1	4	13.745106	2.849470	0.078675	0.183743

Layer	Head	Mean Attention Distance	Attention Entropy	Self Attention	Previous Token Attention	
4	2	1	7.467994	2.461790	0.100245	0.203554
5	2	2	8.523263	2.608120	0.099985	0.185750
6	2	3	7.481506	2.425014	0.099487	0.244219
7	2	4	9.937443	2.516163	0.110755	0.121150
8	3	1	11.259348	2.569336	0.092665	0.104692
9	3	2	8.962164	2.527886	0.111765	0.151356
10	3	3	9.788981	2.440351	0.089573	0.085524
11	3	4	9.251240	2.545817	0.129694	0.137107
12	4	1	10.817175	1.973720	0.076498	0.075266
13	4	2	11.166844	2.624478	0.121597	0.087316
14	4	3	12.569324	2.415189	0.083535	0.084603
15	4	4	11.745436	2.668442	0.126959	0.073193

The output is a table containing the attention statistics for every head in every Transformer layer. The table shows that the attention heads do not behave in exactly the same way.

In Layer 1, the mean attention distances are relatively high, around 13 to 15. This means that the first layer often looks across a fairly wide part of the available context.

Some heads in this layer also show stronger attention to the previous token. For example, Head 2 has a previous token attention of about 0.188, and Head 4 has a value of about 0.184. In comparison, Head 3 has a much smaller value of about 0.067.

Layer 2 shows a different pattern. The mean attention distances are smaller, mostly between 7 and 10. This suggests that these heads focus more on nearby positions. Head 3 in Layer 2 has the highest previous token attention in the

whole table, with a value of about 0.244. This means that this head gives relatively strong importance to the character immediately before the current one.

The later layers again show larger attention distances. For example, Head 3 in Layer 4 has a mean attention distance of about 12.57. This suggests that deeper layers can use information from more distant positions.

The entropy values are also different between heads. Layer 4 Head 1 has the lowest attention entropy, about 1.97. This means that this head is more selective and concentrates its attention on fewer positions. Other heads have higher entropy values, so their attention is more distributed.

The self attention values also change between heads. Some heads give more importance to the current token, while others focus more on previous or distant tokens.

Overall, these results support the idea that different attention heads learn different roles. Some heads focus more on nearby characters, some pay more attention to the previous token, and others use a wider context. This gives numerical evidence for the differences that were already visible in the attention heatmaps.

Cell 12:

```
CONTEXT_LENGTHS = [
    32,
    64,
    128
]
CONTEXT_STEPS = 200
def context_experiment(
    context_length
):
    set_seed()
    model = SmallGPT(
        vocab_size=vocab_size,
        max_context=context_length,
        dim=EMBED_DIM,
        heads=N_HEADS,
        layers=N_LAYERS,
        dropout=DROPOUT
    ).to(device)
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats()
```

```

(
    model,
    history,
    total_time
) = train_language_model(
    model,
    block_size=context_length,
    steps=CONTEXT_STEPS,
    batch_size=32,
    eval_interval=100
)
final_loss = estimate_loss(
    model,
    context_length,
    eval_iters=20,
    batch_size=32
)["val"]
perplexity = np.exp(
    final_loss
)
peak_memory = 0
if device.type == "cuda":
    peak_memory = (
        torch.cuda.max_memory_allocated()
        / (1024 ** 2)
    )
result = {
    "Context Length":
        context_length,
    "Validation Loss":
        final_loss,
    "Perplexity":
        perplexity,
    "Seconds per Step":
        total_time
        / CONTEXT_STEPS,
    "Peak GPU Memory MB":
        peak_memory,

```

```

    "Parameters":
        count_parameters(
            model
        )
    }
    del model
    torch.cuda.empty_cache()
    return result

```

This cell prepares the experiment for studying how the context length affects the Transformer.

Three different context lengths are selected: 32, 64, and 128. Each model is trained for 200 steps so that the comparison stays simple and uses the same training budget.

The `context_experiment` function performs one complete experiment for a given context length.

First, the random seed is reset. Then a new SmallGPT model is created. The model architecture stays the same with 4 layers, 4 attention heads, and an embedding dimension of 128. The main value that changes is the maximum context length.

Before training starts, the peak GPU memory statistics are reset. This makes it possible to measure how much GPU memory is used by each context size.

The model is then trained using the same training function from the previous cells. In this experiment, the batch size is set to 32 and the model is trained for 200 steps.

After training, the final validation loss is measured again using 20 evaluation batches. This gives a more stable final result.

The code also calculates perplexity from the validation loss. Perplexity is another common metric for language models. A lower perplexity usually means that the model is more confident and predicts the next token better.

The peak GPU memory usage is also recorded. This is important because increasing context length makes the attention operation more expensive and usually requires more memory.

The result dictionary stores the context length, validation loss, perplexity, average time per training step, peak GPU memory usage, and number of trainable parameters.

Finally, the model is removed from memory and the CUDA cache is cleared before the next experiment starts. This helps prevent unnecessary GPU memory usage when several models are trained one after another.

This cell does not produce a visible output by itself because it only defines the context length experiment. The next cell runs this function for the three selected context lengths.

Cell 13:

```
context_rows = []
for context_length in (
    CONTEXT_LENGTHS
):
    print(
        "\nContext length:",
        context_length
    )
    result = context_experiment(
        context_length
    )
    context_rows.append(
        result
    )
context_results = pd.DataFrame(
    context_rows
)
context_results
```

This cell runs the context length experiment for all three selected values: 32, 64, and 128.

An empty list called `context_rows` is first created to store the result of each experiment. Then the code loops through the three context lengths and calls the `context_experiment` function that was defined in the previous cell.

For every context length, a new GPT model is created and trained for 200 steps. The training and validation losses are printed during the process. After training is finished, the final validation loss, perplexity, average time per training step, peak GPU memory usage, and number of parameters are stored.

After all three experiments are completed, the results are converted into a Pandas DataFrame. This makes it easier to compare the effect of different context lengths in one table.

Output:

```
Context length: 32
Step 001/200 | Train 4.1918 | Val 4.2042
Step 100/200 | Train 2.6250 | Val 2.6094
Step 200/200 | Train 2.4583 | Val 2.4803
```

```
Context length: 64
Step 001/200 | Train 4.1899 | Val 4.1905
Step 100/200 | Train 2.6502 | Val 2.6375
Step 200/200 | Train 2.5101 | Val 2.5166
```

```
Context length: 128
Step 001/200 | Train 4.1872 | Val 4.1781
Step 100/200 | Train 2.6415 | Val 2.6307
Step 200/200 | Train 2.5188 | Val 2.5203
```

	Context Length	Validation Loss	Perplexity	Seconds per Step	Peak GPU Memory MB	Parameters
0	32	2.481048	11.953791	0.014879	81.597656	814145
1	64	2.510472	12.310745	0.013845	133.531250	818241
2	128	2.515185	12.368897	0.025237	261.891113	826433

The results show that increasing the context length did not improve validation performance in this short experiment.

The model with context length 32 achieved the lowest validation loss, about 2.48, and also the lowest perplexity, about 11.95. The context lengths 64 and 128 produced slightly higher validation losses and perplexities.

This does not mean that a longer context is always worse. All three models were trained for only 200 steps using the same limited training budget. A model with a longer context has to process more information in every sequence and may need more training before it can use this extra context effectively.

The clearest difference can be seen in GPU memory usage. With context length 32, the peak memory usage was about 81.6 MB. It increased to about 133.5 MB for context length 64 and reached about 261.9 MB for context length 128. This shows that longer sequences require noticeably more GPU memory.

The context length of 128 also required about 0.025 seconds per training step, while the other two settings were around 0.014 to 0.015 seconds. The small difference between context lengths 32 and 64 is not very important because execution time can vary slightly between runs. However, the increase for context length 128 is much clearer.

The number of parameters also increases slightly because a longer context requires a larger positional embedding table.

Overall, this experiment shows that increasing the context length increases computational cost and memory usage, but it does not automatically improve the model performance when the training time is limited.

Cell 14:

```
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    context_results[
        "Context Length"
    ],
    context_results[
        "Validation Loss"
    ],
    marker="o"
)
plt.xlabel(
    "Context Length"
)
plt.ylabel(
    "Validation Loss"
)
plt.title(
    "Context Length vs Validation Loss"
)
plt.show()
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    context_results[
        "Context Length"
    ],
    context_results[
```

```

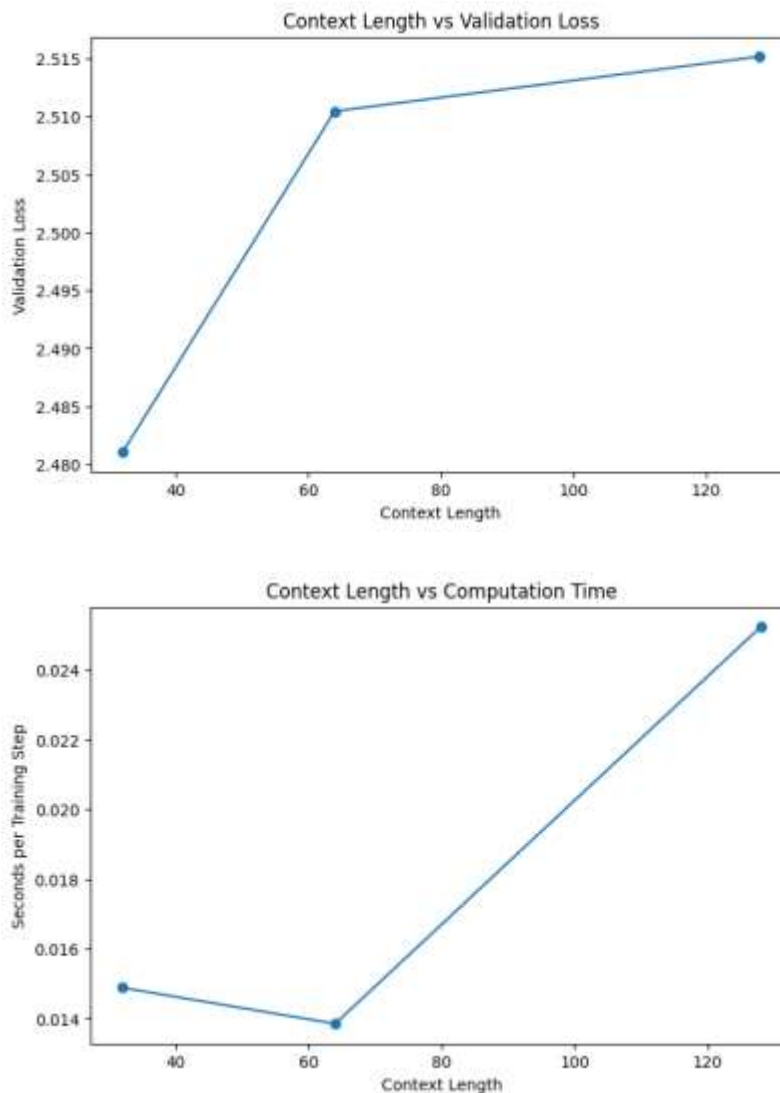
        "Seconds per Step"
    ],
    marker="o"
)
plt.xlabel(
    "Context Length"
)
plt.ylabel(
    "Seconds per Training Step"
)
plt.title(
    "Context Length vs Computation Time"
)
plt.show()
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    context_results[
        "Context Length"
    ],
    context_results[
        "Peak GPU Memory MB"
    ],
    marker="o"
)
plt.xlabel(
    "Context Length"
)
plt.ylabel(
    "Peak GPU Memory (MB) "
)
plt.title(
    "Context Length vs GPU Memory"
)
plt.show()

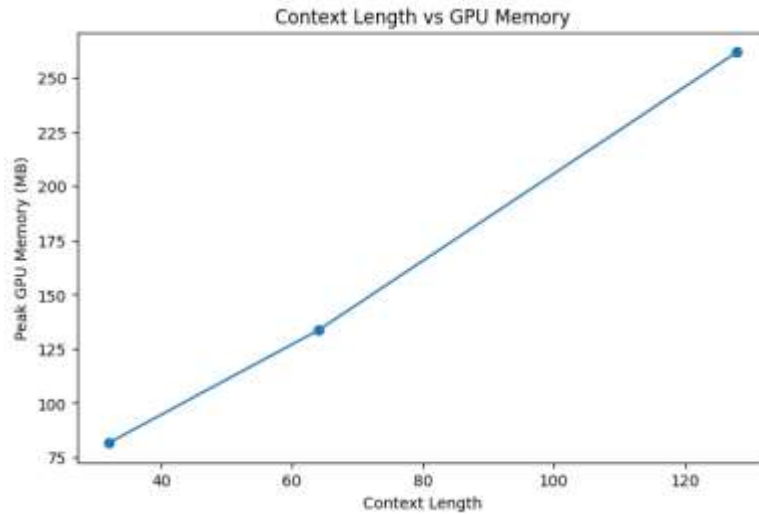
```

This cell creates three graphs to make the context length experiment easier to understand visually.

The first graph compares context length with validation loss. It shows how the model performance changes when the context size is increased from 32 to 64 and then to 128. The second graph compares context length with computation time. The value on the vertical axis is the average number of seconds needed for one training step. The third graph compares context length with peak GPU memory usage. This shows how much GPU memory is required when the sequence becomes longer. These three graphs help us see the trade-off between model performance and computational cost more clearly than the table alone.

Output:





The first graph shows that the validation loss slightly increases as the context length becomes larger. The model with context length 32 has the lowest validation loss, around 2.481. The loss increases to about 2.510 for context length 64 and about 2.515 for context length 128.

This means that a longer context did not improve the validation result in this short experiment. Since all models were trained for only 200 steps, the larger models may not have had enough training time to use the additional context effectively.

The second graph shows the computation time per training step. The values for context lengths 32 and 64 are very close, around 0.015 and 0.014 seconds. This small difference is likely caused by normal runtime variation. However, when the context length increases to 128, the time rises clearly to about 0.025 seconds per step.

The third graph shows the clearest effect of increasing context length. GPU memory usage rises from about 81.6 MB for context length 32 to about 133.5 MB for context length 64. At context length 128, it increases further to about 261.9 MB.

Overall, these graphs show that increasing the context length makes the model more computationally expensive, especially in terms of GPU memory. At the same time, the validation loss did not improve under the same limited training budget. This shows that a larger context is not always automatically better and that there is a trade-off between context size, training cost, and model performance.

Cell 15:

```
digits = load_digits()
images = torch.tensor(
    digits.images,
```

```

        dtype=torch.float32
    ).unsqueeze(1)
    images = images / 16.0
    labels = torch.tensor(
        digits.target,
        dtype=torch.long
    )
    indices = np.arange(
        len(images)
    )
    train_indices, test_indices = (
        train_test_split(
            indices,
            test_size=0.2,
            random_state=SEED,
            stratify=digits.target
        )
    )
    train_indices = torch.tensor(
        train_indices,
        dtype=torch.long
    )
    test_indices = torch.tensor(
        test_indices,
        dtype=torch.long
    )
    patch_train_loader = DataLoader(
        TensorDataset(
            images[train_indices],
            labels[train_indices]
        ),
        batch_size=128,
        shuffle=True
    )
    patch_test_loader = DataLoader(
        TensorDataset(
            images[test_indices],
            labels[test_indices]

```

```

    ),
    batch_size=128,
    shuffle=False
)
print(
    "Training images:",
    len(train_indices)
)
print(
    "Test images:",
    len(test_indices)
)
print(
    "Image shape:",
    images.shape[1:]
)

```

This cell prepares the image dataset that will be used for the spatial locality experiment.

The `load_digits` function loads the Digits dataset from scikit-learn. This dataset contains small grayscale images of handwritten digits from 0 to 9.

The image data is converted into a PyTorch tensor with type `float32`. The `unsqueeze` operation adds a channel dimension, so each image has the shape 1 by 8 by 8. The value 1 represents the single grayscale channel.

The pixel values in the original dataset are between 0 and 16. Dividing the images by 16.0 scales the pixel values to the range from 0 to 1. This makes the data easier to use during training.

The digit labels are also converted into a PyTorch tensor with integer values.

After that, the code creates an array of sample indices and splits them into training and test sets using `train_test_split`. Eighty percent of the images are used for training and twenty percent are used for testing.

The random state is fixed using the same seed value, and `stratify` is used with the digit labels. This keeps the class distribution similar in both the training and test sets.

The selected indices are converted into PyTorch tensors. Then two `DataLoader` objects are created. The training loader uses a batch size of 128 and shuffles the data, while the test loader uses the same batch size but keeps the original order.

Finally, the code prints the number of training images, the number of test images, and the shape of each image.

Output:

```
Training images: 1437
Test images: 360
Image shape: torch.Size([1, 8, 8])
```

The output shows that the Digits dataset was divided into 1,437 training images and 360 test images.

Each image has the shape [1, 8, 8]. The first value represents the grayscale channel, and the 8 by 8 values represent the height and width of the image.

This dataset is much smaller than datasets such as CIFAR-10, so it is useful for a fast experiment. In the next cells, these images will be divided into small patches and processed by a Transformer.

The purpose of using this dataset is not mainly to achieve high image classification performance. It is used so that we can study whether patch representations still keep information about their original spatial position as they move through deeper Transformer layers.

Cell 16:

```
class PatchTransformer(nn.Module):
    def __init__(
        self,
        dim=128,
        heads=4,
        layers=4
    ):
        super().__init__()
        self.patch_embedding = (
            nn.Conv2d(
                1,
                dim,
                kernel_size=2,
                stride=2
            )
        )
```

```

self.position_embedding = (
    nn.Parameter(
        torch.zeros(
            1,
            16,
            dim
        )
    )
)
nn.init.normal_(
    self.position_embedding,
    mean=0.0,
    std=0.02
)
self.layers = nn.ModuleList([
    nn.TransformerEncoderLayer(
        d_model=dim,
        nhead=heads,
        dim_feedforward=4 * dim,
        dropout=0.1,
        batch_first=True
    )
    for _ in range(
        layers
    )
])
self.norm = nn.LayerNorm(
    dim
)
self.classifier = nn.Linear(
    dim,
    10
)
def forward(
    self,
    images,
    use_position=True,
    return_hidden=False

```

```

):
    content = (
        self.patch_embedding(
            images
        )
        .flatten(2)
        .transpose(1, 2)
    )
    if use_position:
        x = (
            content
            +
            self.position_embedding
        )
    else:
        x = content
    hidden_states = []
    for layer in self.layers:
        x = layer(x)
        hidden_states.append(x)
    output = self.norm(x)
    logits = self.classifier(
        output.mean(dim=1)
    )
    if return_hidden:
        return (
            logits,
            content,
            hidden_states
        )
    return logits

```

This cell defines the Patch Transformer model that will be used to study spatial locality in image patches.

The PatchTransformer class receives the handwritten digit images and first divides each image into small patches. Since every image has size 8 by 8 and the patch embedding layer uses a kernel size and stride of 2, each image is divided into 16 patches.

The `patch_embedding` layer is implemented with a convolution. It converts each 2 by 2 image patch into a vector with 128 values. After that, the output is flattened and transposed so that the image becomes a sequence of 16 patch tokens, similar to how a text Transformer receives a sequence of word or character tokens.

The model also contains a learnable positional embedding with shape 1 by 16 by 128. This gives each patch information about its position in the image. For example, the model can distinguish a patch from the top left area from a patch in another location.

The positional embedding is initialized with small random values. During training, these values can be updated along with the other model parameters.

The model contains 4 Transformer encoder layers. Each layer uses an embedding dimension of 128 and 4 attention heads. The feed-forward part inside each Transformer layer has a size of four times the embedding dimension, and dropout is set to 0.1.

In the forward function, the image is first converted into patch tokens. If `use_position` is True, the positional embedding is added to these patch representations. If it is False, the model uses only the patch content without explicit positional information. This option will later be useful for comparing the behavior of the model with and without positional encoding.

The hidden representation after every Transformer layer is stored in `hidden_states`. This allows us to inspect how the representation of each patch changes as it moves through deeper layers.

After the Transformer layers, Layer Normalization is applied. The representations of all 16 patch tokens are then averaged, and a final Linear layer produces scores for the 10 digit classes.

If `return_hidden` is enabled, the model returns the classification result together with the original patch content and the hidden representations from all layers. These values are later used to measure spatial locality.

This cell does not produce a visible output because it only defines the Patch Transformer architecture.

Cell 17:

```
set_seed()
patch_model = PatchTransformer(
    dim=128,
    heads=4,
    layers=4
).to(device)
optimizer = torch.optim.AdamW(
```

```

    patch_model.parameters(),
    lr=0.001
)
criterion = nn.CrossEntropyLoss()
def patch_accuracy(
    model,
    loader
):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)
            output = model(x)
            prediction = (
                output.argmax(dim=1)
            )
            correct += (
                prediction == y
            ).sum().item()

            total += y.size(0)
    return (
        correct / total
    )
PATCH_EPOCHS = 25
for epoch in range(
    1,
    PATCH_EPOCHS + 1
):
    patch_model.train()
    losses = []
    for x, y in (
        patch_train_loader
    ):
        x = x.to(device)

```

```

        y = y.to(device)
        optimizer.zero_grad(
            set_to_none=True
        )
        output = patch_model(x)
        loss = criterion(
            output,
            y
        )
        loss.backward()
        optimizer.step()
        losses.append(
            loss.item()
        )
    accuracy = patch_accuracy(
        patch_model,
        patch_test_loader
    )
    print(
        f"Epoch {epoch:02d}/{PATCH_EPOCHS} | "
        f"Loss {np.mean(losses):.4f} | "
        f"Test Accuracy {accuracy:.4f}"
    )

```

This cell trains the Patch Transformer model on the Digits dataset.

First, the random seed is set again so that the training process is more reproducible. Then the PatchTransformer model is created with an embedding dimension of 128, 4 attention heads, and 4 Transformer layers. The model is moved to the GPU.

AdamW is used as the optimizer with a learning rate of 0.001. Cross Entropy Loss is used because this is a classification problem with 10 possible digit classes.

The patch_accuracy function is defined to measure how well the model performs on the test set. The function switches the model to evaluation mode, goes through all test batches, finds the predicted class for each image, and compares it with the true label. The final accuracy is the number of correct predictions divided by the total number of test images.

The model is trained for 25 epochs. In every epoch, the model is switched to training mode and all training batches are processed. For each batch, the gradients are cleared, the model produces its predictions, the loss is calculated, and backpropagation is used to update the model parameters.

The average training loss is calculated for each epoch. After every epoch, the test accuracy is also measured. Both values are printed so that we can follow the learning process over time.

Output:

```
Epoch 01/25 | Loss 2.4885 | Test Accuracy 0.1000
Epoch 02/25 | Loss 2.3308 | Test Accuracy 0.1028
Epoch 03/25 | Loss 2.3318 | Test Accuracy 0.1000
Epoch 04/25 | Loss 2.3268 | Test Accuracy 0.1056
Epoch 05/25 | Loss 2.3143 | Test Accuracy 0.1000
Epoch 06/25 | Loss 2.2800 | Test Accuracy 0.2194
Epoch 07/25 | Loss 2.0597 | Test Accuracy 0.2194
Epoch 08/25 | Loss 1.8440 | Test Accuracy 0.2583
Epoch 09/25 | Loss 1.7129 | Test Accuracy 0.4639
Epoch 10/25 | Loss 1.4860 | Test Accuracy 0.4528
Epoch 11/25 | Loss 1.2092 | Test Accuracy 0.6917
Epoch 12/25 | Loss 0.9091 | Test Accuracy 0.7389
Epoch 13/25 | Loss 0.7186 | Test Accuracy 0.7917
Epoch 14/25 | Loss 0.5646 | Test Accuracy 0.8083
Epoch 15/25 | Loss 0.5228 | Test Accuracy 0.8167
Epoch 16/25 | Loss 0.4237 | Test Accuracy 0.9056
Epoch 17/25 | Loss 0.3168 | Test Accuracy 0.9167
Epoch 18/25 | Loss 0.2612 | Test Accuracy 0.9194
Epoch 19/25 | Loss 0.2136 | Test Accuracy 0.9333
Epoch 20/25 | Loss 0.1464 | Test Accuracy 0.9528
Epoch 21/25 | Loss 0.1106 | Test Accuracy 0.9222
Epoch 22/25 | Loss 0.1227 | Test Accuracy 0.9528
Epoch 23/25 | Loss 0.1057 | Test Accuracy 0.9500
Epoch 24/25 | Loss 0.0808 | Test Accuracy 0.9444
Epoch 25/25 | Loss 0.0663 | Test Accuracy 0.9556
```

The output shows that the Patch Transformer learned the digit classification task successfully.

The slow improvement during the first few epochs is normal because the model starts with randomly initialized parameters. At this stage, the accuracy is close to 10%, which is approximately random guessing for a problem with 10 classes.

After about 6 to 10 epochs, the model begins to learn useful image patch patterns and the accuracy increases more quickly.

By the later epochs, the model reaches more than 90% test accuracy. The final test accuracy is 0.9556, which means that about 95.56% of the test images are classified correctly.

The training loss also decreases continuously and becomes very small by the end of training. This confirms that the optimization process worked correctly.

There are a few small changes in test accuracy between some epochs, such as the drop at epoch 21. This is normal because test performance does not always improve perfectly at every epoch.

Overall, the final accuracy of 95.56% shows that the Patch Transformer learned meaningful representations from the image patches. This is important because the next experiment studies the hidden patch representations inside this trained model, so it is better to analyze a model that has already learned the task well.

Cell 18:

```
@torch.no_grad()
def measure_locality(
    model,
    loader,
    use_position
):
    model.eval()
    layer_results = {
        i: {
            "same": [],
            "top_left": [],
            "diagonal": [],
            "other": []
        }
        for i in range(4)
    }
    for batch_index, (
```



```

x,
-
) in enumerate(loader):
    if batch_index >= 3:
        break
    x = x.to(device)
    (
        _,
        content,
        hidden_states
    ) = model(
        x,
        use_position=use_position,
        return_hidden=True
    )
    content = F.normalize(
        content,
        dim=-1
    )
    batch_size = (
        content.size(0)
    )
    number_of_patches = (
        content.size(1)
    )
    targets = torch.arange(
        number_of_patches,
        device=device
    ).unsqueeze(0).expand(
        batch_size,
        -1
    )
    for layer_index, hidden in enumerate(
        hidden_states
    ):
        hidden = F.normalize(
            hidden,
            dim=-1

```

```

    )
    similarities = (
        hidden
        @ content.transpose(
            1,
            2
        )
    )
    predicted = (
        similarities.argmax(
            dim=-1
        )
    )
    same_accuracy = (
        predicted
        .eq(targets)
        .float()
        .mean()
        .item()
    )
    top_left_accuracy = (
        predicted[:, 0]
        .eq(0)
        .float()
        .mean()
        .item()
    )
    diagonal = (
        torch.diagonal(
            similarities,
            dim1=1,
            dim2=2
        )
        .mean()
        .item()
    )
    diagonal_sum = (
        torch.diagonal(

```

```

        similarities,
        dim1=1,
        dim2=2
    )
    .sum()
)
other = (
    (
        similarities.sum()
        - diagonal_sum
    )
    /
    (
        batch_size
        * number_of_patches
        * (
            number_of_patches
            - 1
        )
    )
).item()
layer_results[
    layer_index
]["same"].append(
    same_accuracy
)
layer_results[
    layer_index
]["top_left"].append(
    top_left_accuracy
)
layer_results[
    layer_index
]["diagonal"].append(
    diagonal
)
layer_results[
    layer_index

```

```

        ]["other"].append(
            other
        )
    rows = []
    for layer_index in range(4):
        rows.append({
            "Position Encoding":
                "On"
                if use_position
                else "Off",
            "Layer":
                layer_index + 1,
            "Same Patch Retrieval":
                np.mean(
                    layer_results[
                        layer_index
                    ]["same"]
                ),
            "Top Left Retrieval":
                np.mean(
                    layer_results[
                        layer_index
                    ]["top_left"]
                ),
            "Same Patch Similarity":
                np.mean(
                    layer_results[
                        layer_index
                    ]["diagonal"]
                ),
            "Other Patch Similarity":
                np.mean(
                    layer_results[
                        layer_index
                    ]["other"]
                )
        })
    return rows

```

```

locality_rows = (
    measure_locality(
        patch_model,
        patch_test_loader,
        True
    )
    +
    measure_locality(
        patch_model,
        patch_test_loader,
        False
    )
)
locality_results = pd.DataFrame(
    locality_rows
)
locality_results

```

This cell measures how much spatial information is still preserved inside the Patch Transformer as the data moves through deeper layers.

The `measure_locality` function studies the hidden representation of each image patch. It compares the hidden representation in every Transformer layer with the original patch content before it enters the Transformer.

The function is tested in two settings. In the first setting, positional encoding is used normally. In the second setting, positional encoding is turned off during this evaluation. The model is not trained again without positional encoding. This is only an ablation test to see how the internal representations change when the positional information is removed.

The code uses the first three batches from the test set. For every image, the original patch representations and the hidden representations from all four Transformer layers are collected.

Before comparing them, the vectors are normalized. Then cosine similarity is calculated between every hidden patch representation and every original patch representation.

Same Patch Retrieval measures whether a hidden patch is most similar to its own original patch. A higher value means that the Transformer representation still strongly remembers where its original patch content came from.

Top Left Retrieval studies only the first patch, which represents the top left area of the image. It checks whether the hidden representation of this patch is still most similar to the original top left patch.

Same Patch Similarity measures the average similarity between each hidden patch and its own original patch.

Other Patch Similarity measures the average similarity between a hidden patch and all other original patches.

The measurements are calculated separately for all four Transformer layers. The same experiment is then repeated with positional encoding enabled and disabled.

Output:

Position Encoding	Layer	Same Patch Retrieval	Top Left Retrieval	Same Patch Similarity	Other Patch Similarity	
0	On	1	0.611003	0.834936	0.815531	0.538128
1	On	2	0.527181	0.607171	0.504011	0.287064
2	On	3	0.453563	0.412460	0.235571	0.108029
3	On	4	0.212790	0.188301	0.054086	0.006980
4	Off	1	0.695788	0.918470	0.896172	0.604921
5	Off	2	0.624124	0.798678	0.645630	0.382073
6	Off	3	0.593111	0.734375	0.411953	0.222316
7	Off	4	0.475536	0.510216	0.202728	0.096144

The most important pattern is that spatial locality becomes weaker in deeper layers.

With positional encoding enabled, Same Patch Retrieval is about 61% in the first layer but falls to about 21% in the fourth layer. This means that early representations still have a strong connection to their original image patches, but deeper layers mix information from different parts of the image much more strongly.

The same pattern can be seen for the top left patch. Its retrieval value decreases from about 83% in Layer 1 to about 19% in Layer 4. This shows that the top left token gradually becomes less similar to only its original patch because it receives information from other patches through self attention.

The similarity values show the same behavior. Same Patch Similarity becomes much smaller as the layers become deeper. This means that the representation of a patch changes considerably while it moves through the Transformer.

An interesting result appears when positional encoding is turned off. Spatial locality does not disappear. In fact, the retrieval values are higher in this evaluation. For example, Same Patch Retrieval in Layer 4 is about 0.476 without positional encoding compared with about 0.213 when it is enabled.

This result shows that positional encoding is not the only reason why patch representations keep information about their original location. The original patch content and the residual connections inside the Transformer can also help preserve this information.

At the same time, this comparison should be interpreted carefully because the model was trained with positional encoding and only evaluated with it turned off. It is not a separately trained model without positional encoding.

Overall, the experiment shows that deeper Transformer layers gradually mix information from different image regions, so the direct connection between a token and its original patch becomes weaker. However, some spatial locality still remains, even when explicit positional information is removed.

Cell 19:

```
plt.figure(
    figsize=(8, 5)
)
for state in [
    "On",
    "Off"
]:
    values = locality_results[
        locality_results[
            "Position Encoding"
        ] == state
    ]
    plt.plot(
        values["Layer"],
        values[
```

```

        "Same Patch Retrieval"
    ],
    marker="o",
    label=f"Position Encoding {state}"
)
plt.xlabel(
    "Transformer Layer"
)
plt.ylabel(
    "Same Patch Retrieval"
)
plt.title(
    "Spatial Locality Across Layers"
)
plt.xticks(
    [1, 2, 3, 4]
)
plt.legend()
plt.show()
plt.figure(
    figsize=(8, 5)
)
for state in [
    "On",
    "Off"
]:
    values = locality_results[
        locality_results[
            "Position Encoding"
        ] == state
    ]
    plt.plot(
        values["Layer"],
        values[
            "Top Left Retrieval"
        ],
        marker="o",
        label=f"Position Encoding {state}"
    )

```



```

    )
plt.xlabel(
    "Transformer Layer"
)
plt.ylabel(
    "Top Left Patch Retrieval"
)
plt.title(
    "Top Left Patch Locality"
)
plt.xticks(
    [1, 2, 3, 4]
)
plt.legend()
plt.show()

```

This cell creates two graphs to visualize how spatial locality changes across the four Transformer layers.

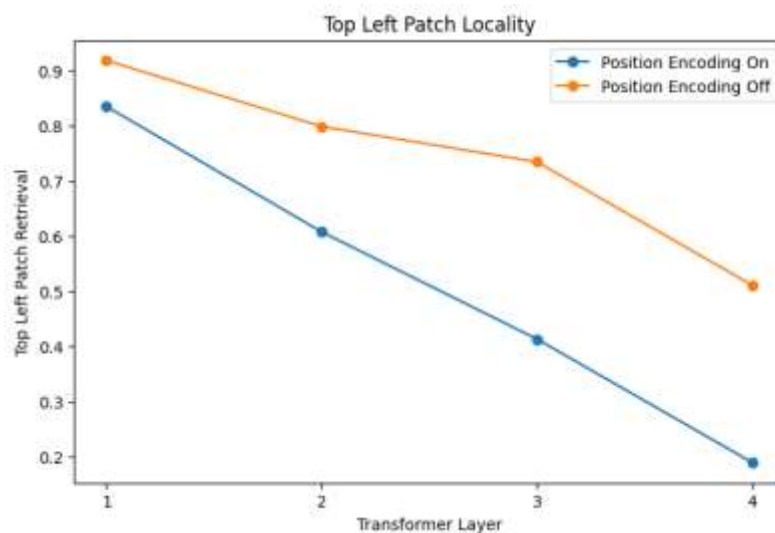
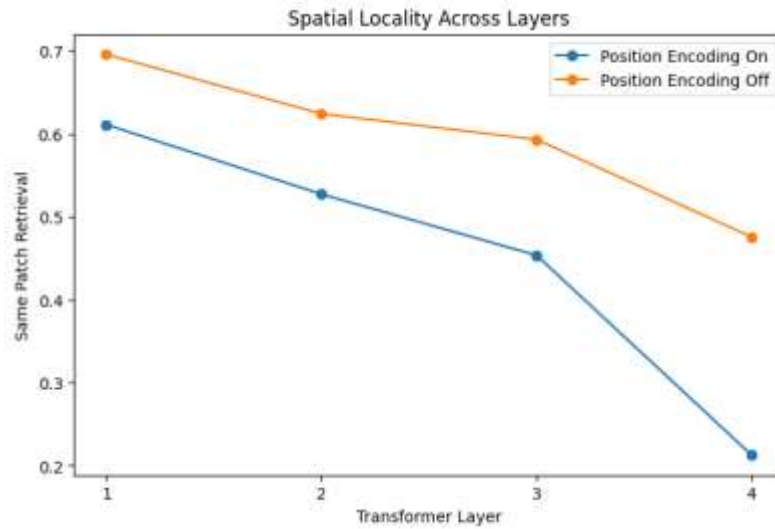
The first graph uses the Same Patch Retrieval values from the previous cell. It compares the results when positional encoding is enabled and when it is disabled.

For each setting, the values from Layers 1 to 4 are plotted as a line with circular markers. This makes it easy to see how strongly each hidden patch representation still matches its own original patch as the model becomes deeper.

The second graph focuses only on the top left patch. It uses the Top Left Retrieval values and again compares positional encoding On and Off across the four Transformer layers.

The x axis in both graphs represents the Transformer layer. The y axis shows the retrieval value. Higher values mean that the hidden representation still has a stronger connection to its original patch.

Output:



The first graph shows that Same Patch Retrieval decreases as the Transformer becomes deeper.

With positional encoding enabled, the value starts at about 0.61 in Layer 1 and falls to about 0.21 in Layer 4.

With positional encoding disabled, the same pattern also appears, but the values remain higher. The retrieval starts at about 0.70 and decreases to about 0.48.

This means that deeper layers gradually mix information from different patches. Because of this, each hidden patch representation becomes less similar to only its own original patch.

The second graph shows the same general pattern for the top left patch.

With positional encoding enabled, Top Left Retrieval decreases from about 0.83 in Layer 1 to about 0.19 in Layer 4.

With positional encoding disabled, it decreases from about 0.92 to about 0.51.

An important point is that locality does not disappear when positional encoding is turned off. In this experiment, the retrieval values are actually higher without positional encoding.

This suggests that positional encoding is not the only reason that patch representations keep information about their original position. The image content itself and the residual structure of the Transformer can also help preserve locality.

At the same time, this experiment only removes positional encoding during evaluation. The model was originally trained with positional encoding, so the comparison should be treated as an ablation test rather than a complete comparison between two separately trained models.

Overall, both graphs show that spatial locality becomes weaker in deeper layers, but it is not completely removed. They also show that positional encoding is only one part of the reason why Transformer tokens can still keep information about their original image region.

Cell 20:

```
final_summary = pd.DataFrame([
    {
        "Experiment":
            "GPT Language Model",
        "Value":
            gpt_history[
                "Validation Loss"
            ].iloc[-1]
    },
    {
        "Experiment":
            "Context 32 Validation Loss",
        "Value":
            context_results.loc[
                context_results[
                    "Context Length"
                ] == 32,
                "Validation Loss"
            ].iloc[0]
    },
])
```

```

{
    "Experiment":
        "Context 64 Validation Loss",
    "Value":
        context_results.loc[
            context_results[
                "Context Length"
            ] == 64,
            "Validation Loss"
        ].iloc[0]
},
{
    "Experiment":
        "Context 128 Validation Loss",
    "Value":
        context_results.loc[
            context_results[
                "Context Length"
            ] == 128,
            "Validation Loss"
        ].iloc[0]
}
])
final_summary

```

This final cell creates a short summary table for the main language modeling and context length experiments. A Pandas DataFrame called `final_summary` is created. The first row stores the final validation loss of the main GPT language model after 600 training steps. The next three rows store the final validation losses from the context length experiment for context sizes 32, 64, and 128. The table does not include every result from Problem 5. Instead, it gives a simple final view of the most important validation loss values from the text experiments.

Output:

Experiment		Value
0	GPT Language Model	2.256198
1	Context 32 Validation Loss	2.481048
2	Context 64 Validation Loss	2.510472
3	Context 128 Validation Loss	2.515185

The main GPT language model achieved the lowest validation loss, with a final value of about 2.256.

This result should not be directly compared with the context experiment as if context length were the only difference. The main GPT model was trained for 600 steps, while the context length models were trained for only 200 steps. Because of this, the main model had more time to learn from the dataset.

Inside the context length experiment, the best validation loss was obtained with context length 32, which reached about 2.481. The loss increased slightly to about 2.510 for context length 64 and about 2.515 for context length 128.

These results confirm what we saw in the previous table and graphs. With the same short training budget, increasing the context length did not improve the validation loss.

The difference between the three context models is small, but the computational cost increased clearly, especially in GPU memory and training time for the longest context.

Overall, this final table gives a compact summary of the language modeling results and shows that the main trained GPT model performed well, while longer context lengths did not provide better validation performance under the limited 200 step experiment.

Analysis Questions

1. Visualize attention patterns using heatmaps. Do certain heads specialize?

The attention heatmaps show that the heads do not all behave in the same way. Some heads give more attention to nearby tokens, while others look at positions that are farther away in the sequence. This difference can also be seen in the numerical attention statistics. For example, Layer 2 Head 3 has a Previous Token Attention value of about 0.244, which is higher than the other heads in that layer. This suggests that this head pays more attention to the

token immediately before the current token. In contrast, some other heads have larger Mean Attention Distance values, which means that they use information from a wider part of the previous context.

The Attention Entropy values are also different between heads. A lower entropy means that a head focuses on a smaller number of positions, while a higher entropy means that its attention is spread more widely. For example, Layer 4 Head 1 has a relatively low entropy of about 1.97, so its attention is more selective. The heatmaps also show that the attention patterns become more varied in deeper layers. Therefore, the results suggest that some specialization does happen between attention heads. However, this does not mean that every head has one simple and fixed role. It only means that different heads learn different ways of using the previous context.

2. Experiment with increasing context length. How does performance degrade?

The experiment compared context lengths of 32, 64, and 128 while keeping the model architecture almost the same and training each model for 200 steps. The model with context length 32 achieved the lowest validation loss, about 2.481, and a perplexity of about 11.95. With context length 64, the validation loss increased slightly to about 2.510 and the perplexity increased to about 12.31. With context length 128, the validation loss was about 2.515 and the perplexity was about 12.37. So, in this experiment, increasing the context length did not improve the validation performance. The change in loss was small, but the computational cost increased much more clearly. Peak GPU memory increased from about 81.6 MB for context length 32 to about 133.5 MB for context length 64 and about 261.9 MB for context length 128. The time per training step also increased noticeably for context length 128.

This does not mean that a longer context is always worse. A longer context gives the model access to more previous information, which can be useful for more difficult dependencies. However, all three models were trained for only 200 steps. Under this limited training budget, the larger context models may not have had enough time to learn how to use the additional information effectively. Therefore, in our experiment, longer context mainly increased memory and computation cost, while the validation performance became slightly worse.

3. Why do tokens in deeper layers maintain spatial locality, where the token that came from the top left patch still encodes information about the top left content?

A patch token can still keep information about its original image region even after passing through several Transformer layers because the representation is updated instead of completely replaced. Self attention allows each token to collect information from other patches, but the Transformer also uses residual connections. These connections add the previous representation back to the new representation. Because of this, some information from the original patch can continue through deeper layers.

The token also keeps its position in the sequence. For example, the first patch token continues to be updated as the first token representation through the network. It can receive information from other patches, but it still contains part of the information that came from its original patch. Our results show that this locality becomes weaker as the

model becomes deeper. With positional encoding enabled, Same Patch Retrieval decreased from about 0.611 in Layer 1 to about 0.213 in Layer 4. Top Left Retrieval also decreased from about 0.835 to about 0.188. This means that deeper layers mix information from different image regions more strongly. However, the original spatial information is not immediately lost. The token gradually becomes a combination of its original patch information and information received from other patches.

Question: What role do positional encodings play in this phenomenon? Can this effect be removed?

Positional encoding gives the Transformer explicit information about where each patch is located. Without positional information, self attention mainly sees a set of patch representations and does not naturally know that one patch is from the top left and another is from the bottom right. Therefore, positional encoding helps the model connect visual content with spatial position. It makes it easier for the model to distinguish different image locations. However, our experiment shows that positional encoding is not the only reason why locality remains. When positional encoding was turned off during evaluation, Same Patch Retrieval was still about 0.476 in Layer 4 and Top Left Retrieval was still about 0.510. This means that some information about the original patch remained even without adding the positional embedding during that forward pass. The reason is that patch content itself can contain useful information, and residual connections also help preserve previous representations through the network. An important limitation is that the model was trained with positional encoding. We only turned positional encoding off during evaluation. Therefore, this experiment does not prove that a model trained completely without positional encoding would behave in exactly the same way. Based on our results, removing positional encoding alone does not completely remove spatial locality. It can reduce the explicit information about position, but other parts of the Transformer can still preserve information from the original patch. To test this more completely, a separate Transformer would need to be trained from the beginning without positional encoding and then compared with the original model.

Problem 6: Denoising Diffusion Probabilistic Models

Problem 6 focuses on Denoising Diffusion Probabilistic Models, also called DDPMs. The main idea of a diffusion model is different from the generative models used in the previous problems. Instead of generating an image directly from a random vector, a diffusion model first learns how noise is added to real images and then learns how to reverse this process. Starting from random noise, the trained model removes the noise gradually until a new image is produced.

For this problem, I used the CIFAR 10 dataset because the problem allows CIFAR 10 when computational resources are limited. CIFAR 10 contains small color images with a size of 32 by 32 pixels and 10 different classes. This dataset is suitable for studying diffusion because the images are small enough to train in Google Colab while still providing a real image generation task.

The first part of the problem is the forward diffusion process. I used 1000 diffusion steps with a linear noise schedule. During this process, a small amount of Gaussian noise is gradually added to a clean image. At the beginning, the original image is still clearly visible, but as the diffusion step increases, the image becomes more noisy. Near the final steps, most of the original visual information is hidden by noise. I visualized this process for five test images at different time steps so that the gradual destruction of the image could be seen clearly.

To learn the reverse process, I implemented a small U Net model. The network receives a noisy image together with the current diffusion time step. Sinusoidal time embeddings are used to represent the time information, which helps the model understand how much noise is expected at each stage of the diffusion process. The training objective is to predict the exact noise that was added to the clean image. Mean Squared Error is used to compare the predicted noise with the real noise.

After training, the model can perform the reverse diffusion process. It starts from random Gaussian noise and removes a small amount of predicted noise at every step. This process is repeated until a final image is produced. I saved intermediate results at diffusion times 1000, 750, 500, 250, 100, and 0. These images make it possible to observe how random noise slowly changes into a more structured image during generation.

I also generated 64 images using the trained unconditional DDPM. To evaluate the generated samples numerically, I calculated the Fréchet Inception Distance, or FID. FID compares features from real images with features from generated images. In general, a lower FID value means that the generated images are closer to the real data distribution. The DDPM result can also be compared with the GAN experiment from Problem 4, although the number of samples used for the two FID calculations is not exactly the same, so this comparison should be interpreted carefully.

The second part of the problem extends the basic diffusion model to conditional generation. In the unconditional model, the network only learns to generate images from the general CIFAR 10 distribution. In the conditional model, class label embeddings are added so that information about the desired image class can also be given to the U Net.

Classifier free guidance is also implemented in this part. During training, some class labels are intentionally removed and replaced with a special null label. Because of this, the same model learns both conditional and unconditional noise prediction. During generation, these two predictions are combined using a guidance scale. This allows the model to produce samples that follow the requested class information more strongly without needing a separate classifier.

After training the conditional model, I generated another set of 64 images and compared the conditional and unconditional generation results. Their training loss, generated samples, sampling time, and FID values were used to study the differences between the two approaches.

One important challenge in this problem is computation time. A DDPM does not generate an image with only one forward pass. In our implementation, generation uses a sequence of 1000 denoising steps. This makes sampling much slower than models such as GANs, even when the U Net itself is relatively small. For this reason, I used a compact U Net and CIFAR 10 to keep the experiment practical while still implementing the main DDPM components required in the problem.

Overall, this problem provides a practical understanding of how diffusion models learn image generation. The forward process shows how clean data is gradually converted into noise, while the reverse process shows how a trained neural network can recover structure from random noise. The conditional experiment also shows how class information and classifier free guidance can give more control over the generation process. Together, these experiments demonstrate the main ideas behind both unconditional and conditional diffusion models.

Explain the code of problem set 6:

The following sections break down the implementation details, the training pipeline, and the experimental setup used for these experiments.

Cell 1:

```
!pip -q install "torchmetrics[image]" torch-fidelity
```

This cell installs two packages that are needed later for evaluating the quality of generated images.

The first package is torchmetrics with the image option. It provides ready to use evaluation metrics for image tasks. In this problem, it is mainly used to calculate the Fréchet Inception Distance, or FID.

The second package is torch-fidelity. This package provides the pretrained Inception model and supporting tools that are needed for FID calculation.

The -q option is used to keep the installation output shorter and cleaner.

Output:

```
85.6/85.6
kB 8.1 MB/s eta 0:00:00
983.4/983.4 kB 61.7 MB/s eta 0:00:00
```

There is no error message in the output, so the installation was completed correctly.

This means the notebook now has the required tools for computing the FID score later in the DDPM experiment.

Cell 2:

```
import os
import time
import math
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import make_grid, save_image
from torchmetrics.image.fid import FrechetInceptionDistance
from google.colab import drive
SEED = 42
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
```

```

    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
set_seed()
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
if device.type != "cuda":
    raise RuntimeError(
        "GPU is not active. Enable GPU in Colab."
    )
torch.backends.cudnn.benchmark = True
print("Device:", device)
print(
    "GPU:",
    torch.cuda.get_device_name(0)
)

```

This cell imports the main libraries that are needed for Problem 6 and prepares the Google Colab environment for training the diffusion models.

The `os` library is used for working with folders and file paths. The `time` library is used to measure training and sampling time. The `math` library is needed for mathematical operations inside the time embedding. The `random` library is used for reproducibility.

NumPy and Pandas are imported for numerical calculations and storing experiment results in tables. Matplotlib is used later for displaying the diffusion process, generated images, and training curves.

PyTorch is imported for building and training the models. The `nn` and `functional` modules provide the neural network layers and operations that are needed for the U Net and diffusion process.

The `DataLoader` class is imported to create mini batches of images during training and testing.

Torchvision and its `transforms` module are used to load and prepare the CIFAR 10 dataset. The `make_grid` and `save_image` functions are used later to display and save groups of generated images.

FrechetInceptionDistance is imported from `torchmetrics`. This metric is used later to calculate the FID score and compare the generated images with real images.

The Google Colab drive module is also imported because important results and trained model files will be saved in Google Drive.

The random seed is set to 42. The `set_seed` function applies the same seed to Python, NumPy, and PyTorch. This helps make the experiments more reproducible.

After that, the code checks whether CUDA is available. If a GPU is active, it is selected as the main device. If CUDA is not available, the code stops and shows an error message because the diffusion experiments would be much slower on the CPU.

The cuDNN benchmark option is also enabled. This allows PyTorch to select efficient GPU operations for the image sizes used in the model and can improve training speed.

Finally, the selected device and GPU name are printed.

Output:

```
Device: cuda
GPU: Tesla T4
```

The output confirms that CUDA is active and the notebook is using a Tesla T4 GPU.

This means that the environment is correctly prepared for the DDPM experiments. The U Net training and the repeated denoising operations can now run on the GPU, which is much faster than using the CPU.

Cell 3:

```
drive.mount(
    "/content/drive"
)
OUTPUT_DIR = (
    "/content/drive/MyDrive/dl_pset6_results"
)
os.makedirs(
    OUTPUT_DIR,
    exist_ok=True
)
T = 1000
BATCH_SIZE = 128
```

```

BASE_CHANNELS = 32
TIME_DIM = 128
UNCOND_EPOCHS = 8
COND_EPOCHS = 8
NUM_CLASSES = 10
FID_SAMPLES = 64
GUIDANCE_SCALE = 2.0
print(
    "Output folder:",
    OUTPUT_DIR
)

```

This cell connects the notebook to Google Drive and defines the main settings that will be used for the DDPM experiments.

First, Google Drive is mounted to the Colab environment. This makes it possible to save trained models, generated images, tables, and other important results directly in Google Drive.

The OUTPUT_DIR variable defines the folder where the results of Problem 6 will be stored. The os.makedirs function creates this folder if it does not already exist. The exist_ok=True option prevents an error if the folder has already been created before.

After that, the main hyperparameters of the experiment are defined.

T is set to 1000, which means that the diffusion process uses 1000 noise steps. This follows the requirement of the problem.

The batch size is set to 128, so the model processes 128 images in each training batch.

BASE_CHANNELS is set to 32. This controls the basic number of feature channels used inside the U Net. Using a relatively small value keeps the model computationally manageable.

TIME_DIM is set to 128. This is the dimension used for the time embeddings that tell the U Net which diffusion step it is processing.

Both the unconditional and conditional models are trained for 8 epochs.

NUM_CLASSES is set to 10 because CIFAR 10 contains 10 image classes.

FID_SAMPLES is set to 64 because the problem asks us to generate 64 samples and calculate their FID score.

GUIDANCE_SCALE is set to 2.0. This value is later used in classifier free guidance to control how strongly the conditional generation follows the class information.

Finally, the output folder path is printed to confirm that the notebook is connected to the correct location in Google Drive.

Output:

```
Mounted at /content/drive
```

```
Output folder: /content/drive/MyDrive/dl_pset6_results
```

The output confirms that Google Drive was mounted successfully.

It also shows that all important files from the DDPM experiments will be saved inside the dl_pset6_results folder in Google Drive.

This is useful because the training results and generated images remain available even after the Colab runtime is disconnected or closed.

Cell 4:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
    )
])
train_dataset = (
    torchvision.datasets.CIFAR10(
        root="./data",
        train=True,
        download=True,
        transform=transform
    )
)
test_dataset = (
    torchvision.datasets.CIFAR10(
```

```

        root="./data",
        train=False,
        download=True,
        transform=transform
    )
)
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True,
    drop_last=True
)
test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True
)
print(
    "Training samples:",
    len(train_dataset)
)
print(
    "Test samples:",
    len(test_dataset)
)

```

This cell downloads and prepares the CIFAR 10 dataset for the diffusion experiments.

First, a transformation pipeline is created. The ToTensor operation converts each image into a PyTorch tensor. After that, Normalize changes the image values from the normal range of 0 to 1 into approximately the range from negative 1 to positive 1.

This normalization is useful for diffusion models because the generated samples and noisy images are usually processed in a centered numerical range. Later, the images can be converted back to the normal display range when we want to visualize them.

The training dataset is then loaded using `torchvision.datasets.CIFAR10`. The `train=True` option selects the training part of CIFAR 10, and `download=True` downloads the dataset if it is not already available.

The test dataset is loaded in the same way, but `train=False` selects the test portion.

After that, `DataLoader` objects are created for both datasets.

The training loader uses the batch size defined earlier, which is 128. The training data is shuffled so that the model sees the images in a different order during each epoch. The `drop_last=True` option removes the final smaller batch so that all training batches have the same size.

The test loader uses a batch size of 64 and does not shuffle the data because random ordering is not necessary for evaluation and visualization.

The `num_workers` value is set to 2 so that data loading can use two background workers. Pin memory is enabled to make data transfer to the GPU more efficient. Persistent workers are also used so that the loading workers stay active between epochs instead of being created again each time.

Finally, the code prints the number of images in the training and test datasets.

Output:

```
100%|██████████| 170M/170M [19:54<00:00, 143kB/s]
Training samples: 50000
Test samples: 10000
```

The first line shows that the CIFAR 10 dataset was downloaded successfully. The downloaded file was about 170 MB.

The training dataset contains 50,000 images, while the test dataset contains 10,000 images.

Each image belongs to one of the 10 CIFAR 10 classes and has a size of 32 by 32 pixels with three color channels.

This output confirms that the dataset was downloaded and prepared correctly and is ready to be used in the forward diffusion process and DDPM training.

Cell 5:

```
betas = torch.linspace(
    1e-4,
    0.02,
    T,
    device=device
)
alphas = (
    1.0 - betas
)
alphaBars = torch.cumprod(
    alphas,
    dim=0
)
alphaBarsPrevious = torch.cat([
    torch.ones(
        1,
        device=device
    ),
    alphaBars[:-1]
])
posterior_variance = (
    betas
    *
    (
        1.0
        - alphaBarsPrevious
    )
    /
    (
        1.0
        - alphaBars
    )
)
def extract(
    values,
    t,
```

```

        shape
    ):
        result = values.gather(
            0,
            t
        )
        return result.view(
            t.size(0),
            *(
                [1]
                * (
                    len(shape)
                    - 1
                )
            )
        )
def q_sample(
    x_start,
    t,
    noise=None
):
    if noise is None:
        noise = torch.randn_like(
            x_start
        )
    sqrt_alpha_bar = torch.sqrt(
        extract(
            alpha_bar,
            t,
            x_start.shape
        )
    )
    sqrt_one_minus = torch.sqrt(
        1.0
        -
        extract(
            alpha_bar,

```

```

        t,
        x_start.shape
    )
)
return (
    sqrt_alpha_bar
    * x_start
    +
    sqrt_one_minus
    * noise
)

```

This cell defines the noise schedule and the main forward diffusion function used in the DDPM.

First, the beta values are created using `torch.linspace`. The values start from 0.0001 and gradually increase to 0.02 across 1000 diffusion steps. This is a linear noise schedule.

The beta values control how much new noise is added at each step. Small beta values mean only a little noise is added, while larger beta values add more noise.

The alpha values are then calculated as 1 minus beta. Alpha represents how much of the previous image information is kept at each step.

After that, `alpha_bar` is calculated using the cumulative product of the alpha values. This value represents how much information from the original clean image remains after several diffusion steps.

The `alpha_bar_previous` tensor stores the cumulative alpha value from the previous time step. This is needed later when calculating the reverse diffusion variance.

The `posterior_variance` value is then calculated. This variance is used during the reverse denoising process to control how much random noise should be added when moving from one noisy step to the previous step.

The `extract` function is a helper function. It selects the correct value for each time step in a batch and reshapes it so that it can be multiplied directly with an image tensor.

The `q_sample` function implements the forward diffusion process. It receives a clean image, a time step, and optionally a noise tensor.

If noise is not provided, random Gaussian noise is generated automatically.

The function then combines the clean image and the noise using the $\alpha_{\bar{t}}$ values. At small time steps, the clean image has a larger effect. At large time steps, the noise becomes much stronger.

This allows the code to directly create the noisy version of an image at any diffusion step without applying all previous steps one by one.

This cell does not produce a visible output because it only defines the diffusion schedule and helper functions. These functions are used in the next cells to visualize and train the DDPM forward process.

Cell 6:

```
forward_images, _ = next(
    iter(test_loader)
)
forward_images = (
    forward_images[:5]
    .to(device)
)
fixed_forward_noise = (
    torch.randn_like(
        forward_images
    )
)
forward_steps = [
    0,
    100,
    250,
    500,
    750,
    1000
]
fig, axes = plt.subplots(
    5,
    len(forward_steps),
    figsize=(12, 10)
)
for column, step in enumerate(
    forward_steps
```

```

):
    if step == 0:
        noisy = forward_images
    else:
        t = torch.full(
            (5,),
            min(
                step - 1,
                T - 1
            ),
            device=device,
            dtype=torch.long
        )
        noisy = q_sample(
            forward_images,
            t,
            fixed_forward_noise
        )
    noisy = (
        (noisy + 1)
        / 2
    ).clamp(
        0,
        1
    )
    for row in range(5):

        axes[
            row,
            column
        ].imshow(
            noisy[row]
            .permute(
                1,
                2,
                0
            )
            .cpu()

```

```

    )
    axes[
        row,
        column
    ].axis(
        "off"
    )
    if row == 0:
        axes[
            row,
            column
        ].set_title(
            f"t = {step}"
        )
plt.tight_layout()
plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "forward_diffusion.png"
    )
)
plt.show()

```

Cell 6 uses the forward diffusion process to show how clean images are gradually destroyed by noise over time.

First, the code takes one batch from the test loader and selects the first 5 images. These images are moved to the GPU and used as fixed examples for visualization.

Then a fixed noise tensor is created with the same shape as the selected images. Using the same noise for all time steps is important, because it makes the change from one column to the next easier to see. This way, only the diffusion step changes, not the random noise pattern itself.

The list called `forward_steps` defines the time steps that will be visualized. These are 0, 100, 250, 500, 750, and 1000. Step 0 shows the original image, while the later steps show increasingly noisy versions.

A figure with 5 rows and 6 columns is then created. Each row corresponds to one image, and each column corresponds to one diffusion step.

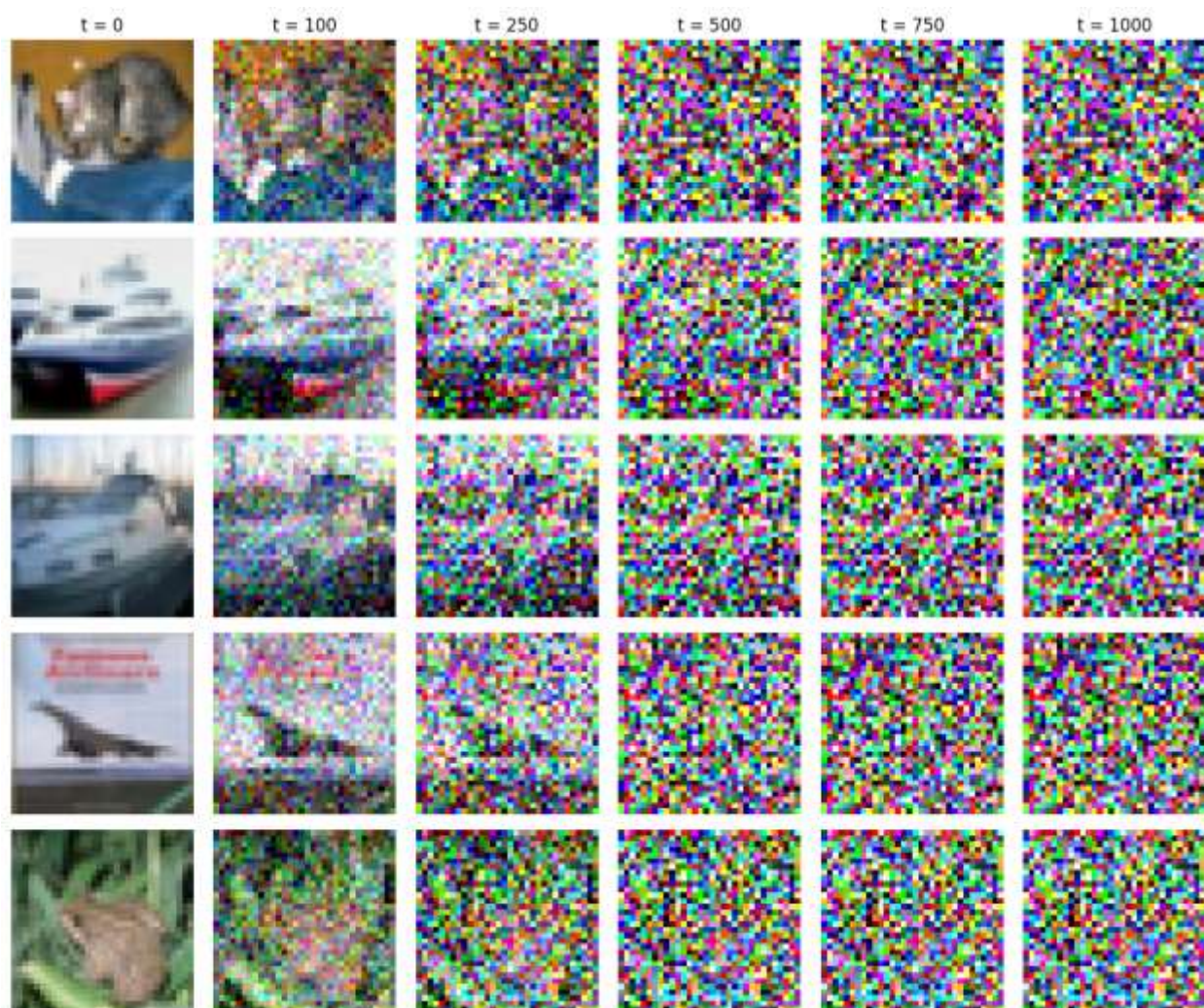
Inside the loop, if the step is 0, the original clean images are used directly. For the other steps, a tensor `t` is created that contains the same time step for all 5 images. The `q_sample` function is then used to generate the noisy version of each image at that step.

After that, the noisy images are converted from the normalized range back to the display range between 0 and 1. This is necessary because matplotlib expects image values in a visible range.

The nested loop displays each image in the correct position of the grid. The axes are turned off so the result looks clean, and the first row receives titles showing the current time step.

At the end, the figure is arranged neatly with `tight_layout`, saved to the output folder as `forward_diffusion.png`, and then displayed.

Output:



The output image clearly shows the behavior of the forward diffusion process. At $t = 0$, the original CIFAR-10 images are still clean and recognizable. At $t = 100$, noise is already strong and many details are damaged. As the time step increases to 250, 500, and 750, the images become more random and harder to identify. By $t = 1000$, almost all visual structure is gone and the images look like pure noise.

This output is important because it confirms that the forward diffusion process works correctly. It also gives a visual understanding of the learning task of the DDPM model, which is to reverse this corruption process and recover meaningful images from noisy inputs.

Cell 7:

```
class SinusoidalTimeEmbedding(
    nn.Module
):
    def __init__(
        self,
        dim
    ):
        super().__init__()
        self.dim = dim
    def forward(
        self,
        t
    ):
        half = (
            self.dim // 2
        )
        frequencies = torch.exp(
            -math.log(10000)
            *
            torch.arange(
                half,
                device=t.device
            )
            /
            (
```



```

        half - 1
    )
)
angles = (
    t.float()[ :, None]
    *
    frequencies[None, :]
)
return torch.cat(
    [
        angles.sin(),
        angles.cos()
    ],
    dim=1
)
class ResBlock(nn.Module):
    def __init__(
        self,
        in_channels,
        out_channels,
        time_dim,
        dropout=0.1
    ):
        super().__init__()

        self.norm1 = nn.GroupNorm(
            8,
            in_channels
        )
        self.conv1 = nn.Conv2d(
            in_channels,
            out_channels,
            3,
            padding=1
        )
        self.time_projection = (
            nn.Linear(
                time_dim,

```

```

        out_channels
    )
)
self.norm2 = nn.GroupNorm(
    8,
    out_channels
)
self.dropout = nn.Dropout(
    dropout
)

self.conv2 = nn.Conv2d(
    out_channels,
    out_channels,
    3,
    padding=1
)
if (
    in_channels
    != out_channels
):
    self.skip = nn.Conv2d(
        in_channels,
        out_channels,
        1
    )

else:
    self.skip = nn.Identity()
def forward(
    self,
    x,
    time_embedding
):
    h = self.conv1(
        F.silu(
            self.norm1(x)

```

```

    )
    )
    h = (
        h
        +
        self.time_projection(
            time_embedding
        )[:, :, None, None]
    )
    h = self.conv2(
        self.dropout(
            F.silu(
                self.norm2(h)
            )
        )
    )
    return (
        h
        +
        self.skip(x)
    )

```

This cell defines two important parts of the DDPM model. The first part creates the time embedding, and the second part creates a residual block that will later be used inside the U Net.

The `SinusoidalTimeEmbedding` class converts the diffusion time step into a vector. The model needs this information because the amount of noise is different at each diffusion step. For example, an image at an early step contains less noise, while an image near step 1000 contains much more noise.

The time step is converted into a set of sine and cosine values with different frequencies. This is similar to positional encoding in Transformer models. The final time embedding has the same dimension that was defined earlier, which is 128.

Using sine and cosine functions gives the model a smooth numerical representation of time. This helps the network understand the relationship between different diffusion steps instead of treating every step as a completely separate value.

The `ResBlock` class defines a residual block that processes image features inside the U Net.

First, Group Normalization is applied to the input. Group Normalization helps keep the feature values stable during training and works well even when the batch size is not very large.

After normalization, the SiLU activation function is applied and the result passes through a convolutional layer.

The time embedding is then passed through a Linear layer so that its size matches the number of output channels. This transformed time information is added to the image features. Because of this, every residual block knows which diffusion step it is currently processing.

The features then pass through another Group Normalization layer, SiLU activation, dropout, and a second convolutional layer.

The block also uses a residual connection. The original input is added to the processed output. If the number of input channels and output channels are different, a 1 by 1 convolution is used to make their shapes compatible. If the channel sizes are already the same, the input can pass directly through the skip connection.

The final output is therefore a combination of the newly learned features and the original input features.

This residual structure helps the model train more reliably and also allows information to pass through deeper parts of the U Net.

This cell does not produce a visible output because it only defines the time embedding and residual block classes that will be used to build the complete diffusion model in the next cell.

Cell 8:

```
class SmallUNet(nn.Module):
    def __init__(
        self,
        base=32,
        time_dim=128,
        num_classes=None
    ):
        super().__init__()
        self.num_classes = (
            num_classes
        )
        self.null_label = (
            num_classes
            if num_classes
```

```

        is not None
        else None
    )
    self.time_mlp = nn.Sequential(
        SinusoidalTimeEmbedding(
            time_dim
        ),
        nn.Linear(
            time_dim,
            time_dim
        ),
        nn.SiLU(),
        nn.Linear(
            time_dim,
            time_dim
        )
    )
    if num_classes is not None:
        self.label_embedding = (
            nn.Embedding(
                num_classes + 1,
                time_dim
            )
        )
    else:
        self.label_embedding = None
    self.input = nn.Conv2d(
        3,
        base,
        3,
        padding=1
    )
    self.down1 = ResBlock(
        base,
        base,
        time_dim
    )

```

```

self.downsample1 = nn.Conv2d(
    base,
    base * 2,
    4,
    2,
    1
)
self.down2 = ResBlock(
    base * 2,
    base * 2,
    time_dim
)
self.downsample2 = nn.Conv2d(
    base * 2,
    base * 4,
    4,
    2,
    1
)
self.middle1 = ResBlock(
    base * 4,
    base * 4,
    time_dim
)
self.middle2 = ResBlock(
    base * 4,
    base * 4,
    time_dim
)
self.upsample1 = (
    nn.ConvTranspose2d(
        base * 4,
        base * 2,
        4,
        2,
        1
    )
)

```

```

        self.up1 = ResBlock(
            base * 4,
            base * 2,
            time_dim
        )
        self.upsample2 = (
            nn.ConvTranspose2d(
                base * 2,
                base,
                4,
                2,
                1
            )
        )
        self.up2 = ResBlock(
            base * 2,
            base,
            time_dim
        )
        self.output_norm = (
            nn.GroupNorm(
                8,
                base
            )
        )
        self.output = nn.Conv2d(
            base,
            3,
            3,
            padding=1
        )
    def forward(
        self,
        x,
        t,
        labels=None
    ):
        time_embedding = (

```

```

        self.time_mlp(t)
    )
    if (
        self.label_embedding
        is not None
    ):
        if labels is None:
            labels = torch.full(
                (
                    x.size(0),
                ),
                self.null_label,
                device=x.device,
                dtype=torch.long
            )
        time_embedding = (
            time_embedding
            +
            self.label_embedding(
                labels
            )
        )
    x = self.input(x)

    h1 = self.down1(
        x,
        time_embedding
    )
    x = self.downsample1(
        h1
    )
    h2 = self.down2(
        x,
        time_embedding
    )
    x = self.downsample2(
        h2
    )

```



```

x = self.middle1(
    x,
    time_embedding
)
x = self.middle2(
    x,
    time_embedding
)
x = self.upsample1(
    x
)
x = torch.cat(
    [
        x,
        h2
    ],
    dim=1
)
x = self.up1(
    x,
    time_embedding
)
x = self.upsample2(
    x
)
x = torch.cat(
    [
        x,
        h1
    ],
    dim=1
)
x = self.up2(
    x,
    time_embedding
)
x = self.output(
    F.silu(

```

```

        self.output_norm(
            x
        )
    )
    )
    return x
def count_parameters(
    model
):
    return sum(
        parameter.numel()
        for parameter
        in model.parameters()
        if parameter.requires_grad
    )

```

This cell defines the complete U Net model that is used for the diffusion experiments.

The SmallUNet class receives three main settings. The base value controls the number of feature channels, time_dim controls the size of the time embedding, and num_classes determines whether the model is unconditional or conditional.

If num_classes is not given, the model works as an unconditional diffusion model. If num_classes is provided, the model also uses class label information.

The model first creates a time processing network called time_mlp. The diffusion step is converted into a sinusoidal time embedding and then passed through two Linear layers with a SiLU activation between them. This gives the network a learned representation of the current diffusion time.

For the conditional version, the model also creates a label embedding. Since CIFAR 10 has 10 classes, one extra label is added for the null condition. This extra label is later used for classifier free guidance when the class information is intentionally removed.

The first convolution receives a 3 channel RGB image and converts it into the basic feature representation.

The downsampling part of the U Net contains two residual blocks and two downsampling layers. The first residual block keeps the number of channels at the base value. Then the first downsampling convolution reduces the spatial size while increasing the number of channels.

The second residual block processes these deeper features, and another downsampling layer reduces the image resolution again while increasing the feature channels to four times the base size.

The middle part contains two residual blocks. This is the deepest part of the U Net, where the model processes compact feature maps with more channels.

After that, the upsampling part begins. A transposed convolution increases the spatial resolution. The result is then joined with the saved feature map from the downsampling path using `torch.cat`.

This connection between the downsampling and upsampling sides is an important part of the U Net architecture. It allows the decoder to reuse detailed information from earlier layers.

The same process happens again in the second upsampling stage. The spatial resolution is increased, and the result is combined with the earlier feature map `h1`.

After the final residual block, Group Normalization and SiLU activation are applied. The last convolution converts the features back into 3 channels.

The output of the network has the same shape as the input image. In this DDPM, the output does not represent the final clean image directly. Instead, it represents the noise that the model predicts was added to the image at the given diffusion step.

Inside the forward function, the time embedding is calculated first. If the model is conditional, the class embedding is added to the time embedding. If no class label is provided, the special null label is used instead.

The image then moves through the downsampling layers, middle blocks, and upsampling layers. The saved feature maps `h1` and `h2` are used as skip connections between the two sides of the U Net.

Finally, the `count_parameters` function is defined. This helper function calculates the total number of trainable parameters in a model.

This cell does not produce a visible output because it only defines the U Net architecture and the parameter counting function.

Cell 9:

```
def train_diffusion(  
    model,  
    loader,  
    epochs,  
    conditional=False,
```

```

label_drop=0.1
):
    model = model.to(
        device
    )
    optimizer = (
        torch.optim.AdamW(
            model.parameters(),
            lr=2e-4,
            weight_decay=1e-4
        )
    )
    scaler = torch.amp.GradScaler(
        "cuda"
    )
    history = []
    total_start = (
        time.perf_counter()
    )
    for epoch in range(
        1,
        epochs + 1
    ):
        model.train()
        losses = []
        epoch_start = (
            time.perf_counter()
        )
        for images, labels in loader:
            images = images.to(
                device,
                non_blocking=True
            )
            labels = labels.to(
                device,
                non_blocking=True
            )
            batch_size = (

```

```

        images.size(0)
    )
    t = torch.randint(
        0,
        T,
        (
            batch_size,
        ),
        device=device
    )
    noise = torch.randn_like(
        images
    )
    noisy_images = q_sample(
        images,
        t,
        noise
    )
    train_labels = None
    if conditional:
        train_labels = (
            labels.clone()
        )
        drop_mask = (
            torch.rand(
                batch_size,
                device=device
            )
            < label_drop
        )
        train_labels[
            drop_mask
        ] = NUM_CLASSES

    optimizer.zero_grad(
        set_to_none=True
    )
    with torch.amp.autocast(

```

```

        "cuda"
    ):
        predicted_noise = model(
            noisy_images,
            t,
            train_labels
        )
        loss = F.mse_loss(
            predicted_noise,
            noise
        )
        scaler.scale(
            loss
        ).backward()
        scaler.step(
            optimizer
        )
        scaler.update()
        losses.append(
            loss.item()
        )
    epoch_time = (
        time.perf_counter()
        - epoch_start
    )
    mean_loss = np.mean(
        losses
    )
    history.append({
        "Epoch": epoch,
        "Loss": mean_loss,
        "Epoch Time":
            epoch_time
    })
    print(
        f"Epoch {epoch:02d}/{epochs} | "
        f"Loss {mean_loss:.4f} | "
        f"time {epoch_time:.1f}s"
    )

```

```

    )
    total_time = (
        time.perf_counter()
        - total_start
    )
    return (
        model,
        pd.DataFrame(
            history
        ),
        total_time
    )

```

This cell defines the complete U Net model that is used for the diffusion experiments.

The SmallUNet class receives three main settings. The base value controls the number of feature channels, time_dim controls the size of the time embedding, and num_classes determines whether the model is unconditional or conditional.

If num_classes is not given, the model works as an unconditional diffusion model. If num_classes is provided, the model also uses class label information.

The model first creates a time processing network called time_mlp. The diffusion step is converted into a sinusoidal time embedding and then passed through two Linear layers with a SiLU activation between them. This gives the network a learned representation of the current diffusion time.

For the conditional version, the model also creates a label embedding. Since CIFAR 10 has 10 classes, one extra label is added for the null condition. This extra label is later used for classifier free guidance when the class information is intentionally removed.

The first convolution receives a 3 channel RGB image and converts it into the basic feature representation.

The downsampling part of the U Net contains two residual blocks and two downsampling layers. The first residual block keeps the number of channels at the base value. Then the first downsampling convolution reduces the spatial size while increasing the number of channels.

The second residual block processes these deeper features, and another downsampling layer reduces the image resolution again while increasing the feature channels to four times the base size.

The middle part contains two residual blocks. This is the deepest part of the U Net, where the model processes compact feature maps with more channels.

After that, the upsampling part begins. A transposed convolution increases the spatial resolution. The result is then joined with the saved feature map from the downsampling path using `torch.cat`.

This connection between the downsampling and upsampling sides is an important part of the U Net architecture. It allows the decoder to reuse detailed information from earlier layers.

The same process happens again in the second upsampling stage. The spatial resolution is increased, and the result is combined with the earlier feature map `h1`.

After the final residual block, Group Normalization and SiLU activation are applied. The last convolution converts the features back into 3 channels.

The output of the network has the same shape as the input image. In this DDPM, the output does not represent the final clean image directly. Instead, it represents the noise that the model predicts was added to the image at the given diffusion step.

Inside the forward function, the time embedding is calculated first. If the model is conditional, the class embedding is added to the time embedding. If no class label is provided, the special null label is used instead.

The image then moves through the downsampling layers, middle blocks, and upsampling layers. The saved feature maps `h1` and `h2` are used as skip connections between the two sides of the U Net.

Finally, the `count_parameters` function is defined. This helper function calculates the total number of trainable parameters in a model.

This cell does not produce a visible output because it only defines the U Net architecture and the parameter counting function.

Cell 10:

```
set_seed()
unconditional_model = SmallUNet(
    base=BASE_CHANNELS,
    time_dim=TIME_DIM
).to(device)
print(
    "Unconditional parameters:",
    count_parameters(
        unconditional_model
    )
)
```



```

)
(
    unconditional_model,
    unconditional_history,
    unconditional_training_time
) = train_diffusion(
    unconditional_model,
    train_loader,
    UNCOND_EPOCHS,
    conditional=False
)
torch.save(
    unconditional_model.state_dict(),
    os.path.join(
        OUTPUT_DIR,
        "unconditional_ddpm.pt"
    )
)
unconditional_history.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "unconditional_history.csv"
    ),
    index=False
)
print(
    "Training time:",
    round(
        unconditional_training_time
        / 60,
        2
    ),
    "minutes"
)

```

This cell creates and trains the unconditional DDPM model.

First, the random seed is set again to keep the experiment reproducible.

The SmallUNet model is then created using the base channel size and time embedding dimension that were defined earlier. Since no number of classes is given, this model is unconditional. This means that it learns to generate images without using class labels.

The model is moved to the GPU, and the `count_parameters` function is used to calculate the number of trainable parameters.

After that, the `train_diffusion` function is called. The model is trained on the CIFAR 10 training set for 8 epochs. The conditional option is set to `False`, so only the noisy image and diffusion time are used during training.

During each training step, random noise is added to the original images. The U Net tries to predict this noise, and Mean Squared Error is used to measure the difference between the real noise and the predicted noise.

The function returns the trained model, the training history, and the total training time.

After training, the model parameters are saved in Google Drive as `unconditional_ddpm.pt`. The training history is also saved as a CSV file. Saving these files is useful because the trained model and loss values can be used again without repeating the full training process.

Finally, the total training time is converted to minutes and printed.

Output:

```
Unconditional parameters: 1254051
Epoch 01/8 | Loss 0.1686 | time 35.9s
Epoch 02/8 | Loss 0.0653 | time 20.5s
Epoch 03/8 | Loss 0.0557 | time 19.0s
Epoch 04/8 | Loss 0.0488 | time 18.9s
Epoch 05/8 | Loss 0.0461 | time 21.1s
Epoch 06/8 | Loss 0.0442 | time 20.3s
Epoch 07/8 | Loss 0.0427 | time 21.2s
Epoch 08/8 | Loss 0.0416 | time 19.5s
Training time: 2.94 minutes
```

The output shows that the unconditional U Net has 1,254,051 trainable parameters.

The training loss decreases strongly during the 8 epochs. In the first epoch, the loss is about 0.1686. By the second epoch, it drops to about 0.0653, and after that it continues to decrease more slowly.

At the final epoch, the loss reaches about 0.0416. This steady decrease shows that the model is learning to predict the added noise more accurately.

The largest improvement happens during the first few epochs, while the later epochs show smaller improvements. This is normal because the model learns the easier patterns first and then makes smaller refinements.

The total training time is about 2.94 minutes on the Tesla T4 GPU. This is relatively short because the U Net was designed to be small enough for the available computational resources.

Overall, the output shows that the unconditional DDPM training was stable and successful. The model learned the noise prediction task and is now ready to be used for the reverse denoising and image generation process.

Cell 11:

```
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    unconditional_history[
        "Epoch"
    ],
    unconditional_history[
        "Loss"
    ],
    marker="o"
)
plt.xlabel(
    "Epoch"
)
plt.ylabel(
    "Noise Prediction Loss"
)
plt.title(
    "Unconditional DDPM Training Loss"
)
plt.show()
```

This cell creates a graph to show how the noise prediction loss changed during the training of the unconditional DDPM.

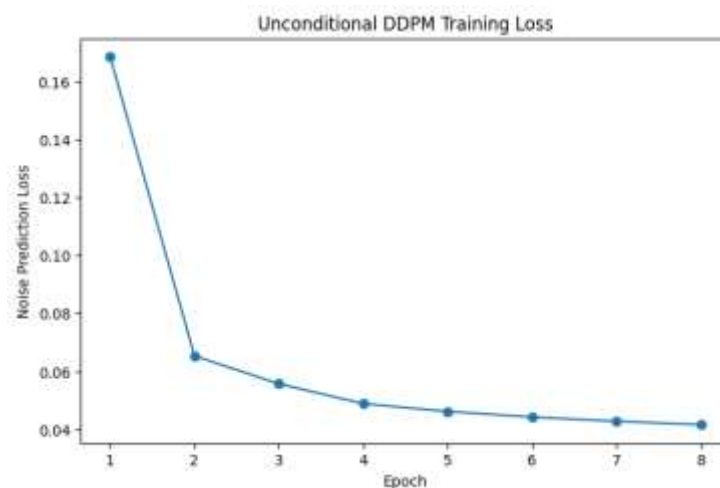
The code uses the training history that was saved in the previous cell. The epoch numbers are placed on the horizontal axis, and the noise prediction loss is placed on the vertical axis.

Circular markers are added to make the loss value of each epoch easier to see.

The title of the figure is Unconditional DDPM Training Loss.

The purpose of this graph is to make the training behavior easier to understand visually. Instead of only reading the numerical loss values, we can clearly see whether the model is improving over time.

Output:



The graph shows that the noise prediction loss decreases during training.

The largest decrease happens between the first and second epochs. The loss starts at about 0.1686 and quickly drops to about 0.0653.

After that, the loss continues to decrease more gradually. It reaches about 0.0557 at epoch 3, about 0.0488 at epoch 4, and finally about 0.0416 at epoch 8.

This pattern shows that the model learned quickly during the beginning of training and then continued to improve with smaller changes in the later epochs.

The curve is smooth and does not show large unstable increases. This suggests that the unconditional DDPM training process was stable.

Overall, the graph confirms the numerical results from the previous cell and shows that the model became better at predicting the added noise as training continued.

Cell 12:

```
def reverse_step(
    x,
    t,
    predicted_noise
):
    beta_t = extract(
        betas,
        t,
        x.shape
    )
    alpha_t = extract(
        alphas,
        t,
        x.shape
    )

    alpha_bar_t = extract(
        alphaBars,
        t,
        x.shape
    )
    variance_t = extract(
        posterior_variance,
        t,
        x.shape
    )
    mean = (
        x
        -
        (
            beta_t
            /
            torch.sqrt(
                1.0
                - alpha_bar_t
            )
        )
        * predicted_noise
    )
```

```

        )
    )
    *
    predicted_noise
)
mean = (
    mean
    /
    torch.sqrt(
        alpha_t
    )
)
noise = torch.randn_like(
    x
)
nonzero = (
    t != 0
).float().view(
    -1,
    1,
    1,
    1
)
return (
    mean
    +
    nonzero
    *
    torch.sqrt(
        variance_t
    )
    *
    noise
)
@torch.no_grad()
def sample_ddpm(
    model,
    n_samples,

```

```

labels=None,
guidance_scale=None,
capture=False
):
    model.eval()
    if labels is not None:

        labels = labels.to(
            device
        )
    x = torch.randn(
        n_samples,
        3,
        32,
        32,
        device=device
    )
    snapshots = {}
    if capture:
        snapshots[1000] = (
            x.detach()
            .cpu()
            .clone()
        )
    capture_map = {
        749: 750,
        499: 500,
        249: 250,
        99: 100
    }
    start = time.perf_counter()
    for step in range(
        T - 1,
        -1,
        -1
    ):
        if (
            capture

```

```

        and step
        in capture_map
    ):
        snapshots[
            capture_map[
                step
            ]
        ] = (
            x.detach()
            .cpu()
            .clone()
        )
    t = torch.full(
        (
            n_samples,
        ),
        step,
        device=device,
        dtype=torch.long
    )
    if (
        labels is not None
        and guidance_scale
        is not None
    ):
        null_labels = (
            torch.full_like(
                labels,
                model.null_label
            )
        )
        unconditional_noise = (
            model(
                x,
                t,
                null_labels
            )
        )

```



```

        conditional_noise = (
            model(
                x,
                t,
                labels
            )
        )
        predicted_noise = (
            unconditional_noise
            +
            guidance_scale
            *
            (
                conditional_noise
                -
                unconditional_noise
            )
        )
    else:
        predicted_noise = model(
            x,
            t,
            labels
        )
    x = reverse_step(
        x,
        t,
        predicted_noise
    )
    sampling_time = (
        time.perf_counter()
        - start
    )
    x = x.clamp(
        -1,
        1
    )
    if capture:

```

```

        snapshots[0] = (
            x.detach()
            .cpu()
            .clone()
        )
    return (
        x.cpu(),
        snapshots,
        sampling_time
    )
def to_image_range(
    images
):
    return (
        (images + 1)
        / 2
    ).clamp(
        0,
        1
    )

```

This cell defines the functions that are used for the reverse diffusion process and image generation.

The first function is `reverse_step`. Its job is to move one step backward in the diffusion process. In the forward process, noise is gradually added to an image. In the reverse process, the model tries to remove this noise step by step.

The function receives the current noisy image `x`, the current diffusion step `t`, and the noise predicted by the U Net.

First, it extracts the beta, alpha, cumulative alpha, and posterior variance values that belong to the current time step. These values come from the diffusion schedule that was created earlier.

The code then calculates the mean of the reverse distribution. The predicted noise is removed from the current image using the DDPM reverse equation. The result is divided by the square root of alpha so that the image moves toward the previous, slightly cleaner diffusion state.

Random Gaussian noise is also created. This random noise is added during most reverse diffusion steps because DDPM sampling is probabilistic.

The nonzero value checks whether the current time step is zero. At the final step, no additional random noise is added because the generation process has reached the final image.

The second function is `sample_ddpm`. This function performs the complete reverse diffusion process.

The model is first switched to evaluation mode. Gradient calculations are disabled because the model is only being used for generation and is not being trained.

The generation process starts from completely random Gaussian noise with the shape of CIFAR 10 images. Each sample therefore begins as a 3 channel image with a size of 32 by 32 pixels.

The function then performs the reverse process from step 999 down to step 0. At every step, the U Net predicts the noise that should be removed, and `reverse_step` creates the next cleaner version of the image.

The capture option is used when we want to save intermediate images during generation. The initial random noise is stored as step 1000, and additional snapshots are saved around the requested stages 750, 500, 250, and 100. The final generated image is stored as step 0. These snapshots are used later to visualize how an image slowly appears during denoising.

The function can also perform conditional generation. If class labels and a guidance scale are provided, the model makes two noise predictions.

One prediction is unconditional and uses the special null label. The second prediction is conditional and uses the requested class label.

These two predictions are combined using classifier free guidance. The guidance scale controls how strongly the final prediction follows the class information. In this experiment, the guidance scale is set to 2.0.

If labels or a guidance scale are not provided, the model simply makes one normal noise prediction. This is the behavior used for the unconditional DDPM.

The total sampling time is also measured because diffusion models require many reverse steps and can be slow during generation.

After all 1000 reverse steps are finished, the generated values are limited to the range from negative 1 to positive 1. The final images, saved snapshots, and total sampling time are then returned.

The last function, `to_image_range`, converts image values from the model range of negative 1 to positive 1 into the normal display range from 0 to 1. This makes the generated images ready to be displayed with Matplotlib or saved as image files.

This cell does not produce a visible output because it only defines the functions needed for reverse diffusion, sampling, classifier free guidance, and image visualization.

Cell 13:

```
(
    reverse_final,
    reverse_snapshots,
    reverse_time
) = sample_ddpm(
    unconditional_model,
    n_samples=5,
    capture=True
)
reverse_steps = [
    1000,
    750,
    500,
    250,
    100,
    0
]
fig, axes = plt.subplots(
    5,
    len(reverse_steps),
    figsize=(12, 10)
)
for column, step in enumerate(
    reverse_steps
):
    images = to_image_range(
        reverse_snapshots[
            step
        ]
    )
    for row in range(5):
        axes[
            row,
            column
        ].imshow(
```

```

        images[row]
        .permute(
            1,
            2,
            0
        )
    )
    axes[
        row,
        column
    ].axis(
        "off"
    )
    if row == 0:
        axes[
            row,
            column
        ].set_title(
            f"t = {step}"
        )
plt.tight_layout()
plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "reverse_denoising.png"
    )
)
plt.show()

```

Cell 13 shows the reverse denoising process of the unconditional DDPM.

In this cell, the model generates 5 images by starting from pure random noise. The function `sample_ddpm` is called with the trained unconditional model, and the capture option is set to `True`. This means the function does not only return the final generated images, but also saves intermediate images at different time steps during the reverse diffusion process.

The variable `reverse_final` contains the final generated samples after the full denoising process is finished. The variable `reverse_snapshots` contains the saved intermediate states of the same samples. The variable `reverse_time` stores the total time needed for sampling.

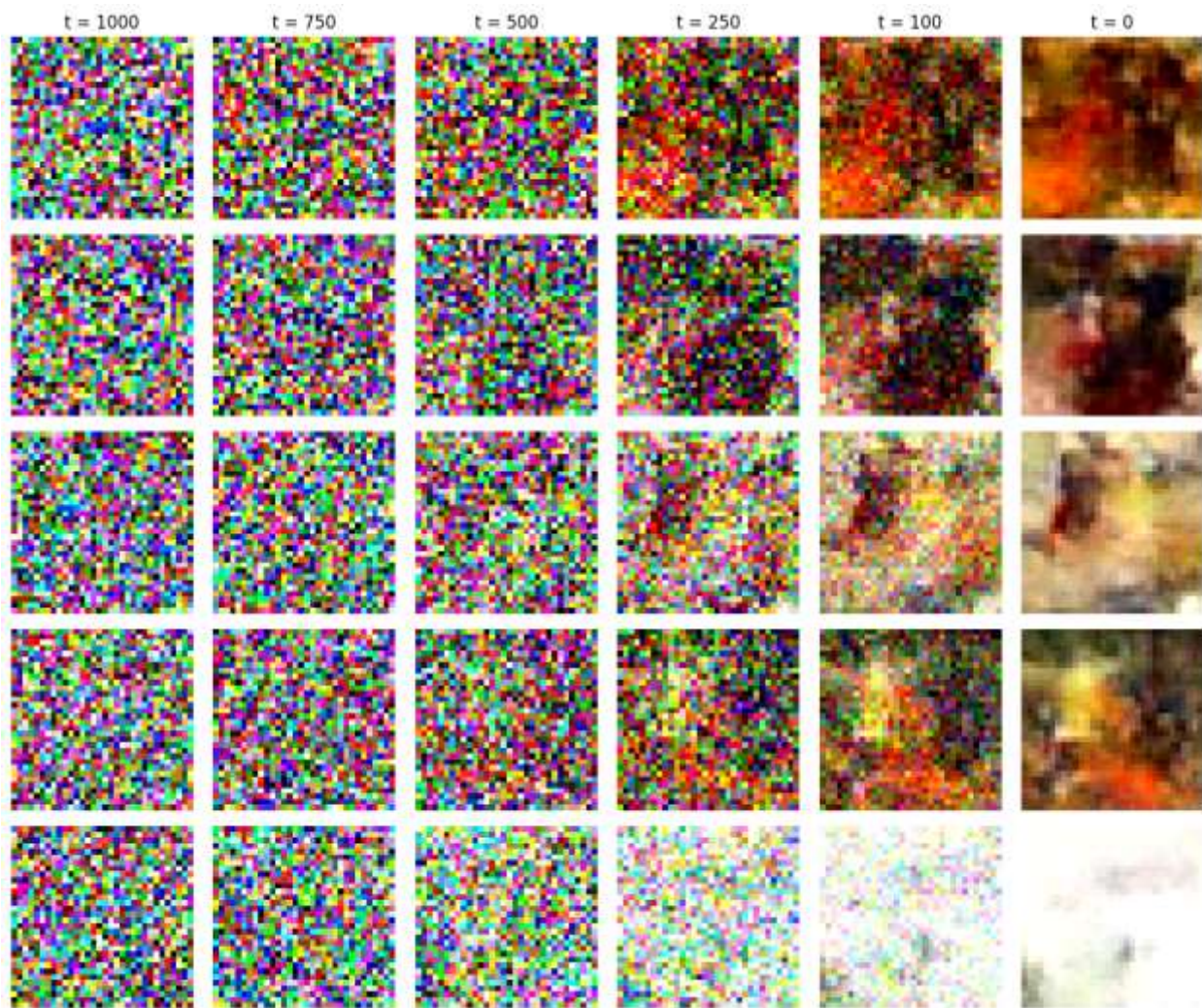
The code then defines the selected reverse steps as 1000, 750, 500, 250, 100, and 0. These values represent different moments during the reverse process. At $t = 1000$, the images are almost completely random noise. As the time step decreases, the model gradually removes noise and tries to recover meaningful visual structure.

A figure with 5 rows and 6 columns is created. Each row represents one generated sample, and each column shows the same sample at a different reverse step. Before displaying the images, the code converts them from the model range into the normal image range by using the `to_image_range` function. This makes the images suitable for visualization.

For each saved step, the corresponding images are plotted. The axes are turned off to keep the figure clean, and the title of the first row shows the time step for each column.

After plotting, the figure is adjusted with `tight_layout` so the images are arranged neatly. Then the result is saved in the output folder with the name `reverse_denoising.png`. Finally, the figure is displayed.

Output:



The output image clearly shows how DDPM generation works in practice. On the left side, the samples are fully noisy. As we move to the right, the noise is reduced step by step, and by the final column at $t = 0$, each sample becomes a blurry but meaningful generated image. This output is important because it directly demonstrates the reverse denoising behavior of the diffusion model.

Cell 14:

```
(
    unconditional_samples,
    _,
    unconditional_sampling_time
) = sample_ddpm(
    unconditional_model,
```

```

        n_samples=64
    )
    grid = make_grid(
        to_image_range(
            unconditional_samples
        ),
        nrow=8
    )
    plt.figure(
        figsize=(8, 8)
    )
    plt.imshow(
        grid.permute(
            1,
            2,
            0
        )
    )
    plt.axis(
        "off"
    )
    plt.title(
        "64 Unconditional DDPM Samples"
    )
    plt.show()
    save_image(
        to_image_range(
            unconditional_samples
        ),
        os.path.join(
            OUTPUT_DIR,
            "unconditional_samples.png"
        ),
        nrow=8
    )
    print(
        "Sampling time:",
        round(

```



```

        unconditional_sampling_time,
        2
    ),
    "seconds"
)

```

Cell 14 generates 64 new images with the trained unconditional DDPM and displays them in one grid. In this cell, the `sample_ddpm` function is used again, but this time it generates 64 images instead of 5. The trained `unconditional_model` starts from random noise and applies the reverse diffusion process step by step until final images are produced. The variable `unconditional_samples` stores the generated images. The second returned value is ignored here because no intermediate snapshots are needed. The variable `unconditional_sampling_time` stores the total time required to generate these 64 samples. After sampling, the generated images are converted to a normal image range by using the `to_image_range` function. Then `make_grid` arranges the 64 images into one large grid with 8 images in each row. This makes the output easier to view and compare. The code then creates a figure with size 8 by 8 and displays the grid with `plt.imshow`. The axes are removed so the focus stays only on the generated samples. A title is added to show that these are 64 unconditional DDPM samples. Next, the same generated images are saved into the output folder with the name `unconditional_samples.png`. The `save_image` function stores them in an 8 by 8 layout, so the saved file matches the displayed result. At the end, the cell prints the total sampling time. In this run, generating the 64 images took about 9.2 seconds.

Output:



Sampling time: 9.2 seconds

The output image shows that the model is able to generate different CIFAR-10-like samples from pure noise. The samples are still blurry and not very sharp, but many of them already contain rough object shapes, colors, and scene structure. This is normal for a small DDPM trained for a limited number of epochs.

Cell 15:

```
def compute_fid(
    real_images,
    fake_images
):
    metric = (
        FrechetInceptionDistance(
            feature=2048,
            normalize=True
        )
        .to(device)
    )
    metric.update(
        to_image_range(
            real_images
        ).to(device),
        real=True
    )
    metric.update(
        to_image_range(
            fake_images
        ).to(device),
        real=False
    )
    score = (
        metric.compute()
        .item()
    )
    del metric
    torch.cuda.empty_cache()
```

```

        return score
    real_fid_images, _ = next(
        iter(test_loader)
    )
    real_fid_images = (
        real_fid_images[
            :FID_SAMPLES
        ]
    )
    unconditional_fid = compute_fid(
        real_fid_images,
        unconditional_samples
    )
    print(
        "Unconditional DDPM FID:",
        unconditional_fid
    )

```

This cell calculates the FID score for the unconditional DDPM samples.

The `compute_fid` function receives two groups of images. The first group contains real CIFAR 10 images, and the second group contains images generated by the diffusion model.

Inside the function, the `FrechetInceptionDistance` metric is created with 2048 feature dimensions. The `normalize` option is enabled because the images are converted to the normal range from 0 to 1 before they are passed to the metric.

The real images are first converted using the `to_image_range` function and added to the metric with `real=True`.

The generated images are processed in the same way and added with `real=False`.

After both groups are added, `metric.compute()` calculates the final FID score.

FID compares the feature distribution of real images with the feature distribution of generated images. In general, a lower FID value means that the generated images are closer to the real images in terms of visual features.

After the score is calculated, the metric object is deleted and the CUDA cache is cleared to free GPU memory.

The code then takes one batch from the test loader and keeps only the first 64 real images, because `FID_SAMPLES` was set to 64.

Finally, the FID score is calculated between these 64 real images and the 64 unconditional DDPM samples generated in the previous cell.

Output:

```
Downloading: "https://github.com/toshas/torch-
fidelity/releases/download/v0.2.0/weights-inception-2015-12-05-6726825d.pth"
to /root/.cache/torch/hub/checkpoints/weights-inception-2015-12-05-
6726825d.pth
100%|██████████| 91.2M/91.2M [00:02<00:00, 33.3MB/s]
Unconditional DDPM FID: 292.8779296875
```

The unconditional DDPM achieved an FID score of about 292.88.

This is a relatively high FID value, which means that the generated image distribution is still quite different from the real CIFAR 10 image distribution.

This result is consistent with the generated samples from the previous cell. The images contain some useful colors and rough object structure, but many of them are still blurry and not very realistic.

One important point is that this FID score was calculated using only 64 real images and 64 generated images. FID is usually more stable when it is calculated with a much larger number of samples. Because only 64 samples were used here, the exact value can change noticeably between different runs.

However, the problem asks us to generate 64 samples and calculate the FID score, so this experiment follows the required setup.

Overall, the FID value gives a numerical confirmation that the small DDPM learned some image structure, but its generation quality is still limited after only 8 training epochs.

Cell 16:

```
set_seed()
conditional_model = SmallUNet(
    base=BASE_CHANNELS,
    time_dim=TIME_DIM,
    num_classes=NUM_CLASSES
).to(device)
```

```
print(
    "Conditional parameters:",
    count_parameters(
        conditional_model
    )
)
```

This cell creates the conditional version of the diffusion model.

First, the random seed is set again to make the model initialization more reproducible.

Then a new SmallUNet model is created using the same base channel size and time embedding dimension as the unconditional model. The main difference is that `num_classes` is now set to `NUM_CLASSES`, which is 10 for CIFAR 10.

Because the number of classes is provided, this version of the U Net also creates a class label embedding. This allows the model to receive information about which CIFAR 10 class should be generated.

The model also includes one extra null label in addition to the 10 normal classes. This special label is needed for classifier free guidance. During training, some class labels are replaced with this null label so that the same network can learn both conditional and unconditional predictions.

After creating the model, it is moved to the GPU.

Finally, the `count_parameters` function is used to calculate the total number of trainable parameters in the conditional model.

Output:

```
Conditional parameters: 1255459
```

The output shows that the conditional U Net contains 1,255,459 trainable parameters.

This number is slightly larger than the unconditional model, which had 1,254,051 parameters.

The difference is small because both models use the same U Net architecture. The additional parameters mainly come from the class label embedding that was added to the conditional model.

This confirms that the conditional DDPM was created successfully and is ready to be trained using both image information and class labels.

Cell 17:

```
(
    conditional_model,
    conditional_history,
    conditional_training_time
) = train_diffusion(
    conditional_model,
    train_loader,
    COND_EPOCHS,
    conditional=True,
    label_drop=0.1
)
torch.save(
    conditional_model.state_dict(),
    os.path.join(
        OUTPUT_DIR,
        "conditional_ddpm.pt"
    )
)
conditional_history.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "conditional_history.csv"
    ),
    index=False
)
print(
    "Conditional training time:",
    round(
        conditional_training_time
        / 60,
        2
    ),
    "minutes"
)
```

This cell trains the conditional DDPM model.

The `train_diffusion` function is called with the conditional model, the CIFAR 10 training loader, and the number of conditional training epochs, which is 8.

The conditional option is set to `True`. This means that the model receives class labels during training.

The `label_drop` value is set to 0.1. This means that about 10% of the class labels are randomly replaced with the special null label during training. This is necessary for classifier free guidance because the same model needs to learn both conditional and unconditional noise prediction.

During each training batch, random diffusion steps are selected and Gaussian noise is added to the clean images. The model then receives the noisy images, the diffusion time steps, and the class information, and it tries to predict the added noise.

The prediction is compared with the real noise using Mean Squared Error. The optimizer then updates the model parameters.

After training is finished, the trained model is saved in Google Drive as `conditional_ddpm.pt`.

The training history is also saved as `conditional_history.csv`. This file contains the loss and epoch time values and can be used later without repeating the training process.

Finally, the total training time is converted to minutes and printed.

Output:

```
Epoch 01/8 | Loss 0.1956 | time 22.6s
Epoch 02/8 | Loss 0.0661 | time 20.4s
Epoch 03/8 | Loss 0.0529 | time 21.4s
Epoch 04/8 | Loss 0.0486 | time 19.8s
Epoch 05/8 | Loss 0.0457 | time 20.0s
Epoch 06/8 | Loss 0.0435 | time 21.4s
Epoch 07/8 | Loss 0.0420 | time 20.1s
Epoch 08/8 | Loss 0.0405 | time 21.5s
Conditional training time: 2.79 minutes
```

The output shows that the conditional DDPM trained successfully.

The loss starts at about 0.1956 in the first epoch and drops quickly to about 0.0661 in the second epoch.

After that, the loss continues to decrease more gradually. It reaches about 0.0529 in epoch 3, about 0.0457 in epoch 5, and finally about 0.0405 in epoch 8.

This shows that the model became better at predicting the added noise while also learning to use the class information.

The final conditional loss of about 0.0405 is slightly lower than the final unconditional loss of about 0.0416. The difference is small, so we should not make a strong conclusion from loss alone. However, it shows that adding the class information did not make the training unstable.

The total training time was about 2.79 minutes, which is very close to the training time of the unconditional model.

Overall, the conditional model trained in a stable way and is now ready for classifier free guidance and class controlled image generation.

Cell 18:

```
plt.figure(
    figsize=(8, 5)
)
plt.plot(
    conditional_history[
        "Epoch"
    ],
    conditional_history[
        "Loss"
    ],
    marker="o"
)
plt.xlabel(
    "Epoch"
)
plt.ylabel(
    "Noise Prediction Loss"
)
plt.title(
    "Conditional DDPM Training Loss"
)
plt.show()
```

This cell creates a graph to show how the noise prediction loss changed during the training of the conditional DDPM.

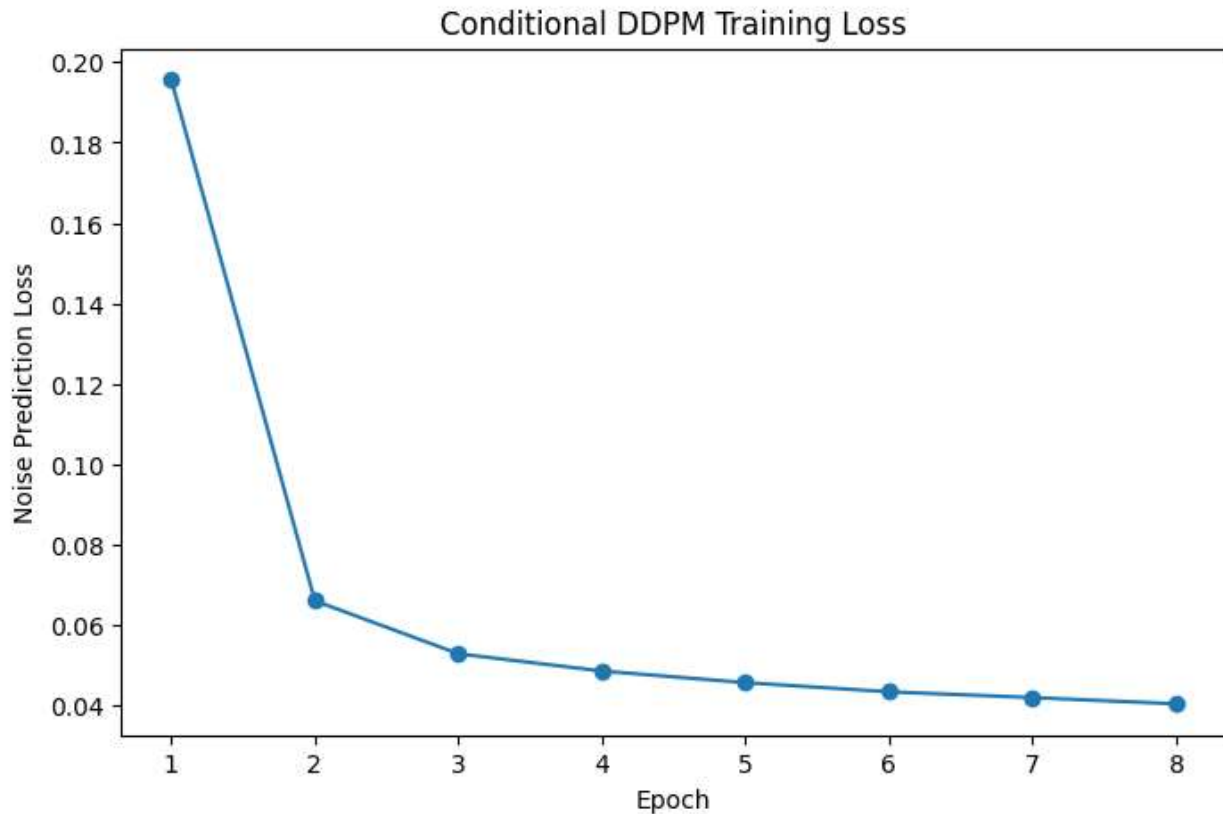
The code uses the `conditional_history` table that was created during the previous training cell. The epoch numbers are placed on the horizontal axis and the noise prediction loss is placed on the vertical axis.

Circular markers are added so that the loss value of each epoch can be seen more clearly.

The title of the graph is Conditional DDPM Training Loss.

The main purpose of this figure is to visually check whether the conditional model learned in a stable way and whether the loss decreased during training.

Output:



The graph shows a clear decrease in the conditional DDPM training loss.

The loss starts at about 0.1956 in the first epoch. It then drops quickly to about 0.0661 in the second epoch.

After that, the loss continues to decrease more slowly. It reaches about 0.0529 at epoch 3, about 0.0486 at epoch 4, and finally about 0.0405 at epoch 8.

The largest improvement happens at the beginning of training, while the later epochs show smaller changes. This is similar to the behavior of the unconditional model.

The curve is smooth and does not show large unstable changes. This suggests that the conditional model trained in a stable way.

Overall, the graph confirms the numerical results from the previous cell and shows that the conditional DDPM became better at predicting the added noise as training continued.

Cell 19:

```
conditional_labels = (  
    torch.arange(  

```

```

        64,
        device=device
    )
    *
    NUM_CLASSES
    //
    64
).long()
(
    conditional_samples,
    _,
    conditional_sampling_time
) = sample_ddpm(
    conditional_model,
    n_samples=64,
    labels=conditional_labels,
    guidance_scale=GUIDANCE_SCALE
)
grid = make_grid(
    to_image_range(
        conditional_samples
    ),
    nrow=8
)
plt.figure(
    figsize=(8, 8)
)
plt.imshow(
    grid.permute(
        1,
        2,
        0
    )
)
plt.axis(
    "off"
)
plt.title(

```

```

        "64 Conditional DDPM Samples with CFG"
    )
plt.show()
save_image(
    to_image_range(
        conditional_samples
    ),
    os.path.join(
        OUTPUT_DIR,
        "conditional_samples.png"
    ),
    nrow=8
)
print(
    "Class labels:"
)
print(
    conditional_labels
    .cpu()
    .tolist()
)
print(
    "Sampling time:",
    round(
        conditional_sampling_time,
        2
    ),
    "seconds"
)

```

This cell generates 64 images from the conditional DDPM model by using class labels and classifier free guidance.

First, the code creates a tensor of 64 labels. These labels are spread across the 10 CIFAR-10 classes, so the model receives different class conditions instead of generating all samples from the same class.

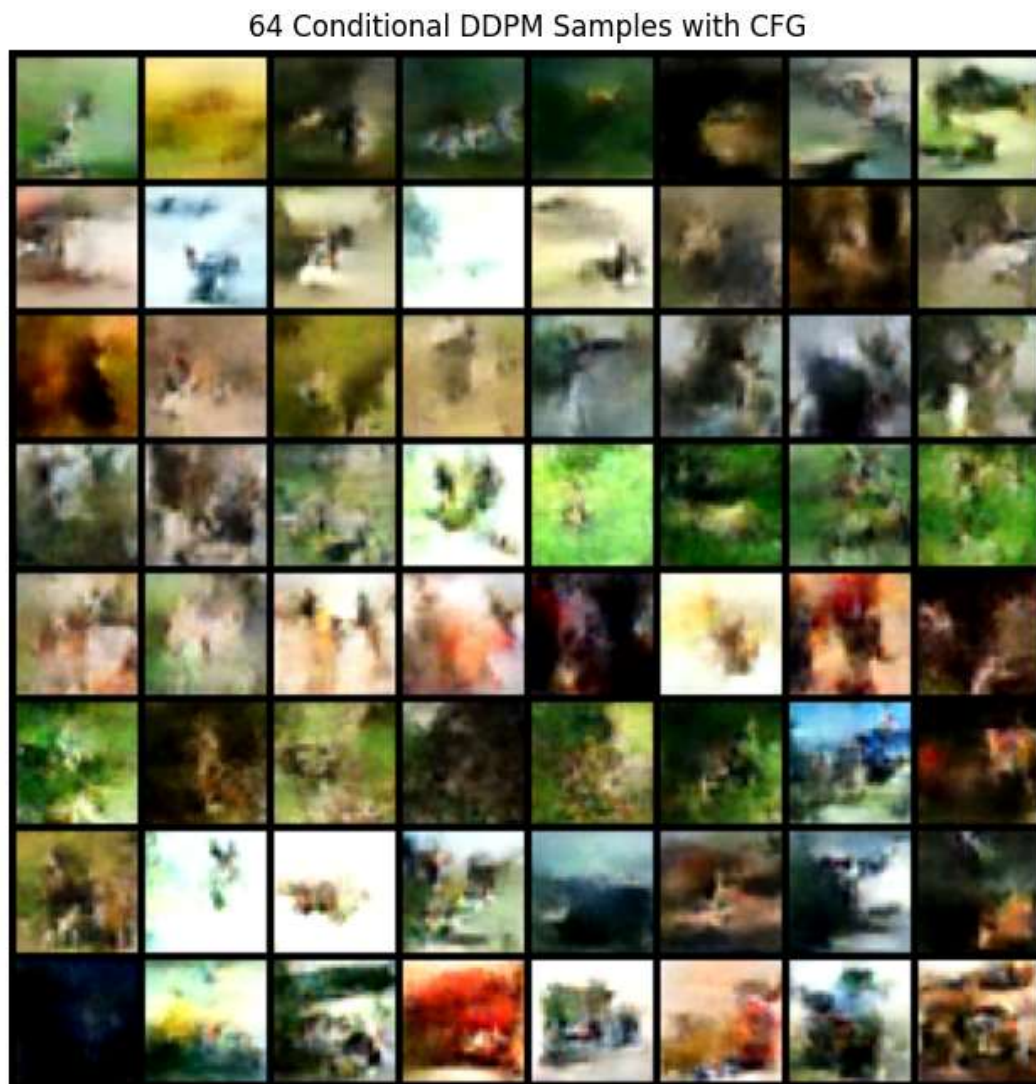
Then the `sample_ddpm` function is called with the trained `conditional_model`. The number of generated images is 64. The class labels are passed to the model, and the guidance scale is also used. This guidance helps the model follow the class condition more strongly during generation.

After sampling, the generated images are moved to the normal image range and arranged into a grid with 8 images in each row. The grid is displayed with matplotlib so we can visually inspect the final conditional results.

The same generated samples are also saved in the output folder as conditional_samples.png.

At the end, the code prints the class labels used for generation and also prints the total sampling time.

Output:



Class labels:

```
[0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3,
3, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 7, 7,
7, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 9]
```

```
Sampling time: 18.09 seconds
```

The image grid shows 64 final samples produced by the conditional diffusion model. Because class labels were given, the model tries to generate images that match those classes.

The printed label list shows that the 64 samples were divided across all 10 classes. This makes the generation more diverse and allows us to check whether the model responds to different class conditions.

The images still look blurry, which is expected because the model was trained for a short time, but many samples already show color and object structure that look more meaningful than pure random noise.

The sampling time is 18.09 seconds, which is higher than the unconditional case. This happens because classifier free guidance needs extra computation during the reverse diffusion process.

Overall, this cell completes the conditional generation part and shows that the model can generate class-conditioned samples with CFG.

Cell 20:

```
conditional_fid = compute_fid(
    real_fid_images,
    conditional_samples
)
quality_comparison = pd.DataFrame([
    {
        "Model":
            "Unconditional DDPM",
        "FID":
            unconditional_fid,
        "Samples":
            FID_SAMPLES
    },
    {
        "Model":
            "Conditional DDPM + CFG",
        "FID":
            conditional_fid,
        "Samples":
            FID_SAMPLES
    }
])
```

```
}  
])  
quality_comparison
```

This cell compares the generation quality of the unconditional DDPM and the conditional DDPM with classifier free guidance.

First, the FID score of the conditional samples is calculated using the same `compute_fid` function that was used for the unconditional model.

The same set of real CIFAR 10 images is used again so that the comparison is more consistent. The conditional samples generated in the previous cell are used as the fake images.

After calculating the conditional FID score, the results are stored in a Pandas DataFrame called `quality_comparison`.

The table contains the model name, the FID score, and the number of samples used for the calculation.

Both models are evaluated using 64 generated samples.

Output:

Model		FID	Samples
0	Unconditional DDPM	292.877930	64
1	Conditional DDPM + CFG	263.604553	64

The unconditional DDPM achieved an FID score of about 292.88, while the conditional DDPM with classifier free guidance achieved a lower FID score of about 263.60.

Since lower FID is generally better, the conditional model performed better in this experiment.

The difference is about 29.27 FID points. This suggests that adding class information and classifier free guidance helped the model generate samples that were closer to the real CIFAR 10 image distribution.

This result is also reasonable because the conditional model receives extra information about the desired class. Instead of learning only the general image distribution, it can use the class label to guide the generation process.

However, the FID values are still high for both models. This means that the generated images are still far from high quality real CIFAR 10 images.

Another important point is that the FID scores were calculated using only 64 samples. With such a small number of images, FID can be unstable and should be interpreted carefully.

Overall, the result shows that the conditional DDPM with classifier free guidance performed better than the unconditional DDPM in this experiment, although both models would need more training and probably a larger model to produce much higher quality samples.

Cell 21:

```
GAN_FID_PROBLEM4 = 68.649940
final_comparison = pd.DataFrame([
    {
        "Model":
            "Unconditional DDPM",
        "FID":
            unconditional_fid,
        "FID Samples":
            FID_SAMPLES,
        "Training Minutes":
            unconditional_training_time
            / 60,
        "Sampling Seconds":
            unconditional_sampling_time
    },
    {
        "Model":
            "Conditional DDPM + CFG",
        "FID":
            conditional_fid,
        "FID Samples":
            FID_SAMPLES,
        "Training Minutes":
            conditional_training_time
            / 60,
        "Sampling Seconds":
            conditional_sampling_time
    },
    {
```



```

        "Model":
            "DCGAN Problem 4",
        "FID":
            GAN_FID_PROBLEM4,
        "FID Samples":
            2000,
        "Training Minutes":
            21.68,
        "Sampling Seconds":
            np.nan
    }
])

final_comparison

```

This cell creates a final comparison between the two diffusion models from Problem 6 and the DCGAN result from Problem 4.

First, the FID score from the DCGAN experiment is entered manually as 68.649940.

Then a Pandas DataFrame called `final_comparison` is created. It contains three models: the Unconditional DDPM, the Conditional DDPM with classifier free guidance, and the DCGAN from Problem 4.

For each model, the table stores the FID score, the number of samples used for FID, the training time in minutes, and the sampling time when available.

For the two diffusion models, the values come directly from the current experiment. For the DCGAN, the FID score and training time come from Problem 4.

The Sampling Seconds value for DCGAN is shown as NaN because a separate sampling time was not recorded in the previous GAN experiment.

Output:

Model	FID	FID Samples	Training Minutes	Sampling Seconds	
0	Unconditional DDPM	292.877930	64	2.938861	9.204343
1	Conditional DDPM + CFG	263.604553	64	2.785827	18.089305
2	DCGAN Problem 4	68.649940	2000	21.680000	NaN

The table shows that the DCGAN has a much lower FID score than both diffusion models.

The DCGAN FID is about 68.65, while the unconditional DDPM FID is about 292.88 and the conditional DDPM FID is about 263.60.

Since lower FID is better, the DCGAN produced better image quality in these experiments.

The conditional DDPM still performed better than the unconditional DDPM, because its FID decreased from about 292.88 to about 263.60.

The training times show a different pattern. The diffusion models were trained for less than 3 minutes, while the DCGAN training took about 21.68 minutes. This happened because the diffusion models were intentionally kept small and were trained for only 8 epochs, while the GAN experiment used a much longer training process.

The unconditional DDPM needed about 9.2 seconds to generate 64 images. The conditional DDPM needed about 18.1 seconds because classifier free guidance requires both conditional and unconditional predictions during every reverse diffusion step.

An important limitation is that the FID comparison is not completely fair. The DDPM FID scores were calculated using only 64 generated samples, while the DCGAN FID was calculated using 2000 samples. FID is more reliable when more samples are used, so the absolute values should be compared carefully.

Also, the models were trained under very different training budgets. Therefore, this table is useful for showing the results of the experiments, but it should not be treated as a perfect comparison of the full capability of GANs and diffusion models.

Overall, the DCGAN achieved the best FID in this setup, while the conditional DDPM performed better than the unconditional DDPM.

Cell 22:

```
quality_comparison.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "conditional_vs_unconditional.csv"
    ),
    index=False
)
final_comparison.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "final_comparison.csv"
    ),
    index=False
)
print(
    "Problem 6 completed."
)
print(
    "All important results saved in:"
)
print(
    OUTPUT_DIR
)
```

This final cell saves the main comparison results from Problem 6 into Google Drive.

First, the `quality_comparison` table is saved as a CSV file named `conditional_vs_unconditional.csv`. This file contains the FID comparison between the unconditional DDPM and the conditional DDPM with classifier free guidance.

Then the `final_comparison` table is saved as `final_comparison.csv`. This file contains the comparison between the unconditional DDPM, the conditional DDPM with CFG, and the DCGAN result from Problem 4.

The `index=False` option is used so that the extra Pandas row numbers are not written into the CSV files.

After saving the files, the code prints a message to confirm that Problem 6 has been completed.

It also prints the Google Drive folder where all important results were saved.

Output:

```
Problem 6 completed.  
All important results saved in:  
/content/drive/MyDrive/dl_pset6_results
```

The output confirms that the final result tables were saved successfully.

The files are stored inside the dl_pset6_results folder in Google Drive.

This means that the important results from the DDPM experiments are safely saved and can be used later for the report without repeating the training process.

Overall, this final cell completes the practical part of Problem 6 and confirms that the main experiment results have been stored successfully.

Analysis Questions**1. Why is DDPM inference time orders of magnitude slower than GANs or VAEs?**

DDPM generation is much slower because it does not create the final image in only one forward pass. It starts from random noise and removes the noise gradually through many reverse diffusion steps. In our implementation, T was set to 1000. This means that the U Net had to be called repeatedly while the image moved from random noise toward the final generated sample. GANs and VAEs work differently. After training, they can usually generate an image with a single forward pass through the generator or decoder. Because of this, their inference process is much faster.

This difference was also visible in our experiment. The unconditional DDPM needed about 9.2 seconds to generate 64 images. The conditional DDPM with classifier free guidance needed about 18.1 seconds. The conditional version was even slower because classifier free guidance calculates both an unconditional prediction and a conditional prediction during each reverse diffusion step. Therefore, the model performs more neural network calculations during sampling. So, the main reason DDPM inference is slow is the large number of sequential denoising steps. The quality is built gradually instead of being produced in one direct network pass.

2. How does T , the number of diffusion steps, affect generation quality?

T controls how many small steps are used in the forward and reverse diffusion processes.

When T is large, noise can be added and removed more gradually. Each reverse step only needs to make a relatively small correction. This can make the denoising process easier for the model and can help produce better quality samples. However, increasing T also makes generation slower because the model must perform more reverse steps before the final image is produced. If T is too small, each reverse step must remove a larger amount of noise. This can make the generation process more difficult and may reduce image quality.

In our implementation, T was fixed at 1000 because this was the value required for the experiment. Therefore, we did not train several models with different T values and cannot give an experimental comparison between different values of T from our own results.

What we can observe is that with T equal to 1000, the reverse process changes the samples gradually. At $t = 1000$ the images are random noise, and as the process moves through $t = 750, 500, 250$, and 100, more visual structure appears until the final image is produced at $t = 0$. So, there is a tradeoff. More diffusion steps can provide a smoother and more gradual denoising process, but they also increase generation time. Fewer steps can make sampling faster, but if too many steps are removed, image quality may decrease.

3. Compare FID scores of your DDPM with your GAN from Problem 4.

The unconditional DDPM achieved an FID score of about 292.88. The conditional DDPM with classifier free guidance achieved a better FID score of about 263.60. In Problem 4, the DCGAN achieved an FID score of about 68.65.

Since a lower FID score is better, the DCGAN performed much better than the DDPM models in our experiments.

However, this comparison has some important limitations. The DDPM FID scores were calculated using only 64 generated images, while the DCGAN FID was calculated using 2000 generated images. FID is more reliable when a larger number of samples is used.

The training conditions were also very different. The DDPM models were small and were trained for only 8 epochs, while the DCGAN was trained for 100 epochs. Therefore, the result should not be interpreted as proof that GANs are generally better than diffusion models.

It only shows that under the training settings used in this project, the DCGAN produced a much lower FID score.

Another useful result is that the conditional DDPM improved the FID from about 292.88 to 263.60 compared with the unconditional DDPM. This suggests that adding class information and classifier free guidance helped the diffusion model produce samples that were closer to the real CIFAR 10 distribution.

Conditional vs Unconditional Generation Quality

The conditional DDPM performed better than the unconditional DDPM in our experiment.

The unconditional model had an FID of about 292.88, while the conditional model with classifier free guidance achieved about 263.60. Since lower FID is better, this is an improvement of about 29.27 FID points.

The conditional model receives information about the desired CIFAR 10 class, so it has more information during generation. Classifier free guidance also pushes the generated sample toward the conditional prediction.

The visual samples are still blurry because both diffusion models were small and trained for only 8 epochs. However, the FID result suggests that conditioning helped improve the overall generation quality.

Question: What is classifier free guidance, and why is it preferred over classifier guidance in practice?

Classifier free guidance is a method used to make conditional diffusion generation follow the requested condition more strongly without using a separate classifier.

During training, the model sometimes receives the real class label and sometimes receives no class information. In our implementation, 10% of the labels were replaced with a special null label.

Because of this, the same U Net learns two behaviors. It learns conditional noise prediction when a class label is provided, and unconditional noise prediction when the null label is provided.

During generation, both predictions are calculated. They are then combined using the guidance scale. In our experiment, the guidance scale was 2.0.

The conditional direction is strengthened by comparing the conditional prediction with the unconditional prediction. This encourages the generated image to follow the requested class more strongly.

Classifier guidance uses a separate pretrained classifier to guide the diffusion process. This requires another model and also requires gradients from that classifier during generation.

Classifier free guidance does not need this separate classifier. The same diffusion model learns both conditional and unconditional behavior. This makes the method simpler and avoids depending on the quality of an external classifier.

It is also convenient because the strength of the condition can be controlled during sampling by changing the guidance scale.

The disadvantage is that classifier free guidance requires both conditional and unconditional predictions during sampling, so it increases computation time. This was visible in our experiment. Generating 64 unconditional samples took about 9.2 seconds, while conditional generation with classifier free guidance took about 18.1 seconds.

Overall, classifier free guidance is widely useful because it provides strong conditional control without requiring a separate classifier, although it makes the sampling process more computationally expensive.

Conclusion

At the end of this project, I would like to sincerely thank my dear professor, Dr. Razzaghi, for teaching us this course and helping us understand the main ideas of Deep Learning. I am also very thankful to the TAs for patiently answering my questions and guiding me whenever I faced a problem.

This project was not always easy. There were many bugs, unexpected errors, long training processes, and different challenges during the implementation. However, these difficulties made the learning experience more real and useful for me.

Before doing these experiments, many of the concepts were mostly theoretical ideas that I had learned in class. By implementing the models myself, training them, checking their outputs, comparing their behavior, and solving the problems that appeared during the process, I was able to understand these concepts much better.

Working with CNNs, RNNs, Transformers, Autoencoders, GANs, and Diffusion Models helped me see how these architectures actually behave in practice. It also showed me that understanding Deep Learning is not only about knowing the formulas or reading the theory. A large part of the learning comes from implementation, testing, making mistakes, finding bugs, and understanding why a model does or does not work as expected.

Overall, despite all the challenges, this project was a very valuable learning experience for me. It helped me connect the theoretical knowledge from the course with real implementation, and I feel that I now understand many Deep Learning concepts more clearly and practically than before.

References

- The main source that I used during this project was the Deep Learning course slides prepared by Dr. Razzaghi. These slides were very helpful for understanding the main concepts, architectures, equations, and ideas discussed in the course, and they were my primary reference throughout the project.
- I also used educational videos from YouTube and the Maktabkhooneh website to understand some algorithms in a simpler and more visual way. These resources were especially useful when I needed another explanation of a concept before starting its implementation.
- For some parts of the Python implementation, I used the Advanced Python Programming lecture PDFs prepared by Dr. Shirazi. These materials helped me with Python syntax, functions, classes, and the general structure of the code.
- I also used ChatGPT during the project as a supporting tool. It helped me understand some parts of the problem statements more clearly, review some concepts, and find the reason for several programming errors and bugs. I mainly used it as a learning and debugging assistant while working through the implementation.
- Some additional books and research papers that were useful as references for the main topics of this project are:
 - Deep Learning, Ian Goodfellow, Yoshua Bengio, and Aaron Courville. ([MIT Press](#))
 - Deep Residual Learning for Image Recognition, Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. ([arXiv](#))
 - Auto-Encoding Variational Bayes, Diederik P. Kingma and Max Welling. ([arXiv](#))
 - Attention Is All You Need, Ashish Vaswani et al. ([arXiv](#))
 - Generative Adversarial Nets, Ian Goodfellow et al.
 - Improved Training of Wasserstein GANs, Ishaan Gulrajani et al. ([arXiv](#))
 - Denoising Diffusion Probabilistic Models, Jonathan Ho, Ajay Jain, and Pieter Abbeel. ([arXiv](#))
- These references helped me connect the practical experiments in the project with the original ideas and architectures that were discussed during the course.